

铁路信号设备数字孪生系统源代码整理稿

1. 项目概述

铁路信号设备数字孪生系统 是面向铁路信号监测与可视化展示场景的软件系统，采用 Vue 3 + Vite 构建单页应用，结合 Cesium、Three.js 与大屏可视化技术，实现山区铁路信号设备地理展示、数字孪生、恶劣天气演练、告警联动、遥测接入和综合分析展示。

2. 源代码整理说明

- 本整理稿仅纳入当前仓库中真实存在且与系统直接相关的自研代码、配置与辅助服务文件。
- 未纳入第三方依赖、构建产物、静态音视频资源及无关文件。
- 本稿共纳入 20 个文件，总计 14361 行，其中 `src` 目录代码量为 14098 行。
- 代码按基础层、数据与服务层、表现层、参考与辅助层分组，以便后续导出 PDF 与人工核对。

3. 目录结构说明

```
railway_sign/
├─ index.html
├─ package.json
├─ vite.config.js
├─ server/
│   └─ telemetry-bridge.js
├─ src/
│   ├─ main.js
│   ├─ App.vue
│   ├─ config/
│   │   └─ demoScenario.js
│   ├─ composables/
│   │   └─ useApiData.js
│   ├─ services/
│   │   ├─ api.js
│   │   ├─ mockDataService.js
│   │   ├─ simulationState.js
│   │   └─ telemetrySocket.js
│   ├─ components/
│   │   ├─ CesiumView.vue
│   │   ├─ ThreeView.vue
│   │   ├─ DataPanel.vue
│   │   └─ datapanel/
│   │       ├─ LeftPanel.vue
│   │       ├─ CenterPanel.vue
│   │       └─ RightPanel.vue
│   └─ js/
│       ├─ cesium.js
│       └─ threejs.js
└─ docs/
    └─ 本次整理输出材料
```

4. 模块划分说明

- 基础层：应用入口、顶层界面组织、场景参数与工程构建配置。
- 数据与服务层：模拟数据接口、趋势生成、全局状态同步、遥测连接与组合式调用。
- 表现层：Cesium 地图、Three.js 数字孪生与大屏可视化组件。
- 参考与辅助层：早期脚本版本及 Node.js 遥测桥接服务。

3. 模块：基础层

基础层包含应用入口、顶层布局、构建配置与演示场景参数，负责前端应用装配、界面入口组织以及全局基础配置。

3.1 文件：index.html

- 文件说明：前端单页应用挂载点与页面基础容器。
- 行数统计：14 行

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" type="image/svg+xml" href="/favicon.svg">
```

```
<title>山区铁路信号灯数字孪生系统</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
```

3.2 文件: vite.config.js

- 文件说明: Vite 构建配置, 集成 Vue 与 Cesium 插件。
- 行数统计: 18 行

```
import { defineConfig } from 'vite'
import cesium from 'vite-plugin-cesium'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue(), cesium()],
  server: {
    port: 4002,
    open: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4002,
  },
  build: {
    target: "esnext",
  },
});
```

3.3 文件: src/main.js

- 文件说明: Vue 应用入口, 挂载根组件。
- 行数统计: 5 行

```
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

3.4 文件: src/App.vue

- 文件说明: 应用顶层布局与三类主视图切换入口。
- 行数统计: 112 行

```
<template>
  <div id="app">
    <div class="tab-container">
      <button
        :class="['tab-btn', { active: currentTab === 'cesium' }]"
        @click="switchTab('cesium')"
      >
        🗺️ 地理信息可视化
      </button>
      <button
        :class="['tab-btn', { active: currentTab === 'three' }]"
        @click="switchTab('three')"
      >
        🚂 铁路信号设备孪生面板
      </button>
      <button
        :class="['tab-btn', { active: currentTab === 'data' }]"
        @click="switchTab('data')"
      >
        📊 数据可视化平台
      </button>
    </div>
```

```

<div class="content-container">
  <CesiumView v-if="currentTab === 'cesium'" />
  <ThreeView v-else-if="currentTab === 'three'" />
  <DataPanel v-else-if="currentTab === 'data'" />
</div>
</div>
</template>

```

```

<script setup>
import { ref } from 'vue'
import CesiumView from './components/CesiumView.vue'
import ThreeView from './components/ThreeView.vue'
import DataPanel from './components/DataPanel.vue'

```

```

const currentTab = ref('three')

```

```

const switchTab = (tab) => {
  currentTab.value = tab
}

```

```

</script>

```

```

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

```

```

html, body, #app {
  width: 100%;
  height: 100%;
  overflow: hidden;
  font-family: 'Microsoft YaHei', Arial, sans-serif;
  background: #000;
}

```

```

.tab-container {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  height: 50px;
  background: rgba(0, 20, 40, 0.95);
  border-bottom: 2px solid rgba(0, 200, 255, 0.3);
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  z-index: 1000;
  backdrop-filter: blur(10px);
  box-shadow: 0 4px 20px rgba(0, 200, 255, 0.2);
}

```

```

.tab-btn {
  padding: 8px 20px;
  background: rgba(0, 100, 150, 0.3);
  border: 2px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  color: #aaa;
  font-size: 15px;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.3s;
}

```

```

display: flex;
align-items: center;
gap: 8px;
}

.tab-btn:hover {
background: rgba(0, 100, 150, 0.5);
color: #00d4ff;
transform: translateY(-2px);
box-shadow: 0 4px 15px rgba(0, 200, 255, 0.3);
}

.tab-btn.active {
background: linear-gradient(135deg, #0066cc, #00d4ff);
color: #fff;
border-color: #00d4ff;
box-shadow: 0 0 20px rgba(0, 200, 255, 0.5);
}

.content-container {
width: 100%;
height: 100%;
padding-top: 50px;
}
</style>

```

3.5 文件: src/config/demoScenario.js

- 文件说明: 演示场景名称、站点、告警文本等基础业务常量。
- 行数统计: 23 行

```

export const DEMO_SCENARIO = {
  regionName: '铁路信号设备数字孪生系统',
  intervalName: '1491-1497 区间演示',
  stationStart: '1491',
  stationEnd: '1497',
  railwayName: '铁路信号设备数字孪生系统',
  weatherLocation: '铁路沿线',
  heroDeviceName: '1495信号机',
  humidityIncident: {
    title: '环境湿度预警',
    alarmContent: '箱体内部湿度偏高, 建议巡检密封状态。',
    analysis: '当前为演示数据, 用于联动孪生面板的环境异常提示。',
    suggestion: '建议核查箱体密封、引线接头与排水条件。'
  },
  nearbyStations: {
    main: '1495信号机',
    freight: '1493信号机',
    hs: '1497信号机'
  }
}

export default DEMO_SCENARIO

```

4. 模块: 数据与服务层

数据与服务层负责本地模拟数据生成、前端数据访问封装、跨视图状态同步、遥测 WebSocket 连接以及组合式数据调用。

4.1 文件: src/services/api.js

- 文件说明: 前端本地内存数据服务, 提供信号设备、天气、地震、统计、对话、日志等接口。
- 行数统计: 301 行

```

const nowIso = () => new Date().toISOString()

const wait = (ms = 80) => new Promise((resolve) => setTimeout(resolve, ms))

const memory = {

```

```

signals: [
  {
    id: 1,
    name: '\u4e3b\u4fe1\u53f7\u706f',
    status: 'red',
    temperature: 35.2,
    humidity: 66.4,
    light_intensity: 920,
    voltage: 220.6,
    current: 2.41,
    signal_strength: -47,
  },
],
trainStats: {
  passed_count: 42,
  total_count: 58,
  on_time_rate: 96.5,
  next_train: 'G1502 14:35',
},
railway: {
  name: 'XX铁路',
  start_station: 'X站',
  end_station: 'Y站',
  length_km: 255,
},
aiConversations: [
  {
    id: 1,
    session_id: 'default',
    role: 'assistant',
    content: '\u7cfb\u7edf\u5df2\u5207\u6362\u5230\u7aef\u672c\u5730\u6570\u636e\u6a21\u5f0f\u3002',
    created_at: nowIso(),
  },
],
logs: [
  {
    id: 1,
    log_type: 'system',
    log_level: 'info',
    message: 'Frontend-only mode enabled',
    details: null,
    created_at: nowIso(),
  },
],
}

const randomFloat = (min, max, precision = 1) => {
  const factor = 10 ** precision
  return Math.round((min + Math.random() * (max - min)) * factor) / factor
}

const generateSeries = (length, min, max, mapValue) => {
  return Array.from({ length }, (_, index) => {
    const value = randomFloat(min, max, 2)
    return mapValue(value, index)
  })
}

const getSeriesLength = (range) => {
  if (range === 'week') return 7
  if (range === 'month') return 30
  return 24
}

```

```
const getSignals = async () => {
  await wait()
  return memory.signals.map((item) => ({ ...item }))
}

const getSignal = async (id) => {
  await wait()
  return memory.signals.find((item) => item.id === Number(id)) || null
}

const createSignal = async (data) => {
  await wait()
  const id = memory.signals.length ? Math.max(...memory.signals.map((item) => item.id)) + 1 : 1
  const created = { id, ...data }
  memory.signals.push(created)
  return created
}

const updateSignal = async (id, data) => {
  await wait()
  const index = memory.signals.findIndex((item) => item.id === Number(id))
  if (index === -1) return null
  memory.signals[index] = { ...memory.signals[index], ...data }
  return memory.signals[index]
}

const deleteSignal = async (id) => {
  await wait()
  const index = memory.signals.findIndex((item) => item.id === Number(id))
  if (index === -1) return { deleted: false }
  memory.signals.splice(index, 1)
  return { deleted: true }
}

const getSeismicData = async (range = '24h') => {
  await wait()
  return generateSeries(getSeriesLength(range), 1.2, 4.2, (value, index) => ({
    id: index + 1,
    level: value,
    location: 'XX区域',
    recorded_at: nowIso(),
  })))
}

const addSeismicData = async (level, location) => {
  await wait()
  return {
    id: Date.now(),
    level,
    location,
    recorded_at: nowIso(),
  }
}

const getWeatherData = async (range = '24h') => {
  await wait()
  return generateSeries(getSeriesLength(range), 16, 34, (value, index) => ({
    id: index + 1,
    temperature: value,
    humidity: Math.round(randomFloat(45, 90, 0)),
    wind_speed: `${randomFloat(1.2, 5.8, 1)}m/s`,
    visibility: `${Math.round(randomFloat(6, 18, 0))}km`,
    pressure: `${Math.round(randomFloat(1002, 1022, 0))}hPa`,
    recorded_at: nowIso(),
  })))
}
```

```

    )))
  }

const getCurrentWeather = async () => {
  await wait()
  return {
    icon: '\u26c5',
    temperature: randomFloat(20, 33, 1),
    description: '\u591a\u4e91',
    location: 'XX区域',
    wind_speed: `${randomFloat(2.0, 4.5, 1)}m/s`,
    humidity: `${Math.round(randomFloat(55, 85, 0))}%`,
    visibility: `${Math.round(randomFloat(8, 16, 0))}km`,
    pressure: `${Math.round(randomFloat(1006, 1020, 0))}hPa`,
  }
}

const getAirQuality = async () => {
  await wait()
  const aqi = Math.round(randomFloat(35, 95, 0))
  return {
    aqi,
    level: aqi <= 50 ? '\u4f18' : '\u826f',
    pm25: Math.round(randomFloat(15, 45, 0)),
    pm10: Math.round(randomFloat(30, 80, 0)),
    o3: Math.round(randomFloat(40, 90, 0)),
    no2: Math.round(randomFloat(20, 50, 0)),
  }
}

const getTrainStats = async () => {
  await wait()
  return { ...memory.trainStats }
}

const updateTrainStats = async (data) => {
  await wait()
  memory.trainStats = { ...memory.trainStats, ...data }
  return { ...memory.trainStats }
}

const getRailway = async () => {
  await wait()
  return { ...memory.railway }
}

const getParamHistory = async (type = 'temperature', range = '24h', signalId = 1) => {
  await wait()
  const settings = {
    temperature: { min: 22, max: 45 },
    humidity: { min: 40, max: 90 },
    light: { min: 100, max: 1800 },
    voltage: { min: 205, max: 230 },
    current: { min: 1.4, max: 3.8 },
    signal: { min: -75, max: -40 },
  }
  const { min, max } = settings[type] || settings.temperature

  return generateSeries(getSeriesLength(range), min, max, (value, index) => ({
    id: index + 1,
    signal_id: signalId,
    param_type: type,
    param_value: value,
    created_at: nowIso(),
  }

```

```

    )))
  }

const getAiConversations = async (sessionId) => {
  await wait()
  if (!sessionId) return [...memory.aiConversations]
  return memory.aiConversations.filter((item) => item.session_id === sessionId)
}

const addAiConversation = async (sessionId, role, content) => {
  await wait()
  const created = {
    id: Date.now(),
    session_id: sessionId || 'default',
    role,
    content,
    created_at: nowIso(),
  }
  memory.aiConversations.push(created)
  return created
}

const getLogs = async (type, limit = 100) => {
  await wait()
  const source = type ? memory.logs.filter((item) => item.log_type === type) : memory.logs
  return source.slice(-Math.max(1, limit))
}

const addLog = async (logType, logLevel, message, details) => {
  await wait()
  const created = {
    id: Date.now(),
    log_type: logType,
    log_level: logLevel,
    message,
    details: details || null,
    created_at: nowIso(),
  }
  memory.logs.push(created)
  return created
}

const getDashboard = async () => {
  const [weather, airQuality, trainStats, railway, seismic] = await Promise.all([
    getCurrentWeather(),
    getAirQuality(),
    getTrainStats(),
    getRailway(),
    getSeismicData('24h'),
  ])

  return {
    weather,
    airQuality,
    trainStats,
    railway,
    seismic,
  }
}

export {
  getSignals,
  getSignal,
  createSignal,

```

```

    updateSignal,
    deleteSignal,
    getSeismicData,
    addSeismicData,
    getWeatherData,
    getCurrentWeather,
    getAirQuality,
    getTrainStats,
    updateTrainStats,
    getRailway,
    getParamHistory,
    getAiConversations,
    addAiConversation,
    getLogs,
    addLog,
    getDashboard,
  }

export default {
  getSignals,
  getSignal,
  createSignal,
  updateSignal,
  deleteSignal,
  getSeismicData,
  addSeismicData,
  getWeatherData,
  getCurrentWeather,
  getAirQuality,
  getTrainStats,
  updateTrainStats,
  getRailway,
  getParamHistory,
  getAiConversations,
  addAiConversation,
  getLogs,
  addLog,
  getDashboard,
}

```

4.2 文件: src/services/mockDataService.js

- 文件说明: 大数据面板模拟数据生成器, 负责设备、天气、趋势、寿命和告警数据构造。
- 行数统计: 775 行

```

/**
 * 大数据面板 Mock 数据服务
 * 提供各种传感器、气象、设备监控等模拟数据
 */

import { severeWeatherEnabled, severeWeatherStartedAt, humidityMitigationAppliedAt, syncedHumidity } from './simulationState.js'

// 随机数生成工具
const random = (min, max, decimal = 0) => {
  const value = Math.random() * (max - min) + min
  return decimal > 0 ? parseFloat(value.toFixed(decimal)) : Math.floor(value)
}

// 可复现的随机数（避免趋势曲线每次刷新都“跳”）
const mulberry32 = (seed) => {
  let t = seed >>> 0
  return () => {
    t += 0x6d2b79f5
    let r = Math.imul(t ^ (t >>> 15), 1 | t)
    r ^= r + Math.imul(r ^ (r >>> 7), 61 | r)
    return ((r ^ (r >>> 14)) >>> 0) / 4294967296
  }
}

```

```

    }
  }

  // 随机选择数组元素
  const randomChoice = (arr) => arr[random(0, arr.length)]

  // 生成时间序列数据
  const generateTimeSeries = (length, min, max, decimal = 1) => {
    return Array.from({ length }, () => random(min, max, decimal))
  }

  // 气象数据
  const risingSeries = (length, start, end, jitter = 0.6, decimal = 1) => {
    if (length <= 1) return [Number(end.toFixed(decimal))]
    return Array.from({ length }, (_, i) => {
      const t = i / (length - 1)
      const base = start + (end - start) * t
      // 增加小幅波动: 在整体上升的基础上加入轻微正弦扰动
      const wave = Math.sin(t * Math.PI * 4) * (jitter * 0.55)
      return Math.max(0, Math.min(100, random(base + wave - jitter, base + wave + jitter, decimal)))
    })
  }

  const clamp = (value, min, max) => Math.max(min, Math.min(max, value))

  const buildHumidityTrend = ({ seed, baselineEnd, peak, current, phaseT, mode }) => {
    const total = 24
    const rng = mulberry32(seed)

    // baseline: 平稳区间 (开场应在 20~40 左右)
    const baselineCount = clamp(
      mode === 'falling' ? 10 : 24 - clamp(Math.floor(2 + phaseT * 10), 2, 12),
      0,
      total - 2
    )
    const baseline = Array.from({ length: baselineCount }, (_, idx) => {
      const n = rng()
      const wave = Math.sin((idx / Math.max(1, baselineCount - 1)) * Math.PI * 2) * 0.6
      const v = 22 + n * 8 + wave // 约 22~30
      return Number(clamp(v, 18, 36).toFixed(1))
    })

    if (!baseline.length) baseline.push(Number(clamp(baselineEnd, 18, 36).toFixed(1)))

    const remain = total - baseline.length
    const riseLen = mode === 'falling' ? clamp(Math.floor(6 + phaseT * 6), 6, 12) : remain
    const fallLen = mode === 'falling' ? clamp(remain - riseLen, 4, remain) : 0

    const riseCount = mode === 'falling' ? clamp(riseLen, 2, total - 2) : remain
    const fallCount = mode === 'falling' ? clamp(fallLen, 2, total - riseCount) : 0

    const startRise = baseline[baseline.length - 1]
    const endRise = Number(clamp(peak, 0, 100).toFixed(1))

    const rise = Array.from({ length: riseCount }, (_, idx) => {
      const t = riseCount <= 1 ? 1 : idx / (riseCount - 1)
      // 轻微加速上升: 前期更平缓, 避免一开场就冲到高值
      const eased = t * t * (3 - 2 * t)
      const v = startRise + (endRise - startRise) * eased
      const jitter = (rng() - 0.5) * 1.2
      return Number(clamp(v + jitter, 0, 100).toFixed(1))
    })

    const fallStart = rise.length ? rise[rise.length - 1] : endRise

```

```

const fallEnd = Number(clamp(current, 0, 100).toFixed(1))
const fall = Array.from({ length: fallCount }, (_, idx) => {
  const t = fallCount <= 1 ? 1 : idx / (fallCount - 1)
  const eased = t * t * (3 - 2 * t)
  const v = fallStart + (fallEnd - fallStart) * eased
  const jitter = (rng() - 0.5) * 0.9
  return Number(clamp(v + jitter, 0, 100).toFixed(1))
})

let out = [...baseline, ...rise, ...fall]
out = out.slice(0, total)

// 强制最后一个点与当前值一致，保证“模拟进行中”时曲线随时间上升/回落
out[total - 1] = fallEnd

// 避免刚打开就出现“高湿度尾巴”：把尾部（近 3 个点）限制在当前值附近
for (let i = total - 3; i < total - 1; i++) {
  if (i < 0) continue
  out[i] = Number(clamp(out[i], 0, fallEnd + 3).toFixed(1))
}

return out
}

export const getWeatherData = () => {
  const enabled = Boolean(severeWeatherEnabled.value)

  if (!enabled) {
    return {
      temperature: random(-5, 35, 1),
      humidity: random(21, 25, 1),
      windSpeed: random(0, 25, 1),
      windDirection: randomChoice(['北', '东北', '东', '东南', '南', '西南', '西', '西北']),
      pressure: random(990, 1030, 1),
      visibility: random(1, 30, 1),
      precipitation: random(0, 50, 1),
      uvIndex: random(0, 11),
      airQuality: random(0, 300),
      pm25: random(0, 150),
      pm10: random(0, 200),
      so2: random(0, 50, 2),
      no2: random(0, 100, 2),
      co: random(0, 2, 2),
      o3: random(0, 200, 1),
      weatherType: randomChoice(['晴', '多云', '阴', '小雨', '中雨', '大雨', '雷阵雨', '小雪', '中雪', '大雪', '雾', '霾']),
      temperatureTrend: generateTimeSeries(24, -5, 35, 1),
      humidityTrend: generateTimeSeries(24, 21, 25, 1),
      forecast: Array.from({ length: 7 }, (_, i) => ({
        date: new Date(Date.now() + i * 86400000).toLocaleDateString('zh-CN', { weekday: 'short' }),
        high: random(15, 35),
        low: random(-5, 20),
        weather: randomChoice(['晴', '多云', '阴', '小雨', '中雨', '雷阵雨'])
      })))
    }
  }
}

// 恶劣天气演练（Early）：
// - 湿度在 4s 内进入红色告警（>=70%），随后缓慢上升至高位平台（~92%）
// - “异常消除”后进入处置回落（约 90s 回落到 35% 左右，进入观察）
const startedAt = severeWeatherStartedAt.value || Date.now()
const now = Date.now()
const elapsed = Math.max(0, now - startedAt)
const stage1Ms = 4000
const stage2Ms = 20000

```

```

const base = 22
const redTarget = 75
const peak = 92
const easeOutQuad = (t) => t * (2 - t)
const lerp = (a, b, t) => a + (b - a) * t
const humidityAtTime = (t0) => {
  const e = Math.max(0, Number(t0) || 0)
  if (e <= stage1Ms) {
    const t = easeOutQuad(clamp(e / stage1Ms, 0, 1))
    return Number(lerp(base, redTarget, t).toFixed(1))
  }
  if (e <= stage1Ms + stage2Ms) {
    const t = easeOutQuad(clamp((e - stage1Ms) / stage2Ms, 0, 1))
    return Number(lerp(redTarget, peak, t).toFixed(1))
  }
  return Number(peak.toFixed(1))
}
const riseT = Math.max(0, Math.min(1, elapsed / stage1Ms))

const mitigationAt = humidityMitigationAppliedAt.value
const synced = Number(syncedHumidity.value)
let humidity = Number.isFinite(synced) ? synced : humidityAtTime(elapsed)
let humidityTrend = []

if (Number.isFinite(Number(mitigationAt)) && mitigationAt >= startedAt) {
  const mitigationElapsed = Math.max(0, now - mitigationAt)
  const humidityAtMitigation = humidityAtTime(mitigationAt - startedAt)
  const fallDurationMs = 90000
  const fallT = clamp(mitigationElapsed / fallDurationMs, 0, 1)

  humidityTrend = buildHumidityTrend({
    seed: Math.floor(startedAt / 1000),
    baselineEnd: 24,
    peak: humidityAtMitigation,
    current: humidity,
    phaseT: fallT,
    mode: 'falling'
  })
} else {
  humidityTrend = buildHumidityTrend({
    seed: Math.floor(startedAt / 1000),
    baselineEnd: 24,
    peak: humidity,
    current: humidity,
    phaseT: riseT,
    mode: 'rising'
  })
}

if (Array.isArray(humidityTrend) && humidityTrend.length) {
  humidityTrend[humidityTrend.length - 1] = humidity
}

return {
  temperature: random(12, 28, 1),
  humidity,
  windSpeed: random(10, 25, 1),
  windDirection: randomChoice(['东北', '东', '东南', '北']),
  pressure: random(985, 1008, 1),
  visibility: random(1, 6, 1),
  precipitation: random(20, 50, 1),
  uvIndex: random(0, 3),
  airQuality: random(160, 260),
  pm25: random(110, 180),

```

```

    pm10: random(160, 240),
    so2: random(10, 35, 2),
    no2: random(60, 120, 2),
    co: random(0.6, 1.6, 2),
    o3: random(80, 160, 1),
    weatherType: randomChoice(['大雨', '暴雨', '雷阵雨', '霾']),
    temperatureTrend: generateTimeSeries(24, 10, 28, 1),
    humidityTrend,
    forecast: Array.from({ length: 7 }, (_, i) => ({
      date: new Date(Date.now() + i * 86400000).toLocaleDateString('zh-CN', { weekday: 'short' }),
      high: random(18, 28),
      low: random(10, 18),
      weather: randomChoice(['中雨', '大雨', '雷阵雨', '阴'])
    })))
  }
}

// 传感器数据
export const getSensorData = () => {
  const sensors = []
  const sensorTypes = [
    { type: '温度传感器', unit: '°C', min: -20, max: 60, icon: 'thermometer' },
    { type: '湿度传感器', unit: '%', min: 0, max: 100, icon: 'droplet' },
    { type: '压力传感器', unit: 'MPa', min: 0, max: 10, decimal: 2, icon: 'gauge' },
    { type: '振动传感器', unit: 'mm/s', min: 0, max: 50, decimal: 2, icon: 'activity' },
    { type: '电流传感器', unit: 'A', min: 0, max: 100, decimal: 2, icon: 'zap' },
    { type: '电压传感器', unit: 'V', min: 200, max: 250, decimal: 1, icon: 'bolt' },
    { type: '位移传感器', unit: 'mm', min: 0, max: 100, decimal: 3, icon: 'move' },
    { type: '流量传感器', unit: 'm³/h', min: 0, max: 1000, decimal: 1, icon: 'flow' },
    { type: '转速传感器', unit: 'RPM', min: 0, max: 3000, icon: 'rotate' },
    { type: '噪音传感器', unit: 'dB', min: 20, max: 120, icon: 'volume' },
    { type: '烟雾传感器', unit: 'ppm', min: 0, max: 500, decimal: 1, icon: 'smoke' },
    { type: '光照传感器', unit: 'lux', min: 0, max: 100000, icon: 'sun' }
  ]

  for (let i = 1; i <= 24; i++) {
    const config = sensorTypes[(i - 1) % sensorTypes.length]
    const status = Math.random() > 0.1 ? 'normal' : (Math.random() > 0.5 ? 'warning' : 'error')
    sensors.push({
      id: `SENSOR-${String(i).padStart(3, '0')}`,
      name: `${config.type}-${i}`,
      type: config.type,
      value: random(config.min, config.max, config.decimal || 0),
      unit: config.unit,
      status,
      icon: config.icon,
      location: randomChoice(['A区', 'B区', 'C区', 'D区', '主站', '信号楼', '调度中心']),
      updateTime: new Date().toLocaleTimeString(),
      threshold: {
        min: config.min + (config.max - config.min) * 0.1,
        max: config.max - (config.max - config.min) * 0.1
      },
      history: generateTimeSeries(30, config.min, config.max, config.decimal || 0)
    })
  }
  return sensors
}

// 设备监控数据
export const getEquipmentData = () => {
  const equipments = []
  const equipmentTypes = [
    { type: '信号机', icon: 'signal', count: 45 },
    { type: '道岔', icon: 'switch', count: 32 },

```

```

    { type: '轨道电路', icon: 'track', count: 128 },
    { type: 'UPS电源', icon: 'battery', count: 16 },
    { type: '通信设备', icon: 'network', count: 24 },
    { type: '监控摄像头', icon: 'camera', count: 86 },
    { type: '列车检测器', icon: 'train', count: 18 },
    { type: '防雷设备', icon: 'lightning', count: 42 }
  ]

  equipmentTypes.forEach(({ type, icon, count }) => {
    const normal = random(Math.floor(count * 0.7), count)
    const warning = random(0, count - normal)
    const error = count - normal - warning
    equipments.push({
      type,
      icon,
      total: count,
      normal,
      warning,
      error,
      onlineRate: ((normal / count) * 100).toFixed(1),
      maintenanceCount: random(0, 5),
      avgRunningTime: random(100, 10000)
    })
  })
  return equipments
}

// 告警数据
export const getAlarmData = () => {
  const alarms = []
  const alarmTypes = [
    { level: 'critical', title: '设备严重故障', desc: '信号机XS-012通信中断' },
    { level: 'critical', title: '轨道电路异常', desc: 'GJ-003区段电压异常' },
    { level: 'warning', title: '温度超限预警', desc: '机房温度超过35°C' },
    { level: 'warning', title: 'UPS电量不足', desc: 'UPS-02电量低于20%' },
    { level: 'info', title: '设备维护提醒', desc: '道岔DC-008即将到期检修' },
    { level: 'info', title: '系统升级通知', desc: '计划于今晚22:00进行系统升级' },
    { level: 'warning', title: '网络延迟预警', desc: '主网络延迟超过50ms' },
    { level: 'critical', title: '安全事件', desc: '检测到异常访问尝试' }
  ]

  for (let i = 0; i < 15; i++) {
    const alarm = randomChoice(alarmTypes)
    alarms.push({
      id: `ALM-${Date.now()}-${i}`,
      ...alarm,
      time: new Date(Date.now() - random(0, 86400000)).toLocaleString('zh-CN'),
      source: randomChoice(['A区', 'B区', 'C区', '调度中心', '信号楼']),
      handled: Math.random() > 0.7
    })
  }

  return alarms.sort((a, b) => new Date(b.time) - new Date(a.time))
}

// 列车运行数据
export const getTrainData = () => ({
  totalTrains: random(50, 200),
  runningTrains: random(20, 80),
  delayedTrains: random(0, 10),
  avgDelay: random(0, 15, 1),
  punctualityRate: random(85, 99.9, 1),
  todayTrips: random(200, 500),
  passengerFlow: random(50000, 200000),

```

```

freightTonnage: random(1000, 10000),
avgSpeed: random(60, 120, 1),
trainsByLine: [
  { line: 'XX1线', count: random(10, 30) },
  { line: 'XX2线', count: random(10, 30) },
  { line: 'XX3线', count: random(10, 30) },
  { line: 'XX4线', count: random(10, 30) },
  { line: 'XX5线', count: random(10, 30) }
],
trainsByType: [
  { type: '高铁', count: random(20, 50), color: '#00d4ff' },
  { type: '动车', count: random(15, 40), color: '#00ff88' },
  { type: '普快', count: random(30, 60), color: '#ffaa00' },
  { type: '货运', count: random(40, 80), color: '#ff6b6b' }
],
realtimePositions: Array.from({ length: 20 }, (_, i) => ({
  id: `G${1000 + i}`,
  line: randomChoice(['XX1', 'XX2', 'XX3', 'XX4', 'XX5']),
  position: [random(115, 125, 4), random(30, 45, 4)],
  speed: random(80, 350),
  status: randomChoice(['正常运行', '即将到站', '正在发车', '临时停车'])
})))
})

// 能耗数据
export const getEnergyData = () => ({
  totalConsumption: random(50000, 100000),
  dailyConsumption: generateTimeSeries(24, 1000, 5000),
  monthlyConsumption: generateTimeSeries(12, 50000, 150000),
  powerLoad: random(1000, 5000, 1),
  peakLoad: random(4000, 6000, 1),
  valleyLoad: random(500, 1500, 1),
  loadRate: random(60, 95, 1),
  energyByType: [
    { type: '牵引供电', value: random(40, 60), color: '#00d4ff' },
    { type: '信号设备', value: random(10, 20), color: '#00ff88' },
    { type: '照明系统', value: random(5, 15), color: '#ffaa00' },
    { type: '空调系统', value: random(15, 25), color: '#ff6b6b' },
    { type: '其他设备', value: random(5, 10), color: '#aa88ff' }
  ],
  carbonEmission: random(100, 500, 1),
  energySavingRate: random(5, 20, 1)
})

// 系统状态数据
export const getSystemStatus = () => ({
  cpuUsage: random(20, 80, 1),
  memoryUsage: random(30, 70, 1),
  diskUsage: random(40, 90, 1),
  networkIn: random(10, 1000, 1),
  networkOut: random(10, 500, 1),
  activeConnections: random(100, 1000),
  requestPerSecond: random(100, 5000),
  avgResponseTime: random(10, 200, 1),
  uptime: random(100, 365),
  services: [
    { name: '主服务器', status: 'running', cpu: random(10, 50, 1), memory: random(20, 60, 1) },
    { name: '数据库服务', status: 'running', cpu: random(10, 40, 1), memory: random(30, 70, 1) },
    { name: '缓存服务', status: 'running', cpu: random(5, 30, 1), memory: random(40, 80, 1) },
    { name: '消息队列', status: 'running', cpu: random(5, 25, 1), memory: random(20, 50, 1) },
    { name: '文件服务', status: Math.random() > 0.1 ? 'running' : 'warning', cpu: random(5, 20, 1), memory: random(10, 40, 1) }
  ],
  recentLogs: Array.from({ length: 10 }, (_, i) => ({
    time: new Date(Date.now() - i * 60000).toLocaleTimeString(),

```

```

    level: randomChoice(['INFO', 'WARN', 'ERROR', 'DEBUG']),
    message: randomChoice([
      '用户登录成功',
      '数据同步完成',
      'API请求超时',
      '数据库连接池已满',
      '缓存命中率: 95.2%',
      '定时任务执行完毕',
      '收到MQTT消息',
      'WebSocket连接建立'
    ])
  })
}))
})

// 安全监控数据
export const getSecurityData = () => ({
  intrusionAttempts: random(0, 50),
  blockedIPs: random(100, 500),
  activeUsers: random(50, 200),
  failedLogins: random(0, 20),
  securityScore: random(70, 100),
  firewallStatus: 'active',
  lastScanTime: new Date(Date.now() - random(0, 3600000)).toLocaleString('zh-CN'),
  vulnerabilities: {
    critical: random(0, 3),
    high: random(0, 10),
    medium: random(0, 20),
    low: random(0, 50)
  },
  recentEvents: Array.from({ length: 8 }, (_, i) => ({
    time: new Date(Date.now() - i * 1800000).toLocaleTimeString(),
    type: randomChoice(['登录', '登出', '权限变更', '配置修改', '文件访问']),
    user: `user${random(1, 100)}`,
    ip: `${random(1, 255)}.${random(1, 255)}.${random(1, 255)}.${random(1, 255)}`,
    status: randomChoice(['成功', '失败', '待审核'])
  })))
})

// 运维工单数据
export const getWorkOrderData = () => ({
  pending: random(5, 20),
  processing: random(10, 30),
  completed: random(50, 100),
  overdue: random(0, 5),
  avgProcessTime: random(2, 48, 1),
  satisfaction: random(85, 99, 1),
  ordersByType: [
    { type: '设备故障', count: random(10, 30), color: '#ff6b6b' },
    { type: '日常巡检', count: random(20, 50), color: '#00d4ff' },
    { type: '预防维护', count: random(15, 40), color: '#00ff88' },
    { type: '升级改造', count: random(5, 15), color: '#ffaa00' },
    { type: '应急抢修', count: random(0, 10), color: '#aa88ff' }
  ],
  recentOrders: Array.from({ length: 8 }, (_, i) => ({
    id: `WO-${Date.now()}-${i}`,
    title: randomChoice([
      '信号机故障维修',
      '道岔润滑保养',
      'UPS电池更换',
      '摄像头清洁',
      '网络设备巡检',
      '软件版本升级',
      '安全漏洞修复',
      '设备参数调整'
    ])
  })))
})

```

```

    ]),
    priority: randomChoice(['紧急', '高', '中', '低']),
    assignee: `工程师${random(1, 20)}`,
    status: randomChoice(['待处理', '处理中', '已完成', '已关闭']),
    createTime: new Date(Date.now() - random(0, 86400000 * 3)).toLocaleString('zh-CN')
  )))
})

// 统计概览数据
export const getOverviewData = () => ({
  totalDevices: 391,
  onlineDevices: random(350, 390),
  alarmCount: random(5, 30),
  todayEvents: random(100, 500),
  dataPoints: random(1000000, 10000000),
  systemHealth: random(90, 99, 1),
  dataQuality: random(95, 99.9, 1),
  predictionAccuracy: random(85, 98, 1),
  keyMetrics: [
    { label: '设备在线率', value: random(95, 99.9, 1), unit: '%', trend: random(-2, 2, 1), color: '#00d4ff' },
    { label: '信号正常率', value: random(98, 99.9, 1), unit: '%', trend: random(-1, 1, 1), color: '#00ff88' },
    { label: '故障响应时间', value: random(5, 30, 0), unit: '分钟', trend: random(-5, 5, 1), color: '#ffaa00' },
    { label: '系统可用性', value: random(99, 99.99, 2), unit: '%', trend: random(-0.1, 0.1, 2), color: '#ff6b6b' }
  ],
  trendData: {
    week: generateTimeSeries(7, 80, 100, 1),
    month: generateTimeSeries(30, 80, 100, 1),
    year: generateTimeSeries(12, 80, 100, 1)
  }
})

// 地图热点数据
export const getMapHotspots = () => Array.from({ length: 20 }, (_, i) => ({
  id: i + 1,
  name: randomChoice(['X站', 'Y站', 'X北站', 'X东站', 'Y北站', 'Y东站', 'X信号所', 'Y信号所', 'XX线路所', 'XX货场']),
  position: [random(100, 130, 4), random(20, 45, 4)],
  value: random(10, 100),
  status: randomChoice(['正常', '繁忙', '拥堵', '维护中']),
  trains: random(5, 30),
  passengers: random(1000, 50000)
}))

// 信号调度系统数据
export const getSignalDispatchData = () => {
  // 线路定义
  const lines = [
    { id: 'L1', name: 'XX1线', color: '#00d4ff', length: 1318 },
    { id: 'L2', name: 'XX2线', color: '#00ff88', length: 2298 },
    { id: 'L3', name: 'XX3线', color: '#ffaa00', length: 2252 },
    { id: 'L4', name: 'XX4线', color: '#ff6b6b', length: 1759 },
    { id: 'L5', name: 'XX5线', color: '#aa88ff', length: 1249 }
  ]

  // 信号机状态
  const signalMachines = []
  const signalStates = ['开放', '关闭', '故障', '维护']
  const signalColors = { '开放': '#00ff88', '关闭': '#ff6b6b', '故障': '#ffaa00', '维护': '#888' }

  for (let i = 1; i <= 32; i++) {
    const state = randomChoice(signalStates)
    signalMachines.push({
      id: `XH-${String(i).padStart(3, '0')}`,
      name: `信号机${i}`,
      line: randomChoice(lines).id,

```

```

    state,
    color: signalColors[state],
    position: { x: random(50, 750), y: random(50, 350) },
    direction: randomChoice(['上行', '下行']),
    lastChange: new Date(Date.now() - random(0, 3600000)).toLocaleTimeString()
  })
}

// 道岔状态
const switches = []
const switchStates = ['定位', '反位', '四开', '故障']

for (let i = 1; i <= 24; i++) {
  const state = randomChoice(switchStates)
  switches.push({
    id: `DC-${String(i).padStart(3, '0')}`,
    name: `道岔${i}`,
    line: randomChoice(lines).id,
    state,
    locked: Math.random() > 0.3,
    position: { x: random(50, 750), y: random(50, 350) },
    operationCount: random(10, 200)
  })
}

// 轨道区段
const trackSections = []
const trackStates = ['空闲', '占用', '锁闭', '故障']

for (let i = 1; i <= 48; i++) {
  const state = randomChoice(trackStates)
  trackSections.push({
    id: `GJ-${String(i).padStart(3, '0')}`,
    name: `区段${i}`,
    line: randomChoice(lines).id,
    state,
    voltage: random(15, 25, 1),
    length: random(500, 2000),
    occupied: state === '占用',
    trainId: state === '占用' ? `G${random(1000, 9999)}` : null
  })
}

// 实时列车调度信息
const trainDispatch = []
const trainStates = ['正常运行', '等待信号', '减速运行', '临时停车', '晚点运行']

for (let i = 1; i <= 15; i++) {
  const line = randomChoice(lines)
  trainDispatch.push({
    id: `G${1000 + i}`,
    line: line.id,
    lineName: line.name,
    lineColor: line.color,
    state: randomChoice(trainStates),
    position: { x: random(50, 750), y: random(50, 350) },
    speed: random(80, 350),
    direction: randomChoice(['上行', '下行']),
    nextStation: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东']),
    eta: random(5, 60),
    delay: random(-5, 15),
    signalAhead: randomChoice(['绿灯', '黄灯', '红灯', '双黄灯']),
    priority: random(1, 5)
  })
}

```

```

}

// 调度命令
const dispatchCommands = []
const commandTypes = ['发车', '停车', '变更进路', '临时限速', '恢复常速', '越站', '扣车']

for (let i = 0; i < 8; i++) {
  dispatchCommands.push({
    id: `CMD-${Date.now()}-${i}`,
    type: randomChoice(commandTypes),
    trainId: `G${random(1000, 9999)}`,
    content: randomChoice([
      '准许从北京南站发车',
      '在济南西站临时停车',
      '变更进路至3道',
      '限速120km/h通过',
      '恢复正常速度运行',
      '越过南京南站不停车',
      '在站扣车等待'
    ]),
    status: randomChoice(['已执行', '执行中', '待执行', '已取消']),
    issuer: `调度员${random(1, 10)}`,
    time: new Date(Date.now() - random(0, 3600000)).toLocaleTimeString()
  })
}

// 进路信息
const routes = []
for (let i = 1; i <= 12; i++) {
  routes.push({
    id: `ROUTE-${String(i).padStart(2, '0')}`,
    name: `进路${i}`,
    from: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东', '西安北']),
    to: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东', '西安北']),
    status: randomChoice(['已建立', '已锁闭', '已解锁', '待建立']),
    signals: [`XH-${random(1, 32).toString().padStart(3, '0')}`, `XH-${random(1, 32).toString().padStart(3, '0')}`],
    switches: [`DC-${random(1, 24).toString().padStart(3, '0')}`, `DC-${random(1, 24).toString().padStart(3, '0')}`],
    sections: [`GJ-${random(1, 48).toString().padStart(3, '0')}`, `GJ-${random(1, 48).toString().padStart(3, '0')}`]
  })
}

return {
  lines,
  signalMachines,
  switches,
  trackSections,
  trainDispatch,
  dispatchCommands,
  routes,
  stats: {
    totalSignals: 32,
    openSignals: signalMachines.filter(s => s.state === '开放').length,
    closedSignals: signalMachines.filter(s => s.state === '关闭').length,
    faultSignals: signalMachines.filter(s => s.state === '故障').length,
    totalSwitches: 24,
    normalSwitches: switches.filter(s => s.state !== '故障').length,
    faultSwitches: switches.filter(s => s.state === '故障').length,
    totalSections: 48,
    occupiedSections: trackSections.filter(s => s.state === '占用').length,
    freeSections: trackSections.filter(s => s.state === '空闲').length,
    lockedSections: trackSections.filter(s => s.state === '锁闭').length,
    runningTrains: trainDispatch.filter(t => t.state === '正常运行').length,
    delayedTrains: trainDispatch.filter(t => t.delay > 0).length,
    activeRoutes: routes.filter(r => r.status === '已建立' || r.status === '已锁闭').length
  }
}

```

```

    }
  }
}

// 信号设备分布（饼图数据）
export const getSignalDistribution = () => ({
  signalStatus: [
    { name: '开放', value: random(20, 28), color: '#00ff88' },
    { name: '关闭', value: random(2, 5), color: '#ff6b6b' },
    { name: '故障', value: random(0, 2), color: '#ffaa00' },
    { name: '维护', value: random(0, 2), color: '#888888' }
  ],
  switchStatus: [
    { name: '定位', value: random(15, 20), color: '#00d4ff' },
    { name: '反位', value: random(4, 8), color: '#00ff88' },
    { name: '四开', value: random(0, 2), color: '#ffaa00' },
    { name: '故障', value: random(0, 1), color: '#ff6b6b' }
  ],
  trackStatus: [
    { name: '空闲', value: random(30, 40), color: '#00ff88' },
    { name: '占用', value: random(5, 12), color: '#ff6b6b' },
    { name: '锁闭', value: random(2, 6), color: '#ffaa00' },
    { name: '故障', value: random(0, 2), color: '#aa88ff' }
  ],
  trainStatus: [
    { name: '正常运行', value: random(8, 12), color: '#00ff88' },
    { name: '等待信号', value: random(1, 3), color: '#ffaa00' },
    { name: '减速运行', value: random(0, 2), color: '#00d4ff' },
    { name: '临时停车', value: random(0, 1), color: '#ff6b6b' }
  ],
  lineLoad: [
    { name: 'XX1线', value: random(25, 35), color: '#00d4ff' },
    { name: 'XX2线', value: random(20, 30), color: '#00ff88' },
    { name: 'XX3线', value: random(15, 25), color: '#ffaa00' },
    { name: 'XX4线', value: random(10, 20), color: '#ff6b6b' },
    { name: 'XX5线', value: random(10, 20), color: '#aa88ff' }
  ]
})

// 调度效率统计
export const getDispatchEfficiency = () => ({
  // 每小时调度列车数
  hourlyDispatch: generateTimeSeries(24, 15, 45),
  // 调度成功率
  successRate: random(95, 99.9, 1),
  // 平均响应时间
  avgResponseTime: random(3, 15, 1),
  // 进路建立时间
  avgRouteTime: random(5, 20, 1),
  // 调度命令执行率
  commandExecutionRate: random(90, 99, 1),
  // 今日调度统计
  todayStats: {
    totalCommands: random(200, 500),
    executedCommands: random(180, 480),
    cancelledCommands: random(5, 20),
    pendingCommands: random(5, 30),
    avgDelay: random(0, 8, 1),
    punctualityRate: 100
  },
  // 近7天趋势
  weeklyTrend: Array.from({ length: 7 }, (_, i) => ({
    date: new Date(Date.now() - (6 - i) * 86400000).toLocaleDateString('zh-CN', { month: 'numeric', day: 'numeric' }),
    trains: random(300, 500),

```

```

        commands: random(400, 700),
        successRate: random(94, 99, 1)
    })))
})

// 实时数据更新（模拟）
export const subscribeToRealtimeData = (callback, interval = 3000) => {
    const timer = setInterval(() => {
        callback({
            timestamp: Date.now(),
            weather: {
                temperature: random(-5, 35, 1),
                humidity: random(30, 95, 1),
                windSpeed: random(0, 25, 1)
            },
            system: {
                cpuUsage: random(20, 80, 1),
                memoryUsage: random(30, 70, 1),
                networkIn: random(10, 1000, 1),
                networkOut: random(10, 500, 1)
            },
            sensors: getSensorData().slice(0, 6).map(s => ({
                id: s.id,
                value: s.value,
                status: s.status
            })))
        })
    }, interval)

    return () => clearInterval(timer)
}

export default {
    getWeatherData,
    getSensorData,
    getEquipmentData,
    getAlarmData,
    getTrainData,
    getEnergyData,
    getSystemStatus,
    getSecurityData,
    getWorkOrderData,
    getOverviewData,
    getMapHotspots,
    getSignalDispatchData,
    getSignalDistribution,
    getDispatchEfficiency,
    subscribeToRealtimeData
}

```

4.3 文件: src/services/simulationState.js

- 文件说明: 跨页面共享的恶劣天气、湿度告警和处置状态同步模块。
- 行数统计: 114 行

```

import { ref } from 'vue'

// 全局模拟状态（用于跨页面共享）
export const severeWeatherEnabled = ref(false)
export const severeWeatherStartedAt = ref(null)
export const humidityMitigationAppliedAt = ref(null)
export const syncedHumidity = ref(22)

// 大数据面板: 湿度异常告警弹窗的全局状态（用于跨页面切换保持一致）
export const humidityAlertDismissed = ref(false)
export const humidityAlertSuppressed = ref(false)

```

```

export const humidityAlertProcessing = ref(false)

let humiditySyncTimer = null

const clamp = (value, min, max) => Math.max(min, Math.min(max, value))

const computeSyncedSevereHumidity = (nowMs = Date.now()) => {
  const startedAt = Number(severeWeatherStartedAt.value) || nowMs
  const now = Number(nowMs) || Date.now()

  // Early 天气演练: 湿度 4 秒内进入红色告警 (>=70%), 随后再缓慢爬升到高位平台
  const stage1Ms = 4000
  const stage2Ms = 20000
  const base = 22
  const redTarget = 75
  const peak = 92

  const easeOutQuad = (t) => t * (2 - t)
  const lerp = (a, b, t) => a + (b - a) * t

  const humidityAtTime = (elapsedMs) => {
    const e = Math.max(0, Number(elapsedMs) || 0)
    if (e <= stage1Ms) {
      const t = easeOutQuad(clamp(e / stage1Ms, 0, 1))
      return Number(lerp(base, redTarget, t).toFixed(1))
    }
    if (e <= stage1Ms + stage2Ms) {
      const t = easeOutQuad(clamp((e - stage1Ms) / stage2Ms, 0, 1))
      return Number(lerp(redTarget, peak, t).toFixed(1))
    }
    return Number(peak.toFixed(1))
  }

  const mitigationAt = Number(humidityMitigationAppliedAt.value)
  const elapsed = Math.max(0, now - startedAt)

  if (Number.isFinite(mitigationAt) && mitigationAt >= startedAt) {
    const mitigationElapsed = Math.max(0, now - mitigationAt)
    const humidityAtMitigation = humidityAtTime(mitigationAt - startedAt)
    const fallDurationMs = 90000
    const fallT = clamp(mitigationElapsed / fallDurationMs, 0, 1)
    const target = 35
    const fallHumidity = humidityAtMitigation + (target - humidityAtMitigation) * fallT
    return Number(clamp(fallHumidity, 0, 100).toFixed(1))
  }

  return humidityAtTime(elapsed)
}

const tickSyncedHumidity = () => {
  if (!severeWeatherEnabled.value) return
  syncedHumidity.value = computeSyncedSevereHumidity(Date.now())
}

const startHumiditySync = () => {
  if (humiditySyncTimer) return
  tickSyncedHumidity()
  humiditySyncTimer = setInterval(tickSyncedHumidity, 250)
}

const stopHumiditySync = () => {
  if (!humiditySyncTimer) return
  clearInterval(humiditySyncTimer)
  humiditySyncTimer = null
}

```

```

}

export const markHumidityMitigationApplied = () => {
  humidityMitigationAppliedAt.value = Date.now()
  tickSyncedHumidity()
}

export const resetHumidityMitigation = () => {
  humidityMitigationAppliedAt.value = null
  tickSyncedHumidity()
}

export const setHumidityAlertDismissed = (dismissed) => {
  humidityAlertDismissed.value = Boolean(dismissed)
}

export const setHumidityAlertSuppressed = (suppressed) => {
  humidityAlertSuppressed.value = Boolean(suppressed)
}

export const setHumidityAlertProcessing = (processing) => {
  humidityAlertProcessing.value = Boolean(processing)
}

export const setSevereWeatherEnabled = (enabled) => {
  const next = Boolean(enabled)
  severeWeatherEnabled.value = next
  severeWeatherStartedAt.value = next ? Date.now() : null
  resetHumidityMitigation()
  setHumidityAlertDismissed(false)
  setHumidityAlertSuppressed(false)
  setHumidityAlertProcessing(false)
  if (next) {
    startHumiditySync()
    return
  }
  stopHumiditySync()
}

```

4.4 文件: `src/services/telemetrySocket.js`

- 文件说明: 浏览器端 WebSocket 遥测连接、重连与最新数据缓存模块。
- 行数统计: 115 行

```

import { ref } from 'vue'

const defaultUrl = () => {
  const envUrl = import.meta?.env?.VITE_TELEMETRY_WS_URL
  if (typeof envUrl === 'string' && envUrl.trim()) return envUrl.trim()
  return 'ws://localhost:8080/ws'
}

export const telemetryConnected = ref(false)
export const lastTelemetry = ref(null)
export const lastTelemetryAt = ref(0)

let socket = null
let reconnectTimer = null
let reconnectAttempt = 0

const scheduleReconnect = () => {
  if (reconnectTimer) return
  const base = 600
  const max = 8000
  const delay = Math.min(max, base * Math.pow(1.6, reconnectAttempt++))
  reconnectTimer = setTimeout(() => {

```

```

    reconnectTimer = null
    connectTelemetry()
  }, delay)
}

export const connectTelemetry = () => {
  const url = defaultUrl()
  try {
    if (socket && (socket.readyState === WebSocket.OPEN || socket.readyState === WebSocket.CONNECTING)) {
      return socket
    }

    socket = new WebSocket(url)

    socket.addEventListener('open', () => {
      telemetryConnected.value = true
      reconnectAttempt = 0
      console.info('[telemetry] ws open', url)
    })

    socket.addEventListener('close', (ev) => {
      telemetryConnected.value = false
      console.warn('[telemetry] ws close', { code: ev?.code, reason: ev?.reason })
      scheduleReconnect()
    })

    socket.addEventListener('error', () => {
      telemetryConnected.value = false
      console.warn('[telemetry] ws error')
      try {
        socket?.close()
      } catch (_) {
        // ignore
      }
    })

    socket.addEventListener('message', (ev) => {
      const text = typeof ev?.data === 'string' ? ev.data : ''
      if (!text) return

      try {
        const parsed = JSON.parse(text)
        if (parsed?.topic === 'telemetry' && parsed?.data) {
          lastTelemetry.value = parsed.data
          lastTelemetryAt.value = Date.now()
          const d = parsed.data
          console.debug('[telemetry] recv', {
            device: d?.device,
            device_id: d?.device_id,
            type: d?.type,
            ts_ms: d?.ts_ms,
            distance_m: d?.distance_m,
            distance_cm: d?.distance_cm,
            water_active: d?.water_active
          })
        }
      } catch (_) {
        // ignore non-json
      }
    })

    return socket
  } catch (_) {
    telemetryConnected.value = false
  }
}

```

```

    scheduleReconnect()
    return null
  }
}

export const disconnectTelemetry = () => {
  if (reconnectTimer) {
    clearTimeout(reconnectTimer)
    reconnectTimer = null
  }
  reconnectAttempt = 0
  telemetryConnected.value = false
  try {
    socket?.close()
  } catch (_) {
    // ignore
  } finally {
    socket = null
  }
}

export default {
  telemetryConnected,
  lastTelemetry,
  lastTelemetryAt,
  connectTelemetry,
  disconnectTelemetry
}

```

4.5 文件: src/composables/useApiData.js

- 文件说明: 组合式 API 数据调用封装, 统一 loading、error 与数据状态。
- 行数统计: 239 行

```

// 数据获取组合函数
import { ref } from 'vue'
import api from '../services/api.js'

// 获取仪表盘汇总数据
export const useDashboardData = () => {
  const loading = ref(false)
  const error = ref(null)
  const dashboardData = ref(null)

  const fetchData = async () => {
    loading.value = true
    error.value = null
    try {
      dashboardData.value = await api.getDashboard()
    } catch (e) {
      error.value = e.message
      console.error('获取仪表盘数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, dashboardData, fetchData }
}

// 获取信号灯数据
export const useSignalsData = () => {
  const loading = ref(false)
  const error = ref(null)
  const signals = ref([])

```

```
const fetchSignals = async () => {
  loading.value = true
  error.value = null
  try {
    signals.value = await api.getSignals()
  } catch (e) {
    error.value = e.message
    console.error('获取信号灯数据失败:', e)
  } finally {
    loading.value = false
  }
}

return { loading, error, signals, fetchSignals }
}

// 获取地震数据
export const useSeismicData = () => {
  const loading = ref(false)
  const error = ref(null)
  const seismicData = ref([])

  const fetchSeismic = async (range = '24h') => {
    loading.value = true
    error.value = null
    try {
      seismicData.value = await api.getSeismicData(range)
    } catch (e) {
      error.value = e.message
      console.error('获取地震数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, seismicData, fetchSeismic }
}

// 获取天气数据
export const useWeatherData = () => {
  const loading = ref(false)
  const error = ref(null)
  const weatherData = ref([])
  const currentWeather = ref(null)

  const fetchWeather = async (range = '24h') => {
    loading.value = true
    error.value = null
    try {
      weatherData.value = await api.getWeatherData(range)
    } catch (e) {
      error.value = e.message
      console.error('获取天气数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  const fetchCurrentWeather = async () => {
    loading.value = true
    error.value = null
    try {
      currentWeather.value = await api.getCurrentWeather()
    } catch (e) {
```

```
      error.value = e.message
      console.error('获取当前天气失败:', e)
    } finally {
      loading.value = false
    }
  }
}

return { loading, error, weatherData, currentWeather, fetchWeather, fetchCurrentWeather }
}
```

// 获取空气质量数据

```
export const useAirQualityData = () => {
  const loading = ref(false)
  const error = ref(null)
  const airQuality = ref(null)

  const fetchAirQuality = async () => {
    loading.value = true
    error.value = null
    try {
      airQuality.value = await api.getAirQuality()
    } catch (e) {
      error.value = e.message
      console.error('获取空气质量失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, airQuality, fetchAirQuality }
}
```

// 获取列车统计数据

```
export const useTrainStatsData = () => {
  const loading = ref(false)
  const error = ref(null)
  const trainStats = ref(null)

  const fetchTrainStats = async () => {
    loading.value = true
    error.value = null
    try {
      trainStats.value = await api.getTrainStats()
    } catch (e) {
      error.value = e.message
      console.error('获取列车统计失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, trainStats, fetchTrainStats }
}
```

// 获取铁道信息

```
export const useRailwayData = () => {
  const loading = ref(false)
  const error = ref(null)
  const railway = ref(null)

  const fetchRailway = async () => {
    loading.value = true
    error.value = null
    try {
```

```

        railway.value = await api.getRailway()
    } catch (e) {
        error.value = e.message
        console.error('获取铁道信息失败:', e)
    } finally {
        loading.value = false
    }
}

return { loading, error, railway, fetchRailway }
}

// 获取参数历史数据
export const useParamsHistory = () => {
    const loading = ref(false)
    const error = ref(null)
    const paramHistory = ref([])

    const fetchParamHistory = async (type = 'temperature', range = '24h', signalId = 1) => {
        loading.value = true
        error.value = null
        try {
            paramHistory.value = await api.getParamHistory(type, range, signalId)
        } catch (e) {
            error.value = e.message
            console.error('获取参数历史失败:', e)
        } finally {
            loading.value = false
        }
    }

    return { loading, error, paramHistory, fetchParamHistory }
}

// 获取AI对话数据
export const useAiConversations = () => {
    const loading = ref(false)
    const error = ref(null)
    const conversations = ref([])

    const fetchConversations = async (sessionId = null) => {
        loading.value = true
        error.value = null
        try {
            conversations.value = await api.getAiConversations(sessionId)
        } catch (e) {
            error.value = e.message
            console.error('获取AI对话失败:', e)
        } finally {
            loading.value = false
        }
    }

    const addConversation = async (sessionId, role, content) => {
        try {
            const newConv = await api.addAiConversation(sessionId, role, content)
            conversations.value.push(newConv)
            return newConv
        } catch (e) {
            console.error('添加AI对话失败:', e)
            throw e
        }
    }
}

```

```

    return { loading, error, conversations, fetchConversations, addConversation }
  }

export default {
  useDashboardData,
  useSignalsData,
  useSeismicData,
  useWeatherData,
  useAirQualityData,
  useTrainStatsData,
  useRailwayData,
  useParamsHistory,
  useAiConversations
}

```

5. 模块：表现层

表现层包括三大主视图及其子面板组件，用于实现 Cesium 地理可视化、Three.js 数字孪生展示和大数据可视化界面。

5.1 文件：src/components/CesiumView.vue

- 文件说明：Cesium 地理信息可视化主界面，含地图、巡航、气象、面板、AI 对话和缓存逻辑。
- 行数统计：2823 行

```

<template>
  <div class="cesium-view">
    <!-- Cesium 容器 -->
    <div id="cesiumContainer" ref="cesiumContainer"></div>

    <div v-if="!networkOnline" class="offline-badge">离线：使用缓存数据</div>

    <!-- 控制按钮 -->
    <div class="control-buttons">
      <button class="reset-btn" @click="resetView">&img alt="Reset icon" data-bbox="375 483 388 493"/> 复位视角</button>
      <button class="toggle-btn" @click="toggleLeftPanel">
        {{ leftPanelVisible ? '◀ 隐藏左侧' : '▶ 显示左侧' }}
      </button>
      <button class="toggle-btn" @click="toggleRightPanel">
        {{ rightPanelVisible ? '隐藏右侧 ▶' : '◀ 显示右侧' }}
      </button>
      <button class="toggle-btn" :class="{ active: cameraCruiseEnabled }" @click="toggleCameraCruise">
        {{ cameraCruiseEnabled ? '🛑 停止巡航' : '🚦 自动巡航' }}
      </button>
      <button class="toggle-btn" :class="{ active: severeWeatherEnabled }" @click="toggleSevereWeather">
        {{ severeWeatherEnabled ? '🌩️ 停止恶劣天气' : '☁️ 恶劣天气模拟' }}
      </button>
    </div>

    <!-- 左侧面板 - 地理信息与铁道数据 -->
    <transition name="slide-left">
      <div v-show="leftPanelVisible" class="side-panel left-panel">
        <div class="panel-card">
          <div class="panel-header">
            <span class="panel-title">📍 地理预告</span>
            <span class="panel-subtitle">GEO BULLETIN</span>
          </div>
          <div class="panel-content">
            <div class="geo-ticker-shell">
              <div class="geo-ticker-track">
                <span
                  v-for="(item, idx) in geoTickerItemsLoop"
                  :key="`${idx}-${item}`"
                  class="geo-ticker-item"
                >
                  {{ item }}
                </span>
              </div>
            </div>
          </div>
        </div>
      </div>
    </transition>
  </div>

```

```

    </div>
  </div>
</div>

<!-- 地理震动监测 -->
<div class="panel-card">
  <div class="panel-header">
    <span class="panel-title">📍 地理震动监测</span>
    <span class="panel-subtitle">SEISMIC MONITORING</span>
  </div>
  <div class="panel-content">
    <div class="seismic-display">
      <div class="seismic-gauge">
        <svg viewBox="0 0 120 80" class="gauge-svg">
          <rect x="10" y="60" width="100" height="15" rx="3" fill="rgba(0,200,255,0.1)" />
          <rect x="10" y="60" :width="seismicLevel * 10" height="15" rx="3" :fill="seismicColor" />
          <line x1="35" y1="55" x2="35" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
          <line x1="60" y1="55" x2="60" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
          <line x1="85" y1="55" x2="85" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
        </svg>
      </div>
      <div class="seismic-info">
        <div class="seismic-value">
          <span class="value-label">震动等级</span>
          <span class="value-num" :style="{ color: seismicColor }">{{ seismicLevel.toFixed(1) }}</span>
        </div>
        <div class="seismic-status" :class="seismicStatusClass">
          {{ seismicStatus }}
        </div>
      </div>
    </div>
    <div class="vibration-history">
      <div class="history-header">
        <span class="history-label">震动曲线</span>
        <select v-model="seismicTimeRange" class="time-select" @change="onSeismicTimeChange">
          <option value="24h">近24小时</option>
          <option value="week">近一周</option>
          <option value="month">近一个月</option>
        </select>
      </div>
      <svg viewBox="0 0 280 60" class="history-svg">
        <defs>
          <linearGradient id="seismicGradient" x1="0%" y1="0%" x2="0%" y2="100%">
            <stop offset="0%" style="stop-color:#00d4ff;stop-opacity:0.3" />
            <stop offset="100%" style="stop-color:#00d4ff;stop-opacity:0" />
          </linearGradient>
        </defs>
        <polygon :points="seismicAreaPoints" fill="url(#seismicGradient)" />
        <polyline :points="seismicLinePoints" fill="none" stroke="#00d4ff" stroke-width="2" />
        <circle v-for="(point, idx) in seismicChartPoints" :key="idx"
          :cx="point.x" :cy="point.y" r="3" fill="#00d4ff"
          @mouseenter="hoveredSeismicPoint = idx"
          @mouseleave="hoveredSeismicPoint = null" />
      </svg>
      <div v-if="hoveredSeismicPoint !== null" class="chart-tooltip">
        {{ seismicTimeLabels[hoveredSeismicPoint] }}: {{ currentSeismicData[hoveredSeismicPoint]?.toFixed(1) }}
      </div>
    </div>
  </div>
</div>

<!-- 当前位置信息 -->
<div class="panel-card">
  <div class="panel-header">

```

```
<span class="panel-title">📍 当前位置</span>
<span class="panel-subtitle">CURRENT POSITION</span>
</div>
<div class="panel-content">
  <div class="position-grid">
    <div class="pos-item">
      <span class="pos-icon">📍</span>
      <span class="pos-label">经度</span>
      <span class="pos-value">{{ currentPosition.lon }}°E</span>
    </div>
    <div class="pos-item">
      <span class="pos-icon">📍</span>
      <span class="pos-label">纬度</span>
      <span class="pos-value">{{ currentPosition.lat }}°N</span>
    </div>
    <div class="pos-item">
      <span class="pos-icon">🏔️</span>
      <span class="pos-label">海拔</span>
      <span class="pos-value">{{ currentPosition.altitude }}m</span>
    </div>
    <div class="pos-item">
      <span class="pos-icon">🧭</span>
      <span class="pos-label">方位角</span>
      <span class="pos-value">{{ currentPosition.heading }}°</span>
    </div>
  </div>
</div>

<!-- 地理信息（脱敏） -->
<div class="panel-card">
  <div class="panel-header">
    <span class="panel-title">🗺️ 地理信息</span>
    <span class="panel-subtitle">GEO STATUS</span>
  </div>
  <div class="panel-content">
    <div class="railway-detail">
      <div class="detail-item">
        <span class="detail-label">场景模式</span>
        <span class="detail-val">3D</span>
      </div>
      <div class="detail-item">
        <span class="detail-label">影像图层</span>
        <span class="detail-val">World Imagery</span>
      </div>
      <div class="detail-item">
        <span class="detail-label">地形图层</span>
        <span class="detail-val">World Terrain</span>
      </div>
      <div class="detail-item">
        <span class="detail-label">地形夸张</span>
        <span class="detail-val">{{ geoStats.terrainExaggeration }}x</span>
      </div>
      <div class="detail-item">
        <span class="detail-label">信标数量</span>
        <span class="detail-val">{{ geoStats.beaconCount }}</span>
      </div>
      <div class="detail-item">
        <span class="detail-label">雾效密度</span>
        <span class="detail-val">{{ geoStats.fogDensity }}</span>
      </div>
    </div>
  </div>
</div>
```

```
<!-- 铁道信息 -->
<div class="panel-card">
  <div class="panel-header">
    <span class="panel-title">🚆 铁道信息</span>
    <span class="panel-subtitle">RAILWAY INFO</span>
  </div>
  <div class="panel-content">
    <div class="railway-info">
      <div class="railway-detail">
        <div class="detail-item">
          <span class="detail-label">起点</span>
          <span class="detail-val">{{ currentRailway.start }}</span>
        </div>
        <div class="detail-item">
          <span class="detail-label">终点</span>
          <span class="detail-val">{{ currentRailway.end }}</span>
        </div>
        <div class="detail-item">
          <span class="detail-label">全程</span>
          <span class="detail-val">{{ currentRailway.length }}km</span>
        </div>
      </div>
    </div>
  </div>
</div>

</div>
</transition>

<!-- 右侧面板 - 天气与AI -->
<transition name="slide-right">
  <div v-show="rightPanelVisible" class="side-panel right-panel">
    <!-- 天气指数 -->
    <div class="panel-card weather-card">
      <div class="panel-header">
        <span class="panel-title">☀️ 天气指数</span>
        <span class="panel-subtitle">WEATHER INFO</span>
      </div>
      <div class="panel-content">
        <div class="weather-compact">
          <div class="weather-left">
            <div class="weather-icon-small">{{ maskedWeather.icon }}</div>
            <div class="weather-info-compact">
              <div class="weather-temp-compact">
                <span class="temp-value-small">{{ maskedWeather.temp }}</span>
                <span class="temp-unit-small">°C</span>
              </div>
              <div class="weather-desc-small">{{ maskedWeather.description }}</div>
              <div class="weather-location-small">{{ maskedWeather.location }}</div>
            </div>
          </div>
          <div class="weather-right">
            <div class="w-item-compact" v-for="item in weatherCompactItems" :key="item.label">
              <span class="w-icon-small">{{ item.icon }}</span>
              <span class="w-info">
                <span class="w-label-small">{{ item.label }}</span>
                <span class="w-value-small">{{ item.value }}</span>
              </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</transition>

<!-- 天气曲线图 -->
```

```

<div class="weather-chart-section">
  <div class="history-header">
    <span class="history-label">{{ weatherTrendTitle }}</span>
    <select v-model="weatherTimeRange" class="time-select" @change="onWeatherTimeChange">
      <option value="24h">近24小时</option>
      <option value="week">近一周</option>
      <option value="month">近一个月</option>
    </select>
  </div>
  <svg viewBox="0 0 280 70" class="history-svg">
    <defs>
      <linearGradient id="weatherGradient" x1="0%" y1="0%" x2="0%" y2="100%">
        <stop offset="0%" :style="`stop-color:${weatherTrendColor};stop-opacity:0.4`" />
        <stop offset="100%" :style="`stop-color:${weatherTrendColor};stop-opacity:0`" />
      </linearGradient>
    </defs>
    <!-- 网格线 -->
    <line x1="10" y1="15" x2="270" y2="15" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <line x1="10" y1="35" x2="270" y2="35" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <line x1="10" y1="55" x2="270" y2="55" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <polygon :points="weatherAreaPoints" fill="url(#weatherGradient)" />
    <polyline :points="weatherLinePoints" fill="none" :stroke="weatherTrendColor" stroke-width="2" />
    <circle v-for="(point, idx) in weatherChartPoints" :key="idx"
      :cx="point.x" :cy="point.y" r="3" :fill="weatherTrendColor"
      @mouseenter="hoveredWeatherPoint = idx"
      @mouseleave="hoveredWeatherPoint = null" />
  </svg>
  <div v-if="hoveredWeatherPoint !== null" class="chart-tooltip weather-tooltip">
    {{ weatherTimeLabels[hoveredWeatherPoint] }}: {{ currentWeatherData[hoveredWeatherPoint] }}{{ weatherTrendUnit }}
  </div>
</div>

<button class="refresh-btn" @click="refreshWeather" :disabled="weatherLoading">
  {{ weatherLoading ? '刷新中...' : '🔄 刷新天气' }}
</button>
</div>
</div>

<!-- 今日列车统计 -->
<div class="panel-card">
  <div class="panel-header">
    <span class="panel-title">🚆 今日列车统计</span>
    <span class="panel-subtitle">TODAY'S TRAINS</span>
  </div>
  <div class="panel-content">
    <div class="train-stats">
      <div class="stat-circle">
        <svg viewBox="0 0 80 80">
          <circle cx="40" cy="40" r="35" fill="none" stroke="rgba(0,200,255,0.2)" stroke-width="6" />
          <circle cx="40" cy="40" r="35" fill="none" stroke="#00d4ff" stroke-width="6"
            stroke-dasharray="220" :stroke-dashoffset="220 - (trainStats.passed / trainStats.total) * 220"
            transform="rotate(-90 40 40)" />
        </svg>
        <div class="stat-num">{{ trainStats.passed }}</div>
        <div class="stat-label">已通过</div>
      </div>
      <div class="stat-details">
        <div class="stat-row">
          <span class="stat-label">计划总数</span>
          <span class="stat-val">{{ trainStats.total }}</span>
        </div>
        <div class="stat-row">
          <span class="stat-label">准点率</span>
          <span class="stat-val green">{{ trainStats.onTime }}%</span>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        </div>
        <div class="stat-row">
            <span class="stat-label">下一班</span>
            <span class="stat-val">{{ trainStats.nextTrain }}</span>
        </div>
    </div>
</div>
</div>
</div>
</div>

<!-- 空气湿度 -->
<div class="panel-card aqi-card" :class="{ 'aqi-flash': airQualitySevereWarning }">
    <div class="panel-header">
        <span class="panel-title">空气湿度</span>
        <span class="panel-subtitle">AIR HUMIDITY</span>
    </div>
    <div class="panel-content">
        <div v-if="airQualitySevereWarning" class="aqi-severe-warning">
            红色告警: 空气湿度过高, 请注意防潮
        </div>
        <div class="aqi-summary">
            <div class="aqi-value" :class="humidityClass" :style="{ color: airQualityColor }">{{ airHumidity }}</div>
            <div class="aqi-level">{{ airHumidityLevel }}</div>
            <div class="aqi-unit">%</div>
        </div>
        <div class="history-header">
            <span class="history-label">空气湿度趋势</span>
            <select v-model="airQualityTimeRange" class="time-select" @change="onAirQualityTimeChange">
                <option value="day">天</option>
                <option value="week">周</option>
            </select>
        </div>
        <svg viewBox="0 0 280 70" class="history-svg aqi-chart">
            <defs>
                <linearGradient id="aqiGradient" x1="0%" y1="0%" x2="0%" y2="100%">
                    <stop offset="0%" :style="{ stop-color: ${airQualityColor}; stop-opacity: 0.35 }" />
                    <stop offset="100%" :style="{ stop-color: ${airQualityColor}; stop-opacity: 0 }" />
                </linearGradient>
            </defs>
            <line x1="10" y1="15" x2="270" y2="15" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
            <line x1="10" y1="35" x2="270" y2="35" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
            <line x1="10" y1="55" x2="270" y2="55" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
            <polygon :points="airQualityAreaPoints" fill="url(#aqiGradient)" />
            <polyline :points="airQualityLinePoints" fill="none" :stroke="airQualityColor" stroke-width="2" />
            <circle v-for="(point, idx) in airQualityChartPoints" :key="idx"
                :cx="point.x" :cy="point.y" r="3" :fill="airQualityColor"
                @mouseenter="hoveredAirQualityPoint = idx"
                @mouseleave="hoveredAirQualityPoint = null" />
        </svg>
        <div v-if="hoveredAirQualityPoint !== null" class="chart-tooltip aqi-tooltip">
            {{ airQualityTimeLabels[hoveredAirQualityPoint] }}: {{ currentAirQualityData[hoveredAirQualityPoint] }}
        </div>
    </div>
</div>

<!-- 地理态势（脱敏） -->
<div class="panel-card">
    <div class="panel-header">
        <span class="panel-title">🗺️ 地理态势</span>
        <span class="panel-subtitle">MAP SITUATION</span>
    </div>
    <div class="panel-content">
        <div class="railway-detail">
            <div class="detail-item">

```

```

        <span class="detail-label">巡航状态</span>
        <span class="detail-val">{{ cameraCruiseEnabled ? '开启' : '关闭' }}</span>
    </div>
    <div class="detail-item">
        <span class="detail-label">视角俯仰</span>
        <span class="detail-val">{{ geoStats.pitchDeg }}°</span>
    </div>
    <div class="detail-item">
        <span class="detail-label">视角翻滚</span>
        <span class="detail-val">{{ geoStats.rollDeg }}°</span>
    </div>
    <div class="detail-item">
        <span class="detail-label">可视范围</span>
        <span class="detail-val">X km</span>
    </div>
    <div class="detail-item">
        <span class="detail-label">区域名称</span>
        <span class="detail-val">XX区域</span>
    </div>
    <div class="detail-item">
        <span class="detail-label">更新频率</span>
        <span class="detail-val">{{ geoStats.refreshHz }}Hz</span>
    </div>
</div>
</div>
</div>
</div>
</transition>
</div>
</template>

<script setup>
import { ref, computed, onMounted, onBeforeUnmount, nextTick, watch } from 'vue'
import * as Cesium from 'cesium'
import 'cesium/Build/Cesium/Widgets/widgets.css'
import { severeWeatherEnabled, setSevereWeatherEnabled } from '../services/simulationState.js'
import { DEMO_SCENARIO } from '../config/demoScenario.js'
import { getWeatherData as getMockWeatherData } from '../services/mockDataService.js'

// Cesium Ion（用于 World Imagery / Terrain 等）。优先使用环境变量，未设置则回退到内置 token（便于开箱即用）。
Cesium.Ion.defaultAccessToken =
    import.meta.env.VITE_CESIUM_ION_TOKEN ||

'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI1MGE2NjE5OC05YmU5LTRlMTctODYxOC1hZWV0YTU0NDJmM2UiLCJpcZCI6Mzg5Mzg5LCJpYXQiOiJE3NzA3NzE2MjR9.X1fsTR
YLQkm1FzS3Z-rGNLnchNdPlNqZUFzdX4SHtWU'

// 响应式变量
const cesiumContainer = ref(null)
const leftPanelVisible = ref(true)
const rightPanelVisible = ref(true)

const networkOnline = ref(typeof navigator !== 'undefined' ? navigator.onLine : true)
let cacheSaveTimer = null
let onlineHandler = null
let offlineHandler = null
const CESIUM_CACHE_KEY = 'railway_sign:cesium_cache:v1'

const readCesiumCache = () => {
    try {
        const raw = localStorage.getItem(CESIUM_CACHE_KEY)
        if (!raw) return null
        const parsed = JSON.parse(raw)
        if (!parsed || parsed.v !== 1) return null
        return parsed
    }

```

```

    } catch {
      return null
    }
  }
}

const writeCesiumCache = () => {
  try {
    const payload = {
      v: 1,
      at: Date.now(),
      weather: weather.value,
      airQuality: airQuality.value,
      airQualitySevereWarning: airQualitySevereWarning.value,
      seismicLevel: seismicLevel.value,
      seismicTimeRange: seismicTimeRange.value,
      seismicData24h: seismicData24h.value,
      seismicDataWeek: seismicDataWeek.value,
      seismicDataMonth: seismicDataMonth.value,
      currentPosition: currentPosition.value,
      geoStats: geoStats.value,
      currentRailway: currentRailway.value,
      trainStats: trainStats.value,
      weatherData24h: weatherData24h.value,
      weatherDataWeek: weatherDataWeek.value,
      weatherDataMonth: weatherDataMonth.value,
      humidityData24h: humidityData24h.value,
      humidityDataWeek: humidityDataWeek.value,
      humidityDataMonth: humidityDataMonth.value
    }
    localStorage.setItem(CESIUM_CACHE_KEY, JSON.stringify(payload))
  } catch {
    // ignore
  }
}

const restoreCesiumCache = () => {
  const cached = readCesiumCache()
  if (!cached) return false
  try {
    if (cached.weather) weather.value = cached.weather
    if (cached.airQuality) airQuality.value = cached.airQuality
    if (typeof cached.airQualitySevereWarning === 'boolean') airQualitySevereWarning.value = cached.airQualitySevereWarning
    if (typeof cached.seismicLevel === 'number') seismicLevel.value = cached.seismicLevel
    if (cached.seismicTimeRange) seismicTimeRange.value = cached.seismicTimeRange
    if (Array.isArray(cached.seismicData24h)) seismicData24h.value = cached.seismicData24h
    if (Array.isArray(cached.seismicDataWeek)) seismicDataWeek.value = cached.seismicDataWeek
    if (Array.isArray(cached.seismicDataMonth)) seismicDataMonth.value = cached.seismicDataMonth
    if (cached.currentPosition) currentPosition.value = cached.currentPosition
    if (cached.geoStats) geoStats.value = cached.geoStats
    if (cached.currentRailway) currentRailway.value = cached.currentRailway
    if (cached.trainStats) trainStats.value = cached.trainStats
    if (Array.isArray(cached.weatherData24h)) weatherData24h.value = cached.weatherData24h
    if (Array.isArray(cached.weatherDataWeek)) weatherDataWeek.value = cached.weatherDataWeek
    if (Array.isArray(cached.weatherDataMonth)) weatherDataMonth.value = cached.weatherDataMonth
    if (Array.isArray(cached.humidityData24h)) humidityData24h.value = cached.humidityData24h
    if (Array.isArray(cached.humidityDataWeek)) humidityDataWeek.value = cached.humidityDataWeek
    if (Array.isArray(cached.humidityDataMonth)) humidityDataMonth.value = cached.humidityDataMonth
    return true
  } catch {
    return false
  }
}

let viewer = null

```

```

let beaconPoints = []
let beaconPopups = []
let selectedBeacon = null
let globalBreathStage = null
let globalBreathStartTime = Date.now()
let rainPostProcessStage = null
let updateDataTimer = null
const defaultFogDensity = 0.0001
let bgmAudio = null
let audioUnlockHandler = null
let cameraAngleChangedHandler = null
let cameraCruiseRafId = null
let cameraCruiseAutoPauseHandler = null
let cameraCruiseStartedAt = 0
let cameraCruiseTarget = null
let cameraCruiseBaseCenter = null
let cameraCruiseEnuToFixed = null
let cameraCruiseRange = 0

let severeWeatherEffectTimer = null
let severeWeatherHumidityTimer = null
let severeWeatherAqiTimer = null
const airQualitySevereWarning = ref(false)
const cameraCruiseEnabled = ref(true)

// ===== 地理震动数据 =====
const seismicLevel = ref(2.3)
const seismicTimeRange = ref('24h')
const hoveredSeismicPoint = ref(null)

// 地震历史数据 - 近24小时（每小时一个点）
const seismicData24h = ref([
  2.1, 1.8, 2.5, 3.1, 2.8, 2.2, 1.9, 2.4, 2.0, 2.3, 2.6, 2.1,
  1.7, 2.0, 2.4, 2.9, 3.2, 2.7, 2.3, 2.0, 1.8, 2.2, 2.5, 2.1
])

// 地震历史数据 - 近一周（每天一个点，7个点）
const seismicDataWeek = ref([2.3, 2.1, 2.8, 3.5, 2.9, 2.4, 2.2])

// 地震历史数据 - 近一个月（每天一个点，30个点）
const seismicDataMonth = ref([
  2.1, 2.3, 1.9, 2.5, 2.8, 3.1, 2.6, 2.2, 1.8, 2.4,
  2.7, 3.0, 2.5, 2.1, 1.7, 2.0, 2.3, 2.6, 2.9, 3.2,
  2.8, 2.4, 2.0, 1.8, 2.2, 2.5, 2.8, 3.1, 2.7, 2.3
])

// 时间标签
const seismicTimeLabels24h = Array.from({ length: 24 }, (_, i) => `${String(i).padStart(2, '0')}:00`)
const seismicTimeLabelsWeek = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
const seismicTimeLabelsMonth = Array.from({ length: 30 }, (_, i) => `${i + 1}日`)

const currentSeismicData = computed(() => {
  switch (seismicTimeRange.value) {
    case 'week': return seismicDataWeek.value
    case 'month': return seismicDataMonth.value
    default: return seismicData24h.value
  }
})

const seismicTimeLabels = computed(() => {
  switch (seismicTimeRange.value) {
    case 'week': return seismicTimeLabelsWeek
    case 'month': return seismicTimeLabelsMonth
    default: return seismicTimeLabels24h
  }
})

```

```

    }
  })

const seismicChartPoints = computed(() => {
  const data = currentSeismicData.value
  const count = data.length
  const width = 260
  const height = 50
  const padding = 10

  return data.map((v, i) => ({
    x: padding + (i / (count - 1)) * width,
    y: padding + height - (v / 8) * height // 假设最大值8
  })))
})

const seismicLinePoints = computed(() => {
  return seismicChartPoints.value.map(p => `${p.x},${p.y}`).join(' ')
})

const seismicAreaPoints = computed(() => {
  const points = seismicChartPoints.value
  if (points.length === 0) return ''
  const bottom = 60
  const linePoints = points.map(p => `${p.x},${p.y}`).join(' ')
  const firstX = points[0].x
  const lastX = points[points.length - 1].x
  return `${firstX},${bottom} ${linePoints} ${lastX},${bottom}`
})

const onSeismicTimeChange = () => {
  hoveredSeismicPoint.value = null
}

const seismicColor = computed(() => {
  if (seismicLevel.value < 3) return '#00d4ff'
  if (seismicLevel.value < 5) return '#ffcc00'
  if (seismicLevel.value < 7) return '#ff9933'
  return '#ff3333'
})

const seismicStatus = computed(() => {
  if (seismicLevel.value < 3) return '稳定'
  if (seismicLevel.value < 5) return '轻微震动'
  if (seismicLevel.value < 7) return '中等震动'
  return '强震动'
})

const seismicStatusClass = computed(() => {
  if (seismicLevel.value < 3) return 'status-normal'
  if (seismicLevel.value < 5) return 'status-warning'
  return 'status-danger'
})

// ===== 当前位置 =====
const currentPosition = ref({
  lon: 'X',
  lat: 'Y',
  altitude: 'X',
  heading: 'X'
})

const geoStats = ref({
  terrainExaggeration: 'X',

```

```

    beaconCount: 0,
    fogDensity: 'X',
    pitchDeg: 'X',
    rollDeg: 'X',
    refreshHz: 30
  })
}

const geoTickerItems = [
  '地理网格 G-07 区段云层底高回落至 820m，山口通视条件保持良好。',
  '巡检轨迹 T2 线回传完成，沿线坡脚监测点位未见异常位移。',
  '北侧缓坡监测面风速提升至 5.1m/s，地表湿度仍处正常区间。',
  '影像拼接通道已刷新，谷地区域热源识别维持低活跃状态。',
  '地理围栏 F-12 完成一次自动校准，边界误差收敛至 0.6m 内。',
  '沿线雾效采样显示能见度稳定，低洼区暂未触发二级遮蔽提示。'
]

const geoTickerItemsLoop = computed(() => [...geoTickerItems, ...geoTickerItems])

// ===== 铁道信息 =====
const currentRailway = ref({
  name: DEMO_SCENARIO.railwayName,
  start: DEMO_SCENARIO.stationStart,
  end: DEMO_SCENARIO.stationEnd,
  length: '255'
})

// ===== 列车统计 =====
const trainStats = ref({
  passed: 42,
  total: 58,
  onTime: 100,
  nextTrain: 'G1502 14:35'
})

// ===== 天气数据 =====
const weather = ref({
  icon: '☁️',
  temp: 26,
  description: '多云',
  location: DEMO_SCENARIO.weatherLocation,
  windSpeed: '3.2m/s',
  humidity: '20%',
  visibility: '15km',
  pressure: '1013hPa'
})

const weatherLoading = ref(false)
const weatherTimeRange = ref('24h')
const hoveredWeatherPoint = ref(null)

const maskedWeather = computed(() => ({
  ...weather.value,
  temp: 'X',
  location: DEMO_SCENARIO.weatherLocation,
  windSpeed: 'X级',
  humidity: 'X%',
  visibility: 'Xkm',
  pressure: 'XhPa'
}))

// 天气紧凑显示项（脱敏展示）
const weatherCompactItems = computed(() => [
  { icon: '🌀', label: '风速', value: maskedWeather.value.windSpeed },
  { icon: '💧', label: '湿度', value: maskedWeather.value.humidity },
  { icon: '👁️', label: '能见度', value: maskedWeather.value.visibility },
  { icon: '🌡️', label: '气压', value: maskedWeather.value.pressure }
])

```

```

// 温度历史数据 - 近24小时
const weatherData24h = ref([
  22, 21, 20, 19, 18, 18, 19, 21, 23, 25, 27, 28,
  29, 30, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21
])

// 温度历史数据 - 近一周
const weatherDataWeek = ref([25, 27, 26, 28, 30, 29, 26])

// 温度历史数据 - 近一个月
const weatherDataMonth = ref([
  24, 25, 26, 27, 28, 27, 26, 25, 24, 26,
  28, 29, 30, 31, 30, 29, 28, 27, 26, 25,
  24, 25, 26, 27, 28, 29, 28, 27, 26, 26
])

// 湿度历史数据（用于恶劣天气模拟的上升折线）
const humidityData24h = ref([
  18, 18, 19, 19, 20, 20, 20, 21, 21, 20, 20, 19,
  19, 19, 20, 20, 21, 21, 22, 22, 21, 21, 20, 20
])
const humidityDataWeek = ref([19, 20, 20, 21, 21, 22, 20])
const humidityDataMonth = ref(Array.from({ length: 30 }, () => 20 + Math.round((Math.random() - 0.5) * 6)))

const weatherTrendMode = computed(() => (severeWeatherEnabled.value ? 'humidity' : 'temperature'))
const weatherTrendTitle = computed(() => (weatherTrendMode.value === 'humidity' ? '湿度趋势' : '温度趋势'))
const weatherTrendUnit = computed(() => (weatherTrendMode.value === 'humidity' ? '%' : '°C'))
const weatherTrendColor = computed(() => (weatherTrendMode.value === 'humidity' ? '#ff4040' : '#ff9900'))

const weatherTimeLabels24h = Array.from({ length: 24 }, (_, i) => `${String(i).padStart(2, '0')}:00`)
const weatherTimeLabelsWeek = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
const weatherTimeLabelsMonth = Array.from({ length: 30 }, (_, i) => `${i + 1}日`)

const currentWeatherData = computed(() => {
  const byRange = (tempData, humidData) => {
    switch (weatherTimeRange.value) {
      case 'week': return weatherTrendMode.value === 'humidity' ? humidData.week : tempData.week
      case 'month': return weatherTrendMode.value === 'humidity' ? humidData.month : tempData.month
      default: return weatherTrendMode.value === 'humidity' ? humidData.h24 : tempData.h24
    }
  }

  return byRange(
    { h24: weatherData24h.value, week: weatherDataWeek.value, month: weatherDataMonth.value },
    { h24: humidityData24h.value, week: humidityDataWeek.value, month: humidityDataMonth.value }
  )
})

const weatherTimeLabels = computed(() => {
  switch (weatherTimeRange.value) {
    case 'week': return weatherTimeLabelsWeek
    case 'month': return weatherTimeLabelsMonth
    default: return weatherTimeLabels24h
  }
})

const weatherChartPoints = computed(() => {
  const data = currentWeatherData.value
  const count = data.length
  const width = 260
  const height = 50
  const padding = 10
  const minVal = weatherTrendMode.value === 'humidity' ? 0 : 15

```

```

const maxVal = weatherTrendMode.value === 'humidity' ? 100 : 35
const range = Math.max(1, maxVal - minVal)

return data.map((v, i) => ({
  x: padding + (i / (count - 1)) * width,
  y: padding + height - ((v - minVal) / range) * height
}))
}))
})

const weatherLinePoints = computed(() => {
  return weatherChartPoints.value.map(p => `${p.x},${p.y}`).join(' ')
})

const weatherAreaPoints = computed(() => {
  const points = weatherChartPoints.value
  if (points.length === 0) return ''
  const bottom = 65
  const linePoints = points.map(p => `${p.x},${p.y}`).join(' ')
  const firstX = points[0].x
  const lastX = points[points.length - 1].x
  return `${firstX},${bottom} ${linePoints} ${lastX},${bottom}`
})

const onWeatherTimeChange = () => {
  hoveredWeatherPoint.value = null
}

const syncWeatherFromMock = () => {
  const w = getMockWeatherData()
  if (!w) return

  // 当前天气（脱敏展示，保留趋势一致性）
  weather.value = {
    icon: w.weatherType === '暴雨' || w.weatherType === '大雨' ? '🌧️' : '☁️',
    temp: Math.round(Number(w.temperature) || 0),
    description: w.weatherType || '多云',
    location: DEMO_SCENARIO.weatherLocation,
    windSpeed: typeof w.windSpeed === 'number' ? `${w.windSpeed.toFixed(1)}m/s` : 'X级',
    humidity: Number.isFinite(Number(w.humidity)) ? `${Number(w.humidity).toFixed(1)}%` : 'X%',
    visibility: typeof w.visibility === 'number' ? `${Number(w.visibility).toFixed(0)}km` : 'Xkm',
    pressure: typeof w.pressure === 'number' ? `${Number(w.pressure).toFixed(0)}hPa` : 'XhPa'
  }
  airQualitySevereWarning.value = Number(w.humidity) >= 70

  // 趋势：优先对齐大数据平台湿度曲线（24h）
  if (Array.isArray(w.humidityTrend) && w.humidityTrend.length) {
    humidityData24h.value = w.humidityTrend.map((v) => Number(v))
    // 简单下采样得到周趋势（7点），避免与 24h 完全不同步
    humidityDataWeek.value = Array.from({ length: 7 }, (_, idx) => {
      const i = Math.round((idx / 6) * (humidityData24h.value.length - 1))
      return Number(humidityData24h.value[i] ?? humidityData24h.value[humidityData24h.value.length - 1] ?? 20)
    })
    // 月趋势：以当前湿度为中心的小幅波动（保持脱敏且不“突变”）
    const last = Number(humidityData24h.value[humidityData24h.value.length - 1] ?? 20)
    humidityDataMonth.value = Array.from({ length: 30 }, () => {
      const n = (Math.random() - 0.5) * 2.5
      return Math.max(0, Math.min(100, Number((last + n).toFixed(1))))
    })
  }
}

const refreshWeather = async () => {
  weatherLoading.value = true
  try {

```

```

    if (severeWeatherEnabled.value) {
      syncWeatherFromMock()
      return
    }
    weather.value = {
      icon: '☁️',
      temp: 24 + Math.floor(Math.random() * 5),
      description: '多云',
      location: DEMO_SCENARIO.weatherLocation,
      windSpeed: `${(2.2 + Math.random() * 1.8).toFixed(1)}m/s`,
      humidity: `${20 + Math.floor(Math.random() * 5)}%`,
      visibility: `${12 + Math.floor(Math.random() * 4)}km`,
      pressure: `${1010 + Math.floor(Math.random() * 6)}hPa`
    }
  } catch (error) {
    console.error('本地天气刷新失败:', error)
    weather.value.temp = 24
    weather.value.humidity = '22%'
  } finally {
    weatherLoading.value = false
  }
}

// ===== 空气质量 =====
const airQuality = ref({
  aqi: 45,
  level: '优',
  pollutants: [
    { name: 'PM2.5', value: '23µg/m³' },
    { name: 'PM10', value: '45µg/m³' },
    { name: 'O3', value: '68µg/m³' },
    { name: 'NO2', value: '32µg/m³' }
  ]
})

const parsePercent = (val, fallback = 0) => {
  const num = Number(String(val).replace('%', '').trim())
  return Number.isFinite(num) ? num : fallback
}

const airHumidity = computed(() => {
  return Math.round(Math.max(0, Math.min(100, parsePercent(weather.value.humidity, 20))))
})

const airHumidityLevel = computed(() => {
  const h = airHumidity.value
  if (h >= 70) return '红色告警'
  if (h >= 60) return '橙色告警'
  return '正常'
})

const humidityClass = computed(() => {
  const h = airHumidity.value
  if (h >= 70) return 'aqi-bad'
  if (h >= 60) return 'aqi-moderate'
  return 'aqi-good'
})

const airQualityTimeRange = ref('day')
const hoveredAirQualityPoint = ref(null)

const airQualityTimeLabelsDay = Array.from({ length: 24 }, (_, i) => `${String(i).padStart(2, '0')}:00`)
const airQualityTimeLabelsWeek = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']

```

```

const currentAirQualityData = computed(() => {
  // 这里展示“空气湿度趋势”，不是污染程度
  return airQualityTimeRange.value === 'week'
    ? humidityDataWeek.value
    : humidityData24h.value
})

const airQualityTimeLabels = computed(() => {
  return airQualityTimeRange.value === 'week'
    ? airQualityTimeLabelsWeek
    : airQualityTimeLabelsDay
})

const airQualityColor = computed(() => {
  const clamp = (v, min, max) => Math.max(min, Math.min(max, v))
  const humidity = clamp(airHumidity.value, 0, 100)

  const hexToRgb = (hex) => {
    const h = String(hex).replace('#', '').trim()
    const full = h.length === 3 ? h.split('').map(c => c + c).join('') : h
    const intVal = parseInt(full, 16)
    return {
      r: (intVal >> 16) & 255,
      g: (intVal >> 8) & 255,
      b: intVal & 255
    }
  }

  const rgbToHex = ({ r, g, b }) => {
    const to2 = (n) => clamp(Math.round(n), 0, 255).toString(16).padStart(2, '0')
    return `#${to2(r)}${to2(g)}${to2(b)}`
  }

  const lerp = (a, b, t) => a + (b - a) * t
  const lerpColor = (a, b, t) => {
    const ca = hexToRgb(a)
    const cb = hexToRgb(b)
    return rgbToHex({
      r: lerp(ca.r, cb.r, t),
      g: lerp(ca.g, cb.g, t),
      b: lerp(ca.b, cb.b, t)
    })
  }

  // <60: 绿; 60~70: 橙; >=70: 红 (湿度告警)
  if (humidity < 60) return lerpColor('#22ff55', '#33ff33', humidity / 60)
  if (humidity < 70) return lerpColor('#ffcc00', '#ff9900', (humidity - 60) / 10)
  return lerpColor('#ff3333', '#ff3333', 1)
})

const airQualityChartPoints = computed(() => {
  const data = currentAirQualityData.value
  if (!data || data.length === 0) return []
  const count = data.length
  const width = 260
  const height = 50
  const padding = 10
  const min = 0
  const max = 100
  const range = Math.max(1, max - min)

  return data.map((v, i) => ({
    x: padding + (i / (count - 1)) * width,
    y: padding + height - ((v - min) / range) * height
  }))
})

```

```

    )))
  })

const airQualityLinePoints = computed(() => {
  return airQualityChartPoints.value.map(p => `${p.x},${p.y}`).join(' ')
})

const airQualityAreaPoints = computed(() => {
  const points = airQualityChartPoints.value
  if (points.length === 0) return ''
  const bottom = 60
  const linePoints = points.map(p => `${p.x},${p.y}`).join(' ')
  const firstX = points[0].x
  const lastX = points[points.length - 1].x
  return `${firstX},${bottom} ${linePoints} ${lastX},${bottom}`
})

const onAirQualityTimeChange = () => {
  hoveredAirQualityPoint.value = null
}

// ===== AI 对话 =====
const aiMessages = ref([
  { role: 'assistant', content: '您好！我是铁道监控AI助手，有什么可以帮助您的吗？' }
])
const aiInput = ref('')
const aiThinking = ref(false)
const aiMessagesRef = ref(null)

const sendAiMessage = async () => {
  if (!aiInput.value.trim() || aiThinking.value) return

  const userMessage = aiInput.value.trim()
  aiMessages.value.push({ role: 'user', content: userMessage })
  aiInput.value = ''
  aiThinking.value = true

  await nextTick()
  scrollToBottom()

  try {
    // 模拟AI响应
    await new Promise(resolve => setTimeout(resolve, 1000 + Math.random() * 1000))

    let response = '抱歉，我暂时无法回答这个问题。'
    if (userMessage.includes('铁道') || userMessage.includes('铁路')) {
      response = `当前您所在的位置是${currentRailway.value.name}，该线路从${currentRailway.value.start}到${currentRailway.value.end}，全程${currentRailway.value.length}公里。目前线路运行正常，今日已通过${trainStats.value.passed}趟列车。`
    } else if (userMessage.includes('天气')) {
      response = `当前${weather.value.location}天气${weather.value.description}，气温${weather.value.temp}°C，湿度${weather.value.humidity}，风速${weather.value.windSpeed}。整体天气条件良好，适合列车运行。`
    } else if (userMessage.includes('站点') || userMessage.includes('附近')) {
      response = `您附近3公里内有以下站点：${DEMO_SCENARIO.nearbyStations.main}（主站）、${DEMO_SCENARIO.nearbyStations.freight}（货运站）、${DEMO_SCENARIO.nearbyStations.hs}（高铁站）。最近的是${DEMO_SCENARIO.nearbyStations.main}，距离约X公里。`
    } else if (userMessage.includes('你好') || userMessage.includes('您好')) {
      response = '您好！我是铁道监控系统AI助手。我可以帮您查询铁道信息、天气状况、列车运行状态等。请问有什么需要帮助的吗？'
    }

    aiMessages.value.push({ role: 'assistant', content: response })
  } catch (error) {
    aiMessages.value.push({ role: 'assistant', content: '抱歉，发生了错误，请稍后重试。' })
  } finally {
    aiThinking.value = false
    await nextTick()
  }
}

```

```

        scrollToBottom()
    }
}

const quickAsk = (question) => {
    aiInput.value = question
    sendAiMessage()
}

const scrollToBottom = () => {
    if (aiMessagesRef.value) {
        aiMessagesRef.value.scrollTop = aiMessagesRef.value.scrollHeight
    }
}

// ===== 面板控制 =====
const toggleLeftPanel = () => {
    leftPanelVisible.value = !leftPanelVisible.value
}

const toggleRightPanel = () => {
    rightPanelVisible.value = !rightPanelVisible.value
}

const startRainFogSimulation = () => {
    if (!viewer || rainPostProcessStage) return
    // 参考 Cesium 雨效常见实现: 全屏后处理雨幕
    rainPostProcessStage = new Cesium.PostProcessStage({
        name: 'railway-rain-effect',
        fragmentShader: `
            uniform sampler2D colorTexture;
            in vec2 v_textureCoordinates;

            float hash(float x) {
                return fract(sin(x * 133.3) * 13.13);
            }

            void main(void) {
                float time = czm_frameNumber / 60.0;
                vec2 resolution = czm_viewport.zw;
                vec2 uv = (gl_FragCoord.xy * 2.0 - resolution.xy) / min(resolution.x, resolution.y);
                vec3 rainColor = vec3(0.62, 0.72, 0.82);

                float angle = -0.4;
                float si = sin(angle), co = cos(angle);
                uv *= mat2(co, -si, si, co);
                uv *= length(uv + vec2(0.0, 4.9)) * 0.3 + 1.0;

                float v = 1.0 - sin(hash(floor(uv.x * 100.0)) * 2.0);
                float b = clamp(abs(sin(20.0 * time * v + uv.y * (5.0 / (2.0 + v)))) - 0.95, 0.0, 1.0) * 20.0;
                rainColor *= v * b;

                vec4 color = texture(colorTexture, v_textureCoordinates);
                out_FragColor = mix(color, vec4(rainColor, 1.0), 0.35);
            }
        `,
    })
    viewer.scene.postProcessStages.add(rainPostProcessStage)

    viewer.scene.fog.enabled = true
    viewer.scene.fog.density = 0.00028
    viewer.scene.fog.minimumBrightness = 0.5
}

```

```
const stopRainFogSimulation = () => {
  if (!viewer) return
  if (rainPostProcessStage) {
    viewer.scene.postProcessStages.remove(rainPostProcessStage)
    rainPostProcessStage = null
  }

  viewer.scene.fog.enabled = true
  viewer.scene.fog.density = defaultFogDensity
  viewer.scene.fog.minimumBrightness = 0.65
}

const startBackgroundMusic = () => {
  if (!bgmAudio) {
    bgmAudio = new Audio('/assets/bgm/d1.mp3')
    bgmAudio.preload = 'auto'
    bgmAudio.loop = true
    bgmAudio.volume = 0.8
    bgmAudio.muted = false
  }

  bgmAudio.muted = false
  return bgmAudio.play()
}

const stopBackgroundMusic = () => {
  if (!bgmAudio) return
  bgmAudio.pause()
  bgmAudio.currentTime = 0
}

const installAudioUnlockFallback = () => {
  if (audioUnlockHandler) return

  audioUnlockHandler = async () => {
    if (!severeWeatherEnabled.value) return
    try {
      await startBackgroundMusic()
    } catch (error) {
      console.warn('背景音乐仍未解锁:', error)
      return
    }
  }

  window.removeEventListener('pointerdown', audioUnlockHandler, true)
  audioUnlockHandler = null
}

// 某些浏览器会阻止首次播放，下一次用户点击后重试
window.addEventListener('pointerdown', audioUnlockHandler, true)
}

const toggleSevereWeather = async () => {
  setSevereWeatherEnabled(!severeWeatherEnabled.value)
}

const stopSevereWeatherUiEffects = () => {
  airQualitySevereWarning.value = false
  if (severeWeatherEffectTimer) {
    clearTimeout(severeWeatherEffectTimer)
    severeWeatherEffectTimer = null
  }
  if (severeWeatherHumidityTimer) {
    clearInterval(severeWeatherHumidityTimer)
    severeWeatherHumidityTimer = null
  }
}
```

```

    }
    if (severeWeatherAqiTimer) {
      clearInterval(severeWeatherAqiTimer)
      severeWeatherAqiTimer = null
    }
  }
}

const applySevereWeatherChange = async (enabled) => {
  if (!viewer) return

  stopSevereWeatherUiEffects()

  if (!enabled) {
    stopRainFogSimulation()
    stopBackgroundMusic()
    if (audioUnlockHandler) {
      window.removeEventListener('pointerdown', audioUnlockHandler, true)
      audioUnlockHandler = null
    }
    // 恢复空气质量为正常（避免一直红色）
    airQuality.value = {
      aqi: 45,
      level: '优',
      pollutants: [
        { name: 'PM2.5', value: '23µg/m³' },
        { name: 'PM10', value: '45µg/m³' },
        { name: 'O3', value: '68µg/m³' },
        { name: 'NO2', value: '32µg/m³' }
      ]
    }
    weather.value.humidity = '20%'
    return
  }

  startRainFogSimulation()
  try {
    await startBackgroundMusic()
  } catch (error) {
    console.warn('背景音乐播放失败:', error)
    installAudioUnlockFallback()
  }

  // 启动后 5 秒：空气湿度加速爬升（默认约20%），60%橙色告警，70%红色告警，最终在92~93%徘徊
  // 恶劣天气下：湿度等指标与大数据平台同步（由 mockDataService + simulationState 统一驱动）
  syncWeatherFromMock()
  severeWeatherHumidityTimer = setInterval(syncWeatherFromMock, 300)
}

// ===== Cesium 初始化 =====
const LIUZHOU_STATION = {
  lon: 109.38871,
  lat: 24.30755,
  height: 500
}

// ===== 相机自动巡航（缓慢前行、避免过低视角）=====
// 用日志角度做关键帧，但剔除“过低”（pitch 接近 0）导致贴地的段落，并补齐回到起点的过渡帧形成闭环
const CAMERA_CRUISE_KEYFRAMES = [
  // 以稳定的俯视（-45°）作为起点
  { heading: 0.0, pitch: -45.0 },
  { heading: 358.8, pitch: -40.5 },
  { heading: 357.8, pitch: -17.1 },
  { heading: 351.9, pitch: -10.1 },
  { heading: 350.9, pitch: -8.7 },

```

```

// 过渡回到俯视，保证循环时不会突然“跳”
{ heading: 356.0, pitch: -14.0 },
{ heading: 0.0, pitch: -22.0 },
{ heading: 0.0, pitch: -32.0 },
{ heading: 0.0, pitch: -40.0 },
{ heading: 0.0, pitch: -45.0 }
]

const CAMERA_CRUISE_LOOP_MS = 45000
const CAMERA_CRUISE_MAX_PITCH_DEG = -8.0
const CAMERA_CRUISE_MIN_RANGE_M = 4500
const CAMERA_CRUISE_TARGET_HEIGHT_M = 180
const CAMERA_CRUISE_PATH_EAST_M = 2200
const CAMERA_CRUISE_PATH_NORTH_M = 700

const normalizeDeg360 = (deg) => {
  const v = deg % 360
  return v < 0 ? v + 360 : v
}

const lerp = (a, b, t) => a + (b - a) * t

const lerpAngleDeg = (a, b, t) => {
  const aN = normalizeDeg360(a)
  const bN = normalizeDeg360(b)
  let delta = bN - aN
  if (delta > 180) delta -= 360
  if (delta < -180) delta += 360
  return normalizeDeg360(aN + delta * t)
}

const smoothstep = (t) => t * t * (3 - 2 * t)

const sampleCruiseOrientation = (u) => {
  const frames = CAMERA_CRUISE_KEYFRAMES
  const n = frames.length
  if (n === 0) return { heading: 0, pitch: -45 }
  if (n === 1) return frames[0]
  const clamped = Math.max(0, Math.min(1, u))
  const scaled = clamped * (n - 1)
  const idx = Math.min(n - 2, Math.max(0, Math.floor(scaled)))
  const localT = smoothstep(scaled - idx)
  const a = frames[idx]
  const b = frames[idx + 1]
  return {
    heading: lerpAngleDeg(a.heading, b.heading, localT),
    pitch: lerp(a.pitch, b.pitch, localT)
  }
}

const clampCruisePitchDeg = (pitchDeg) => {
  // pitch 为负数（向下），越接近 0 越“贴地”，这里限制最大值，避免太低
  return Math.min(pitchDeg, CAMERA_CRUISE_MAX_PITCH_DEG)
}

const cruiseTargetWithEnuOffset = (east, north, up = 0) => {
  if (!cameraCruiseEnuToFixed || !cameraCruiseBaseCenter) return cameraCruiseBaseCenter
  const local = new Cesium.Cartesian3(east, north, up)
  const out = new Cesium.Cartesian3()
  return Cesium.Matrix4.multiplyByPoint(cameraCruiseEnuToFixed, local, out)
}

const stopCameraCruise = () => {
  if (cameraCruiseRafId) {
    cancelAnimationFrame(cameraCruiseRafId)
  }
}

```

```

    cameraCruiseRafId = null
  }
  cameraCruiseBaseCenter = null
  cameraCruiseEnuToFixed = null
  if (viewer) {
    try {
      viewer.camera.lookAtTransform(Cesium.Matrix4.IDENTITY)
    } catch {
      // ignore
    }
  }
}

const startCameraCruise = () => {
  if (!viewer) return
  if (cameraCruiseRafId) return

  cameraCruiseBaseCenter = Cesium.Cartesian3.fromDegrees(
    LIUZHOU_STATION.lon,
    LIUZHOU_STATION.lat,
    CAMERA_CRUISE_TARGET_HEIGHT_M
  )
  cameraCruiseEnuToFixed = Cesium.Transforms.eastNorthUpToFixedFrame(cameraCruiseBaseCenter)
  cameraCruiseTarget = cameraCruiseBaseCenter
  cameraCruiseRange = Math.max(
    CAMERA_CRUISE_MIN_RANGE_M,
    Cesium.Cartesian3.distance(viewer.camera.positionWC, cameraCruiseBaseCenter)
  )
  cameraCruiseStartedAt = performance.now()

  const tick = (now) => {
    if (!viewer || !cameraCruiseEnabled.value) {
      stopCameraCruise()
      return
    }

    const elapsed = now - cameraCruiseStartedAt
    const phase = (elapsed % CAMERA_CRUISE_LOOP_MS) / CAMERA_CRUISE_LOOP_MS
    const o0 = sampleCruiseOrientation(phase)
    const o = { heading: o0.heading, pitch: clampCruisePitchDeg(o0.pitch) }

    // 缓慢“向前行进”：让观察点在站点周围沿椭圆轨迹平滑前行（连续闭环，无跳变）
    const theta = phase * Math.PI * 2
    const east = Math.cos(theta) * CAMERA_CRUISE_PATH_EAST_M
    const north = Math.sin(theta) * CAMERA_CRUISE_PATH_NORTH_M
    cameraCruiseTarget = cruiseTargetWithEnuOffset(east, north, 0)

    viewer.camera.lookAt(
      cameraCruiseTarget,
      new Cesium.HeadingPitchRange(
        Cesium.Math.toRadians(o.heading),
        Cesium.Math.toRadians(o.pitch),
        cameraCruiseRange
      )
    )
    viewer.scene.requestRender?.()
    cameraCruiseRafId = requestAnimationFrame(tick)
  }

  cameraCruiseRafId = requestAnimationFrame(tick)
}

const toggleCameraCruise = () => {
  cameraCruiseEnabled.value = !cameraCruiseEnabled.value

```

```

    if (cameraCruiseEnabled.value) startCameraCruise()
    else stopCameraCruise()
  }

const installCameraCruiseAutoPause = () => {
  if (!viewer || cameraCruiseAutoPauseHandler) return
  // 仅当用户“拖动”相机时才暂停（避免单击就把巡航停掉）
  cameraCruiseAutoPauseHandler = (e) => {
    if (!cameraCruiseEnabled.value) return
    if (!e || (e.button !== 0 && e.pointerType !== 'touch')) return

    const startX = e.clientX
    const startY = e.clientY
    let moved = false

    const onMove = (ev) => {
      if (!cameraCruiseEnabled.value) return
      const dx = Math.abs(ev.clientX - startX)
      const dy = Math.abs(ev.clientY - startY)
      if (dx + dy >= 8) moved = true
      if (!moved) return
      cameraCruiseEnabled.value = false
      stopCameraCruise()
      cleanup()
    }

    const cleanup = () => {
      window.removeEventListener('pointermove', onMove, true)
      window.removeEventListener('pointerup', cleanup, true)
      window.removeEventListener('pointercancel', cleanup, true)
    }

    window.addEventListener('pointermove', onMove, true)
    window.addEventListener('pointerup', cleanup, true)
    window.addEventListener('pointercancel', cleanup, true)
  }

  viewer.scene.canvas.addEventListener('pointerdown', cameraCruiseAutoPauseHandler, { passive: true })
}

const setupGlobalBreathingGlow = () => {
  if (!viewer) return
  if (globalBreathStage) {
    viewer.scene.postProcessStages.remove(globalBreathStage)
    globalBreathStage = null
  }

  globalBreathStartTime = Date.now()
  globalBreathStage = new Cesium.PostProcessStage({
    name: 'global-breathing-glow',
    fragmentShader: `
      uniform sampler2D colorTexture;
      in vec2 v_textureCoordinates;
      uniform float u_time;
      uniform float u_strength;

      void main() {
        vec4 color = texture(colorTexture, v_textureCoordinates);
        vec2 centered = v_textureCoordinates - vec2(0.5, 0.5);
        float radius = length(centered);
        float radial = smoothstep(0.75, 0.0, radius);
        float breath = 0.55 + 0.45 * sin(u_time * 0.45);
        float lift = radial * breath * u_strength;
      }
    `,
  })
}

```

```

    vec3 glow = vec3(0.06, 0.22, 0.36) * lift;
    vec3 boosted = color.rgb + glow;
    boosted = mix(boosted, boosted * (1.0 + 0.08 * lift), 0.8);

    out_FragColor = vec4(boosted, color.a);
  }
},
uniforms: {
  u_time: () => (Date.now() - globalBreathStartTime) * 0.001,
  u_strength: 0.9
}
})
viewer.scene.postProcessStages.add(globalBreathStage)
}

const initCesium = async () => {
  try {
    console.log('正在初始化 Cesium...')

    const viewerOptions = {
      animation: false,
      timeline: false,
      baseLayerPicker: false,
      geocoder: false,
      homeButton: false,
      sceneModePicker: false,
      navigationHelpButton: false,
      fullscreenButton: false,
      infoBox: false,
      selectionIndicator: false,
      // 保持 3D 地形风格, 避免使用工作地图风格底图
      baseLayer: false
    }

    viewer = new Cesium.Viewer('cesiumContainer', viewerOptions)
    if (viewer.cesiumWidget?.creditContainer) {
      viewer.cesiumWidget.creditContainer.style.display = 'none'
    }

    const addBaseLayers = async () => {
      // 恢复为 Cesium Ion 数据源 (非国内底图)
      viewer.imageryLayers.removeAll(true)
      let hasSwitchedToOsm = false
      const switchToOsm = () => {
        if (!viewer) return
        if (hasSwitchedToOsm) return
        hasSwitchedToOsm = true
        viewer.imageryLayers.removeAll(true)
        const osm = new Cesium.UrlTemplateImageryProvider({
          url: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
          maximumLevel: 19,
          credit: '@ OpenStreetMap contributors'
        })
        viewer.imageryLayers.addImageryProvider(osm)
        console.log('已切换底图: OpenStreetMap')
      }

      const installImageryErrorFallback = (provider, providerName) => {
        const ev = provider?.errorEvent
        if (!ev || typeof ev.addEventListener !== 'function') return

        let errCount = 0
        let windowStartedAt = 0
        ev.addEventListener((e) => {

```

```

const now = Date.now()
if (!windowStartedAt || now - windowStartedAt > 12000) {
  windowStartedAt = now
  errCount = 0
}
errCount += 1
if (errCount >= 3) {
  console.warn(`${providerName} 瓦片连续加载失败，已自动切换为 OpenStreetMap:`, e)
  switchToOsm()
}
})
}

try {
  const worldImagery = await Cesium.createWorldImageryAsync({
    style: Cesium.IonWorldImageryStyle.AERIAL
  })
  viewer.imageryLayers.addImageryProvider(worldImagery)
  installImageryErrorFallback(worldImagery, 'Cesium World Imagery')
  console.log('已切换底图: Cesium World Imagery')
} catch (e) {
  console.warn('Cesium World Imagery 初始化失败，切换为 OpenStreetMap:', e)
  switchToOsm()
}
}

await addBaseLayers()

try {
  // 地形依赖 Cesium Ion（国外网络可能不稳定），失败则使用椭球体地形（不依赖网络）
  viewer.terrainProvider = await Cesium.CesiumTerrainProvider.fromIonAssetId(1, {
    requestVertexNormals: true,
    requestWaterMask: true
  })
} catch (error) {
  console.warn('地形加载失败，已切换为默认椭球体地形:', error)
  viewer.terrainProvider = new Cesium.EllipsoidTerrainProvider()
}

try {
  console.log('正在加载 3D 建筑图层...')
  const osmBuildings = await Cesium.createOsmBuildingsAsync({
    enableShowOutline: false,
    showOutline: false
  })
  viewer.scene.primitives.add(osmBuildings)
  console.log('3D 建筑图层加载完成')
} catch (error) {
  console.warn('OSM Buildings 加载失败:', error)
}

viewer.scene.skyAtmosphere.show = true
viewer.scene.fog.enabled = true
viewer.scene.fog.density = defaultFogDensity
viewer.scene.fog.minimumBrightness = 0.65
viewer.scene.globe.enableLighting = true
viewer.terrainExaggeration = 3.0
viewer.scene.globe.depthTestAgainstTerrain = true
viewer.scene.globe.alpha = 1.0
// setupGlobalBreathingGlow() // 暂时去除光晕效果

installCameraAngleLogger()
await applySevereWeatherChange(severeWeatherEnabled.value)

```

```

        console.log('Cesium 初始化完成! ')
        return viewer
    } catch (error) {
        console.error('Cesium 初始化失败:', error)
        throw error
    }
}

const installCameraAngleLogger = () => {
    if (!viewer) return
    if (cameraAngleChangedHandler) {
        viewer.camera.changed.removeEventListener(cameraAngleChangedHandler)
        cameraAngleChangedHandler = null
    }

    let lastLogAt = 0
    let last = { heading: null, pitch: null, roll: null }
    cameraAngleChangedHandler = () => {
        const now = Date.now()
        if (now - lastLogAt < 600) return

        const heading = Cesium.Math.toDegrees(viewer.camera.heading)
        const pitch = Cesium.Math.toDegrees(viewer.camera.pitch)
        const roll = Cesium.Math.toDegrees(viewer.camera.roll)

        const changedEnough =
            last.heading === null ||
            Math.abs(heading - last.heading) > 1 ||
            Math.abs(pitch - last.pitch) > 1 ||
            Math.abs(roll - last.roll) > 1

        if (!changedEnough) return

        lastLogAt = now
        last = { heading, pitch, roll }
        console.log(
            `[相机角度变化] heading=${heading.toFixed(1)}° pitch=${pitch.toFixed(1)}° roll=${roll.toFixed(1)}°`
        )
    }

    viewer.camera.changed.addEventListener(cameraAngleChangedHandler)
}

const flyToLiuZhou = () => {
    if (!viewer) return
    stopCameraCruise()
    viewer.camera.flyTo({
        destination: Cesium.Cartesian3.fromDegrees(
            LIUZHOU_STATION.lon,
            LIUZHOU_STATION.lat,
            LIUZHOU_STATION.height
        ),
        orientation: {
            heading: Cesium.Math.toRadians(0),
            pitch: Cesium.Math.toRadians(-45),
            roll: 0.0
        },
        duration: 3.0,
        complete: () => {
            if (cameraCruiseEnabled.value) startCameraCruise()
        }
    })
}

```

```

const resetView = () => {
  if (!viewer) return
  stopCameraCruise()
  viewer.camera.flyTo({
    destination: Cesium.Cartesian3.fromDegrees(
      LIUZHOU_STATION.lon,
      LIUZHOU_STATION.lat,
      LIUZHOU_STATION.height
    ),
    orientation: {
      heading: Cesium.Math.toRadians(0),
      pitch: Cesium.Math.toRadians(-45),
      roll: 0.0
    },
    duration: 2.0,
    complete: () => {
      if (cameraCruiseEnabled.value) startCameraCruise()
    }
  })
}

const hidePopup = (beaconId) => {
  if (beaconPopups[beaconId]) {
    beaconPopups[beaconId].style.display = 'none'
  }
  selectedBeacon = null
}

const setupInteractions = () => {
  const handler = new Cesium.ScreenSpaceEventHandler(viewer.scene.canvas)

  handler.setInputAction(function(click) {
    const pickedObject = viewer.scene.pick(click.position, 0, 0)

    if (Cesium.defined(pickedObject)) {
      for (let i = 0; i < beaconPoints.length; i++) {
        const beacon = beaconPoints[i]
        // 检查是否点击了光柱相关的任何实体
        const isBeaconEntity =
          pickedObject.id === beacon.cylinder.id ||
          pickedObject.id === beacon.outerCylinder.id ||
          pickedObject.id === beacon.coreCylinder.id ||
          pickedObject.id === beacon.innerCylinder.id ||
          pickedObject.id === beacon.groundPoint.id ||
          pickedObject.id === beacon.groundHalo.id ||
          pickedObject.id === beacon.topBurst.id ||
          beacon.waves.some(w => pickedObject.id === w.main.id || w.trails.some(t => pickedObject.id === t.id))

        if (isBeaconEntity) {
          const popup = beaconPopups[i]
          if (selectedBeacon === i) {
            popup.style.display = 'none'
            selectedBeacon = null
          } else {
            beaconPopups.forEach((p, idx) => {
              if (idx !== i) p.style.display = 'none'
            })
            popup.style.display = 'block'
            selectedBeacon = i
          }
          break
        }
      }
    }
  }, click)
} else {

```

```

        beaconPopups.forEach(p => p.style.display = 'none')
        selectedBeacon = null
    }
}, Cesium.ScreenSpaceEventType.LEFT_CLICK)

viewer.scene.postRender.addEventListener(() => {
    updatePositionDisplay()
    updatePopupPositions()
})
}

const updatePositionDisplay = () => {
    if (!viewer) return
    const cameraPosition = viewer.camera.positionCartographic
    // 脱敏：不展示具体经纬度，仅保留与视图相关的非地点信息
    currentPosition.value.lon = 'X'
    currentPosition.value.lat = 'Y'
    currentPosition.value.altitude = Math.round(cameraPosition.height)
    currentPosition.value.heading = Math.round(Cesium.Math.toDegrees(viewer.camera.heading))

    geoStats.value.terrainExaggeration = viewer?.terrainExaggeration ?? 'X'
    geoStats.value.beaconCount = Array.isArray(beaconPoints) ? beaconPoints.length : 0
    geoStats.value.fogDensity = Number(viewer?.scene?.fog?.density ?? defaultFogDensity).toFixed(5)
    geoStats.value.pitchDeg = Math.round(Cesium.Math.toDegrees(viewer.camera.pitch))
    geoStats.value.rollDeg = Math.round(Cesium.Math.toDegrees(viewer.camera.roll))
}

const updatePopupPositions = () => {
    beaconPoints.forEach((beacon, index) => {
        const popup = beaconPopups[index]
        if (!popup || popup.style.display === 'none') return

        const position = Cesium.Cartesian3.fromDegrees(beacon.lon, beacon.lat, beacon.basePosition.height)
        const windowCoord = Cesium.SceneTransforms.wgs84ToWindowCoordinates(viewer.scene, position)

        if (Cesium.defined(windowCoord)) {
            const x = windowCoord.x - popup.offsetWidth / 2
            const y = windowCoord.y - popup.offsetHeight - 50
            popup.style.transform = `translate3d(${Math.round(x)}px, ${Math.round(y)}px, 0)`
        }
    })
}

const createBeaconPoints = () => {
    const nearCount = Math.floor(Math.random() * 3) + 6
    const horizonCount = 5
    const pointCount = nearCount + horizonCount

    for (let i = 0; i < pointCount; i++) {
        const isHorizon = i >= nearCount
        const spread = isHorizon ? 0.35 : 0.1
        const lon = LIUZHOU_STATION.lon + (Math.random() - 0.5) * spread
        const lat = LIUZHOU_STATION.lat + (Math.random() - 0.5) * spread
        const pillarHeight = isHorizon ? 2200 + Math.random() * 1000 : 1400 + Math.random() * 700
        const radiusScale = isHorizon ? 1.35 : 1
        const waveCount = isHorizon ? 3 : 4
        const maxWaveRadius = isHorizon ? 2200 : 1300
        const waveAlpha = isHorizon ? 0.32 : 0.45
        const wavePhases = Array.from({ length: waveCount }, (_, idx) => idx / waveCount)

        const eventTypes = ['设备正常', '温度异常', '维护中', '离线', '电压异常']
        const eventMessages = [
            '信号灯运行正常',
            '温度超过阈值',

```

```

    '设备正在维护',
    '设备离线',
    '电压异常'
]
const randomEventIndex = Math.floor(Math.random() * eventTypes.length)
const eventType = eventTypes[randomEventIndex]
const eventMessage = eventMessages[randomEventIndex]

const groundPosition = Cesium.Cartesian3.fromDegrees(lon, lat, 0)
const basePosition = Cesium.Cartesian3.fromDegrees(lon, lat, pillarHeight / 2)
const topPosition = Cesium.Cartesian3.fromDegrees(lon, lat, pillarHeight)

const pillarAlphaBase = isHorizon ? 0.42 : 0.55
const outerAlphaBase = isHorizon ? 0.18 : 0.24
const coreAlphaBase = isHorizon ? 0.68 : 0.82
const innerAlphaBase = isHorizon ? 0.78 : 0.9

// 主光柱
const cylinder = viewer.entities.add({
  position: basePosition,
  cylinder: {
    length: pillarHeight,
    topRadius: 26 * radiusScale,
    bottomRadius: 14 * radiusScale,
    material: Cesium.Color.fromCssColorString('#37d7ff').withAlpha(pillarAlphaBase),
    outline: true,
    outlineColor: Cesium.Color.CYAN.withAlpha(0.35),
    outlineWidth: 2
  }
})

// 外层体积光
const outerCylinder = viewer.entities.add({
  position: basePosition,
  cylinder: {
    length: pillarHeight * 1.05,
    topRadius: 52 * radiusScale,
    bottomRadius: 36 * radiusScale,
    material: Cesium.Color.fromCssColorString('#1c8bff').withAlpha(outerAlphaBase),
    outline: false
  }
})

// 核心高亮光柱
const coreCylinder = viewer.entities.add({
  position: Cesium.Cartesian3.fromDegrees(lon, lat, pillarHeight * 0.52),
  cylinder: {
    length: pillarHeight * 1.04,
    topRadius: 8 * radiusScale,
    bottomRadius: 4 * radiusScale,
    material: Cesium.Color.fromCssColorString('#8bffff').withAlpha(coreAlphaBase),
    outline: false
  }
})

// 内层亮芯
const innerCylinder = viewer.entities.add({
  position: Cesium.Cartesian3.fromDegrees(lon, lat, pillarHeight * 0.45),
  cylinder: {
    length: pillarHeight * 0.92,
    topRadius: 5 * radiusScale,
    bottomRadius: 2 * radiusScale,
    material: Cesium.Color.fromCssColorString('#00ffff').withAlpha(innerAlphaBase),
    outline: false
  }
})

```

```

    }
  })

  // 地面核心发光点
  const groundPoint = viewer.entities.add({
    position: groundPosition,
    point: {
      pixelSize: isHorizon ? 24 : 34,
      color: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.9),
      outlineColor: Cesium.Color.WHITE,
      outlineWidth: 3,
      scaleByDistance: new Cesium.NearFarScalar(500, 2, 5000, 1)
    }
  })

  // 地面常驻光晕
  const groundHalo = viewer.entities.add({
    position: groundPosition,
    ellipse: {
      semiMinorAxis: isHorizon ? 180 : 120,
      semiMajorAxis: isHorizon ? 180 : 120,
      height: 0,
      material: Cesium.Color.fromCssColorString('#00b7ff').withAlpha(isHorizon ? 0.18 : 0.25),
      outline: false
    }
  })

  // 顶部爆光层
  const topBurst = viewer.entities.add({
    position: topPosition,
    ellipse: {
      semiMinorAxis: isHorizon ? 95 : 70,
      semiMajorAxis: isHorizon ? 95 : 70,
      height: pillarHeight,
      material: Cesium.Color.fromCssColorString('#a7ffff').withAlpha(isHorizon ? 0.28 : 0.35),
      outline: false
    }
  })

  // 地表扩散冲击波
  const waves = []

  for (let w = 0; w < waveCount; w++) {
    const mainWave = viewer.entities.add({
      position: groundPosition,
      ellipse: {
        semiMinorAxis: 30 + w * 120,
        semiMajorAxis: 30 + w * 120,
        height: 0,
        material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(waveAlpha - w * 0.15),
        outline: true,
        outlineColor: Cesium.Color.CYAN.withAlpha(0.75 - w * 0.15),
        outlineWidth: 3
      }
    })
  }

  const trails = Array.from({ length: 2 }, (_, trailIndex) => viewer.entities.add({
    position: groundPosition,
    ellipse: {
      semiMinorAxis: 30 + w * 120,
      semiMajorAxis: 30 + w * 120,
      height: 0,
      material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.0),
      outline: true,
      outlineColor: Cesium.Color.CYAN.withAlpha(0.0),

```

```

        outlineWidth: 2 - trailIndex * 0.5
    }
}))

waves.push({
  main: mainWave,
  trails
})
}

beaconPoints.push({
  cylinder,
  outerCylinder,
  coreCylinder,
  innerCylinder,
  pillarAlphaBase,
  outerAlphaBase,
  coreAlphaBase,
  innerAlphaBase,
  groundPoint,
  groundHalo,
  topBurst,
  waves,
  basePosition: { lon, lat, height: pillarHeight },
  maxRadius: maxWaveRadius,
  waveSpeed: 0.18 + Math.random() * 0.1,
  waveAlpha,
  waveStartTime: Date.now() + i * 600,
  wavePhases,
  id: i, lon, lat,
  eventType, eventMessage
})

const popupDiv = document.createElement('div')
popupDiv.className = 'beacon-popup'
popupDiv.innerHTML = `
  <div class="popup-content">
    <div class="popup-title">🚦 信号灯 ${i + 1} - ${eventType}</div>
    <div class="popup-message">${eventMessage}</div>
    <div class="popup-coord">📍 ${lon.toFixed(4)}, ${lat.toFixed(4)}</div>
    <div class="popup-close" onclick="event.stopPropagation(); window.vueHidePopup(${i})">X</div>
  </div>
`

popupDiv.style.cssText = 'position:absolute;left:0;top:0;display:none;z-index:1000:pointer-events:auto;'
viewer.container.appendChild(popupDiv)
beaconPopups.push(popupDiv)
}

const style = document.createElement('style')
style.textContent = `
  .beacon-popup { background:rgba(0,20,40,0.95);border:2px solid rgba(0,200,255,0.6);border-radius:8px;padding:15px;min-width:200px;backdrop-
  filter:blur(10px); }
  .popup-content { color:#fff;font-size:13px; }
  .popup-title { color:#00d4ff;font-weight:bold;margin-bottom:8px; }
  .popup-close { position:absolute;top:5px;right:8px;cursor:pointer;color:#ff6666; }
`

document.head.appendChild(style)
}

const updateWaveAnimation = () => {
  const currentTime = Date.now()

  beaconPoints.forEach((beacon) => {
    if (currentTime < beacon.waveStartTime) return

```

```

const baseTime = (currentTime - beacon.waveStartTime) * 0.001

beacon.waves.forEach((waveLayer, wIndex) => {
  const phase = beacon.wavePhases[wIndex] || 0
  const progress = ((baseTime * beacon.waveSpeed) + phase) % 1.0
  const minRadius = 40
  const currentRadius = minRadius + progress * (beacon.maxRadius - minRadius)

  waveLayer.main.ellipse.semiMinorAxis = currentRadius
  waveLayer.main.ellipse.semiMajorAxis = currentRadius

  const alpha = beacon.waveAlpha * (1 - progress * 0.92)
  waveLayer.main.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(alpha)

  const outlineAlpha = 0.75 * (1 - progress * 0.86)
  waveLayer.main.ellipse.outlineColor = Cesium.Color.CYAN.withAlpha(outlineAlpha)

  waveLayer.trails.forEach((trailWave, trailIndex) => {
    const trailOffset = (trailIndex + 1) * 0.11
    const trailProgress = progress - trailOffset
    if (trailProgress <= 0) {
      trailWave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.0)
      trailWave.ellipse.outlineColor = Cesium.Color.CYAN.withAlpha(0.0)
      return
    }

    const trailRadius = minRadius + trailProgress * (beacon.maxRadius - minRadius)
    trailWave.ellipse.semiMinorAxis = trailRadius
    trailWave.ellipse.semiMajorAxis = trailRadius
    const trailAlpha = beacon.waveAlpha * 0.55 * (1 - trailProgress * 0.92) * (1 - trailIndex * 0.22)
    trailWave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(Math.max(0, trailAlpha))
    trailWave.ellipse.outlineColor = Cesium.Color.CYAN.withAlpha(Math.max(0, trailAlpha * 0.9))
  })
})

const pulseTime = currentTime * 0.0022
const pulseA = 0.5 + 0.5 * Math.sin(pulseTime + beacon.id * 0.7)
const pulseB = 0.5 + 0.5 * Math.sin(pulseTime * 1.35 + beacon.id * 1.1)

const groundPulseSize = 26 + pulseA * 16
beacon.groundPoint.point.pixelSize = groundPulseSize

beacon.cylinder.cylinder.material = Cesium.Color.fromCssColorString('#37d7ff')
  .withAlpha(beacon.pillarAlphaBase * (0.8 + pulseA * 0.3))
beacon.outerCylinder.cylinder.material = Cesium.Color.fromCssColorString('#1c8bff')
  .withAlpha(beacon.outerAlphaBase * (0.7 + pulseB * 0.35))
beacon.coreCylinder.cylinder.material = Cesium.Color.fromCssColorString('#9effff')
  .withAlpha(beacon.coreAlphaBase * (0.82 + pulseA * 0.25))
beacon.innerCylinder.cylinder.material = Cesium.Color.fromCssColorString('#00ffff')
  .withAlpha(beacon.innerAlphaBase * (0.85 + pulseB * 0.2))

const haloSize = 90 + pulseA * 120
beacon.groundHalo.ellipse.semiMajorAxis = haloSize
beacon.groundHalo.ellipse.semiMinorAxis = haloSize
beacon.groundHalo.ellipse.material = Cesium.Color.fromCssColorString('#00b7ff')
  .withAlpha(0.14 + pulseB * 0.18)

const burstSize = 56 + pulseB * 55
beacon.topBurst.ellipse.semiMajorAxis = burstSize
beacon.topBurst.ellipse.semiMinorAxis = burstSize
beacon.topBurst.ellipse.material = Cesium.Color.fromCssColorString('#b9ffff')
  .withAlpha(0.2 + pulseA * 0.26)
})

```

```

}

// 数据更新定时器
const updateData = () => {
  // 保持地震震动总体良好且波动平稳
  const target = 2.2 + (Math.random() - 0.5) * 0.08
  const smoothed = seismicLevel.value * 0.86 + target * 0.14
  seismicLevel.value = Math.max(1.8, Math.min(3.0, Number(smoothed.toFixed(2))))
  // 更新24小时数据
  seismicData24h.value.shift()
  seismicData24h.value.push(seismicLevel.value)

  // 模拟列车数据更新
  if (Math.random() > 0.95 && trainStats.value.passed < trainStats.value.total) {
    trainStats.value.passed++
  }

  // 恶劣天气：同步湿度趋势（与大数据平台保持一致）
  if (severeWeatherEnabled.value) {
    syncWeatherFromMock()
  }
}

// 从API加载仪表盘数据
const loadDashboardData = async () => {
  await refreshWeather()
  airQuality.value = {
    aqi: 45,
    level: '优',
    pollutants: [
      { name: 'PM2.5', value: '23µg/m³' },
      { name: 'PM10', value: '45µg/m³' },
      { name: 'O3', value: '68µg/m³' },
      { name: 'NO2', value: '32µg/m³' }
    ]
  }
  trainStats.value = {
    passed: 42,
    total: 58,
    onTime: 100,
    nextTrain: 'G1502 14:35'
  }
  currentRailway.value = {
    name: DEMO_SCENARIO.railwayName,
    start: DEMO_SCENARIO.stationStart,
    end: DEMO_SCENARIO.stationEnd,
    length: '255'
  }
  seismicLevel.value = 2.2
}

// 从API加载地震历史数据
const loadSeismicData = async (range = '24h') => {
  if (range === '24h') {
    seismicData24h.value = [2.1, 2.0, 2.2, 2.1, 2.3, 2.2, 2.1, 2.2, 2.3, 2.2, 2.1, 2.2, 2.1, 2.0, 2.1, 2.2, 2.3, 2.2, 2.1, 2.2, 2.1, 2.0, 2.1, 2.2]
  } else if (range === 'week') {
    seismicDataWeek.value = [2.2, 2.1, 2.3, 2.4, 2.2, 2.1, 2.2]
  } else if (range === 'month') {
    seismicDataMonth.value = Array.from({ length: 30 }, (_, idx) => Number((2.0 + Math.sin(idx / 4) * 0.18).toFixed(2)))
  }
}

// 从API加载天气历史数据

```

```

const loadWeatherHistory = async (range = '24h') => {
  if (range === '24h') {
    weatherData24h.value = [22, 22, 21, 21, 20, 20, 21, 22, 23, 24, 25, 26, 27, 27, 26, 26, 25, 24, 24, 23, 23, 22, 22, 21]
  } else if (range === 'week') {
    weatherDataWeek.value = [23, 24, 23, 25, 24, 23, 24]
  } else if (range === 'month') {
    weatherDataMonth.value = Array.from({ length: 30 }, (_, idx) => 22 + Math.round(Math.sin(idx / 5) * 3))
  }
}

onMounted(async () => {
  onlineHandler = () => { networkOnline.value = true }
  offlineHandler = () => { networkOnline.value = false }
  window.addEventListener('online', onlineHandler)
  window.addEventListener('offline', offlineHandler)

  restoreCesiumCache()
  cacheSaveTimer = setInterval(writeCesiumCache, 1500)

  try {
    // 本地加载数据（即使 Cesium 初始化失败也不影响面板展示）
    await loadDashboardData()
    await loadSeismicData('24h')
    await loadWeatherHistory('24h')
  } catch (error) {
    console.warn('面板数据初始化失败，将尝试使用缓存数据:', error)
    restoreCesiumCache()
  }

  try {
    await initCesium()
    installCameraCruiseAutoPause()
    setTimeout(() => flyToLiuZhou(), 500)
    setupInteractions()
  } catch (error) {
    console.warn('Cesium 初始化失败（可能离线），面板将继续使用缓存数据:', error)
  }

  updateDataTimer = setInterval(updateData, 2000)
  window.vueHidePopup = hidePopup
})

onBeforeUnmount(() => {
  if (onlineHandler) {
    window.removeEventListener('online', onlineHandler)
    onlineHandler = null
  }
  if (offlineHandler) {
    window.removeEventListener('offline', offlineHandler)
    offlineHandler = null
  }
  if (cacheSaveTimer) {
    clearInterval(cacheSaveTimer)
    cacheSaveTimer = null
  }
  writeCesiumCache()
  if (updateDataTimer) {
    clearInterval(updateDataTimer)
    updateDataTimer = null
  }
  stopRainFogSimulation()
  if (bgmAudio) {
    bgmAudio.pause()
    bgmAudio.currentTime = 0
  }

```

```

    bgmAudio = null
  }
  if (audioUnlockHandler) {
    window.removeEventListener('pointerdown', audioUnlockHandler, true)
    audioUnlockHandler = null
  }
  if (viewer && globalBreathStage) {
    viewer.scene.postProcessStages.remove(globalBreathStage)
    globalBreathStage = null
  }
  stopSevereWeatherUiEffects()
  stopCameraCruise()
  if (viewer && cameraCruiseAutoPauseHandler) {
    viewer.scene.canvas.removeEventListener('pointerdown', cameraCruiseAutoPauseHandler)
    cameraCruiseAutoPauseHandler = null
  }
  if (viewer && cameraAngleChangedHandler) {
    viewer.camera.changed.removeEventListener(cameraAngleChangedHandler)
    cameraAngleChangedHandler = null
  }
  if (viewer) viewer.destroy()
})

```

```

watch(severeWeatherEnabled, (enabled) => {
  // 允许从其它页面切换恶劣天气模拟后，同步到 Cesium 视图
  applySevereWeatherChange(enabled)
  if (enabled) {
    syncWeatherFromMock()
  }
})
</script>

```

```

<style scoped>
.cesium-view {
  width: 100%;
  height: 100%;
  position: relative;
}

```

```

#cesiumContainer {
  width: 100%;
  height: 100%;
}

```

```

.offline-badge {
  position: absolute;
  top: 14px;
  left: 50%;
  transform: translateX(-50%);
  z-index: 120;
  padding: 6px 12px;
  border-radius: 999px;
  background: rgba(120, 0, 0, 0.75);
  border: 1px solid rgba(255, 90, 90, 0.55);
  color: #fff;
  font-size: 14px;
  letter-spacing: 0.5px;
  box-shadow: 0 0 18px rgba(255, 60, 60, 0.22);
  backdrop-filter: blur(8px);
}

```

```

/* 控制按钮 */
.control-buttons {
  position: absolute;

```

```

    bottom: 20px;
    left: 50%;
    transform: translateX(-50%);
    display: flex;
    gap: 12px;
    z-index: 100;
}

.reset-btn, .toggle-btn {
    padding: 7px 12px;
    background: rgba(0, 20, 40, 0.85);
    border: 2px solid rgba(0, 200, 255, 0.3);
    border-radius: 8px;
    color: #00d4ff;
    cursor: pointer;
    font-size: 12px;
    font-weight: bold;
    transition: all 0.3s;
    backdrop-filter: blur(10px);
}

.reset-btn:hover, .toggle-btn:hover {
    background: rgba(0, 40, 80, 0.9);
    box-shadow: 0 0 15px rgba(0, 200, 255, 0.5);
}

.toggle-btn.active {
    border-color: rgba(120, 190, 255, 0.9);
    color: #d8e8ff;
    background: rgba(25, 70, 120, 0.9);
    box-shadow: 0 0 20px rgba(110, 180, 255, 0.45);
}

:deep(.cesium-viewer-toolbar),
:deep(.cesium-viewer-fullscreenContainer),
:deep(.cesium-viewer-animationContainer),
:deep(.cesium-viewer-timelineContainer),
:deep(.cesium-viewer-bottom) {
    display: none !important;
}

/* 侧边面板 */
.side-panel {
    position: absolute;
    top: 56px;
    width: 340px;
    max-height: calc(100% - 66px);
    overflow-y: auto;
    z-index: 50;
}

.left-panel { left: 15px; }
.right-panel { right: 15px; }

.side-panel::-webkit-scrollbar { width: 4px; }
.side-panel::-webkit-scrollbar-track { background: rgba(0,0,0,0.2); }
.side-panel::-webkit-scrollbar-thumb { background: rgba(0,200,255,0.4); border-radius: 2px; }

/* 历史记录头部 */
.history-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 8px;

```

```
}

.time-select {
  background: rgba(0, 100, 150, 0.3);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 4px;
  color: #00d4ff;
  font-size: 11px;
  padding: 4px 8px;
  cursor: pointer;
  outline: none;
}

.time-select:hover {
  background: rgba(0, 100, 150, 0.5);
}

.time-select option {
  background: rgba(0, 20, 40, 0.95);
  color: #fff;
}

/* 图表提示 */
.chart-tooltip {
  position: relative;
  display: inline-block;
  background: rgba(0, 40, 80, 0.95);
  border: 1px solid rgba(0, 200, 255, 0.5);
  border-radius: 4px;
  padding: 4px 8px;
  font-size: 11px;
  color: #fff;
  margin-top: 4px;
}

.vibration-history {
  margin-top: 10px;
  position: relative;
}

.history-svg {
  width: 100%;
  height: 60px;
  cursor: crosshair;
}

/* 面板卡片 */
.panel-card {
  background: rgba(0, 20, 40, 0.85);
  border: 1px solid rgba(0, 200, 255, 0.25);
  border-radius: 10px;
  margin-bottom: 12px;
  backdrop-filter: blur(10px);
  overflow: hidden;
}

.panel-header {
  padding: 12px 15px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.2);
  background: rgba(0, 200, 255, 0.08);
}

.panel-title {
  font-size: 14px;
```

```
font-weight: bold;
color: #fff;
}

.panel-subtitle {
font-size: 10px;
color: #666;
letter-spacing: 0.5px;
}

.panel-content {
padding: 15px;
}

.geo-ticker-shell {
position: relative;
overflow: hidden;
border: 1px solid rgba(0, 200, 255, 0.2);
border-radius: 8px;
background: linear-gradient(90deg, rgba(0, 24, 46, 0.9), rgba(0, 44, 78, 0.78));
}

.geo-ticker-shell::before,
.geo-ticker-shell::after {
content: '';
position: absolute;
top: 0;
width: 28px;
height: 100%;
z-index: 1;
pointer-events: none;
}

.geo-ticker-shell::before {
left: 0;
background: linear-gradient(90deg, rgba(0, 18, 35, 0.96), rgba(0, 18, 35, 0));
}

.geo-ticker-shell::after {
right: 0;
background: linear-gradient(270deg, rgba(0, 18, 35, 0.96), rgba(0, 18, 35, 0));
}

.geo-ticker-track {
display: flex;
align-items: center;
gap: 36px;
width: max-content;
padding: 10px 0;
animation: geo-ticker-scroll 30s linear infinite;
}

.geo-ticker-item {
color: #dff7ff;
font-size: 14px;
white-space: nowrap;
text-shadow: 0 0 12px rgba(0, 212, 255, 0.28);
}

@keyframes geo-ticker-scroll {
from {
transform: translateX(0);
}
to {
```

```
        transform: translateX(-50%);
    }
}

/* 震动监测 */
.seismic-display {
    display: flex;
    gap: 15px;
    margin-bottom: 12px;
}

.seismic-gauge { flex: 1; }
.gauge-svg { width: 100%; height: auto; }

.seismic-info { flex: 1; }
.seismic-value { margin-bottom: 8px; }
.value-label { font-size: 11px; color: #888; display: block; }
.value-num { font-size: 28px; font-weight: bold; }

.seismic-status {
    display: inline-block;
    padding: 4px 12px;
    border-radius: 12px;
    font-size: 12px;
    font-weight: bold;
}

.status-normal { background: rgba(0,200,255,0.2); color: #00d4ff; }
.status-warning { background: rgba(255,200,0,0.2); color: #ffcc00; }
.status-danger { background: rgba(255,50,50,0.2); color: #ff3333; }

.vibration-history { margin-top: 10px; }
.history-label { font-size: 11px; color: #888; display: block; margin-bottom: 5px; }
.history-svg { width: 100%; height: 50px; }

/* 位置信息 */
.position-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
}

.pos-item {
    background: rgba(0, 200, 255, 0.08);
    padding: 10px;
    border-radius: 6px;
    display: flex;
    flex-direction: column;
    align-items: center;
}

.pos-icon { font-size: 18px; margin-bottom: 4px; }
.pos-label { font-size: 11px; color: #888; }
.pos-value { font-size: 14px; color: #00d4ff; font-weight: bold; margin-top: 2px; }

/* 铁道信息 */
.railway-name {
    background: rgba(0, 200, 255, 0.1);
    padding: 12px;
    border-radius: 6px;
    text-align: center;
    margin-bottom: 12px;
}
```

```
.rail-label { font-size: 11px; color: #888; display: block; }
.rail-value { font-size: 18px; color: #00d4ff; font-weight: bold; margin-top: 4px; }

.railway-detail { display: flex; flex-direction: column; gap: 8px; }
.detail-item { display: flex; justify-content: space-between; padding: 6px 0; border-bottom: 1px solid rgba(255,255,255,0.05); }
.detail-label { color: #888; font-size: 12px; }
.detail-val { color: #fff; font-size: 12px; }

/* 列车统计 */
.train-stats {
  display: flex;
  gap: 20px;
  align-items: center;
}

.stat-circle {
  position: relative;
  width: 80px;
  height: 80px;
}

.stat-circle svg { width: 100%; height: 100%; }
.stat-circle .stat-num {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  font-size: 20px;
  font-weight: bold;
  color: #00d4ff;
}

.stat-circle .stat-label {
  position: absolute;
  bottom: -9px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 10px;
  color: #888;
}

.stat-details { flex: 1; }
.stat-row { display: flex; justify-content: space-between; padding: 5px 0; font-size: 12px; }
.stat-row .stat-label { color: #888; }
.stat-row .stat-val { color: #fff; }
.stat-row .stat-val.green { color: #33ff33; }

/* 天气紧凑布局 */
.weather-compact {
  display: flex;
  gap: 12px;
  margin-bottom: 12px;
}

.weather-left {
  display: flex;
  align-items: center;
  gap: 10px;
  flex: 1;
}

.weather-icon-small {
  font-size: 36px;
}
```

```
.weather-info-compact {
  flex: 1;
}

.weather-temp-compact {
  display: flex;
  align-items: baseline;
  gap: 2px;
}

.temp-value-small {
  font-size: 28px;
  font-weight: bold;
  color: #fff;
}

.temp-unit-small {
  font-size: 14px;
  color: #888;
}

.weather-desc-small {
  font-size: 12px;
  color: #00d4ff;
}

.weather-location-small {
  font-size: 10px;
  color: #888;
  margin-top: 2px;
}

.weather-right {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 6px;
  flex: 1;
}

.w-item-compact {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 6px 8px;
  background: rgba(255,255,255,0.03);
  border-radius: 4px;
}

.w-icon-small {
  font-size: 14px;
}

.w-info {
  display: flex;
  flex-direction: column;
}

.w-label-small {
  font-size: 9px;
  color: #888;
}

.w-value-small {
  font-size: 11px;
```

```
    color: #fff;
}

/* 天气曲线图区域 */
.weather-chart-section {
    margin-top: 10px;
}

.weather-tooltip {
    background: rgba(80, 40, 0, 0.95);
    border-color: rgba(255, 153, 0, 0.5);
}

/* 天气 - 保留原样式用于其他地方 */
.weather-main {
    text-align: center;
    padding: 15px;
    background: rgba(0, 200, 255, 0.05);
    border-radius: 8px;
    margin-bottom: 12px;
}

.weather-icon { font-size: 48px; }
.weather-temp { margin: 8px 0; }
.temp-value { font-size: 36px; font-weight: bold; color: #fff; }
.temp-unit { font-size: 16px; color: #888; }
.weather-desc { font-size: 14px; color: #00d4ff; }
.weather-location { font-size: 12px; color: #888; margin-top: 4px; }

.weather-details {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 8px;
}

.w-item {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 8px;
    background: rgba(255, 255, 255, 0.03);
    border-radius: 6px;
}

.w-icon { font-size: 16px; }
.w-label { flex: 1; font-size: 11px; color: #888; }
.w-value { font-size: 12px; color: #fff; }

.refresh-btn {
    width: 100%;
    margin-top: 12px;
    padding: 8px;
    background: rgba(0, 200, 255, 0.15);
    border: 1px solid rgba(0, 200, 255, 0.3);
    border-radius: 6px;
    color: #00d4ff;
    cursor: pointer;
    font-size: 12px;
    transition: all 0.3s;
}

.refresh-btn:hover:not(:disabled) { background: rgba(0, 200, 255, 0.25); }
.refresh-btn:disabled { opacity: 0.5; cursor: not-allowed; }
```

```
/* 空气质量 */
.aqi-severe-warning {
  margin-bottom: 10px;
  padding: 8px 10px;
  border-radius: 8px;
  border: 1px solid rgba(255, 60, 60, 0.85);
  background: linear-gradient(135deg, rgba(180, 20, 20, 0.85), rgba(110, 10, 10, 0.75));
  color: #fff;
  font-size: 12px;
  font-weight: bold;
  text-align: center;
  box-shadow: 0 0 18px rgba(255, 40, 40, 0.55);
  animation: aqiWarningPulse 0.95s ease-in-out infinite;
}

@keyframes aqiWarningPulse {
  0%, 100% { filter: brightness(1); }
  50% { filter: brightness(1.22); }
}

.aqi-card.aqi-flash {
  animation: aqiCardFlash 0.85s ease-in-out infinite;
}

@keyframes aqiCardFlash {
  0%, 100% {
    box-shadow: 0 0 0 rgba(255, 40, 40, 0);
    border-color: rgba(255, 80, 80, 0.55);
  }
  50% {
    box-shadow: 0 0 26px rgba(255, 50, 50, 0.55);
    border-color: rgba(255, 60, 60, 0.95);
  }
}

.aqi-summary {
  display: flex;
  align-items: baseline;
  justify-content: center;
  gap: 8px;
  padding: 4px 0 8px;
}

.aqi-unit { font-size: 11px; color: #888; }
.aqi-display { text-align: center; padding: 8px 0; }
.aqi-value { font-size: 32px; font-weight: bold; }
.aqi-value.aqi-good { color: #33ff33; }
.aqi-value.aqi-moderate { color: #ffcc00; }
.aqi-value.aqi-bad { color: #ff3333; }
.aqi-level { font-size: 14px; color: #888; margin-top: 4px; }

.aqi-bar {
  height: 8px;
  background: linear-gradient(90deg, #33ff33, #ffcc00, #ff9933, #ff3333);
  border-radius: 4px;
  margin: 10px 0;
  position: relative;
}

.aqi-indicator {
  position: absolute;
  top: -3px;
  width: 14px;
  height: 14px;
  background: #fff;
```

```
border-radius: 50%;
transform: translateX(-50%);
box-shadow: 0 0 5px rgba(0,0,0,0.3);
}

.aqi-items { display: grid; grid-template-columns: 1fr 1fr; gap: 6px; }
.aqi-item { display: flex; justify-content: space-between; font-size: 11px; padding: 4px 8px; background: rgba(255,255,255,0.03); border-radius: 4px; }
.item-name { color: #888; }
.item-value { color: #fff; }

.aqi-chart { height: 70px; }
.aqi-tooltip { background: rgba(0, 40, 80, 0.95); border-color: rgba(0, 200, 255, 0.5); }

/* AI 面板 */
.ai-panel .panel-content { padding: 12px; }

.ai-messages {
  height: 200px;
  overflow-y: auto;
  background: rgba(0,0,0,0.2);
  border-radius: 8px;
  padding: 10px;
  margin-bottom: 10px;
}

.ai-messages::-webkit-scrollbar { width: 4px; }
.ai-messages::-webkit-scrollbar-thumb { background: rgba(0,200,255,0.3); border-radius: 2px; }

.ai-msg {
  display: flex;
  gap: 8px;
  margin-bottom: 10px;
}

.ai-msg.user { flex-direction: row-reverse; }

.msg-avatar {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  background: rgba(0,200,255,0.2);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 14px;
  flex-shrink: 0;
}

.ai-msg.user .msg-avatar { background: rgba(255,200,0,0.2); }

.msg-content {
  max-width: 80%;
  padding: 8px 12px;
  border-radius: 12px;
  font-size: 12px;
  line-height: 1.5;
  background: rgba(0, 200, 255, 0.15);
  color: #fff;
}

.ai-msg.user .msg-content { background: rgba(255, 200, 0, 0.15); }

.ai-msg.thinking .msg-content { opacity: 0.6; }
```

```
.ai-input {
  display: flex;
  gap: 8px;
  margin-bottom: 8px;
}

.ai-input-field {
  flex: 1;
  padding: 10px 12px;
  background: rgba(0,0,0,0.3);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 6px;
  color: #fff;
  font-size: 12px;
  outline: none;
}

.ai-input-field:focus { border-color: #00d4ff; }
.ai-input-field::placeholder { color: #666; }

.ai-send-btn {
  padding: 10px 16px;
  background: rgba(0, 200, 255, 0.2);
  border: 1px solid rgba(0, 200, 255, 0.4);
  border-radius: 6px;
  color: #00d4ff;
  cursor: pointer;
  font-size: 12px;
  transition: all 0.3s;
}

.ai-send-btn:hover:not(:disabled) { background: rgba(0, 200, 255, 0.3); }
.ai-send-btn:disabled { opacity: 0.5; cursor: not-allowed; }

.ai-quick-actions { display: flex; gap: 6px; flex-wrap: wrap; }
.quick-btn {
  padding: 6px 10px;
  background: rgba(255,255,255,0.05);
  border: 1px solid rgba(255,255,255,0.1);
  border-radius: 12px;
  color: #aaa;
  cursor: pointer;
  font-size: 10px;
  transition: all 0.3s;
}

.quick-btn:hover {
  background: rgba(0, 200, 255, 0.15);
  border-color: rgba(0, 200, 255, 0.3);
  color: #00d4ff;
}

/* 过渡动画 */
.slide-left-enter-active, .slide-left-leave-active,
.slide-right-enter-active, .slide-right-leave-active {
  transition: all 0.3s ease;
}

.slide-left-enter-from, .slide-left-leave-to {
  transform: translateX(-100%);
  opacity: 0;
}
```

```

.slide-right-enter-from, .slide-right-leave-to {
  transform: translateX(100%);
  opacity: 0;
}

/* 加载动画 */
.loading {
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background: #000;
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 9999;
}

.loading-content { text-align: center; }

.loading-spinner {
  width: 60px; height: 60px;
  border: 4px solid rgba(0, 200, 255, 0.3);
  border-top-color: #00d4ff;
  border-radius: 50%;
  animation: spin 1s linear infinite;
  margin: 0 auto 20px;
}

@keyframes spin { to { transform: rotate(360deg); } }

.loading-text { color: #00d4ff; font-size: 18px; }

</style>

```

5.2 文件: src/components/ThreeView.vue

- 文件说明: Three.js 数字孪生主界面, 含信号机模型、列车动画、视频监控和遥测联动。
- 行数统计: 3959 行

```

<template>
  <div class="three-view">
    <!-- Three.js 容器 -->
    <div id="threeContainer" ref="threeContainer"></div>

    <!-- 控制按钮 -->
    <div class="control-panel">
      <button class="control-btn" @click="resetView">🔄 复位视角</button>
      <button class="control-btn" @click="toggleSignals">🚦 切换信号</button>
      <div class="train-control-group">
        <button class="control-btn" @click="trainAnimation">🚆 列车行进</button>
        <div class="speed-inline">
          <span class="speed-label">列车速度</span>
          <select v-model.number="trainSpeed" class="speed-select">
            <option v-for="option in speedOptions" :key="option.value" :value="option.value">
              {{ option.label }}
            </option>
          </select>
        </div>
      </div>
      <button class="control-btn" @click="toggleRotation">🌀 自动旋转</button>
      <button class="control-btn" @click="openDeviceModal('power')">⚡ 电源屏</button>
      <button class="control-btn" @click="restoreNormalState">🔧 恢复异常</button>
    </div>

    <!-- 左侧数据面板 -->

```

```

<div class="info-panel">
  <!-- 装饰边框 -->
  <div class="panel-border top-left"></div>
  <div class="panel-border top-right"></div>
  <div class="panel-border bottom-left"></div>
  <div class="panel-border bottom-right"></div>
  <div class="panel-border glow-line"></div>

  <!-- 行人距离感应器 -->
  <div class="panel-section" :class="{ 'panel-alert': pedestrianDesiredVisible, 'panel-boost': pedestrianHotFlash }">
    <h2 class="panel-title">
      <span class="title-icon">🚶</span>
      <span class="title-text">行人距离感应器</span>
      <span class="title-decorator"></span>
    </h2>
    <div class="humanoid-sensor">
      <div class="sensor-row">
        <span class="sensor-label">电源</span>
        <span class="sensor-value online">{{ humanoidSensor.power }}</span>
        <span class="sensor-meta">{{ humanoidSensor.voltage }}</span>
      </div>
      <div class="sensor-row">
        <span class="sensor-label">GPS</span>
        <span class="sensor-value">{{ humanoidSensor.gps }}</span>
      </div>
      <div class="sensor-row">
        <span class="sensor-label">行人距离</span>
        <span class="sensor-value sensor-distance" :class="{ warning: pedestrianDesiredVisible, 'sensor-boost': pedestrianHotFlash }">{{
humanoidSensor.pedestrianDistance }}</span>
      </div>
    </div>
  </div>

  <div class="panel-section">
    <h2 class="panel-title">
      <span class="title-icon">📶</span>
      <span class="title-text">信号机清单</span>
      <span class="title-decorator"></span>
    </h2>
    <div id="signalList"></div>
    <div class="signal-kpis">
      <div class="kpi-item">
        <div class="kpi-label">当前选中</div>
        <div class="kpi-value ok">{{ heroDeviceName }}</div>
      </div>
      <div class="kpi-item">
        <div class="kpi-label">信号状态</div>
        <div class="kpi-value" :class="activeSignalState === 'off' ? 'danger' : 'ok'">{{ activeSignalState === 'off' ? '异常灭灯' : '正常' }}
</div>
      </div>
      <div class="kpi-item">
        <div class="kpi-label">温度</div>
        <div
          class="kpi-value"
          :class="[
            twinEnvVisible ? (signalParams.temperature > 50 ? 'warn' : 'ok') : 'masked',
            twinEnvVisible ? 'env-glow' : ''
          ]"
        >
          {{ twinEnvVisible ? `${signalParams.temperature}°C` : '--' }}
        </div>
      </div>
      <div class="kpi-item">
        <div class="kpi-label">湿度</div>

```

```
<div
  class="kpi-value"
  :class="[twinEnvVisible ? humidityClass : 'masked', twinEnvVisible ? 'env-glow' : '']"
>
  {{ twinEnvVisible ? `${signalParams.humidity}%` : '--' }}
</div>
</div>
<div class="kpi-item">
  <div class="kpi-label">电压</div>
  <div class="kpi-value" :class="voltageClass === 'voltage-normal' ? 'ok' : 'warn'">{{ signalParams.voltage }}V</div>
</div>
<div class="kpi-item">
  <div class="kpi-label">信号强度</div>
  <div class="kpi-value" :class="signalParams.signal < -55 ? 'warn' : 'ok'">{{ signalParams.signal }}dBm</div>
</div>
</div>
<div class="twin-metrics">
  <div class="tm-item">
    <span class="tm-label">设备名称</span>
    <span class="tm-value">{{ heroDeviceName }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">孪生体ID</span>
    <span class="tm-value">{{ signalTwin.twinId }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">资产编码</span>
    <span class="tm-value">{{ signalTwin.assetId }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">设备类型</span>
    <span class="tm-value">{{ signalTwin.deviceType }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">互锁状态</span>
    <span class="tm-value" :class="signalTwin.interlocking === '正常' ? 'ok' : 'warn'">{{ signalTwin.interlocking }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">工作模式</span>
    <span class="tm-value">{{ signalTwin.workMode }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">同步状态</span>
    <span class="tm-value" :class="signalTwin.syncStatus === '已同步' ? 'ok' : 'warn'">{{ signalTwin.syncStatus }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">同步延迟</span>
    <span class="tm-value">{{ signalTwin.syncLatencyMs }}ms</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">数据新鲜度</span>
    <span class="tm-value">{{ signalTwin.dataFreshnessS }}s</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">健康评分</span>
    <span class="tm-value" :class="signalTwin.healthScore >= 85 ? 'ok' : (signalTwin.healthScore >= 70 ? 'warn' : 'danger')">{{
signalTwin.healthScore }}</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">预测剩余寿命</span>
    <span class="tm-value">{{ signalTwin.predictedRulDays }}天</span>
  </div>
  <div class="tm-item">
    <span class="tm-label">固件版本</span>
  </div>
```

```

        <span class="tm-value">{{ signalTwin.firmware }}</span>
    </div>
    <div class="tm-item">
        <span class="tm-label">上次检修</span>
        <span class="tm-value">{{ signalTwin.lastMaintenance }}</span>
    </div>
</div>
</div>

<div class="panel-section">
    <h2 class="panel-title">
        <span class="title-icon">🚨</span>
        <span class="title-text">告警联动</span>
        <span class="title-decorator"></span>
    </h2>
    <div class="alarm-feed">
        <div v-for="item in alertFeed" :key="item.id" class="alarm-card" :class="`alarm-card-${item.level}`">
            <div class="alarm-title">{{ item.title }}</div>
            <div class="alarm-desc">{{ item.description }}</div>
        </div>
        <div v-if="latestVoiceText" class="alarm-voice">最近语音播报: {{ latestVoiceText }}</div>
    </div>
</div>

<!-- 设备状态 -->
<div class="panel-section">
    <h2 class="panel-title">
        <span class="title-icon">🔌</span>
        <span class="title-text">设备状态</span>
        <span class="title-decorator"></span>
    </h2>
    <div class="device-status">
        <div v-for="row in deviceStatusRows" :key="row.label" class="status-row">
            <span class="status-dot" :class="row.className"></span>
            <span class="status-label">{{ row.label }}</span>
            <span class="status-value" :class="row.className">{{ row.value }}</span>
        </div>
    </div>
</div>

</div>

<!-- 右侧数据面板 -->
<div class="stats-panel">
    <!-- 装饰边框 -->
    <div class="panel-border top-left"></div>
    <div class="panel-border top-right"></div>
    <div class="panel-border bottom-left"></div>
    <div class="panel-border bottom-right"></div>
    <div class="panel-border glow-line"></div>

    <div class="panel-section">
        <h2 class="panel-title">
            <span class="title-icon">📷</span>
            <span class="title-text">铁路信号设备数字孪生系统</span>
            <span class="title-decorator"></span>
        </h2>
        <div class="stat-item">
            <span class="stat-label">相机位置:</span>
            <span class="stat-value" id="cameraPos">--</span>
        </div>
        <div class="stat-item">
            <span class="stat-label">运行时间:</span>
            <span class="stat-value" id="uptime">0s</span>
        </div>
    </div>
</div>

```

```
<div class="stat-item">
  <span class="stat-label">最后更新:</span>
  <span class="stat-value" id="lastUpdate">--</span>
</div>
<div class="stat-item">
  <span class="stat-label">系统时间:</span>
  <span class="stat-value">{{ panelTimeText }}</span>
</div>
</div>

<div class="panel-section">
  <h2 class="panel-title">
    <span class="title-icon">🔌</span>
    <span class="title-text">接入状态</span>
    <span class="title-decorator"></span>
  </h2>
  <div class="stat-item">
    <span class="stat-label">遥测接入:</span>
    <span class="stat-value" :class="telemetryStatusClass">{{ telemetryStatusText }}</span>
  </div>
  <div class="stat-item">
    <span class="stat-label">最近上报:</span>
    <span class="stat-value">{{ telemetryLastSeenText }}</span>
  </div>
  <div class="stat-item">
    <span class="stat-label">硬件安装:</span>
    <span class="stat-value" :class="installStatusClass">{{ installStatusText }}</span>
  </div>
  <div class="stat-item">
    <span class="stat-label">进水检测:</span>
    <span class="stat-value" :class="telemetryWaterClass">{{ telemetryWaterText }}</span>
  </div>
  <div class="stat-item">
    <span class="stat-label">行人距离:</span>
    <span class="stat-value" :class="{ warn: pedestrianDesiredVisible, 'stat-boost': pedestrianHotFlash }">{{
humanoidSensor.pedestrianDistance }}</span>
  </div>
</div>

<div class="panel-section">
  <h2 class="panel-title">
    <span class="title-icon">🔌</span>
    <span class="title-text">设备交互</span>
    <span class="title-decorator"></span>
  </h2>
  <div class="shortcut-grid">
    <button class="shortcut-card" @click="openDeviceModal('power')">
      <span class="shortcut-title">电源屏</span>
      <span class="shortcut-desc">动态电压线 + 系统时钟</span>
    </button>
    <button class="shortcut-card" @click="openDeviceModal('turnout')">
      <span class="shortcut-title">道岔监测</span>
      <span class="shortcut-desc">查看道岔电压参数</span>
    </button>
    <button class="shortcut-card" @click="openDeviceModal('track')">
      <span class="shortcut-title">轨道电路</span>
      <span class="shortcut-desc">查看调谐单元电容值</span>
    </button>
    <button class="shortcut-card" @click="openDeviceModal('signal', selectedSignalId)">
      <span class="shortcut-title">{{ heroDeviceName }}</span>
      <span class="shortcut-desc">查看当前信号机监测曲线</span>
    </button>
  </div>
</div>
</div>
```

```
<div class="panel-section">
  <h2 class="panel-title">
    <span class="title-icon">■</span>
    <span class="title-text">键盘控制</span>
    <span class="title-decorator"></span>
  </h2>
  <div class="key-grid">
    <div class="key-chip"><b>K</b><span>接入状态改为已接入</span></div>
    <div class="key-chip"><b>M</b><span>硬件安装完成，信号机恢复正常</span></div>
    <div class="key-chip"><b>P</b><span>1495 信号机故障掉压并灭灯</span></div>
    <div class="key-chip"><b>L</b><span>1495 轨道电路故障掉压</span></div>
  </div>
</div>

<!-- 主信号灯实时参数 -->
<div class="panel-section">
  <h2 class="panel-title">
    <span class="title-icon">🚦</span>
    <span class="title-text">实时参数</span>
    <span class="title-decorator"></span>
  </h2>
  <!-- 参数选择器 -->
  <div class="param-selectors">
    <select v-model="selectedParam" class="param-select" @change="onParamChange">
      <option value="temperature">🔥 温度</option>
      <option value="humidity">💧 湿度</option>
      <option value="light">☀️ 光照</option>
      <option value="voltage">⚡ 电压</option>
      <option value="current">🔌 电流</option>
      <option value="signal">🚦 信号</option>
    </select>

    <select v-model="paramTimeRange" class="param-select" @change="onParamTimeChange">
      <option value="1h">近1小时</option>
      <option value="24h">近24小时</option>
      <option value="week">近一周</option>
    </select>
  </div>
  <!-- 当前值显示 -->
  <div class="current-value-display">
    <div class="current-param-icon">{{ paramConfig.icon }}</div>
    <div class="current-param-info">
      <div class="current-param-label">{{ paramConfig.label }}</div>
      <div class="current-param-value" :style="{ color: paramConfig.color }">
        {{ currentParamValue }}<span class="param-unit">{{ paramConfig.unit }}</span>
      </div>
    </div>
  </div>
</div>

<!-- 曲线图 -->
<div class="param-chart-container">
  <svg viewBox="0 0 280 100" class="param-chart-svg">
    <defs>
      <linearGradient id="'paramGradient' + selectedParam" x1="0%" y1="0%" x2="0%" y2="100%">
        <stop offset="0%" :style="{ stop-color: ${paramConfig.color}; stop-opacity: 0.4 }" />
        <stop offset="100%" :style="{ stop-color: ${paramConfig.color}; stop-opacity: 0 }" />
      </linearGradient>
    </defs>
    <!-- 网格线 -->
    <line x1="10" y1="20" x2="270" y2="20" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <line x1="10" y1="50" x2="270" y2="50" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <line x1="10" y1="80" x2="270" y2="80" stroke="rgba(255,255,255,0.1)" stroke-width="1" />
    <!-- 数据区域 -->
    <polygon :points="paramAreaPoints" :fill="'url(#paramGradient${selectedParam})'" />
    <!-- 数据线 -->
```

```

<polyline :points="paramLinePoints" fill="none" :stroke="paramConfig.color" stroke-width="2" />
<!-- 数据点 -->
<circle v-for="(point, idx) in paramChartPoints" :key="idx"
  :cx="point.x" :cy="point.y" r="3" :fill="paramConfig.color"
  @mouseenter="hoveredParamPoint = idx"
  @mouseleave="hoveredParamPoint = null" />
</svg>
<div v-if="hoveredParamPoint !== null" class="param-tooltip">
  {{ paramTimeLabels[hoveredParamPoint] }}: {{ currentParamData[hoveredParamPoint] }}{{ paramConfig.unit }}
</div>
</div>
<!-- 参数快捷列表 -->
<div class="param-quick-list">
  <div class="param-quick-item" v-for="(p, key) in paramConfigs" :key="key"
    :class="{ active: selectedParam === key }" @click="selectedParam = key">
    <span class="pq-icon">{{ p.icon }}</span>
    <span class="pq-value">{{ signalParams[key] }}{{ p.unit }}</span>
  </div>
</div>
</div>

<div v-if="deviceModal.visible" class="device-modal-backdrop" @click.self="closeDeviceModal">
  <div class="device-modal">
    <div class="device-modal-header">
      <div>
        <div class="device-modal-title">{{ modalTypeText }}</div>
        <div class="device-modal-subtitle">系统时间: {{ panelTimeText }}</div>
      </div>
      <button class="device-modal-close" @click="closeDeviceModal">关闭</button>
    </div>
    <div class="device-modal-stats">
      <div v-for="item in modalPreviewStats" :key="item.label" class="device-modal-stat">
        <span>{{ item.label }}</span>
        <strong>{{ item.value }}</strong>
      </div>
    </div>
    <div class="device-modal-chart">
      <svg viewBox="0 0 320 140" class="device-modal-svg">
        <defs>
          <linearGradient id="deviceModalGradient" x1="0%" y1="0%" x2="0%" y2="100%">
            <stop offset="0%" :style="`stop-color:${modalChartColor};stop-opacity:0.42`" />
            <stop offset="100%" :style="`stop-color:${modalChartColor};stop-opacity:0.02`" />
          </linearGradient>
        </defs>
        <line x1="16" y1="30" x2="304" y2="30" class="device-grid-line" />
        <line x1="16" y1="74" x2="304" y2="74" class="device-grid-line" />
        <line x1="16" y1="118" x2="304" y2="118" class="device-grid-line" />
        <polygon :points="modalAreaPoints" fill="url(#deviceModalGradient)" />
        <polyline :points="modalLinePoints" :stroke="modalChartColor" class="device-modal-polyline" />
        <circle v-for="(point, index) in modalChartPoints" :key="`${point.time}-${index}`" :cx="point.x" :cy="point.y" r="2.4"
          :fill="modalChartColor" />
      </svg>
      <div class="device-modal-time-row">
        <span>{{ modalChartPoints[0]?.time }}</span>
        <span>{{ modalChartPoints[Math.floor(modalChartPoints.length / 2)]?.time }}</span>
        <span>{{ modalChartPoints[modalChartPoints.length - 1]?.time }}</span>
      </div>
    </div>
    <div class="device-modal-footer">
      <div class="device-modal-note">{{ modalAlertText }}</div>
      <div class="device-modal-actions">
        <button class="control-btn" @click="restoreNormalState">异常消除</button>
      </div>
    </div>
  </div>
</div>

```

```

        <button class="control-btn" @click="closeDeviceModal">继续观察</button>
      </div>
    </div>
  </div>
</div>
</template>

<script setup>
import { ref, computed, onMounted, onBeforeUnmount, watch } from 'vue'
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
import { clone as cloneSkinned } from 'three/examples/jsm/utils/SkeletonUtils.js'
import { CSS2DRenderer, CSS2DObject } from 'three/examples/jsm/renderers/CSS2DRenderer.js'
import gsap from 'gsap'
import { DEMO_SCENARIO } from '../config/demoScenario.js'
import { severeWeatherEnabled, syncedHumidity } from '../services/simulationState.js'

const getWorldYaw = (obj) => {
  if (!obj) return 0
  const q = new THREE.Quaternion()
  obj.getWorldQuaternion(q)
  const e = new THREE.Euler().setFromQuaternion(q, 'YXZ')
  return e.y || 0
}

const threeContainer = ref(null)
const SIGNAL_IDS = ['1491', '1493', '1495', '1497']
const signalCatalog = [
  { id: '1491', assetId: 'SIG-1491', twinId: 'TWIN-SIG-1491', gps: '109.3881, 24.3071', voltageBase: 221, currentBase: 2.3, tempBase: 31.5 },
  { id: '1493', assetId: 'SIG-1493', twinId: 'TWIN-SIG-1493', gps: '109.3884, 24.3074', voltageBase: 220, currentBase: 2.2, tempBase: 32.2 },
  { id: '1495', assetId: 'SIG-1495', twinId: 'TWIN-SIG-1495', gps: '109.3887, 24.3076', voltageBase: 220, currentBase: 2.4, tempBase: 35.0 },
  { id: '1497', assetId: 'SIG-1497', twinId: 'TWIN-SIG-1497', gps: '109.3890, 24.3079', voltageBase: 219, currentBase: 2.1, tempBase: 31.8 }
]

const selectedSignalId = ref('1495')
const selectedSignal = computed(() => signalCatalog.find((item) => item.id === selectedSignalId.value) || signalCatalog[2])
const heroDeviceName = computed(() => `${selectedSignal.value.id}信号机`)
const HUMIDITY_WARNING = 45
const HUMIDITY_ALARM = 75
const uiNowMs = ref(Date.now())
const telemetryConnected = ref(false)
const signalHardwareReady = ref(false)
const signalFaultActive = ref(false)
const trackFaultActive = ref(false)
const observedTrackCapacitance = ref(18.6)
const deviceModal = ref({
  visible: false,
  type: 'signal',
  signalId: '1495'
})
const latestVoiceText = ref('')

// 信号灯实时参数
const signalParams = ref({
  temperature: 35,
  humidity: 23,
  light: 850,
  voltage: 220,
  current: 2.5,
  signal: -45
})

const humidityClass = computed(() => {

```

```

const h = Number(signalParams.value.humidity)
if (!Number.isFinite(h)) return ''
if (h >= HUMIDITY_ALARM) return 'danger'
if (h >= HUMIDITY_WARNING) return 'warn'
return 'ok'
})

// 信号孪生体参数（行业常用：资产/互锁/同步/健康/寿命预测等）
const signalTwin = ref({
  twinId: selectedSignal.value.twinId,
  assetId: selectedSignal.value.assetId,
  deviceType: `${selectedSignal.value.id}信号机`,
  interlocking: '正常',
  workMode: '自动',
  syncStatus: '已同步',
  syncLatencyMs: 120,
  dataFreshnessS: 2,
  healthScore: 92,
  predictedRulDays: 180,
  firmware: 'FW-X.Y',
  lastMaintenance: '2026-XX-XX'
})

const telemetryDistanceM = ref(null)
const telemetryWaterActive = ref(null)
const pedestrianDesiredVisible = ref(false)
const pedestrianHotFlash = ref(false)

const telemetrySimEnabled = ref(false)
const telemetrySimLastAt = ref(0)
let telemetrySimTimer = null
let telemetrySimPassTimer = null
let pedestrianHotFlashTimer = null
let pedestrianLoopEnabled = false

const telemetryActive = computed(() => telemetrySimEnabled.value || telemetryConnected.value)

const twinEnvVisible = ref(false)

const telemetryStatusText = computed(() => {
  if (telemetryConnected.value) return '已接入'
  if (telemetrySimEnabled.value) return '手动模拟'
  return '待接入'
})

const telemetryStatusClass = computed(() => {
  if (telemetryConnected.value) return 'ok'
  if (telemetrySimEnabled.value) return 'ok'
  return 'warn'
})

const telemetryLastSeenText = computed(() => {
  const at = telemetrySimLastAt.value
  if (!telemetryActive.value || !at) return '--'
  const deltaS = Math.max(0, Math.round((uiNowMs.value - at) / 1000))
  return `${deltaS}s`
})

const telemetryWaterText = computed(() => {
  if (!telemetryActive.value) return '--'
  if (telemetryWaterActive.value === 1) return '告警'
  if (telemetryWaterActive.value === 0) return '正常'
  return '--'
})

```

```
const telemetryWaterClass = computed(() => {
  if (!telemetryActive.value) return ''
  if (telemetryWaterActive.value === 1) return 'danger'
  if (telemetryWaterActive.value === 0) return 'ok'
  return ''
})

const installStatusText = computed(() => (signalHardwareReady.value ? '已安装' : '待安装'))
const installStatusClass = computed(() => (signalHardwareReady.value ? 'ok' : 'warn'))

const activeSignalState = computed(() => {
  const target = signals.find((item) => item.id === selectedSignalId.value)
  return target?.state || 'red'
})

const alertFeed = computed(() => {
  const items = []
  if (signalFaultActive.value) {
    items.push({
      id: 'signal',
      level: 'danger',
      title: '1495信号机异常灭灯',
      description: '检测到信号机内部温度异常, 漏电严重, 电压值已降为 0V。'
    })
  }
  if (trackFaultActive.value) {
    items.push({
      id: 'track',
      level: 'danger',
      title: '1495调谐单元异常',
      description: 'FS 调谐单元内部电容受潮, 谐振频率偏离设计值, 轨道电压降为 0V。'
    })
  }
  if (!items.length) {
    items.push({
      id: 'normal',
      level: 'ok',
      title: '系统运行正常',
      description: '当前未检测到信号机或轨道电路异常。'
    })
  }
  return items
})

const deviceStatusRows = computed(() => ([
  { label: '主控板', value: '正常', className: 'online' },
  { label: '通信模块', value: telemetryConnected.value ? '在线' : '离线', className: telemetryConnected.value ? 'online' : 'warning' },
  { label: '散热系统', value: signalFaultActive.value ? '异常' : '正常', className: signalFaultActive.value ? 'warning' : 'online' },
  { label: '电源模块', value: signalHardwareReady.value ? '已安装' : '待安装', className: signalHardwareReady.value ? 'online' : 'warning' }
]))

const panelTimeText = computed(() => new Date(uiNowMs.value).toLocaleString('zh-CN'))

const modalSignalId = computed(() => deviceModal.value.signalId || selectedSignalId.value)
const modalSignalName = computed(() => `${modalSignalId.value}信号机`)
const modalTypeText = computed(() => {
  const typeMap = {
    power: '电源屏',
    signal: `${modalSignalId.value}信号机监测`,
    turnout: '道岔监测',
    track: '1495轨道电路监测'
  }
  return typeMap[deviceModal.value.type] || '设备监测'
})
```

```

const modalCurrentMetric = computed(() => {
  switch (deviceModal.value.type) {
    case 'power':
      return telemetryConnected.value ? 380 : 0
    case 'turnout':
      return signalHardwareReady.value ? 110 : 0
    case 'track':
      return trackFaultActive.value ? 0 : (signalHardwareReady.value ? 72 : 0)
    case 'signal':
    default:
      return signalFaultActive.value ? 0 : (signalHardwareReady.value ? signalParams.value.voltage : 0)
  }
})

const modalMetricUnit = computed(() => (deviceModal.value.type === 'track' ? 'V / uF' : 'V'))

const modalAlertText = computed(() => {
  if (deviceModal.value.type === 'signal' && signalFaultActive.value) {
    return '1495信号机异常灭灯，左侧已同步推送告警。'
  }
  if (deviceModal.value.type === 'track' && trackFaultActive.value) {
    return '1495调谐单元故障持续中，请按要求携带备件前往 K149+525 处处理。'
  }
  if (deviceModal.value.type === 'power' && !telemetryConnected.value) {
    return '当前电源屏尚未接入，可按 K 键切换为已接入状态。'
  }
  return '当前设备运行正常，曲线按系统时间持续刷新。'
})

const modalChartColor = computed(() => {
  if ((deviceModal.value.type === 'signal' && signalFaultActive.value) || (deviceModal.value.type === 'track' && trackFaultActive.value)) {
    return '#ff5c5c'
  }
  return '#00d4ff'
})

const modalSeriesData = computed(() => {
  const total = 24
  const now = uiNowMs.value
  const points = []
  for (let index = 0; index < total; index += 1) {
    const ratio = index / Math.max(1, total - 1)
    const wave = Math.sin((now / 1000) * 0.9 + index * 0.55)
    const ripple = Math.cos((now / 1400) + index * 0.35)
    let value = 0

    if (deviceModal.value.type === 'power') {
      value = telemetryConnected.value ? 370 + wave * 8 + ripple * 3 : 0
    } else if (deviceModal.value.type === 'turnout') {
      value = signalHardwareReady.value ? 108 + wave * 5 + ripple * 2 : 0
    } else if (deviceModal.value.type === 'track') {
      value = trackFaultActive.value ? 0 : (signalHardwareReady.value ? 71 + wave * 2.8 + ripple : 0)
    } else {
      value = signalFaultActive.value ? 0 : (signalHardwareReady.value ? selectedSignal.value.voltageBase + wave * 4 + ripple * 1.5 : 0)
    }

    points.push({
      time: new Date(now - (total - 1 - index) * 60 * 1000).toLocaleTimeString('zh-CN', { hour: '2-digit', minute: '2-digit' }),
      value: Math.round(value * 10) / 10,
      ratio
    })
  }
  return points
})

```

```

    })

const modalChartPoints = computed(() => {
  const data = modalSeriesData.value
  if (!data.length) return []
  const values = data.map((item) => item.value)
  const max = Math.max(...values, 1)
  const min = Math.min(...values, 0)
  const range = Math.max(1, max - min)

  return data.map((item, index) => {
    const x = 16 + index * ((320 - 32) / Math.max(1, data.length - 1))
    const normalized = (item.value - min) / range
    const y = 118 - normalized * 88
    return { x, y, value: item.value, time: item.time }
  })
})

const modallinePoints = computed(() => modalChartPoints.value.map((point) => `${point.x},${point.y}`).join(' '))
const modalAreaPoints = computed(() => {
  const points = modalChartPoints.value
  if (!points.length) return ''
  const bottom = 128
  return `${points[0].x},${bottom} ${points.map((point) => `${point.x},${point.y}`).join(' ')} ${points[points.length - 1].x},${bottom}`
})
const modalPreviewStats = computed(() => {
  if (deviceModal.value.type === 'track') {
    return [
      { label: '轨道电压', value: `${trackFaultActive.value ? 0 : (signalHardwareReady.value ? 72 : 0)}V` },
      { label: '调谐电容', value: `${observedTrackCapacitance.value.toFixed(1)}uF` },
      { label: '系统时间', value: new Date(uiNowMs.value).toLocaleTimeString('zh-CN') }
    ]
  }
  return [
    { label: '当前电压', value: `${Math.round(modalCurrentMetric.value * 10) / 10}V` },
    { label: '接入状态', value: telemetryConnected.value ? '已接入' : '未接入' },
    { label: '系统时间', value: new Date(uiNowMs.value).toLocaleTimeString('zh-CN') }
  ]
})

const syncSignalTwin = () => {
  signalTwin.value.twinId = selectedSignal.value.twinId
  signalTwin.value.assetId = selectedSignal.value.assetId
  signalTwin.value.deviceType = `${selectedSignal.value.id}信号机`
  signalTwin.value.firmware = `FW-${selectedSignal.value.id}`
  signalTwin.value.lastMaintenance = '2026-04-20'
}

const openDeviceModal = (type, signalId = selectedSignalId.value) => {
  if (type === 'signal') {
    selectedSignalId.value = signalId
  }
  deviceModal.value = {
    visible: true,
    type,
    signalId
  }
}

const closeDeviceModal = () => {
  deviceModal.value.visible = false
}

const speakAlarm = (text) => {

```

```

latestVoiceText.value = text
if (!('speechSynthesis' in window)) return
try {
  window.speechSynthesis.cancel()
  const utterance = new SpeechSynthesisUtterance(text)
  utterance.lang = 'zh-CN'
  utterance.rate = 1
  utterance.pitch = 1
  window.speechSynthesis.speak(utterance)
} catch (error) {
  console.warn('语音播报失败:', error)
}
}

const syncSignalVisual = (signalId, state) => {
  const target = signals.find((item) => item.id === signalId)
  if (!target) return
  target.state = state
  const color = getColorByState(state)
  if (target.light) {
    target.light.color.setHex(color)
    target.light.intensity = state === 'off' ? 0 : 3
  }
  if (target.mesh) {
    target.mesh.traverse((child) => {
      if (!child.isMesh || !child.material) return
      if (child.material.emissive) {
        child.material.emissive.setHex(color)
        if ('emissiveIntensity' in child.material) {
          child.material.emissiveIntensity = state === 'off' ? 0 : 0.7
        }
      }
      if (child.material.color) {
        child.material.color.setHex(state === 'off' ? 0x555555 : color)
      }
    })
  }
}

const restoreNormalState = () => {
  signalFaultActive.value = false
  trackFaultActive.value = false
  observedTrackCapacitance.value = 18.6
  SIGNAL_IDS.forEach((id) => syncSignalVisual(id, id === '1495' ? 'red' : 'green'))
  updateSignalUI()
  closeDeviceModal()
}

const setPedestrianVisible = (visible) => {
  pedestrianDesiredVisible.value = Boolean(visible)
  if (pedestrianObject) {
    pedestrianObject.visible = pedestrianDesiredVisible.value
  }
}

const triggerPedestrianHotFlash = () => {
  pedestrianHotFlash.value = true
  if (pedestrianHotFlashTimer) {
    clearTimeout(pedestrianHotFlashTimer)
  }
  pedestrianHotFlashTimer = setTimeout(() => {
    pedestrianHotFlash.value = false
    pedestrianHotFlashTimer = null
  }, 1200)
}

```

```

}

const applyTwinEnvVisibilityToLabels = () => {
  if (!modellabels?.length) return
  for (const entry of modellabels) {
    if (!entry?.div) continue
    entry.div.classList.toggle('env-visible', twinEnvVisible.value)
    entry.div.classList.toggle('env-hidden', !twinEnvVisible.value)
  }
}

const handleSignalAction = (signalId) => {
  selectedSignalId.value = signalId
  openDeviceModal('signal', signalId)
}

const onGlobalKeydown = (event) => {
  if (!event) return
  if (event.repeat) return
  const key = event.key
  const tag = String(event.target?.tagName || '').toLowerCase()
  if (tag === 'input' || tag === 'textarea' || tag === 'select') return
  if (key === 'k' || key === 'K') {
    telemetryConnected.value = true
    telemetrySimLastAt.value = Date.now()
    telemetryWaterActive.value = 0
    return
  }
  if (key === 'm' || key === 'M') {
    telemetryConnected.value = true
    signalHardwareReady.value = true
    signalFaultActive.value = false
    observedTrackCapacitance.value = trackFaultActive.value ? 8.3 : 18.6
    syncSignalVisual('1495', 'red')
    openDeviceModal('signal', '1495')
    return
  }
  if (key === 'p' || key === 'P') {
    telemetryConnected.value = true
    signalHardwareReady.value = true
    signalFaultActive.value = true
    selectedSignalId.value = '1495'
    syncSignalVisual('1495', 'off')
    openDeviceModal('signal', '1495')
    speakAlarm('检测到1495信号机内部温度异常，漏电严重，建议携带工具箱、密封圈及LED信号机点灯单元前往处理。')
    return
  }
  if (key === 'l' || key === 'L') {
    telemetryConnected.value = true
    signalHardwareReady.value = true
    trackFaultActive.value = true
    observedTrackCapacitance.value = 8.3
    openDeviceModal('track', '1495')
    speakAlarm('监测到1495调谐单元内部电容受潮容值改变，现谐振频率偏离设计值，请携带调谐单元备件及工具箱前往K149加S25处进行更换作业。')
    return
  }
  if (key === 'i' || key === 'I') {
    twinEnvVisible.value = !twinEnvVisible.value
    applyTwinEnvVisibilityToLabels()
    return
  }
  if (key === 'o' || key === 'O') {
    telemetrySimEnabled.value = !telemetrySimEnabled.value
    pedestrianLoopEnabled = telemetrySimEnabled.value
  }
}

```

```

    if (pedestrianLoopEnabled) {
      playPedestrianNotice()
      triggerPedestrianHotFlash()
      startFrontendTelemetrySim()
      return
    }
    stopFrontendTelemetrySim()
  }
}

const updateTelemetrySim = (distanceM) => {
  telemetryDistanceM.value = distanceM
  telemetryWaterActive.value = 0
  telemetrySimLastAt.value = Date.now()
  setPedestrianVisible(Boolean(Number.isFinite(distanceM) && distanceM > 0 && distanceM < 2))
}

const stopFrontendTelemetrySim = () => {
  pedestrianLoopEnabled = false
  pedestrianHotFlash.value = false
  if (pedestrianHotFlashTimer) {
    clearTimeout(pedestrianHotFlashTimer)
    pedestrianHotFlashTimer = null
  }
  if (telemetrySimTimer) {
    clearTimeout(telemetrySimTimer)
    telemetrySimTimer = null
  }
  if (telemetrySimPassTimer) {
    clearInterval(telemetrySimPassTimer)
    telemetrySimPassTimer = null
  }
  updateTelemetrySim(0)
}

const startFrontendTelemetrySim = () => {
  if (!pedestrianLoopEnabled) return

  const PASS_MIN_MS = 3000
  const PASS_MAX_MS = 5000
  const IDLE_MIN_MS = 6000
  const IDLE_MAX_MS = 18000

  const scheduleNextPass = (delayMs) => {
    if (!pedestrianLoopEnabled) return
    if (telemetrySimTimer) clearTimeout(telemetrySimTimer)
    telemetrySimTimer = setTimeout(runPass, Math.max(0, delayMs))
  }

  const runPass = () => {
    if (!pedestrianLoopEnabled) return
    if (telemetrySimPassTimer) {
      clearInterval(telemetrySimPassTimer)
      telemetrySimPassTimer = null
    }

    const passMs = PASS_MIN_MS + Math.floor(Math.random() * (PASS_MAX_MS - PASS_MIN_MS + 1))
    const startAt = Date.now()

    const jitterDistance = () => {
      const base = 0.25 + Math.random() * 1.35
      const meters = Math.round(base * 100) / 100
      updateTelemetrySim(meters)
    }
  }
}

```

```

jitterDistance()
telemetrySimPassTimer = setInterval(() => {
  if (!pedestrianLoopEnabled) {
    stopFrontendTelemetrySim()
    return
  }

  const elapsed = Date.now() - startAt
  if (elapsed >= passMs) {
    clearInterval(telemetrySimPassTimer)
    telemetrySimPassTimer = null
    updateTelemetrySim(0)
    const idleMs = IDLE_MIN_MS + Math.floor(Math.random() * (IDLE_MAX_MS - IDLE_MIN_MS + 1))
    scheduleNextPass(idleMs)
    return
  }
  jitterDistance()
}, 600)
}

updateTelemetrySim(0)
runPass()
}

// ===== 参数曲线图相关 =====
const selectedParam = ref('temperature')
const paramTimeRange = ref('24h')
const hoveredParamPoint = ref(null)

// 参数配置
const paramConfigs = {
  temperature: { icon: '🌡️', label: '温度', unit: '°C', color: '#ff6b6b', min: 0, max: 60 },
  humidity: { icon: '💧', label: '湿度', unit: '%', color: '#4ecdc4', min: 0, max: 100 },
  light: { icon: '☀️', label: '光照强度', unit: 'Lux', color: '#ffe66d', min: 0, max: 2000 },
  voltage: { icon: '⚡️', label: '电压', unit: 'V', color: '#a8e6cf', min: 180, max: 250 },
  current: { icon: '🔌', label: '电流', unit: 'A', color: '#dda0dd', min: 0, max: 5 },
  signal: { icon: '📶', label: '信号强度', unit: 'dBm', color: '#87ceeb', min: -100, max: 0 }
}

const paramConfig = computed(() => paramConfigs[selectedParam.value] || paramConfigs.temperature)
const currentParamValue = computed(() => signalParams.value[selectedParam.value])

// 参数历史数据 - 1小时（每5分钟一个点，12个点）
const paramData1h = ref({
  temperature: [32, 33, 34, 35, 34, 33, 35, 36, 35, 34, 35, 35],
  humidity: [22, 22, 23, 23, 24, 24, 23, 23, 22, 22, 23, 23],
  light: [800, 850, 900, 880, 860, 870, 850, 840, 850, 860, 850, 850],
  voltage: [218, 220, 222, 219, 221, 220, 218, 220, 221, 220, 219, 220],
  current: [2.3, 2.4, 2.5, 2.4, 2.5, 2.6, 2.5, 2.4, 2.5, 2.5, 2.4, 2.5],
  signal: [-48, -46, -45, -47, -45, -44, -46, -45, -47, -45, -46, -45]
})

// 参数历史数据 - 24小时（每小时一个点，24个点）
const paramData24h = ref({
  temperature: [28, 27, 26, 25, 24, 24, 25, 27, 30, 33, 36, 38, 40, 41, 40, 39, 37, 35, 33, 31, 30, 29, 28, 27],
  humidity: [21, 21, 20, 20, 20, 21, 21, 22, 22, 23, 23, 24, 24, 24, 23, 23, 22, 22, 22, 21, 21, 21, 20, 20],
  light: [0, 0, 0, 0, 0, 50, 200, 500, 800, 1100, 1400, 1600, 1800, 1700, 1500, 1200, 900, 600, 300, 100, 20, 0, 0, 0],
  voltage: [215, 216, 218, 218, 219, 220, 220, 221, 222, 220, 219, 218, 217, 218, 219, 220, 221, 220, 219, 218, 217, 216, 215, 215],
  current: [2.0, 1.9, 1.8, 1.8, 1.9, 2.0, 2.2, 2.4, 2.6, 2.8, 3.0, 3.1, 3.2, 3.1, 3.0, 2.8, 2.6, 2.4, 2.2, 2.1, 2.0, 2.0, 1.9, 1.9],
  signal: [-55, -52, -50, -48, -46, -45, -44, -43, -42, -43, -44, -45, -46, -45, -44, -43, -44, -45, -47, -48, -50, -52, -53, -54]
})

// 参数历史数据 - 一周（每天一个点，7个点）

```

```

const paramDataWeek = ref({
  temperature: [32, 35, 38, 36, 34, 33, 35],
  humidity: [22, 23, 22, 24, 23, 22, 23],
  light: [850, 900, 950, 880, 820, 800, 850],
  voltage: [220, 218, 222, 219, 221, 220, 220],
  current: [2.5, 2.6, 2.7, 2.5, 2.4, 2.5, 2.5],
  signal: [-45, -48, -42, -44, -46, -47, -45]
})

// 时间标签
const paramTimeLabels1h = ['00:00', '00:05', '00:10', '00:15', '00:20', '00:25', '00:30', '00:35', '00:40', '00:45', '00:50', '00:55']
const paramTimeLabels24h = Array.from({ length: 24 }, (_, i) => `${String(i).padStart(2, '0')}:00`)
const paramTimeLabelsWeek = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']

const currentParamData = computed(() => {
  const dataMap = {
    '1h': paramData1h.value,
    '24h': paramData24h.value,
    'week': paramDataWeek.value
  }
  return dataMap[paramTimeRange.value]?.[selectedParam.value] || []
})

const paramTimeLabels = computed(() => {
  const labelsMap = {
    '1h': paramTimeLabels1h,
    '24h': paramTimeLabels24h,
    'week': paramTimeLabelsWeek
  }
  return labelsMap[paramTimeRange.value] || []
})

const paramChartPoints = computed(() => {
  const data = currentParamData.value
  if (!data || data.length === 0) return []

  const count = data.length
  const width = 260
  const height = 70
  const padding = 10
  const config = paramConfig.value

  return data.map((v, i) => ({
    x: padding + (i / (count - 1)) * width,
    y: padding + 10 + height - ((v - config.min) / (config.max - config.min)) * height
  })))
})

const paramLinePoints = computed(() => {
  return paramChartPoints.value.map(p => `${p.x},${p.y}`).join(' ')
})

const paramAreaPoints = computed(() => {
  const points = paramChartPoints.value
  if (points.length === 0) return ''
  const bottom = 90
  const linePoints = points.map(p => `${p.x},${p.y}`).join(' ')
  const firstX = points[0].x
  const lastX = points[points.length - 1].x
  return `${firstX},${bottom} ${linePoints} ${lastX},${bottom}`
})

const onParamChange = () => {
  hoveredParamPoint.value = null

```

```

}

const onParamTimeChange = () => {
  hoveredParamPoint.value = null
}

// ===== 视频监控相关 =====
const STREAM_URL = (() => {
  const v = import.meta?.env?.VITE_STREAM_URL
  return typeof v === 'string' ? v.trim() : ''
})();

const retryTimers = new Map()
const logVideo = (index, action, detail = '') => {
  const prefix = `[video-${index + 1}] ${action}`
  console.log(detail ? `${prefix}: ${detail}` : prefix)
}

const videoList = ref([
  {
    title: '监控点 1 - 区间A',
    streamPath: STREAM_URL,
    src: '',
    loaded: false,
    status: STREAM_URL ? '待播放' : '未配置'
  },
  {
    title: '监控点 2 - 区间B',
    streamPath: STREAM_URL,
    src: '',
    loaded: false,
    status: STREAM_URL ? '待播放' : '未配置'
  },
  {
    title: '监控点 3 - 区间C',
    streamPath: STREAM_URL,
    src: '',
    loaded: false,
    status: STREAM_URL ? '待播放' : '未配置'
  }
])

const buildStreamUrl = (streamPath) => {
  if (!streamPath) return ''
  const sep = streamPath.includes('?') ? '&' : '?'
  return `${streamPath}${sep}_t=${Date.now()}`
}

const loadVideo = (index) => {
  const video = videoList.value[index]
  if (!video?.streamPath) {
    video.status = '未配置'
    logVideo(index, 'skip', 'VITE_STREAM_URL not set')
    return
  }
  if (!video.loaded) {
    video.src = buildStreamUrl(video.streamPath)
    video.loaded = true
    video.status = '连接中...'
    logVideo(index, 'load', video.src)
  }
}

const onVideoLoad = (index) => {
  const video = videoList.value[index]

```

```

    if (!video) return
    video.status = '已连接'
    logVideo(index, 'connected')
  }

const reloadVideo = (index) => {
  const video = videoList.value[index]
  if (!video?.loaded) {
    return
  }

  video.status = '重连中...'
  logVideo(index, 'error', 'stream load failed, retry in 1.2s')
  clearTimeout(retryTimers.get(index))
  const timer = setTimeout(() => {
    video.src = buildStreamUrl(video.streamPath)
    logVideo(index, 'retry', video.src)
  }, 1200)
  retryTimers.set(index, timer)
}

// 计算属性
const tempPercent = computed(() => Math.min(signalParams.value.temperature / 60 * 100, 100))
const lightPercent = computed(() => Math.min(signalParams.value.light / 2000 * 100, 100))
const voltagePercent = computed(() => Math.min(signalParams.value.voltage / 250 * 100, 100))
const currentPercent = computed(() => Math.min(signalParams.value.current / 5 * 100, 100))
const signalPercent = computed(() => Math.min((100 + signalParams.value.signal) / 100 * 100, 100))

const tempClass = computed(() => {
  if (signalParams.value.temperature < 30) return 'temp-low'
  if (signalParams.value.temperature < 50) return 'temp-medium'
  return 'temp-high'
})

const voltageClass = computed(() => {
  if (signalParams.value.voltage >= 210 && signalParams.value.voltage <= 230) return 'voltage-normal'
  return 'voltage-warning'
})

let scene, camera, renderer, controls, labelRenderer
let signals = []
let train = null
let isTrainRunning = false
let autoRotate = false
let rafId = 0
let animationStopped = false
const speedOptions = [
  { label: '慢速', value: 0.08 },
  { label: '标准', value: 0.12 },
  { label: '快速', value: 0.18 },
  { label: '极速', value: 0.25 }
]

// 默认速度稍微慢一点
const trainSpeed = ref(0.08)
let clock = new THREE.Clock()
let startTime = Date.now()
let mixer = null
let gltfLoader = null
const gltfAssetCache = new Map()
let modellabels = []
let trainLabelEntry = null
let signalLabelEntry = null
let updateUiTimer = null
let updateSignalParamsTimer = null

```

```

let signalObject = null
let pedestrianObject = null
let workerObject = null
let boxObject = null
let boxObjectSize = null
let boxSensorLabelEntry = null
let boxSensorLight = null
let boxSensorWave = null
let boxSensorLabelAnchor = null
let autoDemoStarted = false
let autoDemoWaitTimer = null
let autoDemoFlyTween = null
let skyParticles = null
let skyParticlesMaterial = null
let endpointGlowTexture = null
let endpointGlows = []
let pedestrianNoticeAudio = null

const playPedestrianNotice = async () => {
  try {
    if (!pedestrianNoticeAudio) {
      pedestrianNoticeAudio = new Audio('/assets/audio/pedestrian_notice.wav')
      pedestrianNoticeAudio.preload = 'auto'
      pedestrianNoticeAudio.volume = 0.85
    }
    pedestrianNoticeAudio.currentTime = 0
    await pedestrianNoticeAudio.play()
  } catch (e) {
    console.warn('行人语音播放失败（可能被浏览器拦截）:', e)
  }
}

const ensureGltfLoader = () => {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }
  return gltfLoader
}

const getCachedModelClone = async (url) => {
  const cachedAsset = gltfAssetCache.get(url)
  if (cachedAsset) {
    const gltf = await cachedAsset
    return {
      scene: cloneSkinned(gltf.scene),
      animations: gltf.animations || []
    }
  }
}

const loader = ensureGltfLoader()
const assetPromise = new Promise((resolve, reject) => {
  loader.load(
    url,
    (gltf) => resolve(gltf),
    undefined,
    (error) => reject(error)
  )
})

gltfAssetCache.set(url, assetPromise)
const gltf = await assetPromise.catch((error) => {
  gltfAssetCache.delete(url)
  throw error
})

```

```

return {
  scene: cloneSkinned(gltf.scene),
  animations: gltf.animations || []
}
}

// 人形感应器面板数据
const humanoidSensor = ref({
  power: '正常',
  voltage: '5V',
  gps: '--, --',
  pedestrianDistance: '--'
})
const boxSensor = ref({
  power: '正常',
  battery: '86%',
  gps: '--, --',
  pedestrianDistance: '--'
})
let trainProgress = 0
const trainPath = new THREE.CatmullRomCurve3([
  new THREE.Vector3(-168, 3, -168),
  new THREE.Vector3(-100, 3, -100),
  new THREE.Vector3(-84, 3, -84),
  new THREE.Vector3(-50, 3, -50),
  new THREE.Vector3(0, 3, 0),
  new THREE.Vector3(50, 3, 50),
  new THREE.Vector3(100, 3, 100),
  new THREE.Vector3(168, 3, 168),
])

const createRadialGlowTexture = () => {
  const size = 192
  const canvas = document.createElement('canvas')
  canvas.width = size
  canvas.height = size
  const ctx = canvas.getContext('2d')
  if (!ctx) return null

  const cx = size / 2
  const cy = size / 2
  const radius = size / 2

  const g = ctx.createRadialGradient(cx, cy, 0, cx, cy, radius)
  g.addColorStop(0.0, 'rgba(255,255,255,0.00)')
  g.addColorStop(0.28, 'rgba(120,220,255,0.00)')
  g.addColorStop(0.45, 'rgba(120,220,255,0.18)')
  g.addColorStop(0.62, 'rgba(120,220,255,0.55)')
  g.addColorStop(0.78, 'rgba(120,220,255,0.16)')
  g.addColorStop(1.0, 'rgba(120,220,255,0.00)')

  ctx.fillStyle = g
  ctx.fillRect(0, 0, size, size)

  const texture = new THREE.CanvasTexture(canvas)
  texture.colorSpace = THREE.SRGBColorSpace
  texture.needsUpdate = true
  return texture
}

const createSkyParticles = () => {
  const count = 1400
  const width = 520
  const depth = 520

```

```

const minY = 70
const maxY = 200

const positions = new Float32Array(count * 3)
for (let i = 0; i < count; i++) {
  const idx = i * 3
  positions[idx] = (Math.random() - 0.5) * width
  positions[idx + 1] = minY + Math.random() * (maxY - minY)
  positions[idx + 2] = (Math.random() - 0.5) * depth
}

const geometry = new THREE.BufferGeometry()
geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3))

skyParticlesMaterial = new THREE.PointsMaterial({
  color: 0x9fe6ff,
  size: 1.35,
  sizeAttenuation: true,
  transparent: true,
  opacity: 0.22,
  depthWrite: false,
  blending: THREE.AdditiveBlending
})

skyParticles = new THREE.Points(geometry, skyParticlesMaterial)
skyParticles.frustumCulled = false
scene.add(skyParticles)
}

const createRailEndpointGlows = () => {
  if (!endpointGlowTexture) {
    endpointGlowTexture = createRadialGlowTexture()
  }
  if (!endpointGlowTexture) return

  const start = trainPath.getPoint(0)
  const end = trainPath.getPoint(1)

  const makeGlow = (position, baseScale, baseOpacity, phase) => {
    const material = new THREE.SpriteMaterial({
      map: endpointGlowTexture,
      color: 0x9fe6ff,
      transparent: true,
      opacity: baseOpacity,
      depthWrite: false,
      depthTest: true,
      blending: THREE.AdditiveBlending
    })
    const sprite = new THREE.Sprite(material)
    sprite.position.set(position.x, 0.18, position.z)
    sprite.scale.set(baseScale, baseScale, 1)
    scene.add(sprite)
    endpointGlows.push({ sprite, baseScale, baseOpacity, phase })
  }

  endpointGlows = []
  makeGlow(start, 22, 0.55, 0.0)
  makeGlow(start, 14, 0.38, 1.2)
  makeGlow(end, 22, 0.55, 2.4)
  makeGlow(end, 14, 0.38, 3.6)
}

const getClosestPathProgress = (position, samples = 300) => {
  let bestProgress = 0

```

```

let minDistance = Infinity

for (let index = 0; index <= samples; index++) {
  const progress = index / samples
  const point = trainPath.getPoint(progress)
  const distance = point.distanceTo(position)
  if (distance < minDistance) {
    minDistance = distance
    bestProgress = progress
  }
}

return bestProgress
}

const getColorByState = (state) => {
  switch (state) {
    case 'red': return 0xff3333
    case 'green': return 0x33ff33
    case 'yellow': return 0xffff33
    case 'off': return 0x333333
    default: return 0x666666
  }
}

const init = () => {
  animationStopped = false
  if (rafId) {
    cancelAnimationFrame(rafId)
    rafId = 0
  }

  scene = new THREE.Scene()
  scene.background = new THREE.Color(0xf0f0f0)
  scene.fog = new THREE.Fog(0xf0f0f0, 100, 800)

  camera = new THREE.PerspectiveCamera(
    60,
    window.innerWidth / window.innerHeight,
    0.1,
    2000
  )
  camera.position.set(60, 60, 80)

  renderer = new THREE.WebGLRenderer({ antialias: true })
  renderer.setSize(window.innerWidth, window.innerHeight)
  renderer.setPixelRatio(window.devicePixelRatio)
  renderer.shadowMap.enabled = true
  renderer.shadowMap.type = THREE.PCFSoftShadowMap
  const containerEl = document.getElementById('threeContainer')
  containerEl.appendChild(renderer.domElement)

  labelRenderer = new CSS2DRenderer()
  labelRenderer.setSize(window.innerWidth, window.innerHeight)
  labelRenderer.domElement.style.position = 'absolute'
  labelRenderer.domElement.style.top = '0'
  labelRenderer.domElement.style.left = '0'
  labelRenderer.domElement.style.width = '100%'
  labelRenderer.domElement.style.height = '100%'
  labelRenderer.domElement.style.pointerEvents = 'none'
  containerEl.appendChild(labelRenderer.domElement)

  controls = new OrbitControls(camera, renderer.domElement)
  controls.enableDamping = true

```

```

controls.dampingFactor = 0.05
controls.maxPolarAngle = Math.PI / 2
controls.minDistance = 20
controls.maxDistance = 200

setupLights()
createAxisHelper()
createGround()
createMountains()
createRailway()
createSkyParticles()
createRailEndpointGlows()
createSignalLights()
createTrain()
createTrees()

window.addEventListener('resize', onWindowResize)

animate()
updateUI()
updateUiTimer = setInterval(updateUI, 1000)

console.log('Three.js 小地图初始化完成')
}

const setupLights = () => {
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.8)
  scene.add(ambientLight)

  const mainLight = new THREE.DirectionalLight(0xffffff, 1.5)
  mainLight.position.set(50, 100, 50)
  mainLight.castShadow = true
  mainLight.shadow.camera.left = -100
  mainLight.shadow.camera.right = 100
  mainLight.shadow.camera.top = 100
  mainLight.shadow.camera.bottom = -100
  mainLight.shadow.mapSize.width = 2048
  mainLight.shadow.mapSize.height = 2048
  scene.add(mainLight)

  const fillLight = new THREE.DirectionalLight(0xffeedd, 0.5)
  fillLight.position.set(-50, 50, -50)
  scene.add(fillLight)

  const hemiLight = new THREE.HemisphereLight(0xffffff, 0x444444, 0.6)
  hemiLight.position.set(0, 200, 0)
  scene.add(hemiLight)
}

const createAxisHelper = () => {
  const axisGroup = new THREE.Group()
  const axisLength = 16.5
  const arrowHeadLength = 2.7
  const arrowHeadRadius = 1
  const lineRadius = 0.35

  const xAxisGroup = new THREE.Group()
  const xLineGeom = new THREE.CylinderGeometry(lineRadius, lineRadius, axisLength, 8)
  const xLineMat = new THREE.MeshBasicMaterial({ color: 0xff4444, transparent: true, opacity: 0.45 })
  const xLine = new THREE.Mesh(xLineGeom, xLineMat)
  xLine.rotation.z = -Math.PI / 2
  xLine.position.x = axisLength / 2
  xAxisGroup.add(xLine)
  const xArrowGeom = new THREE.ConeGeometry(arrowHeadRadius, arrowHeadLength, 8)

```

```

const xArrowMat = new THREE.MeshBasicMaterial({ color: 0xff4444, transparent: true, opacity: 0.45 })
const xArrow = new THREE.Mesh(xArrowGeom, xArrowMat)
xArrow.rotation.z = -Math.PI / 2
xArrow.position.x = axisLength + arrowHeadLength / 2
xAxisGroup.add(xArrow)
axisGroup.add(xAxisGroup)

const yAxisGroup = new THREE.Group()
const yLineGeom = new THREE.CylinderGeometry(lineRadius, lineRadius, axisLength, 8)
const yLineMat = new THREE.MeshBasicMaterial({ color: 0x44ff44, transparent: true, opacity: 0.45 })
const yLine = new THREE.Mesh(yLineGeom, yLineMat)
yLine.position.y = axisLength / 2
yAxisGroup.add(yLine)
const yArrowGeom = new THREE.ConeGeometry(arrowHeadRadius, arrowHeadLength, 8)
const yArrowMat = new THREE.MeshBasicMaterial({ color: 0x44ff44, transparent: true, opacity: 0.45 })
const yArrow = new THREE.Mesh(yArrowGeom, yArrowMat)
yArrow.position.y = axisLength + arrowHeadLength / 2
yAxisGroup.add(yArrow)
axisGroup.add(yAxisGroup)

// Z 轴
const zAxisGroup = new THREE.Group()
const zLineGeom = new THREE.CylinderGeometry(lineRadius, lineRadius, axisLength, 8)
const zLineMat = new THREE.MeshBasicMaterial({ color: 0x4488ff, transparent: true, opacity: 0.45 })
const zLine = new THREE.Mesh(zLineGeom, zLineMat)
zLine.rotation.x = Math.PI / 2
zLine.position.z = axisLength / 2
zAxisGroup.add(zLine)
const zArrowGeom = new THREE.ConeGeometry(arrowHeadRadius, arrowHeadLength, 8)
const zArrowMat = new THREE.MeshBasicMaterial({ color: 0x4488ff, transparent: true, opacity: 0.45 })
const zArrow = new THREE.Mesh(zArrowGeom, zArrowMat)
zArrow.rotation.x = Math.PI / 2
zArrow.position.z = axisLength + arrowHeadLength / 2
zAxisGroup.add(zArrow)
axisGroup.add(zAxisGroup)

const createAxisLabel = (text, color, position) => {
  const div = document.createElement('div')
  div.className = 'axis-label-3d'
  div.textContent = text
  div.style.cssText = `
    color: ${color};
    font-size: 12px;
    font-weight: bold;
    text-shadow: 0 0 4px rgba(0,0,0,0.8);
    pointer-events: none;
  `
  const label = new CSS2DObject(div)
  label.position.set(position.x, position.y, position.z)
  return label
}

axisGroup.add(createAxisLabel('X', '0xff4444', { x: axisLength + 5, y: 0, z: 0 }))
axisGroup.add(createAxisLabel('Y', '0x44ff44', { x: 0, y: axisLength + 5, z: 0 }))
axisGroup.add(createAxisLabel('Z', '0x4488ff', { x: 0, y: 0, z: axisLength + 5 }))
axisGroup.position.set(120, -10, -100)
scene.add(axisGroup)
}

const createGround = () => {
  const groundGeometry = new THREE.PlaneGeometry(500, 500, 50, 50)
  const groundMaterial = new THREE.MeshStandardMaterial({
    // 地面用灰色填充
    color: 0x808080,
  })

```

```

        roughness: 0.9,
        metalness: 0.1
    })

    const vertices = groundGeometry.attributes.position.array
    for (let i = 0; i < vertices.length; i += 3) {
        const x = vertices[i]
        const z = vertices[i + 2]
        vertices[i + 1] = Math.sin(x * 0.05) * Math.cos(z * 0.05) * 5
    }
    groundGeometry.computeVertexNormals()

    const ground = new THREE.Mesh(groundGeometry, groundMaterial)
    ground.rotation.x = -Math.PI / 2
    ground.receiveShadow = true
    scene.add(ground)
}

const createMountains = () => {
    const mountainGeometry = new THREE.ConeGeometry(30, 60, 8)
    const mountainMaterial = new THREE.MeshStandardMaterial({
        color: 0x8a9a8a,
        roughness: 0.95,
        metalness: 0.05
    })

    const positions = [
        { x: -100, z: 50 },
        { x: 80, z: -100 },
        { x: 100, z: 60 },
        { x: -60, z: 120 }
    ]

    positions.forEach(pos => {
        const mountain = new THREE.Mesh(mountainGeometry, mountainMaterial)
        mountain.position.set(pos.x, 30, pos.z)
        mountain.scale.y = 1 + Math.random() * 0.5
        mountain.castShadow = true
        mountain.receiveShadow = true
        scene.add(mountain)
    })
}

const createRailway = () => {
    ensureGltfLoader()

    // 铁轨方向: X轴旋转45度 (Math.PI / 4 = 45度)
    // 沿着X-Z对角线方向放置铁轨, 确保在同一条直线上
    const railAngle = Math.PI / 4 // 45度

    // 铁轨模型缩放12倍后, 假设实际有效长度约为15单位
    // 在45度角方向, X和Z方向的分量相等
    const segmentSpacing = 8 // 在当前基础上再缩小约 1.5 倍

    // 生成铁轨位置, 沿着45度对角线方向 (z = x)
    const railPositions = []
    // 在现有基础上首末各再复制 5 段
    for (let i = -17; i <= 17; i++) {
        railPositions.push({
            x: i * segmentSpacing,
            z: i * segmentSpacing, // 45度角时 z = x
            rotation: railAngle
        })
    }
}

```

```

console.log('铁轨位置 (45度角, 紧密排列):', railPositions)

getCachedModelClone('/assets/models/railway.glb')
  .then(({ scene: template }) => {
    template.traverse((child) => {
      if (child.isMesh) {
        child.castShadow = true
        child.receiveShadow = true
      }
    })

    railPositions.forEach((pos, index) => {
      const railway = cloneSkinned(template)
      railway.scale.set(12, 12, 12)
      railway.position.set(pos.x, 0, pos.z)
      railway.rotation.y = pos.rotation
      scene.add(railway)
      console.log(`铁轨段 ${index + 1} 加载成功, 位置: (${pos.x}, ${pos.z})`)
    })
  })
  .catch((error) => {
    console.error('铁轨模型加载失败:', error)
  })
}

```

```

const createSignallights = () => {
  ensureGltfLoader()

```

```

  const getObjectSize = (object3d) => {
    const box = new THREE.Box3().setFromObject(object3d)
    const size = new THREE.Vector3()
    box.getSize(size)
    return { box, size }
  }

```

```

  const placeOnGround = (object3d) => {
    const box = new THREE.Box3().setFromObject(object3d)
    const yOffset = -box.min.y
    object3d.position.y += yOffset
  }

```

```

  const createBoxSensorLabel = (model) => {
    const div = document.createElement('div')
    div.className = 'model-label'
    // 行人检测信息栏: 稍微缩小一点 (更紧凑)
    div.style.maxWidth = '200px'
    div.style.minWidth = '160px'
    div.style.transform = 'translate(-50%, -100%) scale(0.9)'
    div.innerHTML = `
      <div class="label-title">行人检测传感器</div>
      <div class="label-row">
        <span>🔌 电源:</span>
        <span class="label-value" data-field="power">正常</span>
      </div>
      <div class="label-row">
        <span>🔋 电量:</span>
        <span class="label-value" data-field="battery">86%</span>
      </div>
      <div class="label-row">
        <span>📶 GPS:</span>
        <span class="label-value" data-field="gps">--, --</span>
      </div>
      <div class="label-row">

```

```

    <span>🚶 行人距离:</span>
    <span class="label-value" data-field="dist">--</span>
  </div>
  、

const label = new CSS2DObject(div)
// 不直接挂到 model 上，避免模型旋转导致 label 偏离“正上方”
const anchor = new THREE.Object3D()
anchor.position.copy(model.position)
scene.add(anchor)
anchor.add(label)

return {
  object: anchor,
  label,
  div,
  fields: {
    power: div.querySelector('[data-field="power"]'),
    battery: div.querySelector('[data-field="battery"]'),
    gps: div.querySelector('[data-field="gps"]'),
    dist: div.querySelector('[data-field="dist"]')
  }
}
}

const createSensorWave = () => {
  // 扇形光波（超声波）：最终需要在 X-Z 平面（法线对齐 Y 轴）
  // CircleGeometry 默认在 XY 平面，这里直接烘焙旋转到 XZ，避免后续叠加旋转导致“看起来不垂直”
  const radius = 8
  const angle = Math.PI / 2.6 // 扇形开角
  const geometry = new THREE.CircleGeometry(radius, 48, -angle / 2, angle)
  geometry.rotateX(-Math.PI / 2)
  const material = new THREE.MeshBasicMaterial({
    color: 0xff3333,
    transparent: true,
    opacity: 0.28,
    side: THREE.DoubleSide,
    depthWrite: false
  })

  const mesh = new THREE.Mesh(geometry, material)
  mesh.renderOrder = 5
  return mesh
}

const getWorldYaw = (obj) => {
  if (!obj) return 0
  const q = new THREE.Quaternion()
  obj.getWorldQuaternion(q)
  const e = new THREE.Euler().setFromQuaternion(q, 'YXZ')
  return e.y || 0
}

const loadAndPlaceNear = async ({ url, desiredHeight, position, rotation = {}, afterPlace }) => {
  const { scene: obj } = await getCachedModelClone(url)
  if (rotation.x !== null) obj.rotation.x = rotation.x
  if (rotation.y !== null) obj.rotation.y = rotation.y
  if (rotation.z !== null) obj.rotation.z = rotation.z

  const { size } = getObjectSize(obj)
  const baseHeight = Math.max(0.0001, size.y || 0.0001)
  const scale = desiredHeight / baseHeight
  obj.scale.setScalar(scale)

```

```

obj.position.set(position.x, position.y ?? 0, position.z)
placeOnGround(obj)

if (typeof afterPlace === 'function') {
  try {
    afterPlace(obj, getObjectSize(obj).size)
  } catch (e) {
    console.warn('模型后处理失败:', e)
  }
}

obj.traverse((child) => {
  if (child.isMesh) {
    child.castShadow = true
    child.receiveShadow = true
  }
})

scene.add(obj)
return obj
}

const signalPositions = [
  { x: -96, z: -114 },
  { x: -56, z: -74 },
  { x: -16, z: -34 },
  { x: 24, z: 6 }
]

const signalStates = ['green', 'green', 'red', 'green']
const signalNames = SIGNAL_IDS.map((id) => `${id}信号机`)

signalPositions.forEach((pos, index) => {
  getCacheModelClone('/assets/models/sign.glb')
    .then(({ scene: signGroup }) => {
      // 信号灯在现有基础上缩放为当前的 1/3
      const signScale = (11.2 * 0.8) / 3
      signGroup.scale.set(signScale, signScale, signScale)
      signGroup.position.set(pos.x, 0, pos.z)

      signGroup.traverse((child) => {
        if (child.isMesh) {
          child.castShadow = true
          child.receiveShadow = true
        }
      })

      scene.add(signGroup)
      const isHeroSignal = signalNames[index] === '1495信号机'
      if (isHeroSignal) {
        signalObject = signGroup
      }

      // 在 1495 主信号灯旁边放置 box / 行人 / 工作者
      if (isHeroSignal) {
        try {
          // 信号灯由 Z 向 X 轴旋转 120° (绕 Y 轴), 并朝 X 轴反方向移动两个身位
          signGroup.rotation.y = -Math.PI * 2 / 3

          const { size: signSize } = getObjectSize(signGroup)
          const signStepX = Math.max(2.0, signSize.x || 0)
          signGroup.position.x -= signStepX * 2
          const signHeight = Math.max(1, signSize.y)

```

```

// 主信号灯整体朝 X/Z 夹角的反方向 (-X/-Z) 移动 (3 + 4) 个身位
const signDiagStep = Math.max(2.0, signSize.x || 0, signSize.z || 0)
const signDiagMove = (signDiagStep * 7) / Math.SQRT2
signGroup.position.x -= signDiagMove
signGroup.position.z -= signDiagMove

const base = { x: signGroup.position.x, z: signGroup.position.z }
const dx = Math.max(2.2, signSize.x * 0.9)
const dz = Math.max(1.2, signSize.z * 0.4)

// Box: 信号灯的 1/4 大小 (按高度比例)
loadAndPlaceNear({
  url: '/assets/models/box.glb',
  // box ×0.5
  desiredHeight: signHeight * 0.25 * 0.5,
  position: { x: base.x + dx, z: base.z + dz },
  // 由 X 朝 Y 方向旋转 90° (绕 Z 轴)
  rotation: { y: 0.2, z: Math.PI / 2 },
  // 盒子改为红色
  afterPlace: (obj) => {
    // box 向 Z 轴方向移动两个身位
    const { size } = getObjectSize(obj)
    const stepZ = Math.max(1.0, size.z || 0)
    obj.position.z += stepZ * 2
    boxObject = obj
    boxObjectSize = size

    // box 上方信息栏: 固定在传感器正上方 (使用包围盒高度)
    boxSensorLabelEntry = createBoxSensorLabel(obj)
    const { size: bSize } = getObjectSize(obj)
    boxSensorLabelEntry.label.position.set(0, Math.max(2.5, bSize.y + 1.4), 0)
    boxSensorLabelAnchor = boxSensorLabelEntry.object

    // 扇形超声波: 沿 X-Y 夹角方向 (45°) 发射
    if (boxSensorWave) {
      scene.remove(boxSensorWave)
      boxSensorWave.geometry?.dispose?.()
      if (Array.isArray(boxSensorWave.material)) boxSensorWave.material.forEach(m => m.dispose?.())
      else boxSensorWave.material?.dispose?.()
      boxSensorWave = null
    }
    boxSensorWave = createSensorWave()
    boxSensorWave.position.set(obj.position.x, obj.position.y + 1.6, obj.position.z)
    // 光波应该垂直于 Y 轴: 已烘焙到 X-Z 平面, 这里只需要设置朝向 (绕 Y 轴)
    boxSensorWave.rotation.set(0, getWorldYaw(obj), 0)
    scene.add(boxSensorWave)

    // 行人传感器红色发光 (点光源)
    if (boxSensorLight) {
      scene.remove(boxSensorLight)
      boxSensorLight = null
    }
    boxSensorLight = new THREE.PointLight(0xff2222, 2.4, 40)
    boxSensorLight.position.set(obj.position.x, obj.position.y + 4.2, obj.position.z)
    scene.add(boxSensorLight)

    obj.traverse((child) => {
      if (!child.isMesh || !child.material) return
      const mats = Array.isArray(child.material) ? child.material : [child.material]
      mats.forEach((m) => {
        if (m.color) m.color.setHex(0xff3333)
        if (m.emissive) m.emissive.setHex(0xff0000)
        if ('emissiveIntensity' in m) m.emissiveIntensity = 0.85
        m.needsUpdate = true
      })
    })
  }
})

```

```

    })
  })
}
}).catch((e) => console.warn('Box 模型加载失败:', e))

// 行人/工作者：放在信号灯旁边，大小略小于信号灯（按高度比例）
loadAndPlaceNear({
  url: '/assets/models/man.glb',
  // 人物 ×1.2
  desiredHeight: signHeight * 0.55 * 1.2,
  position: { x: base.x + dx * 0.35, z: base.z - dz * 0.85 },
  rotation: { y: -Math.PI / 6 },
  // 行人沿 X-Z 45° 方向移动 6 个身位，并再沿 X-Y 45° 方向移动 2 个身位
  afterPlace: (obj, objSize) => {
    const step = Math.max(1.2, objSize.x || 0, objSize.z || 0)
    const move = step * 6
    const d = move / Math.SQRT2
    obj.position.x += d
    obj.position.z += d
    const moveXY = step * 2
    const dxy = moveXY / Math.SQRT2
    obj.position.x += dxy
    obj.position.y += dxy
    // 行人朝 X 轴方向移动两个身位
    const stepX = Math.max(1.2, objSize.x || 0)
    obj.position.x += stepX * 2
    pedestrianObject = obj
    pedestrianObject.visible = pedestrianDesiredVisible.value
  }
}).catch((e) => console.warn('行人模型加载失败:', e))

// 用户口述为 word_man.glb，仓库实际文件名为 work_man.glb
loadAndPlaceNear({
  url: '/assets/models/work_man.glb',
  // 人物 ×1.2
  desiredHeight: signHeight * 0.6 * 1.2,
  position: { x: base.x + dx * 0.75, z: base.z - dz * 0.35 },
  rotation: { y: Math.PI / 10 },
  // 检修人员往 Z 轴移动一个身位
  afterPlace: (obj, objSize) => {
    // 检修人员朝 X 轴反方向移动两身位
    const stepX = Math.max(1.2, objSize.x || 0)
    obj.position.x -= stepX * 2
    const stepZ = Math.max(1.2, objSize.z || 0)
    obj.position.z += stepZ * 1

    // 工作者朝 X/Z 夹角的反方向（-X/-Z）移动（3 + 4）个身位
    const diagStep = Math.max(1.2, objSize.x || 0, objSize.z || 0)
    const diagMove = (diagStep * 7) / Math.SQRT2
    obj.position.x -= diagMove
    obj.position.z -= diagMove
    workerObject = obj

    // 让主信号灯（及其面板）跟随工作人员靠近一些（仅在 XZ 平面移动）
    try {
      const vx = workerObject.position.x - signGroup.position.x
      const vz = workerObject.position.z - signGroup.position.z
      const dist = Math.hypot(vx, vz)
      const targetDist = Math.max(2.2, (signSize.x || 0) * 0.9, (signSize.z || 0) * 0.9)
      if (dist > targetDist) {
        const move = dist - targetDist
        const nx = vx / dist
        const nz = vz / dist
        signGroup.position.x += nx * move
      }
    } catch {}
  }
})

```

```

        signGroup.position.z += nz * move
    }
    } catch (e) {
        console.warn('主信号灯跟随工作人员移动失败:', e)
    }
    }
    }).catch((e) => console.warn('工作者模型加载失败:', e))
} catch (e) {
    console.warn('主信号灯旁模型摆放失败:', e)
}
}

const color = getColorByState(signalStates[index])
const light = new THREE.PointLight(color, 3, 80)
// 绑定到信号灯, 保证信号灯移动时灯光也跟随
light.position.set(0, 10, 0)
signGroup.add(light)

signals.push({
    id: signalNames[index].replace('信号机', ''),
    mesh: signGroup,
    light: light,
    state: signalStates[index],
    name: signalNames[index]
})

// 主信号灯信息栏高度在现有基础上再加一倍 (湿度与恶劣天气演练同步)
if (!signalLabelEntry) {
    const t0 = Math.round(Number(signalParams.value.temperature) * 10) / 10
    const h0 = Math.round(Number(signalParams.value.humidity) * 10) / 10
    signalLabelEntry = createModelLabel(signGroup, signalNames[index], t0, h0, '109.3887', '24.3076', 2.4)
}
updateSignalUI()
console.log(`信号灯 ${signalNames[index]} 加载成功`)
})
.catch((error) => {
    console.error(`信号灯 ${signalNames[index]} 加载失败:`, error)
})
})
}

const createTrain = () => {
    ensureGltfLoader()

    getCacheModelClone('/assets/models/locomotive.glb')
    .then(({ scene: trainScene, animations }) => {
        train = trainScene
        train.scale.set(12, 12, 12)
        // 列车方向: 由X轴向Z轴旋转45度, 与铁轨方向一致
        // 铁轨方向是 Math.PI / 4 = 45度
        // 火车头模型默认朝向+X方向, 需要旋转到朝向X-Z对角线方向
        const railAngle = Math.PI / 4 // 45度
        // 火车头往Y轴方向抬高, 与铁轨平齐
        // 铁轨位置: z = x (45度对角线), 从 -168 到 168
        // 火车头放在铁轨上的位置, z 必须等于 x
        train.position.set(-84, 3, -84) // 放在铁轨上, Y轴抬高到3, z=x保证在铁轨线上
        // 火车头由 Z 向 X 方向旋转 90° (在原有对齐铁轨方向的基础上偏转 90°)
        train.rotation.y = -railAngle + Math.PI / 2

        train.traverse((child) => {
            if (child.isMesh) {
                child.castShadow = true
                child.receiveShadow = true
            }
        })
    })
}

```

```

    })

    scene.add(train)

    if (animations.length > 0) {
        mixer = new THREE.AnimationMixer(train)
        const action = mixer.clipAction(animations[0])
        action.play()
    }

    // 火车头信息板高度降低: 与主体模型更贴合 (剩余约 1/5 高度)
    trainLabelEntry = createModelLabel(train, '火车头', 28, 70, '109.3900', '24.3100', 1.2)

    console.log('火车头模型加载成功 - 45度角方向, Y轴抬高')
})
.catch((error) => {
    console.error('火车头模型加载失败:', error)
})
}

const createModelLabel = (model, name, temperature, humidity, gpsLon, gpsLat, labelHeight = 8) => {
    const div = document.createElement('div')
    div.className = 'model-label'
    div.classList.toggle('env-visible', twinEnvVisible.value)
    div.classList.toggle('env-hidden', !twinEnvVisible.value)

    div.innerHTML = `
        <div class="label-title">${name}</div>
        <div class="label-row env-row">
            <span>🌡 温度:</span>
            <span class="label-value" data-field="temperature">${temperature}°C</span>
        </div>
        <div class="label-row env-row">
            <span>💧 湿度:</span>
            <span class="label-value" data-field="humidity">${humidity}%</span>
        </div>
        <div class="label-row">
            <span>📶 GPS:</span>
            <span class="label-value" data-field="gps">${gpsLon}, ${gpsLat}</span>
        </div>
    `

    const label = new CSS2DObject(div)
    // 文本框位置: 使用传入的labelHeight参数
    label.position.set(0, labelHeight, 0)
    model.add(label)
    // 保存标签信息用于跟随相机
    const entry = {
        object: model,
        label,
        div,
        name,
        fields: {
            temperature: div.querySelector('[data-field="temperature"]'),
            humidity: div.querySelector('[data-field="humidity"]'),
            gps: div.querySelector('[data-field="gps"]')
        }
    }
    modelLabels.push(entry)

    // 确保样式被添加到文档中 (因为 CSS2DRenderer 的元素不在 Vue scoped 样式作用域内)
    if (!document.querySelector('#model-label-styles')) {
        const style = document.createElement('style')
        style.id = 'model-label-styles'
    }
}

```

```

style.textContent = `
  .model-label {
    position: absolute;
    background: rgba(0, 20, 40, 0.75) !important;
    border: 1px solid rgba(0, 200, 255, 0.4) !important;
    border-radius: 6px;
    padding: 6px 10px;
    color: #fff;
    font-size: 13px;
    pointer-events: none;
    backdrop-filter: blur(8px);
    box-shadow: 0 3px 10px rgba(0, 150, 200, 0.25);
    max-width: 200px;
    min-width: 150px;
    transform: translate(-50%, -100%); /* 居中并在上方 */
    z-index: 0; /* 确保在模型后面 */
  }
  .model-label.env-hidden .env-row {
    display: none;
  }
  .model-label.env-visible .env-row .label-value {
    color: #e9fbff !important;
    text-shadow:
      0 0 8px rgba(0, 212, 255, 0.75),
      0 0 16px rgba(0, 200, 255, 0.45),
      0 0 28px rgba(0, 160, 255, 0.25);
  }
  .model-label.sensor-warning {
    border-color: rgba(255, 40, 40, 0.95) !important;
    background: linear-gradient(135deg, rgba(180, 20, 20, 0.88), rgba(90, 10, 10, 0.78)) !important;
    box-shadow: 0 0 22px rgba(255, 40, 40, 0.55) !important;
    animation: sensorWarningFlash 0.85s ease-in-out infinite;
  }
  @keyframes sensorWarningFlash {
    0%, 100% { filter: brightness(1); }
    50% { filter: brightness(1.28); }
  }
  .label-title {
    font-size: 15px;
    font-weight: bold;
    color: #00d4ff;
    margin-bottom: 6px;
    padding-bottom: 6px;
    border-bottom: 1px solid rgba(0, 200, 255, 0.25);
  }
  .label-row {
    display: flex;
    align-items: center;
    margin: 4px 0;
    font-size: 12px;
  }
  .label-value {
    flex: 1;
    color: #00d4ff;
    font-weight: bold;
  }
  `

document.head.appendChild(style)
}

return entry
}

const updateLabelFields = (entry, next) => {

```

```

    if (!entry?.fields) return
    if (entry.fields.temperature && next.temperature !== null) {
      entry.fields.temperature.textContent = `${next.temperature}°C`
    }
    if (entry.fields.humidity && next.humidity !== null) {
      entry.fields.humidity.textContent = `${next.humidity}%`
    }
    if (entry.fields.gps && next.gps !== null) {
      entry.fields.gps.textContent = next.gps
    }
  }
}

const toTrainGps = () => {
  // 地点脱敏：不展示真实坐标
  return { lon: 'X', lat: 'Y' }
}

const getPedestrianDistanceMeters = () => {
  const telemetry = telemetryDistanceM.value
  if (!Number.isFinite(telemetry)) return null
  // 约定：后端未回传/无有效数据时 distance 为 0（此时不应触发告警/飘红）
  if (telemetry <= 0) return null
  return telemetry
}

const updateHumanoidSensorPanel = () => {
  if (!telemetryActive.value) {
    humanoidSensor.value.pedestrianDistance = '--'
    return
  }

  const meters = getPedestrianDistanceMeters()
  if (meters == null || meters >= 2) {
    humanoidSensor.value.pedestrianDistance = '--'
    return
  }

  const { lon, lat } = toTrainGps()
  humanoidSensor.value.gps = `${lon}, ${lat}`
  humanoidSensor.value.pedestrianDistance = `${Number(meters).toFixed(2)}m`
}

const updateBoxSensorLabel = () => {
  if (!boxObject || !boxSensorLabelEntry?.fields) return
  const meters = getPedestrianDistanceMeters()
  const hasPedestrian = telemetryActive.value && meters !== null && meters < 2

  // 行人检测传感器与右侧“人形感应器”为同一套数据
  boxSensor.value.gps = humanoidSensor.value.gps
  boxSensor.value.pedestrianDistance = hasPedestrian ? humanoidSensor.value.pedestrianDistance : '--'

  // 行人经过预警：距离近时闪烁增强发光（无人行人则不告警）
  const isWarning = hasPedestrian && meters < 2
  if (boxSensorLight && boxObject) {
    boxSensorLight.position.set(boxObject.position.x, boxObject.position.y + 4.2, boxObject.position.z)
    const t = Date.now() * 0.01
    boxSensorLight.intensity = isWarning ? (4.2 + Math.sin(t * 7) * 2.0) : 0.9
  }

  if (boxObject) {
    boxObject.traverse((child) => {
      if (!child.isMesh || !child.material) return
      const mats = Array.isArray(child.material) ? child.material : [child.material]
      mats.forEach((m) => {
        if (m.emissive) m.emissive.setHex(0xff0000)
      })
    })
  }
}

```

```

        if ('emissiveIntensity' in m) {
            m.emissiveIntensity = isWarning ? (1.25 + Math.sin(Date.now() * 0.02) * 0.6) : 0.85
        }
        m.needsUpdate = true
    })
})
}

// 传感器信息栏始终在 box 正上方（世界坐标系）
if (boxSensorLabelAnchor && boxObject) {
    const bSize = boxObjectSize
    const lift = Math.max(1.2, (bSize?.y || 0) * 1.15)
    boxSensorLabelAnchor.position.set(boxObject.position.x, boxObject.position.y + lift, boxObject.position.z)
    boxSensorLabelAnchor.rotation.set(0, 0, 0)
}

// 扇形超声波动画（缩放 + 淡出循环），并跟随传感器位置
if (boxSensorWave && boxObject) {
    const bSize = boxObjectSize
    const waveY = boxObject.position.y + Math.max(0.2, (bSize?.y || 0) * 0.55)
    boxSensorWave.position.set(boxObject.position.x, waveY, boxObject.position.z)
    // 强制保持在 X-Z 平面（垂直于 Y 轴）：几何已烘焙旋转，这里只跟随朝向（绕 Y 轴）
    boxSensorWave.rotation.set(0, getWorldYaw(boxObject), 0)
    const period = 1300
    const t = (Date.now() % period) / period
    const scale = 0.6 + t * 2.4
    boxSensorWave.scale.set(scale, scale, scale)
    const baseOpacity = isWarning ? 0.45 : 0.12
    boxSensorWave.material.opacity = Math.max(0, (1 - t) * baseOpacity)
    boxSensorWave.material.color.setHex(isWarning ? 0xff1111 : 0xaa3333)
}

// 信息栏红色告警（距离 < 10m）
if (boxSensorLabelEntry?.div) {
    boxSensorLabelEntry.div.classList.toggle('sensor-warning', isWarning)
}

boxSensorLabelEntry.fields.power.textContent = boxSensor.value.power
boxSensorLabelEntry.fields.battery.textContent = boxSensor.value.battery
boxSensorLabelEntry.fields.gps.textContent = boxSensor.value.gps
boxSensorLabelEntry.fields.dist.textContent = boxSensor.value.pedestrianDistance
}

const startAutoDemo = () => {
    if (autoDemoStarted) return
    autoDemoStarted = true

    autoDemoWaitTimer = setInterval(() => {
        if (!workerObject || !controls || !camera || !train || !signalObject) return
        clearInterval(autoDemoWaitTimer)
        autoDemoWaitTimer = null

        const workerPos = new THREE.Vector3()
        workerObject.getWorldPosition(workerPos)

        const camTo = workerPos.clone().add(new THREE.Vector3(22, 14, 22))

        const state = {
            camX: camera.position.x,
            camY: camera.position.y,
            camZ: camera.position.z,
            tx: controls.target.x,
            ty: controls.target.y,
            tz: controls.target.z
        }
    }, 1000)
}

```

```

}

autoDemoFlyTween = gsap.to(state, {
  camX: camTo.x,
  camY: camTo.y,
  camZ: camTo.z,
  tx: workerPos.x,
  ty: workerPos.y,
  tz: workerPos.z,
  duration: 7,
  ease: 'power2.inOut',
  onUpdate: () => {
    camera.position.set(state.camX, state.camY, state.camZ)
    controls.target.set(state.tx, state.ty, state.tz)
    controls.update()
  },
  onComplete: () => {
    // 保持一定距离后开始旋转
    autoRotate = true

    // 自动触发火车头行进，并从信号灯附近开始
    const nearSignal = new THREE.Vector3(signalObject.position.x, 3, signalObject.position.z)
    trainProgress = getClosestPathProgress(nearSignal, 800)
    const startPoint = trainPath.getPoint(trainProgress)
    train.position.copy(startPoint)
    const nextPoint = trainPath.getPoint((trainProgress + 0.01) % 1)
    train.lookAt(nextPoint)
    isTrainRunning = true
  }
})
}, 200)
}

const createTrees = () => {
  const treeCount = 30

  for (let i = 0; i < treeCount; i++) {
    const treeGroup = new THREE.Group()

    const trunkGeometry = new THREE.CylinderGeometry(0.3, 0.5, 3, 8)
    const trunkMaterial = new THREE.MeshStandardMaterial({ color: 0x6a5a4a })
    const trunk = new THREE.Mesh(trunkGeometry, trunkMaterial)
    trunk.position.y = 1.5
    trunk.castShadow = true
    treeGroup.add(trunk)

    const leavesGeometry = new THREE.ConeGeometry(2, 4, 8)
    const leavesMaterial = new THREE.MeshStandardMaterial({ color: 0x4a8a4a })
    const leaves = new THREE.Mesh(leavesGeometry, leavesMaterial)
    leaves.position.y = 5
    leaves.castShadow = true
    treeGroup.add(leaves)

    // 树木大小随机分布
    const treeScale = 0.7 + Math.random() * 1.1
    treeGroup.scale.set(treeScale, treeScale * (0.85 + Math.random() * 0.4), treeScale)
    leaves.scale.set(1, 0.8 + Math.random() * 0.8, 1)

    // 树木位置不能落在铁轨上（铁轨沿 z = x 的 45° 对角线分布）
    // 点到直线 z=x 的距离: |z-x|/sqrt(2)
    const railClearance = 10
    let x = 0
    let z = 0
    let attempts = 0

```

```

do {
  x = (Math.random() - 0.5) * 200
  z = (Math.random() - 0.5) * 200
  attempts++
} while ((Math.abs(z - x) / Math.SQRT2) < railClearance && attempts < 80)

treeGroup.position.set(x, 0, z)

scene.add(treeGroup)
}
}

const updateSignalUI = () => {
  const signalList = document.getElementById('signallist')
  if (!signalList) return

  const stateText = {
    'red': '禁止通行',
    'green': '允许通行',
    'yellow': '减速通行',
    'off': '异常灭灯'
  }

  signalList.innerHTML = signals.map(signal => `
    <div class="signal-item ${selectedSignalId.value === signal.id ? 'signal-item-active' : ''}" data-signal-id="${signal.id}">
      <div class="signal-light signal-${signal.state}"></div>
      <div class="signal-info">
        <div class="signal-line">
          <span class="signal-name">${signal.name}</span>
          <span class="signal-status signal-status-${signal.state}">${stateText[signal.state]}</span>
        </div>
        <div class="signal-meta-row">
          <span>点击查看监测</span>
          <span>${signal.id === '1495' && signalFaultActive.value ? '异常处理中' : '在线'}</span>
        </div>
      </div>
    </div>
  `).join('')

  signalList.onclick = (event) => {
    const target = event.target instanceof Element ? event.target.closest('[data-signal-id]') : null
    const signalId = target?.getAttribute('data-signal-id')
    if (!signalId) return
    handleSignalAction(signalId)
  }
}

const toggleSignals = () => {
  const states = ['red', 'green', 'yellow']
  signals.forEach(signal => {
    if (signal.id === '1495' && signalFaultActive.value) {
      syncSignalVisual(signal.id, 'off')
      return
    }
    const currentStateIndex = states.indexOf(signal.state)
    signal.state = states[(currentStateIndex + 1) % states.length]
    const color = getColorByState(signal.state)

    signal.light.color.setHex(color)
    signal.light.intensity = signal.state === 'off' ? 0 : 3

    if (signal.mesh) {
      signal.mesh.traverse((child) => {
        if (child.isMesh && child.material) {

```

```

        if (child.material.emissive) {
            child.material.emissive.setHex(color)
        }
        if (child.material.color) {
            child.material.color.setHex(color)
        }
    }
})
}
})
updateSignalUI()
}

const trainAnimation = () => {
    const nextState = !isTrainRunning
    if (nextState && train) {
        trainProgress = getClosestPathProgress(train.position, 800)
    }
    isTrainRunning = nextState
}

const toggleRotation = () => {
    autoRotate = !autoRotate
}

const resetView = () => {
    camera.position.set(60, 60, 80)
    controls.target.set(0, 0, 0)
    controls.update()
}

const onWindowResize = () => {
    camera.aspect = window.innerWidth / window.innerHeight
    camera.updateProjectionMatrix()
    renderer.setSize(window.innerWidth, window.innerHeight)
    labelRenderer.setSize(window.innerWidth, window.innerHeight)
}

const updateUI = () => {
    uiNowMs.value = Date.now()
    const lastUpdate = document.getElementById('lastUpdate')
    if (lastUpdate) {
        lastUpdate.textContent = new Date().toLocaleString('zh-CN')
    }

    const uptime = document.getElementById('uptime')
    if (uptime) {
        const seconds = Math.floor((Date.now() - startTime) / 1000)
        uptime.textContent = `${seconds}s`
    }

    const camPos = document.getElementById('cameraPos')
    if (camPos) {
        camPos.textContent = `${camera.position.x.toFixed(0)}, ${camera.position.y.toFixed(0)}, ${camera.position.z.toFixed(0)}`
    }
}

// 更新信号灯参数（本地模拟）
const updateSignalParams = async () => {
    const base = selectedSignal.value
    signalParams.value.temperature = Math.round((base.tempBase + Math.random() * 4) * 10) / 10
    signalParams.value.humidity = Math.round((21 + Math.random() * 4) * 10) / 10
    signalParams.value.light = Math.round(550 + Math.random() * 950)
    signalParams.value.voltage = signalFaultActive.value

```

```

    ? 0
    : (signalHardwareReady.value ? Math.round((base.voltageBase - 2 + Math.random() * 4) * 10) / 10 : 0)
signalParams.value.current = signalFaultActive.value
    ? 0
    : (signalHardwareReady.value ? Math.round((base.currentBase - 0.2 + Math.random() * 0.4) * 100) / 100 : 0)
signalParams.value.signal = Math.round(-58 + Math.random() * 10)

// 恶劣天气模拟时: 湿度与其它界面同步
if (severeWeatherEnabled.value) {
    const h = Number(syncedHumidity.value)
    if (Number.isFinite(h)) {
        signalParams.value.humidity = Math.round(h * 10) / 10
    }
}

// 同步更新孪生体参数 (与实时参数联动, 便于演示)
const latency = Math.round(80 + Math.random() * 260)
const freshness = Math.max(1, Math.round(latency / 100))
const highTempPenalty = Math.max(0, signalParams.value.temperature - 45) * 0.9
const voltagePenalty = Math.max(0, Math.abs(signalParams.value.voltage - 220) - 8) * 0.6
const signalPenalty = Math.max(0, -55 - signalParams.value.signal) * 0.35
const baseHealth = 95 - highTempPenalty - voltagePenalty - signalPenalty
const health = Math.max(55, Math.min(99, Math.round(baseHealth)))

signalTwin.value.syncLatencyMs = latency
signalTwin.value.dataFreshnessS = freshness
signalTwin.value.healthScore = health
signalTwin.value.syncStatus = telemetryConnected.value ? (latency > 320 ? '延迟偏高' : '已同步') : '未接入'
signalTwin.value.interlocking = signalFaultActive.value || trackFaultActive.value ? '预警' : (health < 70 ? '预警' : '正常')
signalTwin.value.workMode = signalFaultActive.value || trackFaultActive.value ? '故障演练' : (latency > 320 ? '降级' : '自动')
signalTwin.value.predictedRulDays = Math.max(15, Math.round(220 - (99 - health) * 3.2))
}

watch(selectedSignalId, () => {
    syncSignalTwin()
    updateSignalUI()
}, { immediate: true })

watch(
    [severeWeatherEnabled, syncedHumidity],
    async ([enabled, h]) => {
        if (!enabled) {
            await updateSignalParams()
            return
        }
        const humidity = Number(h)
        if (!Number.isFinite(humidity)) return
        signalParams.value.humidity = Math.round(humidity * 10) / 10
    },
    { immediate: true }
)

// 正常态参数历史数据保持在较低湿度, 避免页面初始即出现异常
const loadParamHistory = async () => {
    paramData1h.value.humidity = [22, 22, 23, 23, 24, 24, 23, 23, 22, 22, 23, 23]
    paramData24h.value.humidity = [21, 21, 20, 20, 20, 21, 21, 22, 22, 23, 23, 24, 24, 24, 23, 23, 22, 22, 22, 21, 21, 21, 20, 20]
    paramDataWeek.value.humidity = [22, 23, 22, 24, 23, 22, 23]
}

const animate = () => {
    if (animationStopped) return
    rafId = requestAnimationFrame(animate)

    if (!renderer || !scene || !camera || !controls) {

```

```

    return
  }

  const delta = clock.getDelta()
  const t = clock.getElapsedTime()

  controls.update()

  if (autoRotate) {
    controls.autoRotate = true
    controls.autoRotateSpeed = 2.0
  } else {
    controls.autoRotate = false
  }

  if (isTrainRunning && train) {
    trainProgress = (trainProgress + delta * trainSpeed.value) % 1
    const position = trainPath.getPoint(trainProgress)
    train.position.copy(position)

    const nextPoint = trainPath.getPoint((trainProgress + 0.01) % 1)
    train.lookAt(nextPoint)

    // 列车行进: GPS 跟随变化, 同时展示温度
    if (trainLabelEntry) {
      const { lon, lat } = toTrainGps()
      const temp = Math.round((26 + Math.sin(trainProgress * Math.PI * 2) * 3 + Math.random() * 0.6) * 10) / 10
      const hum = Math.round((22 + Math.cos(trainProgress * Math.PI * 2) * 1.4 + Math.random() * 0.6) * 10) / 10
      updateLabelFields(trainLabelEntry, {
        temperature: temp,
        humidity: hum,
        gps: `${lon}, ${lat}`
      })
    }
  }

  if (signalLabelEntry) {
    updateLabelFields(signalLabelEntry, {
      temperature: Math.round(Number(signalParams.value.temperature) * 10) / 10,
      humidity: Math.round(Number(signalParams.value.humidity) * 10) / 10
    })
  }

  updateHumanoidSensorPanel()
  updateBoxSensorLabel()

  if (skyParticles) {
    skyParticles.rotation.y += delta * 0.03
    if (skyParticlesMaterial) {
      skyParticlesMaterial.opacity = 0.16 + (Math.sin(t * 0.7) + 1) * 0.06
    }
  }

  if (endpointGlows.length) {
    endpointGlows.forEach((item, index) => {
      const p = (Math.sin(t * 1.6 + item.phase) + 1) * 0.5
      const scale = item.baseScale * (1 + 0.18 * p)
      item.sprite.scale.set(scale, scale, 1)
      item.sprite.material.opacity = Math.max(0, Math.min(1, item.baseOpacity * (0.35 + 0.65 * p)))
      item.sprite.material.rotation = (t * 0.15 + index * 0.4) % (Math.PI * 2)
    })
  }

  if (mixer) {

```

```

    mixer.update(delta)
  }

  renderer.render(scene, camera)
  if (labelRenderer) {
    labelRenderer.render(scene, camera)
  }
}

onMounted(async () => {
  await loadParamHistory()
  await updateSignalParams()

  init()
  // 页面加载完成后自动演示：飞向工作人员 -> 到位后旋转 -> 自动触发行进
  startAutoDemo()

  window.addEventListener('keydown', onGlobalKeydown)
  applyTwinEnvVisibilityToLabels()
  stopFrontendTelemetrySim()

  // 定期更新信号灯参数
  updateSignalParamsTimer = setInterval(updateSignalParams, 5000)
})

onBeforeUnmount(() => {
  animationStopped = true
  if (rafId) {
    cancelAnimationFrame(rafId)
    rafId = 0
  }

  for (const timer of retryTimers.values()) {
    clearTimeout(timer)
  }
  retryTimers.clear()

  if (updateUiTimer) {
    clearInterval(updateUiTimer)
    updateUiTimer = null
  }
  if (updateSignalParamsTimer) {
    clearInterval(updateSignalParamsTimer)
    updateSignalParamsTimer = null
  }
  if (telemetrySimTimer) {
    clearTimeout(telemetrySimTimer)
    telemetrySimTimer = null
  }
  if (telemetrySimPassTimer) {
    clearInterval(telemetrySimPassTimer)
    telemetrySimPassTimer = null
  }
  if (pedestrianHotFlashTimer) {
    clearTimeout(pedestrianHotFlashTimer)
    pedestrianHotFlashTimer = null
  }
  if (pedestrianNoticeAudio) {
    pedestrianNoticeAudio.pause()
    pedestrianNoticeAudio.currentTime = 0
    pedestrianNoticeAudio = null
  }

  if (autoDemoWaitTimer) {

```

```

clearInterval(autoDemoWaitTimer)
autoDemoWaitTimer = null
}
if (autoDemoFlyTween) {
  autoDemoFlyTween.kill()
  autoDemoFlyTween = null
}

if (renderer) {
  renderer.dispose()
}
if (renderer?.domElement?.parentNode) {
  renderer.domElement.parentNode.removeChild(renderer.domElement)
}
renderer = null

if (labelRenderer?.domElement?.parentNode) {
  labelRenderer.domElement.parentNode.removeChild(labelRenderer.domElement)
}
labelRenderer = null
if (boxSensorLight && scene) {
  scene.remove(boxSensorLight)
  boxSensorLight = null
}
if (boxSensorWave && scene) {
  scene.remove(boxSensorWave)
  boxSensorWave.geometry?.dispose?().
  if (Array.isArray(boxSensorWave.material)) boxSensorWave.material.forEach(m => m.dispose?().)
  else boxSensorWave.material?.dispose?().
  boxSensorWave = null
}
if (boxSensorLabelAnchor && scene) {
  scene.remove(boxSensorLabelAnchor)
  boxSensorLabelAnchor = null
  boxSensorLabelEntry = null
}

if (skyParticles && scene) {
  scene.remove(skyParticles)
  skyParticles.geometry?.dispose?().
  skyParticlesMaterial?.dispose?().
  skyParticles = null
  skyParticlesMaterial = null
}

if (endpointGlows.length && scene) {
  endpointGlows.forEach(({ sprite }) => {
    scene.remove(sprite)
    sprite.material?.dispose?().
  })
  endpointGlows = []
}
if (endpointGlowTexture) {
  endpointGlowTexture.dispose?().
  endpointGlowTexture = null
}
window.removeEventListener('resize', onWindowResize)
window.removeEventListener('keydown', onGlobalKeydown)

controls = null
camera = null
scene = null
})
</script>

```

```
<style scoped>
.three-view {
  width: 100%;
  height: 100%;
  position: relative;
}

#threeContainer {
  width: 100%;
  height: 100%;
  position: relative;
}

.control-panel {
  position: absolute;
  bottom: 20px;
  left: 50%;
  transform: translateX(-50%);
  background: rgba(0, 20, 40, 0.85);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 10px 16px;
  color: #fff;
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  backdrop-filter: blur(10px);
  z-index: 100;
}

.control-btn {
  padding: 6px 12px;
  background: linear-gradient(135deg, #0066cc, #00d4ff);
  border: none;
  border-radius: 5px;
  color: #fff;
  cursor: pointer;
  font-size: 13px;
  transition: all 0.3s;
}

.control-btn:hover {
  background: linear-gradient(135deg, #0088ff, #00ffff);
  box-shadow: 0 0 15px rgba(0, 200, 255, 0.5);
}

.train-control-group {
  display: flex;
  align-items: center;
  gap: 8px;
}

.speed-inline {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 6px 10px;
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 6px;
  background: rgba(0, 40, 80, 0.45);
}

.speed-label {
```

```
font-size: 13px;
color: #a8ddff;
}

.speed-select {
  min-width: 78px;
  padding: 3px 6px;
  border: 1px solid rgba(0, 200, 255, 0.4);
  border-radius: 4px;
  background: rgba(0, 20, 40, 0.9);
  color: #d8f3ff;
  font-size: 13px;
}

.info-panel {
  position: absolute;
  top: 62px;
  left: 20px;
  bottom: 64px;
  width: 320px;
  background: rgba(0, 20, 40, 0.55);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 20px;
  color: #fff;
  overflow-y: auto;
  backdrop-filter: blur(10px);
  box-shadow: 0 4px 20px rgba(0, 200, 255, 0.2),
              inset 0 0 30px rgba(0, 200, 255, 0.03);
}

.stats-panel {
  position: absolute;
  top: 62px;
  right: 20px;
  bottom: 64px;
  width: 320px;
  background: rgba(0, 20, 40, 0.55);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 20px;
  color: #fff;
  overflow-y: auto;
  backdrop-filter: blur(10px);
  box-shadow: 0 4px 20px rgba(0, 200, 255, 0.2),
              inset 0 0 30px rgba(0, 200, 255, 0.03);
}

/* 面板装饰边框 */
.panel-border {
  position: absolute;
  pointer-events: none;
}

.panel-border.top-left {
  top: -1px;
  left: -1px;
  width: 30px;
  height: 30px;
  border-top: 3px solid #00d4ff;
  border-left: 3px solid #00d4ff;
  border-radius: 8px 0 0 0;
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.5);
  animation: corner-float-tl 3.2s ease-in-out infinite;
```

```

}

.panel-border.top-right {
  top: -1px;
  right: -1px;
  width: 30px;
  height: 30px;
  border-top: 3px solid #00d4ff;
  border-right: 3px solid #00d4ff;
  border-radius: 0 8px 0 0;
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.5);
  animation: corner-float-tr 3.6s ease-in-out infinite;
}

.panel-border.bottom-left {
  bottom: -1px;
  left: -1px;
  width: 30px;
  height: 30px;
  border-bottom: 3px solid #00d4ff;
  border-left: 3px solid #00d4ff;
  border-radius: 0 0 0 8px;
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.5);
  animation: corner-float-bl 3.4s ease-in-out infinite;
}

.panel-border.bottom-right {
  bottom: -1px;
  right: -1px;
  width: 30px;
  height: 30px;
  border-bottom: 3px solid #00d4ff;
  border-right: 3px solid #00d4ff;
  border-radius: 0 0 8px 0;
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.5);
  animation: corner-float-br 3.8s ease-in-out infinite;
}

.panel-border.glow-line {
  top: 0;
  left: 50%;
  transform: translateX(-50%);
  width: 60%;
  height: 2px;
  background: linear-gradient(90deg, transparent, #00d4ff, transparent);
  opacity: 0.6;
  animation: glow-pulse 2s ease-in-out infinite;
}

@keyframes glow-pulse {
  0%, 100% { opacity: 0.4; width: 50%; }
  50% { opacity: 0.8; width: 70%; }
}

@keyframes corner-float-tl {
  0%, 100% { transform: translate(0, 0); opacity: 0.85; }
  50% { transform: translate(-2px, -2px); opacity: 1; }
}

@keyframes corner-float-tr {
  0%, 100% { transform: translate(0, 0); opacity: 0.85; }
  50% { transform: translate(2px, -2px); opacity: 1; }
}

```

```
@keyframes corner-float-bl {
  0%, 100% { transform: translate(0, 0); opacity: 0.85; }
  50% { transform: translate(-2px, 2px); opacity: 1; }
}

@keyframes corner-float-br {
  0%, 100% { transform: translate(0, 0); opacity: 0.85; }
  50% { transform: translate(2px, 2px); opacity: 1; }
}

.panel-section {
  margin-bottom: 25px;
}

.panel-section:last-child {
  margin-bottom: 0;
}

.panel-title {
  display: flex;
  align-items: center;
  gap: 8px;
  color: #00d4ff;
  font-size: 16px;
  margin-bottom: 15px;
  padding-bottom: 8px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.3);
  position: relative;
}

.title-icon {
  font-size: 18px;
}

.title-text {
  flex: 1;
}

.title-decorator {
  width: 40px;
  height: 2px;
  background: linear-gradient(90deg, #00d4ff, transparent);
}

.signal-item {
  display: flex;
  align-items: center;
  margin: 12px 0;
  padding: 10px;
  background: rgba(0, 100, 150, 0.2);
  border-radius: 5px;
  border-left: 3px solid #00d4ff;
}

.signal-light {
  width: 20px;
  height: 20px;
  border-radius: 50%;
  margin-right: 12px;
  box-shadow: 0 0 10px currentColor;
  animation: blink 2s infinite;
}

.signal-red { background: #ff3333; color: #ff3333; }
```

```
.signal-green { background: #33ff33; color: #33ff33; }
.signal-yellow { background: #ffff33; color: #ffff33; }

@keyframes blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.6; }
}

.signal-info {
  flex: 1;
}

.signal-name {
  font-size: 13px;
  font-weight: bold;
}

.signal-status {
  display: inline-flex;
  align-items: center;
  padding: 2px 8px;
  border-radius: 10px;
  font-size: 12px;
  font-weight: 600;
  line-height: 1.2;
}

.signal-line {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
}

.signal-status-red {
  color: #ffd7d7;
  background: rgba(255, 70, 70, 0.28);
  border: 1px solid rgba(255, 120, 120, 0.5);
}

.signal-status-green {
  color: #dbffe1;
  background: rgba(46, 204, 113, 0.24);
  border: 1px solid rgba(102, 255, 178, 0.45);
}

.signal-status-yellow {
  color: #fff5cf;
  background: rgba(255, 193, 7, 0.22);
  border: 1px solid rgba(255, 225, 119, 0.45);
}

.signal-off {
  background: #585858;
  color: #585858;
  box-shadow: none;
}

.signal-status-off {
  color: #f8d6d6;
  background: rgba(120, 120, 120, 0.26);
  border: 1px solid rgba(180, 180, 180, 0.35);
}
```

```
.signal-item-active {
  border-left-color: #7ef9ff;
  background: rgba(0, 160, 210, 0.26);
  box-shadow: 0 0 18px rgba(0, 212, 255, 0.16);
}

.signal-meta-row {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  margin-top: 6px;
  font-size: 12px;
  color: rgba(255, 255, 255, 0.64);
}

.twin-metrics {
  margin-top: 12px;
  padding: 10px;
  background: rgba(0, 80, 120, 0.18);
  border: 1px solid rgba(0, 200, 255, 0.18);
  border-radius: 6px;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 8px 10px;
}

.tm-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 6px 8px;
  border-radius: 6px;
  background: rgba(0, 20, 40, 0.35);
}

.tm-label {
  font-size: 12px;
  color: rgba(255, 255, 255, 0.7);
  white-space: nowrap;
}

.tm-value {
  font-size: 12px;
  color: #d8f3ff;
  font-weight: 700;
  text-align: right;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.tm-value.ok { color: #33ff99; }
.tm-value.warn { color: #ffcc66; }
.tm-value.danger { color: #ff5c5c; }

.signal-kpis {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
  margin-top: 10px;
  padding: 10px;
  border-radius: 10px;
  background: rgba(0, 100, 150, 0.10);
```

```
border: 1px solid rgba(0, 200, 255, 0.18);
}

.kpi-item {
  display: flex;
  justify-content: space-between;
  align-items: baseline;
  gap: 8px;
}

.kpi-label {
  font-size: 12px;
  color: rgba(255, 255, 255, 0.72);
}

.kpi-value {
  font-size: 13px;
  font-weight: 800;
  color: #d8f3ff;
  text-shadow: 0 0 10px rgba(0, 212, 255, 0.15);
}

.kpi-value.masked {
  color: rgba(216, 243, 255, 0.38);
  font-weight: 700;
  text-shadow: none;
}

.kpi-value.env-glow {
  animation: envGlowPulse 1.55s ease-in-out infinite;
}

.kpi-value.ok { color: #33ff99; }
.kpi-value.warn { color: #ffcc66; }
.kpi-value.danger { color: #ff5c5c; }

@keyframes envGlowPulse {
  0%, 100% {
    text-shadow:
      0 0 10px rgba(0, 212, 255, 0.35),
      0 0 18px rgba(0, 200, 255, 0.20),
      0 0 30px rgba(0, 160, 255, 0.12);
  }
  50% {
    text-shadow:
      0 0 12px rgba(0, 212, 255, 0.65),
      0 0 24px rgba(0, 200, 255, 0.35),
      0 0 42px rgba(0, 160, 255, 0.18);
  }
}

.panel-alert {
  padding: 10px;
  border-radius: 8px;
  background: linear-gradient(135deg, rgba(255, 40, 40, 0.22), rgba(120, 0, 0, 0.12));
  border: 1px solid rgba(255, 80, 80, 0.35);
  box-shadow: 0 0 22px rgba(255, 50, 50, 0.22);
  animation: panel-alert-pulse 1.2s ease-in-out infinite;
}

.panel-alert .panel-title {
  color: #ff4b4b;
  border-bottom-color: rgba(255, 80, 80, 0.45);
}
```

```
.panel-alert .title-decorator {
  background: linear-gradient(90deg, #ff4b4b, transparent);
}

.panel-boost {
  animation: panel-boost-flash 0.32s ease-out 3;
}

@keyframes panel-boost-flash {
  0% {
    transform: scale(1);
    box-shadow: 0 0 0 rgba(0, 212, 255, 0);
  }
  50% {
    transform: scale(1.015);
    box-shadow:
      0 0 18px rgba(0, 212, 255, 0.45),
      0 0 34px rgba(0, 212, 255, 0.22);
  }
  100% {
    transform: scale(1);
    box-shadow: 0 0 0 rgba(0, 212, 255, 0);
  }
}

@keyframes panel-alert-pulse {
  0%, 100% { box-shadow: 0 0 18px rgba(255, 50, 50, 0.18); }
  50% { box-shadow: 0 0 28px rgba(255, 50, 50, 0.32); }
}

.alarm-feed {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.alarm-card {
  padding: 10px 12px;
  border-radius: 8px;
  border: 1px solid rgba(0, 200, 255, 0.18);
  background: rgba(0, 90, 130, 0.12);
}

.alarm-card-danger {
  border-color: rgba(255, 92, 92, 0.4);
  background: rgba(120, 12, 12, 0.28);
}

.alarm-card-ok {
  border-color: rgba(51, 255, 153, 0.25);
  background: rgba(0, 80, 48, 0.18);
}

.alarm-title {
  font-size: 13px;
  font-weight: 700;
  color: #f0fbff;
}

.alarm-desc {
  margin-top: 4px;
  font-size: 12px;
  line-height: 1.5;
}
```

```
    color: rgba(255, 255, 255, 0.76);
}

.alarm-voice {
  font-size: 12px;
  color: #9ceeff;
  line-height: 1.6;
}

.stat-item {
  display: flex;
  justify-content: space-between;
  margin: 8px 0;
  font-size: 13px;
}

.stat-label { color: #aaa; }
.stat-value { color: #00d4ff; font-weight: bold; }
.stat-value.ok { color: #33ff99; }
.stat-value.warn { color: #ffcc66; }
.stat-value.danger { color: #ff5c5c; }
.stat-value.highlight {
  color: #33ff33;
  text-shadow: 0 0 5px rgba(51, 255, 51, 0.5);
}

.shortcut-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
}

.shortcut-card {
  padding: 12px;
  text-align: left;
  border: 1px solid rgba(0, 200, 255, 0.24);
  border-radius: 10px;
  background: rgba(0, 70, 120, 0.16);
  color: #dff9ff;
  cursor: pointer;
  transition: 0.25s ease;
}

.shortcut-card:hover {
  transform: translateY(-2px);
  border-color: rgba(0, 212, 255, 0.45);
  box-shadow: 0 8px 18px rgba(0, 140, 200, 0.18);
}

.shortcut-title {
  display: block;
  font-size: 13px;
  font-weight: 700;
  color: #fff;
}

.shortcut-desc {
  display: block;
  margin-top: 6px;
  font-size: 12px;
  color: rgba(255, 255, 255, 0.7);
}

.key-grid {
```

```
    display: grid;
    gap: 8px;
}

.key-chip {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 8px 10px;
    border-radius: 8px;
    background: rgba(0, 100, 150, 0.12);
    border: 1px solid rgba(0, 200, 255, 0.16);
    font-size: 12px;
    color: #d7f7ff;
}

.key-chip b {
    min-width: 18px;
    color: #00d4ff;
}

.progress-bar {
    width: 100%;
    height: 8px;
    background: rgba(0, 100, 150, 0.3);
    border-radius: 4px;
    overflow: hidden;
    margin: 8px 0;
}

.progress-fill {
    height: 100%;
    background: linear-gradient(90deg, #0066cc, #00d4ff);
    border-radius: 4px;
    transition: width 0.5s;
}

.loading {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: #000;
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 9999;
}

.loading-content {
    text-align: center;
}

.loading-spinner {
    width: 60px;
    height: 60px;
    border: 4px solid rgba(0, 200, 255, 0.3);
    border-top-color: #00d4ff;
    border-radius: 50%;
    animation: spin 1s linear infinite;
    margin: 0 auto 20px;
}
```

```
@keyframes spin {
  to { transform: rotate(360deg); }
}

.loading-text {
  color: #00d4ff;
  font-size: 18px;
}

.model-label {
  position: absolute;
  background: rgba(0, 20, 40, 0.9);
  border: 2px solid rgba(0, 200, 255, 0.5);
  border-radius: 8px;
  padding: 12px 16px;
  color: #fff;
  font-size: 14px;
  pointer-events: none;
  backdrop-filter: blur(5px);
  box-shadow: 0 4px 15px rgba(0, 150, 200, 0.3);
  max-width: 280px;
}

.label-title {
  font-size: 16px;
  font-weight: bold;
  color: #00d4ff;
  margin-bottom: 8px;
  padding-bottom: 8px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.3);
}

.label-row {
  display: flex;
  align-items: center;
  margin: 6px 0;
  font-size: 13px;
}

.label-value {
  flex: 1;
  color: #00d4ff;
  font-weight: bold;
}

/* 参数网格 */
.param-grid {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.param-card {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px;
  background: rgba(0, 100, 150, 0.15);
  border-radius: 8px;
  border-left: 3px solid #00d4ff;
}

.param-icon {
  font-size: 20px;
```

```
width: 30px;
text-align: center;
}

.param-info {
  flex: 1;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.param-label {
  font-size: 13px;
  color: #aaa;
}

.param-value {
  font-size: 14px;
  font-weight: bold;
  color: #00d4ff;
}

.param-bar {
  width: 60px;
  height: 6px;
  background: rgba(0, 100, 150, 0.3);
  border-radius: 3px;
  overflow: hidden;
}

.bar-fill {
  height: 100%;
  border-radius: 3px;
  transition: width 0.5s ease;
}

.bar-fill.temp { background: linear-gradient(90deg, #4CAF50, #FFC107, #FF5722); }
.bar-fill.humidity { background: linear-gradient(90deg, #00d4ff, #0066cc); }
.bar-fill.light { background: linear-gradient(90deg, #FFC107, #FF9800); }
.bar-fill.voltage { background: linear-gradient(90deg, #33ff33, #00d4ff); }
.bar-fill.current { background: linear-gradient(90deg, #00d4ff, #ff9900); }
.bar-fill.signal { background: linear-gradient(90deg, #ff3333, #FFC107, #33ff33); }

/* 参数值颜色 */
.temp-low { color: #4CAF50 !important; }
.temp-medium { color: #FFC107 !important; }
.temp-high { color: #FF5722 !important; }
.voltage-normal { color: #33ff33 !important; }
.voltage-warning { color: #FFC107 !important; }

/* 设备状态 */
.device-status {
  margin-bottom: 15px;
}

.humanoid-sensor {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-top: 6px;
}

.sensor-row {
  display: flex;
```

```

    align-items: center;
    gap: 10px;
    padding: 8px 10px;
    background: rgba(0, 100, 150, 0.1);
    border-radius: 6px;
    border: 1px solid rgba(0, 200, 255, 0.18);
}

.sensor-label {
    font-size: 13px;
    color: #aaa;
    min-width: 60px;
}

.sensor-value {
    font-size: 13px;
    color: #fff;
    font-weight: bold;
    flex: 1;
}

.sensor-distance {
    font-size: 18px;
    letter-spacing: 0.5px;
}

.sensor-boost,
.stat-boost {
    color: #8ffbff !important;
    text-shadow:
        0 0 10px rgba(0, 212, 255, 0.75),
        0 0 20px rgba(0, 212, 255, 0.4);
    animation: telemetry-hot-flash 0.32s ease-out 3;
}

.sensor-meta {
    font-size: 12px;
    color: rgba(255, 255, 255, 0.75);
}

.sensor-value.online { color: #33ff33; }
.sensor-value.warning { color: #ffaa00; }

@keyframes telemetry-hot-flash {
    0%,
    100% {
        opacity: 1;
        transform: scale(1);
    }
    50% {
        opacity: 0.68;
        transform: scale(1.08);
    }
}

.status-row {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 8px 10px;
    background: rgba(0, 100, 150, 0.1);
    border-radius: 6px;
    margin-bottom: 6px;
}

```

```
.status-dot {
  width: 10px;
  height: 10px;
  border-radius: 50%;
}

.status-dot.online {
  background: #33ff33;
  box-shadow: 0 0 8px rgba(51, 255, 51, 0.6);
}

.status-dot.warning {
  background: #FFC107;
  box-shadow: 0 0 8px rgba(255, 193, 7, 0.6);
  animation: pulse-warning 1s infinite;
}

@keyframes pulse-warning {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

.status-label {
  flex: 1;
  font-size: 13px;
  color: #aaa;
}

.status-value {
  font-size: 13px;
  font-weight: bold;
}

.status-value.online { color: #33ff33; }
.status-value.warning { color: #FFC107; }

/* 视频网格 */
.video-grid {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.video-card {
  background: rgba(0, 100, 150, 0.15);
  border-radius: 8px;
  overflow: hidden;
  border: 1px solid rgba(0, 200, 255, 0.2);
}

.video-header {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 10px;
  background: rgba(0, 200, 255, 0.1);
  border-bottom: 1px solid rgba(0, 200, 255, 0.2);
}

.video-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
```

```
    background: #666;
}

.video-dot.live {
    background: #ff3333;
    box-shadow: 0 0 6px rgba(255, 51, 51, 0.8);
    animation: live-pulse 1.5s infinite;
}

@keyframes live-pulse {
    0%, 100% { opacity: 1; }
    50% { opacity: 0.4; }
}

.video-title {
    font-size: 12px;
    color: #00d4ff;
    font-weight: bold;
}

.video-container {
    position: relative;
    width: 100%;
    padding-bottom: 56.25%; /* 16:9 */
    background: #000;
}

.video-container .stream-frame {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;
}

.video-status {
    position: absolute;
    right: 8px;
    bottom: 8px;
    padding: 2px 8px;
    border-radius: 10px;
    font-size: 12px;
    color: #d8f3ff;
    background: rgba(0, 20, 40, 0.75);
    border: 1px solid rgba(0, 200, 255, 0.35);
}

/* 视频占位符 */
.video-placeholder {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: linear-gradient(135deg, rgba(0, 40, 80, 0.8), rgba(0, 20, 40, 0.9));
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    cursor: pointer;
    transition: all 0.3s;
}
```

```
.video-placeholder:hover {
  background: linear-gradient(135deg, rgba(0, 60, 100, 0.8), rgba(0, 30, 60, 0.9));
}

.video-placeholder:hover .play-button {
  transform: scale(1.1);
  box-shadow: 0 0 30px rgba(0, 200, 255, 0.6);
}

.play-button {
  width: 50px;
  height: 50px;
  background: rgba(0, 200, 255, 0.3);
  border: 2px solid #00d4ff;
  border-radius: 50%;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 20px;
  color: #00d4ff;
  transition: all 0.3s;
  box-shadow: 0 0 15px rgba(0, 200, 255, 0.4);
}

.video-hint {
  margin-top: 10px;
  font-size: 13px;
  color: #888;
}

/* 参数选择器 */
.param-selectors {
  display: flex;
  gap: 10px;
  margin-bottom: 12px;
}

.param-select {
  flex: 1;
  background: rgba(0, 100, 150, 0.3);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 6px;
  color: #00d4ff;
  font-size: 13px;
  padding: 8px 10px;
  cursor: pointer;
  outline: none;
}

.param-select:hover {
  background: rgba(0, 100, 150, 0.5);
}

.param-select option {
  background: rgba(0, 20, 40, 0.95);
  color: #fff;
}

/* 当前值显示 */
.current-value-display {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px;
```

```
background: rgba(0, 100, 150, 0.15);
border-radius: 8px;
margin-bottom: 12px;
border-left: 3px solid #00d4ff;
}

.current-param-icon {
  font-size: 32px;
}

.current-param-info {
  flex: 1;
}

.current-param-label {
  font-size: 13px;
  color: #aaa;
  margin-bottom: 4px;
}

.current-param-value {
  font-size: 28px;
  font-weight: bold;
}

.param-unit {
  font-size: 14px;
  color: #888;
  margin-left: 4px;
}

/* 参数曲线图容器 */
.param-chart-container {
  background: rgba(0, 50, 80, 0.2);
  border-radius: 8px;
  padding: 10px;
  margin-bottom: 12px;
}

.param-chart-svg {
  width: 100%;
  height: 100px;
  cursor: crosshair;
}

.param-tooltip {
  background: rgba(0, 40, 80, 0.95);
  border: 1px solid rgba(0, 200, 255, 0.5);
  border-radius: 4px;
  padding: 4px 8px;
  font-size: 12px;
  color: #fff;
  text-align: center;
  margin-top: 4px;
}

/* 参数快捷列表 */
.param-quick-list {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 8px;
}

.param-quick-item {
```

```
display: flex;
flex-direction: column;
align-items: center;
padding: 8px 6px;
background: rgba(0, 100, 150, 0.15);
border-radius: 6px;
cursor: pointer;
transition: all 0.3s;
border: 1px solid transparent;
}

.param-quick-item:hover {
  background: rgba(0, 100, 150, 0.3);
}

.param-quick-item.active {
  background: rgba(0, 200, 255, 0.2);
  border-color: rgba(0, 200, 255, 0.5);
}

.pq-icon {
  font-size: 16px;
  margin-bottom: 4px;
}

.pq-value {
  font-size: 13px;
  font-weight: bold;
  color: #00d4ff;
}

.device-modal-backdrop {
  position: absolute;
  inset: 0;
  background: rgba(0, 10, 20, 0.68);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 240;
  backdrop-filter: blur(8px);
}

.device-modal {
  width: min(760px, calc(100vw - 80px));
  border-radius: 18px;
  border: 1px solid rgba(0, 200, 255, 0.3);
  background: linear-gradient(180deg, rgba(3, 28, 48, 0.96), rgba(2, 18, 32, 0.96));
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.45);
  padding: 22px;
  color: #fff;
}

.device-modal-header,
.device-modal-footer,
.device-modal-actions,
.device-modal-time-row,
.device-modal-stat {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.device-modal-header {
  gap: 16px;
```

```
}

.device-modal-title {
  font-size: 22px;
  font-weight: 700;
  color: #e8fbff;
}

.device-modal-subtitle {
  margin-top: 4px;
  font-size: 12px;
  color: rgba(255, 255, 255, 0.65);
}

.device-modal-close {
  padding: 8px 14px;
  border-radius: 999px;
  border: 1px solid rgba(0, 200, 255, 0.25);
  background: rgba(0, 120, 170, 0.16);
  color: #dff9ff;
  cursor: pointer;
}

.device-modal-stats {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
  margin: 18px 0;
}

.device-modal-stat {
  padding: 10px 12px;
  border-radius: 10px;
  border: 1px solid rgba(0, 200, 255, 0.14);
  background: rgba(0, 85, 130, 0.12);
  font-size: 12px;
  gap: 8px;
}

.device-modal-stat strong {
  color: #7ef9ff;
  font-size: 14px;
}

.device-modal-chart {
  border-radius: 14px;
  padding: 14px;
  background: rgba(0, 20, 40, 0.6);
  border: 1px solid rgba(0, 200, 255, 0.12);
}

.device-grid-line {
  stroke: rgba(255, 255, 255, 0.1);
  stroke-width: 1;
}

.device-modal-svg {
  width: 100%;
  height: 240px;
}

.device-modal-polyline {
  fill: none;
  stroke-width: 3;
}
```

```

    stroke-linecap: round;
    stroke-dasharray: 6 8;
    animation: modallineFlow 1.6s linear infinite;
  }

  @keyframes modallineFlow {
    from { stroke-dashoffset: 0; }
    to { stroke-dashoffset: -28; }
  }

  .device-modal-time-row {
    margin-top: 8px;
    font-size: 12px;
    color: rgba(255, 255, 255, 0.62);
  }

  .device-modal-footer {
    gap: 16px;
    margin-top: 16px;
  }

  .device-modal-note {
    flex: 1;
    font-size: 13px;
    line-height: 1.6;
    color: #d7f7ff;
  }

  .device-modal-actions {
    gap: 10px;
  }
</style>

```

5.3 文件：src/components/DataPanel.vue

- 文件说明：大数据可视化总控页面，负责顶部信息、全局告警和左右中子面板联动。
- 行数统计：1042 行

```

<template>
  <div class="data-panel-container">
    <!-- 背景粒子效果 -->
    <div class="particles-bg">
      <div
        v-for="i in 50"
        :key="i"
        class="particle"
        :style="particleStyle(i)"
      ></div>
    </div>

    <!-- 网格背景 -->
    <div class="grid-bg"></div>

    <!-- 头部标题 -->
    <header class="panel-header">
      <div class="header-left">
        <div class="logo">
          <span class="logo-icon"><img alt="Logo icon" data-bbox="271 833 284 841"/></span>
          <span class="logo-text">铁路信号智能监控平台</span>
        </div>
      </div>
      <div class="header-center">
        <h1 class="main-title">
          <span class="title-decorator">【</span>
            大数据可视化分析平台

```

```
<span class="title-decorator">] </span>
</h1>
<div class="sub-title">RAILWAY SIGNAL INTELLIGENT MONITORING PLATFORM</div>
</div>
<div class="header-right">
  <div class="datetime">
    <div class="date">{{ currentDate }}</div>
    <div class="time">{{ currentTime }}</div>
  </div>
  <div class="weather-mini">
    <span class="weather-icon">☁</span>
    <span class="weather-temp">{{ weather.temperature }}°C</span>
  </div>
  <button class="severe-weather-btn" :class="{ active: severeWeatherEnabled }" @click="toggleSevereWeather">
    {{ severeWeatherEnabled ? '☁ 停止恶劣天气' : '☁ 恶劣天气模拟' }}
  </button>
  <div class="anomaly-controls">
    <button class="anomaly-btn clear-btn" :disabled="!humidityAnomaly" @click="clearHumidityAnomaly">
      异常消除
    </button>
  </div>
</div>
</header>

<div class="global-alerts">
  <transition name="alert-pop">
    <div v-if="showHumidityAlert" class="global-alert humidity" :class="humidityAlertClass">
      <div class="alert-title">湿度异常告警</div>
      <div class="alert-desc">
        <div class="alert-target">{{ heroDeviceName }}</div>
        <div class="alert-hint">
          <div class="alert-line">告警内容: 盒内湿度持续升高(当前 {{ currentHumidityText }})</div>
          <div class="alert-line">关联分析: 疑似密封件胶套老化渗水</div>
          <div class="alert-line">建议处置: 立即现场核查盒体密封状态、胶套老化情况及内部受潮情况。</div>
        </div>
      </div>
      <div class="alert-chart">
        <div class="chart-caption">24小时湿度变化曲线</div>
        <svg viewBox="0 0 560 150" class="alert-chart-svg">
          <defs>
            <linearGradient id="humidityAlertGradient" x1="0%" y1="0%" x2="0%" y2="100%">
              <stop offset="0%" style="stop-color:#ff2f2f;stop-opacity:0.38" />
              <stop offset="100%" style="stop-color:#ff2f2f;stop-opacity:0" />
            </linearGradient>
          </defs>
          <line x1="10" y1="30" x2="550" y2="30" stroke="rgba(255,255,255,0.12)" stroke-width="1" />
          <line x1="10" y1="70" x2="550" y2="70" stroke="rgba(255,255,255,0.12)" stroke-width="1" />
          <line x1="10" y1="110" x2="550" y2="110" stroke="rgba(255,255,255,0.12)" stroke-width="1" />
          <polygon :points="humidityAlertAreaPoints" fill="url(#humidityAlertGradient)" />
          <polyline :points="humidityAlertLinePoints" fill="none" stroke="#ff2f2f" stroke-width="3" />
          <g v-for="(p, idx) in humidityAlertLabelPoints" :key="idx">
            <circle :cx="p.x" :cy="p.y" r="3.2" fill="#ffb0b0" />
            <text :x="p.x" :y="p.y - 8" text-anchor="middle" fill="ffff" font-size="11" font-weight="bold">
              {{ p.value }}
            </text>
          </g>
        </svg>
      </div>
      <div class="alert-actions">
        <button class="alert-handle" :disabled="humidityAlertProcessing" @click="markHumidityAlertProcessing">
          {{ humidityAlertProcessing ? '处理中...' : '处理中' }}
        </button>
        <button class="alert-close" @click="dismissHumidityAlert">关闭弹框</button>
      </div>
    </div>
  </transition>
</div>
```

```
</div>
</transition>

<transition name="alert-pop">
  <div v-if="showHumidityObservation" class="global-alert observe">
    <div class="observe-left">
      <div class="observe-title">观察中</div>
      <div class="observe-desc">
        {{ heroDeviceName }}: 盒内湿度回落中 (当前 {{ currentHumidityText }})
      </div>
    </div>
    <button class="observe-close" @click="dismissHumidityObservation">关闭</button>
  </div>
</transition>
</div>

<!-- 主内容区 -->
<main class="panel-main">
  <!-- 左侧面板 -->
  <aside class="left-aside">
    <LeftPanel
      :humidity-anomaly="humidityAnomaly"
      :temperature-anomaly="false"
    />
  </aside>

  <!-- 中间面板 -->
  <section class="center-section">
    <CenterPanel />
  </section>

  <!-- 右侧面板 -->
  <aside class="right-aside">
    <RightPanel />
  </aside>
</main>

<!-- 底部状态栏 -->
<footer class="panel-footer">
  <div class="footer-left">
    <span class="status-item">
      <span class="status-dot online"></span>
      系统状态: 正常
    </span>
    <span class="status-item">
      <span class="status-dot online"></span>
      数据同步: 实时
    </span>
    <span class="status-item">
      数据点: {{ formatNumber(dataPoints) }}
    </span>
  </div>
  <div class="footer-center">
    <span class="version">v2.0.0</span>
    <span class="copyright">© 2024 铁路信号监控系统</span>
  </div>
  <div class="footer-right">
    <span class="update-time">数据更新: {{ lastUpdate }}</span>
    <span class="fps">FPS: {{ fps }}</span>
  </div>
</footer>

<!-- 装饰线条 -->
<div class="decorations">
```

```

        <div class="corner corner-tl"></div>
        <div class="corner corner-tr"></div>
        <div class="corner corner-bl"></div>
        <div class="corner corner-br"></div>
    </div>
</div>
</template>

<script setup>
import { ref, onMounted, onUnmounted, computed, watch } from 'vue'
import LeftPanel from './datapanel/LeftPanel.vue'
import CenterPanel from './datapanel/CenterPanel.vue'
import RightPanel from './datapanel/RightPanel.vue'
import { getWeatherData, getOverviewData } from '../services/mockDataService'
import {
    severeWeatherEnabled,
    setSevereWeatherEnabled,
    markHumidityMitigationApplied,
    syncedHumidity,
    humidityAlertDismissed,
    humidityAlertSuppressed,
    humidityAlertProcessing,
    setHumidityAlertDismissed,
    setHumidityAlertSuppressed,
    setHumidityAlertProcessing
} from '../services/simulationState.js'
import { DEMO_SCENARIO } from '../config/demoScenario.js'

const currentDate = ref('')
const currentTime = ref('')
const weather = ref(getWeatherData())
const overview = ref(getOverviewData())
const lastUpdate = ref('')
const fps = ref(60)
const dataPoints = ref(12345678)
const humidityAnomaly = ref(false)
const showHumidityAlert = ref(false)
const showHumidityObservation = ref(false)
const heroDeviceName = ref(DEMO_SCENARIO.heroDeviceName)

const HUMIDITY_WARNING = 45
const HUMIDITY_ALARM = 75

let observationAutoHideTimer = null

let timeTimer = null
let updateTimer = null
let fpsTimer = null
let alarmAudio = null
let alarmLastPlayedAt = 0

const ensureAlarmAudio = () => {
    if (alarmAudio) return alarmAudio
    alarmAudio = new Audio('/assets/audio/alarm.wav')
    alarmAudio.preload = 'auto'
    alarmAudio.volume = 0.95
    return alarmAudio
}

const playAlarmOnce = async () => {
    const audio = ensureAlarmAudio()
    try {
        audio.currentTime = 0
        await audio.play()
    }

```

```

    } catch (error) {
      // 可能被浏览器自动播放策略拦截：用户下一次交互后可再次触发
      console.warn('告警音频播放被拦截/失败:', error)
    }
  }

const maybePlayAlarmOnEnterOrShow = async () => {
  if (!severeWeatherEnabled.value) return
  if (!showHumidityAlert.value) return
  if (humidityAlertDismissed.value) return
  if (humidityAlertProcessing.value) return

  const now = Date.now()
  if (now - alarmLastPlayedAt < 500) return
  alarmLastPlayedAt = now
  await playAlarmOnce()
}

// 格式化数字
const formatNumber = (num) => {
  if (num >= 10000000) return (num / 10000000).toFixed(1) + '千万'
  if (num >= 10000) return (num / 10000).toFixed(1) + '万'
  return num.toLocaleString()
}

// 粒子样式
const particleStyle = (index) => {
  const size = Math.random() * 3 + 1
  return {
    width: size + 'px',
    height: size + 'px',
    left: Math.random() * 100 + '%',
    top: Math.random() * 100 + '%',
    animationDelay: Math.random() * 5 + 's',
    animationDuration: (Math.random() * 10 + 10) + 's'
  }
}

// 更新时间
const updateTime = () => {
  const now = new Date()
  currentDate.value = now.toLocaleDateString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    weekday: 'long'
  })
  currentTime.value = now.toLocaleTimeString('zh-CN', {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  })
}

// 更新数据
const updateData = () => {
  weather.value = getWeatherData()
  overview.value = getOverviewData()
  lastUpdate.value = new Date().toLocaleTimeString('zh-CN')
  dataPoints.value += Math.floor(Math.random() * 1000)
}

const toggleSevereWeather = () => {
  setSevereWeatherEnabled(!severeWeatherEnabled.value)
}

```

```

}

const clearHumidityAnomaly = () => {
  humidityAnomaly.value = false
  showHumidityAlert.value = false
  setHumidityAlertDismissed(false)
  showHumidityObservation.value = true
  markHumidityMitigationApplied()
  // 恶劣天气仍在模拟时，允许手动“消除异常”并保持不自动反复弹出
  if (severeWeatherEnabled.value) {
    setHumidityAlertSuppressed(true)
  }
}

const dismissHumidityAlert = () => {
  showHumidityAlert.value = false
  setHumidityAlertDismissed(true)
}

const markHumidityAlertProcessing = () => {
  setHumidityAlertProcessing(true)
  if (alarmAudio) {
    alarmAudio.pause()
    alarmAudio.currentTime = 0
  }
}

const dismissHumidityObservation = () => {
  showHumidityObservation.value = false
}

const currentHumidityText = computed(() => {
  const h = Number(weather.value?.humidity)
  if (!Number.isFinite(h)) return '--'
  return `${h.toFixed(1)}%`
})

const humidityStage = computed(() => {
  const h = Number(weather.value?.humidity)
  if (!Number.isFinite(h)) return 'normal'
  if (h >= HUMIDITY_ALARM) return 'alarm'
  if (h >= HUMIDITY_WARNING) return 'warning'
  return 'normal'
})

const humidityAlertClass = computed(() => {
  if (humidityStage.value === 'alarm') return 'alarm'
  if (humidityStage.value === 'warning') return 'warning'
  return 'normal'
})

watch(severeWeatherEnabled, (enabled) => {
  if (enabled) {
    if (!humidityAlertSuppressed.value) {
      humidityAnomaly.value = true
      if (!humidityAlertDismissed.value) {
        showHumidityAlert.value = true
      }
      showHumidityObservation.value = false
    }
    return
  }
})

// 退出恶劣天气模拟：恢复默认状态

```

```

humidityAnomaly.value = false
showHumidityAlert.value = false
showHumidityObservation.value = false
}, { immediate: true })

watch(
  showHumidityAlert,
  (visible, prev) => {
    if (visible && !prev) {
      maybePlayAlarmOnEnterOrShow()
    }
  },
  { immediate: false }
)

onMounted(() => {
  // 规则：恶劣天气时，进入大数据面板若弹窗未关闭且未“处理中”，每次切换进入都播放一次告警语音
  maybePlayAlarmOnEnterOrShow()
})

watch(
  syncedHumidity,
  (value) => {
    if (!severeWeatherEnabled.value) return
    const h = Number(value)
    if (!Number.isFinite(h)) return
    if (!weather.value) return
    weather.value.humidity = h
    if (Array.isArray(weather.value.humidityTrend) && weather.value.humidityTrend.length) {
      const next = [...weather.value.humidityTrend]
      next[next.length - 1] = h
      weather.value.humidityTrend = next
    }
  },
  { immediate: true }
)

watch(
  () => weather.value?.humidity,
  (value) => {
    if (!showHumidityObservation.value) return
    const h = Number(value)
    if (!Number.isFinite(h)) return
    if (h >= HUMIDITY_WARNING) return

    if (observationAutoHideTimer) return
    observationAutoHideTimer = setTimeout(() => {
      observationAutoHideTimer = null
      const latest = Number(weather.value?.humidity)
      if (showHumidityObservation.value && Number.isFinite(latest) && latest < HUMIDITY_WARNING) {
        showHumidityObservation.value = false
      }
    }, 8000)
  }
)

const humidityAlertChartPoints = computed(() => {
  const data = weather.value?.humidityTrend || []
  if (!Array.isArray(data) || data.length === 0) return []
  const count = data.length
  const width = 540
  const height = 100
  const paddingLeft = 10
  const paddingTop = 20

```

```

const min = 0
const max = 100
const range = Math.max(1, max - min)

return data.map((v, i) => ({
  x: paddingLeft + (i / (count - 1)) * width,
  y: paddingTop + height - ((v - min) / range) * height
}))
})

const humidityAlertLinePoints = computed(() => humidityAlertChartPoints.value.map(p => `${p.x},${p.y}`).join(' '))
const humidityAlertAreaPoints = computed(() => {
  const points = humidityAlertChartPoints.value
  if (points.length === 0) return ''
  const bottom = 130
  const firstX = points[0].x
  const lastX = points[points.length - 1].x
  const line = points.map(p => `${p.x},${p.y}`).join(' ')
  return `${firstX},${bottom} ${line} ${lastX},${bottom}`
})

const humidityAlertLabelPoints = computed(() => {
  const data = weather.value?.humidityTrend || []
  const points = humidityAlertChartPoints.value
  if (!Array.isArray(data) || !data.length || points.length !== data.length) return []

  // 每 3 个点标一个数值 (24h -> 8 个标注), 避免过密
  const step = 3
  const out = []
  for (let i = 0; i < data.length; i += step) {
    out.push({ ...points[i], value: data[i] })
  }
  // 末尾再补一个“最新值”
  const lastIdx = data.length - 1
  if (lastIdx % step !== 0) {
    out.push({ ...points[lastIdx], value: data[lastIdx] })
  }
  return out
})

onMounted(() => {
  updateTime()
  timeTimer = setInterval(updateTime, 1000)
  updateTimer = setInterval(updateData, 5000)

  // 模拟 FPS
  fpsTimer = setInterval(() => {
    fps.value = Math.floor(55 + Math.random() * 10)
  }, 1000)
})

onUnmounted(() => {
  if (timeTimer) clearInterval(timeTimer)
  if (updateTimer) clearInterval(updateTimer)
  if (fpsTimer) clearInterval(fpsTimer)
  if (observationAutoHideTimer) clearTimeout(observationAutoHideTimer)
  if (alarmAudio) {
    alarmAudio.pause()
    alarmAudio.currentTime = 0
    alarmAudio = null
  }
})
</script>

```

```
<style scoped>
.data-panel-container {
  width: 100%;
  height: 100%;
  background: linear-gradient(135deg, #0a1628 0%, #0d2137 50%, #0a1628 100%);
  position: relative;
  overflow: hidden;
  display: flex;
  flex-direction: column;
}

/* 粒子背景 */
.particles-bg {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  pointer-events: none;
  overflow: hidden;
}

.particle {
  position: absolute;
  background: #00d4ff;
  border-radius: 50%;
  opacity: 0.3;
  animation: particleFloat 15s infinite linear;
}

@keyframes particleFloat {
  0% {
    transform: translateY(100vh) translateX(0);
    opacity: 0;
  }
  10% {
    opacity: 0.3;
  }
  90% {
    opacity: 0.3;
  }
  100% {
    transform: translateY(-100vh) translateX(100px);
    opacity: 0;
  }
}

/* 网格背景 */
.grid-bg {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background-image:
    linear-gradient(rgba(0, 200, 255, 0.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0, 200, 255, 0.03) 1px, transparent 1px);
  background-size: 50px 50px;
  pointer-events: none;
}

/* 头部 */
.panel-header {
  height: 70px;
```

```
display: flex;
justify-content: space-between;
align-items: center;
padding: 0 20px;
background: linear-gradient(180deg, rgba(0, 50, 100, 0.8) 0%, transparent 100%);
border-bottom: 1px solid rgba(0, 200, 255, 0.3);
position: relative;
z-index: 10;
}

.panel-header::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  right: 0;
  height: 1px;
  background: linear-gradient(90deg, transparent, #00d4ff, transparent);
}

.header-left, .header-right {
  display: flex;
  align-items: center;
  gap: 20px;
  min-width: 250px;
}

.logo {
  display: flex;
  align-items: center;
  gap: 10px;
}

.logo-icon {
  font-size: 28px;
}

.logo-text {
  font-size: 14px;
  color: #00d4ff;
  font-weight: bold;
}

.header-center {
  text-align: center;
}

.main-title {
  font-size: 24px;
  color: #fff;
  font-weight: bold;
  text-shadow: 0 0 20px rgba(0, 200, 255, 0.5);
  letter-spacing: 8px;
  margin: 0;
}

.title-decorator {
  color: #00d4ff;
  margin: 0 10px;
}

.sub-title {
  font-size: 10px;
  color: rgba(0, 200, 255, 0.6);
}
```

```
    letter-spacing: 3px;
    margin-top: 5px;
}

.header-right {
    justify-content: flex-end;
}

.severe-weather-btn {
    padding: 6px 12px;
    border-radius: 14px;
    border: 1px solid rgba(0, 200, 255, 0.35);
    background: rgba(0, 40, 80, 0.35);
    color: #d6f6ff;
    font-size: 12px;
    cursor: pointer;
    transition: all 0.25s ease;
    white-space: nowrap;
}

.severe-weather-btn:hover {
    border-color: rgba(0, 200, 255, 0.6);
    background: rgba(0, 60, 120, 0.45);
}

.severe-weather-btn.active {
    border-color: rgba(255, 80, 80, 1);
    background: rgba(255, 45, 45, 0.45);
    box-shadow: 0 0 18px rgba(255, 45, 45, 0.55);
    color: #fff;
}

.anomaly-controls {
    display: flex;
    gap: 8px;
}

.anomaly-btn {
    padding: 6px 12px;
    border-radius: 14px;
    border: 1px solid rgba(255, 120, 120, 0.45);
    background: rgba(80, 20, 20, 0.35);
    color: #ffd5d5;
    font-size: 12px;
    cursor: pointer;
    transition: all 0.25s ease;
}

.anomaly-btn:disabled {
    opacity: 0.45;
    cursor: not-allowed;
    filter: grayscale(0.4);
}

.anomaly-btn:hover {
    border-color: rgba(255, 140, 140, 0.7);
    background: rgba(120, 30, 30, 0.45);
}

.anomaly-btn.clear-btn {
    border-color: rgba(255, 120, 120, 0.65);
    background: rgba(120, 20, 20, 0.35);
    color: #fff;
}
```

```
.anomaly-btn.active {
  color: #fff;
  border-color: rgba(255, 80, 80, 1);
  background: rgba(255, 45, 45, 0.45);
  box-shadow: 0 0 18px rgba(255, 45, 45, 0.55);
}

.global-alerts {
  position: absolute;
  top: 84px;
  left: 50%;
  transform: translateX(-50%);
  z-index: 60;
  display: flex;
  flex-direction: column;
  gap: 10px;
  pointer-events: none;
}

.global-alert {
  min-width: 860px;
  max-width: 980px;
  border-radius: 14px;
  border: 2px solid rgba(255, 185, 80, 0.95);
  background: linear-gradient(135deg, rgba(200, 40, 40, 0.96), rgba(210, 150, 30, 0.92));
  box-shadow:
    0 0 26px rgba(255, 60, 60, 0.55),
    0 0 22px rgba(255, 200, 90, 0.45);
  padding: 18px 22px;
  pointer-events: auto;
  animation: globalAlertFlash 0.9s ease-in-out infinite;
}

.global-alert.humidity.warning {
  border-color: rgba(255, 210, 90, 0.95);
  background: linear-gradient(135deg, rgba(180, 90, 20, 0.96), rgba(220, 170, 40, 0.92));
  box-shadow:
    0 0 22px rgba(255, 180, 60, 0.55),
    0 0 18px rgba(255, 230, 120, 0.35);
}

.global-alert.humidity.alarm {
  border-color: rgba(255, 90, 90, 0.98);
  background: linear-gradient(135deg, rgba(200, 25, 25, 0.96), rgba(210, 145, 30, 0.9));
  box-shadow:
    0 0 32px rgba(255, 35, 35, 0.72),
    0 0 18px rgba(255, 220, 110, 0.32);
}

.alert-line {
  margin-top: 6px;
  line-height: 1.5;
}

.global-alert.observe {
  min-width: 860px;
  max-width: 980px;
  border-radius: 14px;
  border: 1px solid rgba(255, 210, 90, 0.55);
  background: linear-gradient(135deg, rgba(40, 60, 90, 0.78), rgba(140, 95, 20, 0.55));
  box-shadow:
    0 0 18px rgba(255, 210, 90, 0.22),
    0 0 18px rgba(0, 212, 255, 0.18);
}
```

```
padding: 12px 18px;
animation: none;
display: flex;
align-items: center;
justify-content: space-between;
gap: 16px;
}

.observe-left {
display: flex;
flex-direction: column;
gap: 4px;
}

.observe-title {
font-size: 16px;
font-weight: 900;
letter-spacing: 1px;
color: rgba(255, 230, 160, 0.98);
}

.observe-desc {
font-size: 14px;
color: rgba(255, 255, 255, 0.92);
}

.observe-close {
border: 1px solid rgba(255, 255, 255, 0.35);
background: rgba(255, 255, 255, 0.10);
color: #fff;
border-radius: 8px;
font-size: 13px;
padding: 6px 14px;
cursor: pointer;
white-space: nowrap;
}

.alert-title {
font-size: 22px;
font-weight: bold;
color: #fff;
margin-bottom: 8px;
letter-spacing: 1px;
}

.alert-desc {
font-size: 16px;
color: rgba(255, 235, 235, 0.95);
}

.alert-target {
font-size: 18px;
font-weight: bold;
color: #fff;
margin-bottom: 6px;
letter-spacing: 0.5px;
text-shadow: 0 0 14px rgba(255, 70, 70, 0.6);
}

.alert-hint {
font-size: 15px;
color: rgba(255, 235, 235, 0.92);
}
```

```
.alert-close {
  margin-top: 14px;
  border: 1px solid rgba(255, 255, 255, 0.4);
  background: rgba(255, 255, 255, 0.12);
  color: #fff;
  border-radius: 8px;
  font-size: 14px;
  padding: 6px 16px;
  cursor: pointer;
}

.alert-handle {
  margin-top: 14px;
  border: 1px solid rgba(0, 212, 255, 0.55);
  background: rgba(0, 212, 255, 0.14);
  color: rgba(235, 252, 255, 0.98);
  border-radius: 8px;
  font-size: 14px;
  padding: 6px 16px;
  cursor: pointer;
}

.alert-handle:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.alert-chart {
  margin-top: 14px;
}

.chart-caption {
  font-size: 13px;
  color: rgba(255, 255, 255, 0.92);
  margin-bottom: 8px;
  font-weight: bold;
  letter-spacing: 1px;
}

.alert-chart-svg {
  width: 100%;
  height: 150px;
  border-radius: 10px;
  background: rgba(0, 0, 0, 0.18);
  border: 1px solid rgba(255, 110, 110, 0.35);
}

.alert-actions {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
}

.alert-pop-enter-active,
.alert-pop-leave-active {
  transition: all 0.25s ease;
}

.alert-pop-enter-from,
.alert-pop-leave-to {
  opacity: 0;
  transform: translateY(-14px) scale(0.96);
}
```

```
@keyframes globalAlertFlash {
  0%, 100% { filter: brightness(1); }
  50% { filter: brightness(1.22); }
}

.datetime {
  text-align: right;
}

.date {
  font-size: 14px;
  color: #888;
}

.time {
  font-size: 22px;
  color: #00d4ff;
  font-weight: bold;
  font-family: monospace;
}

.weather-mini {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 15px;
  background: rgba(0, 50, 100, 0.5);
  border-radius: 20px;
  border: 1px solid rgba(0, 200, 255, 0.3);
}

.weather-icon {
  font-size: 18px;
}

.weather-temp {
  font-size: 16px;
  color: #fff;
  font-weight: bold;
}

/* 主内容 */
.panel-main {
  flex: 1;
  display: flex;
  padding: 10px;
  gap: 10px;
  overflow: hidden;
  position: relative;
  z-index: 5;
}

.left-aside, .right-aside {
  width: 320px;
  flex-shrink: 0;
}

.center-section {
  flex: 1;
  min-width: 0;
}

/* 底部 */
.panel-footer {
```

```
height: 35px;
display: flex;
justify-content: space-between;
align-items: center;
padding: 0 20px;
background: linear-gradient(0deg, rgba(0, 50, 100, 0.8) 0%, transparent 100%);
border-top: 1px solid rgba(0, 200, 255, 0.2);
position: relative;
z-index: 10;
}

.footer-left, .footer-right {
display: flex;
align-items: center;
gap: 20px;
}

.status-item {
display: flex;
align-items: center;
gap: 6px;
font-size: 13px;
color: #888;
}

.status-dot {
width: 8px;
height: 8px;
border-radius: 50%;
animation: statusPulse 2s infinite;
}

.status-dot.online {
background: #00ff88;
box-shadow: 0 0 5px #00ff88;
}

@keyframes statusPulse {
0%, 100% { opacity: 1; }
50% { opacity: 0.5; }
}

.footer-center {
display: flex;
align-items: center;
gap: 15px;
}

.version {
font-size: 12px;
color: #666;
padding: 2px 8px;
background: rgba(0, 50, 100, 0.5);
border-radius: 10px;
}

.copyright {
font-size: 12px;
color: #666;
}

.update-time, .fps {
font-size: 13px;
color: #888;
}
```

```

}

.fps {
  color: #00ff88;
  font-family: monospace;
}

/* 装饰 */
.decorations {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  pointer-events: none;
  z-index: 1;
}

.corner {
  position: absolute;
  width: 30px;
  height: 30px;
  border: 2px solid rgba(0, 200, 255, 0.5);
}

.corner-tl {
  top: 5px;
  left: 5px;
  border-right: none;
  border-bottom: none;
}

.corner-tr {
  top: 5px;
  right: 5px;
  border-left: none;
  border-bottom: none;
}

.corner-bl {
  bottom: 5px;
  left: 5px;
  border-right: none;
  border-top: none;
}

.corner-br {
  bottom: 5px;
  right: 5px;
  border-left: none;
  border-top: none;
}
}
</style>

```

5.4 文件：src/components/datapanel/LeftPanel.vue

- 文件说明：左侧环境与传感器状态面板。
- 行数统计：726 行

```

<template>
  <div class="left-panel">
    <!-- 气象监测 -->
    <div class="panel-section weather-section">
      <div class="section-header">
        <span class="section-icon">🌤️</span>
        <span class="section-title">气象监测</span>

```

```

    <span class="section-badge">{{ weather.weatherType }}</span>
</div>
<div class="weather-grid">
    <div class="weather-item main">
        <div class="weather-icon">
            <span class="temp-display">{{ weather.temperature }}</span>
            <span class="temp-unit">°C</span>
        </div>
        <div class="weather-label">实时温度</div>
    </div>
    <div class="weather-item">
        <div class="weather-value">{{ weather.humidity }}<span class="unit">%</span></div>
        <div class="weather-label">湿度</div>
    </div>
    <div class="weather-item">
        <div class="weather-value">{{ weather.windSpeed }}<span class="unit">m/s</span></div>
        <div class="weather-label">{{ weather.windDirection }}风</div>
    </div>
    <div class="weather-item">
        <div class="weather-value">{{ weather.pressure }}<span class="unit">hPa</span></div>
        <div class="weather-label">气压</div>
    </div>
    <div class="weather-item">
        <div class="weather-value">{{ weather.visibility }}<span class="unit">km</span></div>
        <div class="weather-label">能见度</div>
    </div>
    <div class="weather-item">
        <div class="weather-value">{{ weather.precipitation }}<span class="unit">mm</span></div>
        <div class="weather-label">降水量</div>
    </div>
</div>
<!-- 趋势图（恶劣天气/湿度异常时切换为湿度趋势） -->
<div class="trend-chart">
    <div class="chart-title">{{ trendTitle }}</div>
    <div class="chart-container">
        <svg viewBox="0 0 280 60" class="trend-svg">
            <defs>
                <linearGradient id="tempGradient" x1="0%" y1="0%" x2="0%" y2="100%">
                    <stop offset="0%" :style="`stop-color:${trendColor};stop-opacity:0.3`" />
                    <stop offset="100%" :style="`stop-color:${trendColor};stop-opacity:0`" />
                </linearGradient>
            </defs>
            <path d="trendAreaPath" fill="url(#tempGradient)" />
            <path d="trendLinePath" fill="none" :stroke="trendColor" stroke-width="2.5" />
            <circle v-for="(point, i) in trendPoints" :key="i"
                :cx="point.x" :cy="point.y" r="2" :fill="trendColor" class="chart-dot" />
        </svg>
        <div class="chart-labels">
            <span>00:00</span>
            <span>06:00</span>
            <span>12:00</span>
            <span>18:00</span>
            <span>24:00</span>
        </div>
    </div>
</div>
<!-- 空气湿度 -->
<div class="air-quality">
    <div class="air-quality-header">
        <span class="air-quality-title">空气湿度</span>
    </div>
    <div class="aqi-display">
        <span class="aqi-value" :style="{ color: aqiColor }">{{ humidityShown }}%</span>
        <span class="aqi-label">{{ aqiLabel }}</span>
    </div>
</div>

```

```

    </div>
  </div>
</div>

<!-- 传感器状态 -->
<div class="panel-section sensor-section">
  <div class="section-header">
    <span class="section-icon">📡</span>
    <span class="section-title">传感器监控</span>
    <span class="section-badge online">{{ onlineSensors }}/{{ processedSensors.length }} 在线</span>
  </div>
  <div class="sensor-list">
    <div
      v-for="sensor in displayedSensors"
      :key="sensor.id"
      :class="['sensor-item', sensor.status, { 'anomaly-flash': sensor.isAnomaly }]"
    >
      <div class="sensor-header">
        <span class="sensor-icon">{{ sensorIcon(sensor.icon) }}</span>
        <span class="sensor-name">{{ sensor.name }}</span>
        <span :class="['sensor-status', sensor.status]">{{ statusText(sensor.status) }}</span>
      </div>
      <div class="sensor-value-row">
        <span class="sensor-value" :class="{ abnormal: sensor.isAnomaly }">{{ sensor.value }}</span>
        <span class="sensor-unit">{{ sensor.unit }}</span>
      </div>
      <div class="sensor-meta">
        <span class="sensor-location">📍 {{ sensor.location }}</span>
        <span class="sensor-time">🕒 {{ sensor.updateTime }}</span>
      </div>
      <!-- 传感器迷你趋势图 -->
      <div class="sensor-mini-chart">
        <svg viewBox="0 0 80 20">
          <polyline
            :points="sensorHistoryPoints(sensor.history)"
            fill="none"
            :stroke="sensor.status === 'normal' ? '#00ff88' : sensor.status === 'warning' ? '#ffaa00' : '#ff6b6b'"
            stroke-width="1.5"
          />
        </svg>
      </div>
    </div>
  </div>
</div>

<!-- 环境监测 -->
<div class="panel-section env-section">
  <div class="section-header">
    <span class="section-icon">🌱</span>
    <span class="section-title">环境监测</span>
  </div>
  <div class="env-grid">
    <div class="env-item" v-for="item in envData" :key="item.label">
      <div class="env-gauge">
        <svg viewBox="0 0 60 60">
          <circle cx="30" cy="30" r="25" fill="none" stroke="rgba(255,255,255,0.1)" stroke-width="5" />
          <circle
            cx="30" cy="30" r="25"
            fill="none"
            :stroke="item.color"
            stroke-width="5"
            stroke-linecap="round"
            :stroke-dasharray="157"
            :stroke-dashoffset="157 - (item.value / item.max * 157)"
          />
        </svg>
      </div>
    </div>
  </div>
</div>

```

```

        class="gauge-circle"
      />
      <text x="30" y="30" text-anchor="middle" dominant-baseline="middle" fill="#fff" font-size="12">
        {{ item.value }}
      </text>
    </svg>
  </div>
  <div class="env-label">{{ item.label }}</div>
  <div class="env-unit">{{ item.unit }}</div>
</div>
</div>
</div>
</div>
</template>

<script setup>
import { ref, computed, onMounted, onUnmounted } from 'vue'
import { getWeatherData, getSensorData } from '../services/mockDataService'

const weather = ref(getWeatherData())
const sensors = ref(getSensorData())
const updateTimer = ref(null)

const props = defineProps({
  humidityAnomaly: {
    type: Boolean,
    default: false
  },
  temperatureAnomaly: {
    type: Boolean,
    default: false
  }
})

const processedSensors = computed(() => {
  return sensors.value.map((sensor) => {
    const humidityAnomaly = props.humidityAnomaly && sensor.icon === 'droplet'
    const temperatureAnomaly = props.temperatureAnomaly && sensor.icon === 'thermometer'
    const isAnomaly = humidityAnomaly || temperatureAnomaly

    if (!isAnomaly) {
      return { ...sensor, isAnomaly: false }
    }

    return {
      ...sensor,
      status: 'error',
      isAnomaly: true
    }
  })
})

// 计算在线传感器数量
const onlineSensors = computed(() => {
  return processedSensors.value.filter(s => s.status === 'normal').length
})

// 显示前8个传感器
const displayedSensors = computed(() => processedSensors.value.slice(0, 8))

const trendMode = computed(() => (props.humidityAnomaly ? 'humidity' : 'temperature'))
const trendTitle = computed(() => (trendMode.value === 'humidity' ? '24小时湿度趋势' : '24小时温度趋势'))
const trendColor = computed(() => (trendMode.value === 'humidity' ? '#ff2f2f' : '#00d4ff'))

```

```

const trendData = computed(() => {
  return trendMode.value === 'humidity'
    ? (weather.value.humidityTrend || [])
    : (weather.value.temperatureTrend || [])
})

const trendPointY = (v) => {
  if (trendMode.value === 'humidity') {
    const vv = Math.max(0, Math.min(100, Number(v)))
    return 55 - (vv / 100) * 50
  }
  const vv = Number(v)
  return 55 - ((vv + 5) / 40) * 50
}

const trendLinePath = computed(() => {
  const data = trendData.value
  if (!data.length) return ''
  const points = data.map((v, i) => {
    const x = (i / (data.length - 1)) * 280
    const y = trendPointY(v)
    return `${x},${y}`
  })
  return `M${points.join(' L')}`
})

const trendAreaPath = computed(() => {
  const data = trendData.value
  if (!data.length) return ''
  const points = data.map((v, i) => {
    const x = (i / (data.length - 1)) * 280
    const y = trendPointY(v)
    return `${x},${y}`
  })
  return `M0,60 L${points.join(' L')} L280,60 Z`
})

const trendPoints = computed(() => {
  const data = trendData.value
  if (!data.length) return []
  // 取 6 个点用于 hover 显示（保持原来“稀疏点”效果）
  const step = Math.max(1, Math.floor(data.length / 6))
  return data.filter((_, i) => i % step === 0).map((v, i) => ({
    x: (i * step / (data.length - 1)) * 280,
    y: trendPointY(v)
  }))
})

const humidityShown = computed(() => Math.round(Number(weather.value.humidity) || 0))

// 湿度告警颜色和标签 (<60 正常; 60~70 橙; >=70 红)
const aqiColor = computed(() => {
  const h = humidityShown.value
  if (h < 60) return '#00ff88'
  if (h < 70) return '#ffaa00'
  return '#ff2f2f'
})

const aqiLabel = computed(() => {
  const h = humidityShown.value
  if (h < 60) return '正常'
  if (h < 70) return '橙色告警'
  return '红色告警'
})

```

```

// 传感器图标
const sensorIcon = (type) => {
  const icons = {
    thermometer: '🌡️',
    droplet: '💧',
    gauge: '📊',
    activity: '📍',
    zap: '⚡',
    bolt: '⚡',
    move: '🏠',
    flow: '🌊',
    rotate: '🌀',
    volume: '🔊',
    smoke: '👉',
    sun: '☀️'
  }
  return icons[type] || '🏠'
}

// 状态文本
const statusText = (status) => {
  const texts = {
    normal: '正常',
    warning: '预警',
    error: '故障'
  }
  return texts[status] || status
}

// 传感器历史数据点
const sensorHistoryPoints = (history) => {
  return history.map((v, i) => {
    const x = (i / (history.length - 1)) * 80
    const min = Math.min(...history)
    const max = Math.max(...history)
    const range = max - min || 1
    const y = 18 - ((v - min) / range) * 16
    return `${x},${y}`
  }).join(' ')
}

// 环境监测数据
const envData = computed(() => [
  { label: '噪音', value: random(30, 80), unit: 'dB', max: 120, color: '#00d4ff' },
  { label: '光照', value: random(100, 800), unit: 'lux', max: 1000, color: '#ffaa00' },
  { label: 'CO₂', value: random(300, 600), unit: 'ppm', max: 1000, color: '#00ff88' },
  { label: '粉尘', value: random(0, 50), unit: 'mg/m³', max: 100, color: '#ff6b6b' }
])

const random = (min, max) => Math.floor(Math.random() * (max - min) + min)

// 定时更新数据
onMounted(() => {
  updateTimer.value = setInterval(() => {
    weather.value = getWeatherData()
    sensors.value = getSensorData()
  }, 5000)
})

onUnmounted(() => {
  if (updateTimer.value) {
    clearInterval(updateTimer.value)
  }
})

```

```
})
</script>

<style scoped>
.left-panel {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  gap: 15px;
  overflow-y: auto;
  padding-right: 5px;
}

.left-panel::-webkit-scrollbar {
  width: 4px;
}

.left-panel::-webkit-scrollbar-thumb {
  background: rgba(0, 200, 255, 0.3);
  border-radius: 2px;
}

.panel-section {
  background: linear-gradient(135deg, rgba(0, 30, 60, 0.9) 0%, rgba(0, 50, 100, 0.6) 100%);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 10px;
  padding: 15px;
  backdrop-filter: blur(10px);
}

.section-header {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 15px;
  padding-bottom: 10px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.2);
}

.section-icon {
  font-size: 20px;
}

.section-title {
  font-size: 16px;
  font-weight: bold;
  color: #00d4ff;
  flex: 1;
}

.section-badge {
  font-size: 13px;
  padding: 2px 8px;
  background: rgba(0, 200, 255, 0.2);
  border: 1px solid rgba(0, 200, 255, 0.4);
  border-radius: 10px;
  color: #00d4ff;
}

.section-badge.online {
  background: rgba(0, 255, 136, 0.2);
  border-color: rgba(0, 255, 136, 0.4);
  color: #00ff88;
}
```

```
}

/* 气象区域 */
.weather-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  margin-bottom: 15px;
}

.weather-item {
  background: rgba(0, 50, 100, 0.3);
  border-radius: 8px;
  padding: 10px;
  text-align: center;
}

.weather-item.main {
  grid-column: span 3;
  background: linear-gradient(135deg, rgba(0, 100, 200, 0.3), rgba(0, 200, 255, 0.1));
  border: 1px solid rgba(0, 200, 255, 0.3);
}

.weather-icon {
  display: flex;
  align-items: baseline;
  justify-content: center;
}

.temp-display {
  font-size: 36px;
  font-weight: bold;
  color: #00d4ff;
  text-shadow: 0 0 20px rgba(0, 200, 255, 0.5);
}

.temp-unit {
  font-size: 16px;
  color: #aaa;
  margin-left: 3px;
}

.weather-value {
  font-size: 20px;
  font-weight: bold;
  color: #fff;
}

.weather-value .unit {
  font-size: 10px;
  color: #888;
  margin-left: 2px;
}

.weather-label {
  font-size: 13px;
  color: #888;
  margin-top: 5px;
}

/* 趋势图 */
.trend-chart {
  margin-bottom: 15px;
}
```

```
.chart-title {
  font-size: 14px;
  color: #aaa;
  margin-bottom: 8px;
}

.chart-container {
  position: relative;
}

.trend-svg {
  width: 100%;
  height: 60px;
}

.chart-dot {
  opacity: 0;
  transition: opacity 0.3s;
}

.trend-svg:hover .chart-dot {
  opacity: 1;
}

.chart-labels {
  display: flex;
  justify-content: space-between;
  font-size: 12px;
  color: #666;
  margin-top: 5px;
}

/* 空气湿度 */
.air-quality {
  display: flex;
  flex-direction: column;
  gap: 10px;
  align-items: flex-start;
}

.air-quality-header {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.air-quality-title {
  font-size: 12px;
  font-weight: bold;
  color: #00d4ff;
  letter-spacing: 1px;
}

.aqi-display {
  text-align: left;
  min-width: 0;
}

.aqi-value {
  font-size: 34px;
  font-weight: bold;
  display: block;
```

```
}

.aqi-label {
  font-size: 14px;
  color: #888;
}

.pollutants {
  flex: 1;
}

.pollutant-item {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 6px;
}

.pollutant-label {
  font-size: 10px;
  color: #888;
  width: 35px;
}

.pollutant-bar {
  flex: 1;
  height: 4px;
  background: rgba(255, 255, 255, 0.1);
  border-radius: 2px;
  overflow: hidden;
}

.pollutant-fill {
  height: 100%;
  border-radius: 2px;
  transition: width 0.5s ease;
}

.pollutant-value {
  font-size: 10px;
  color: #aaa;
  width: 30px;
  text-align: right;
}

/* 传感器列表 */
.sensor-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
  max-height: 300px;
  overflow-y: auto;
}

.sensor-list::-webkit-scrollbar {
  width: 3px;
}

.sensor-list::-webkit-scrollbar-thumb {
  background: rgba(0, 200, 255, 0.3);
  border-radius: 2px;
}

.sensor-item {
```

```
background: rgba(0, 50, 100, 0.2);
border-radius: 8px;
padding: 10px;
border-left: 3px solid;
transition: all 0.3s;
}

.sensor-item.normal {
  border-left-color: #00ff88;
}

.sensor-item.warning {
  border-left-color: #ffaa00;
  background: rgba(255, 170, 0, 0.1);
}

.sensor-item.error {
  border-left-color: #ff6b6b;
  background: rgba(255, 107, 107, 0.1);
}

.sensor-item.anomaly-flash {
  border-left-color: #ff2424;
  background: rgba(255, 30, 30, 0.24);
  box-shadow: 0 0 18px rgba(255, 40, 40, 0.55), inset 0 0 12px rgba(255, 20, 20, 0.3);
  animation: anomalyFlash 0.95s infinite;
}

.sensor-value.abnormal {
  color: #ff4040;
  text-shadow: 0 0 10px rgba(255, 64, 64, 0.7);
}

@keyframes anomalyFlash {
  0%, 100% { filter: brightness(1); }
  50% { filter: brightness(1.35); }
}

.sensor-header {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 5px;
}

.sensor-icon {
  font-size: 14px;
}

.sensor-name {
  font-size: 14px;
  color: #fff;
  flex: 1;
}

.sensor-status {
  font-size: 12px;
  padding: 2px 6px;
  border-radius: 4px;
}

.sensor-status.normal {
  background: rgba(0, 255, 136, 0.2);
  color: #00ff88;
```

```
}

.sensor-status.warning {
  background: rgba(255, 170, 0, 0.2);
  color: #ffaa00;
}

.sensor-status.error {
  background: rgba(255, 107, 107, 0.2);
  color: #ff6b6b;
}

.sensor-value-row {
  margin-bottom: 5px;
}

.sensor-value {
  font-size: 22px;
  font-weight: bold;
  color: #00d4ff;
}

.sensor-unit {
  font-size: 13px;
  color: #888;
  margin-left: 4px;
}

.sensor-meta {
  display: flex;
  justify-content: space-between;
  font-size: 12px;
  color: #666;
  margin-bottom: 5px;
}

.sensor-mini-chart {
  height: 20px;
}

.sensor-mini-chart svg {
  width: 100%;
  height: 100%;
}

/* 环境监测 */
.env-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
}

.env-item {
  text-align: center;
}

.env-gauge {
  width: 50px;
  height: 50px;
  margin: 0 auto 5px;
}

.gauge-circle {
  transform: rotate(-90deg);
```

```

transform-origin: center;
transition: stroke-dashoffset 1s ease;
}

.env-label {
  font-size: 12px;
  color: #aaa;
}

.env-unit {
  font-size: 11px;
  color: #666;
}
</style>

```

5.5 文件: src/components/datapanel/CenterPanel.vue

- 文件说明: 中部调度总览、寿命预测和线路态势面板。
- 行数统计: 1631 行

```

<template>
  <div class="center-panel">
    <!-- 顶部核心指标 -->
    <div class="top-metrics">
      <div class="metric-card" v-for="metric in keyMetrics" :key="metric.label">
        <div class="metric-icon">{{ metric.icon }}</div>
        <div class="metric-content">
          <div class="metric-value">
            <span class="value">{{ metric.value }}</span>
            <span class="unit">{{ metric.unit }}</span>
          </div>
          <div class="metric-label">{{ metric.label }}</div>
          <div class="metric-trend" :class="metric.trend >= 0 ? 'up' : 'down'">
            <span class="trend-icon">{{ metric.trend >= 0 ? '↑' : '↓' }}</span>
            <span class="trend-value">{{ Math.abs(metric.trend) }}%</span>
          </div>
        </div>
      </div>
    </div>

    <!-- 中间设备寿命预测区域 -->
    <div class="main-visualization">
      <!-- 标题 -->
      <div class="viz-header">
        <h2 class="viz-title">设备寿命预测</h2>
        <div class="viz-controls">
          <!-- 缩放控制 -->
          <div class="zoom-controls">
            <button class="zoom-btn" @click="zoomIn" title="放大">
              <svg viewBox="0 0 24 24" width="16" height="16">
                <path fill="currentColor" d="M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/>
              </svg>
            </button>
            <span class="zoom-level">{{ Math.round(zoomLevel * 100) }}%</span>
            <button class="zoom-btn" @click="zoomOut" title="缩小">
              <svg viewBox="0 0 24 24" width="16" height="16">
                <path fill="currentColor" d="M19 13H5v-2h14v2z"/>
              </svg>
            </button>
            <button class="zoom-btn reset-btn" @click="resetZoom" title="重置">
              <svg viewBox="0 0 24 24" width="16" height="16">
                <path fill="currentColor" d="M12 5V1L7 6L5 7C3.31 0 6 2.69 6 6s-2.69 6-6 6-6-2.69-6-6H4c0 4.42 3.58 8 8 8s8-3.58 8-8-3.58-8-8-8z"/>
              </svg>
            </button>
            <button class="zoom-btn fullscreen-btn" @click="toggleFullscreen" title="全屏">

```

```

        <svg viewBox="0 0 24 24" width="16" height="16">
            <path fill="currentColor" d="M7 14H5v5h5v-2H7v-3zm-2-4h2V7h3V5H5v5zm12 7h-3v2h5v-5h-2v3zM14 5v2h3v3h2V5h-5z"/>
        </svg>
    </button>
</div>
</div>
<div class="viz-time">{{ currentTime }}</div>
</div>

<!-- 设备寿命预测 - 可缩放表格 -->
<div class="life-container">
    <div class="canvas-wrapper" :style="canvasStyle">
        <div class="life-table">
            <div class="life-table-header">
                <div class="life-tip">
                    <span class="dot"></span>
                    <span>基于启用时间、磨损模型与历史维护记录推算（演示数据）</span>
                </div>
                <div class="life-summary">
                    <div class="sum-item">
                        <div class="sum-value warning">{{ lifeSummary.riskCount }}</div>
                        <div class="sum-label">高风险</div>
                    </div>
                    <div class="sum-item">
                        <div class="sum-value">{{ lifeSummary.normalCount }}</div>
                        <div class="sum-label">正常</div>
                    </div>
                    <div class="sum-item">
                        <div class="sum-value">{{ lifeSummary.avgWear }}%</div>
                        <div class="sum-label">平均磨损</div>
                    </div>
                </div>
            </div>
        </div>

        <div
            class="life-table-scroll"
            ref="lifeScrollRef"
            @mouseenter="pauselifeAutoScroll"
            @mouseleave="resumeLifeAutoScroll"
        >
            <table class="life-table-grid">
                <thead>
                    <tr>
                        <th class="col-name">部件</th>
                        <th class="col-time">启用时间</th>
                        <th class="col-duration">使用时长</th>
                        <th class="col-wear">磨损程度</th>
                        <th class="col-life">预计寿命</th>
                        <th class="col-status">状态</th>
                    </tr>
                </thead>
                <tbody>
                    <tr v-for="(row, idx) in lifeRowsLoop" :key="'${row.id}-${idx}'" :class="row.level">
                        <td class="col-name">
                            <div class="name-main">{{ row.name }}</div>
                            <div class="name-sub">{{ row.note }}</div>
                        </td>
                        <td class="col-time">{{ row.enableAt }}</td>
                        <td class="col-duration">{{ row.usage }}</td>
                        <td class="col-wear">
                            <div class="wear-wrap">
                                <div class="wear-bar">
                                    <div class="wear-fill" :style="{ width: row.wear + '%' }"></div>
                                </div>
                            </div>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>

```

```

        <div class="wear-text">{{ row.wear }}%</div>
      </div>
    </td>
    <td class="col-life">
      <div class="life-main">{{ row.eta }}</div>
      <div class="life-sub">剩余 {{ row.remaining }}</div>
    </td>
    <td class="col-status">
      <span class="status-pill" :class="row.level">{{ row.status }}</span>
    </td>
  </tr>
</tbody>
</table>
</div>
</div>
</div>
</div>

<!-- 信号调度实时监控（已取消，保留代码便于回退） -->
<template v-if="false">
<!-- 信号调度图 - 可缩放画布 -->
<div class="dispatch-container" ref="dispatchContainer">
  @wheel.prevent="handleWheel"
  @mousedown="startDrag"
  @mousemove="onDrag"
  @mouseup="endDrag"
  @mouseleave="endDrag">
  <div class="canvas-wrapper">
    :style="canvasStyle"
    ref="canvasWrapper">
    <svg :viewBox="'0 0 ${canvasWidth} ${canvasHeight}'" class="dispatch-svg" ref="dispatchSvg">
    <defs>
      <!-- 信号灯发光效果 -->
      <filter id="glow-green">
        <feGaussianBlur stdDeviation="3" result="coloredBlur"/>
        <feMerge>
          <feMergeNode in="coloredBlur"/>
          <feMergeNode in="SourceGraphic"/>
        </feMerge>
      </filter>
      <filter id="glow-red">
        <feGaussianBlur stdDeviation="3" result="coloredBlur"/>
        <feMerge>
          <feMergeNode in="coloredBlur"/>
          <feMergeNode in="SourceGraphic"/>
        </feMerge>
      </filter>
      <!-- 列车动画 -->
      <linearGradient id="trainGradient" x1="0%" y1="0%" x2="100%" y2="0%">
        <stop offset="0%" style="stop-color:#00d4ff;stop-opacity:1" />
        <stop offset="100%" style="stop-color:#00ff88;stop-opacity:1" />
      </linearGradient>
    </defs>

    <!-- 线路背景网格 -->
    <g class="grid-lines">
      <line v-for="i in 9" :key="'h'+i" :x1="50" :y1="i * 40 + 20" :x2="750" :y2="i * 40 + 20"
        stroke="rgba(0,200,255,0.1)" stroke-width="1" />
      <line v-for="i in 15" :key="'v'+i" :x1="i * 50" :y1="20" :x2="i * 50" :y2="380"
        stroke="rgba(0,200,255,0.1)" stroke-width="1" />
    </g>

    <!-- 线路 -->
    <g class="rail-lines">

```

```
<!-- 线路示意 -->
<g class="line-group" v-for="(line, idx) in displayLines" :key="line.id">
  <line :x1="60" :y1="60 + idx * 70" :x2="740" :y2="60 + idx * 70"
    :stroke="line.color" stroke-width="4" stroke-linecap="round"
    :opacity="0.6" />
  <text :x="30" :y="65 + idx * 70" fill="#888" font-size="10">{{ line.shortName }}</text>
</g>
</g>

<!-- 轨道区段 -->
<g class="track-sections">
  <rect v-for="section in displaySections" :key="section.id"
    :x="section.x" :y="section.y" :width="section.width" :height="8"
    :fill="getSectionColor(section.state)" :opacity="0.8" rx="2"
    class="track-section" :class="section.state">
    <animate v-if="section.state === '占用'" attributeName="opacity"
      values="0.8;1;0.8" dur="1s" repeatCount="indefinite" />
  </rect>
</g>

<!-- 信号机 -->
<g class="signals">
  <g v-for="signal in displaySignals" :key="signal.id"
    :transform="`translate(${signal.x}, ${signal.y})`"
    class="signal-group" @click="showSignalInfo(signal)">
    <!-- 信号机柱 -->
    <line x1="0" y1="0" x2="0" y2="-20" stroke="#555" stroke-width="2" />
    <!-- 信号灯 -->
    <circle cx="0" cy="-25" r="6" :fill="signal.color"
      :filter="signal.state === '开放' ? 'url(#glow-green)' : 'url(#glow-red)'">
      <animate v-if="signal.state === '开放'" attributeName="r"
        values="6;7;6" dur="1.5s" repeatCount="indefinite" />
    </circle>
    <circle cx="0" cy="-38" r="4" :fill="signal.state === '开放' ? '#333' : '#ffaa00'" />
    <circle cx="0" cy="-48" r="4" :fill="signal.state === '关闭' ? '#ff6b6b' : '#333'" />
    <!-- 信号机编号 -->
    <text x="8" y="-30" fill="#888" font-size="8">{{ signal.id }}</text>
  </g>
</g>

<!-- 道岔 -->
<g class="switches">
  <g v-for="sw in displaySwitches" :key="sw.id"
    :transform="`translate(${sw.x}, ${sw.y})`" class="switch-group">
    <!-- 道岔表示 -->
    <polygon v-if="sw.state === '定位'" points="-8,0 8,0 0,-12"
      :fill="sw.locked ? '#00ff88' : '#00d4ff'" />
    <polygon v-else-if="sw.state === '反位'" points="-8,0 8,0 8,-12"
      :fill="sw.locked ? '#00ff88' : '#ffaa00'" />
    <polygon v-else points="-8,0 8,0 0,-12" fill="#ff6b6b" />
    <circle cx="0" cy="4" r="3" :fill="sw.locked ? '#ffaa00' : '#555'" />
    <text x="10" y="0" fill="#888" font-size="7">{{ sw.id }}</text>
  </g>
</g>

<!-- 列车 -->
<g class="trains">
  <g v-for="train in displayTrains" :key="train.id"
    :transform="`translate(${train.x}, ${train.y})`"
    class="train-group" :class="train.state">
    <!-- 列车车身 -->
    <rect x="-15" y="-8" width="30" height="16" rx="4"
      :fill="train.color" class="train-body">
      <animate attributeName="x" values="-15;-12;-15" dur="0.5s"

```

```

        repeatCount="indefinite" v-if="train.state === '正常运行'" />
    </rect>
    <!-- 列车编号 -->
    <text x="0" y="3" text-anchor="middle" fill="#fff" font-size="8" font-weight="bold">
        {{ train.id }}
    </text>
    <!-- 状态指示灯 -->
    <circle cx="12" cy="-5" r="3" :fill="getTrainStatusColor(train.state)" />
    <!-- 运动轨迹 -->
    <line v-if="train.direction === '上行'" x1="-20" y1="0" x2="-30" y2="0"
        stroke="rgba(0,212,255,0.5)" stroke-width="2" stroke-dasharray="4,2">
        <animate attributeName="stroke-dashoffset" values="0;-6" dur="0.5s" repeatCount="indefinite" />
    </line>
</g>
</g>

<!-- 进路线 -->
<g class="routes">
    <line v-for="route in activeRoutes" :key="route.id"
        :x1="route.x1" :y1="route.y1" :x2="route.x2" :y2="route.y2"
        stroke="#00ff88" stroke-width="2" stroke-dasharray="8,4" opacity="0.6">
        <animate attributeName="stroke-dashoffset" values="0;-12" dur="1s" repeatCount="indefinite" />
    </line>
</g>

<!-- 图例 -->
<g class="legend" transform="translate(620, 350)">
    <text x="0" y="0" fill="#888" font-size="10">图例:</text>
    <circle cx="10" cy="15" r="4" fill="#00ff88" />
    <text x="20" y="18" fill="#888" font-size="9">信号开放</text>
    <circle cx="70" cy="15" r="4" fill="#ff6b6b" />
    <text x="80" y="18" fill="#888" font-size="9">信号关闭</text>
    <rect x="5" y="28" width="20" height="6" fill="#00ff88" rx="2" />
    <text x="30" y="34" fill="#888" font-size="9">空闲</text>
    <rect x="60" y="28" width="20" height="6" fill="#ff6b6b" rx="2" />
    <text x="85" y="34" fill="#888" font-size="9">占用</text>
</g>
</svg>
</div>

<!-- 小地图导航 -->
<div class="minimap">
    <svg viewBox="0 0 800 400" class="minimap-svg">
        <!-- 简化的线路 -->
        <g class="minimap-lines" opacity="0.5">
            <line v-for="(line, idx) in displayLines" :key="line.id"
                x1="60" :y1="60 + idx * 70" x2="740" :y2="60 + idx * 70"
                :stroke="line.color" stroke-width="2" />
        </g>
        <!-- 视口指示器 -->
        <rect class="viewport-indicator"
            :x="viewportX" :y="viewportY"
            :width="viewportWidth" :height="viewportHeight"
            fill="rgba(0, 200, 255, 0.2)"
            stroke="#00d4ff"
            stroke-width="2"
            @mousedown="startMinimapDrag" />
    </svg>
</div>
</div>

<!-- 调度统计栏 -->
<div class="dispatch-stats">
    <div class="stat-item" v-for="stat in dispatchStats" :key="stat.label">

```

```

        <span class="stat-value" :style="{ color: stat.color }">{{ stat.value }}</span>
        <span class="stat-label">{{ stat.label }}</span>
    </div>
</div>
</template>
</div>

<!-- 底部数据面板 -->
<div class="bottom-panels">
    <!-- 饼图区域 -->
    <div class="bottom-panel pie-panel">
        <div class="panel-title">信号设备状态分布</div>
        <div class="pie-charts">
            <div class="pie-item" v-for="pie in pieData" :key="pie.title">
                <div class="pie-chart-wrapper">
                    <svg viewBox="0 0 100 100" class="pie-svg">
                        <!-- 背景圆 -->
                        <circle cx="50" cy="50" r="35" fill="none" stroke="rgba(255,255,255,0.1)" stroke-width="15" />
                        <!-- 数据扇形 -->
                        <g class="pie-slices">
                            <circle v-for="(slice, i) in pie.data" :key="i"
                                cx="50" cy="50" r="35"
                                fill="none"
                                :stroke="slice.color"
                                stroke-width="15"
                                :stroke-dasharray="slice.dashArray"
                                :stroke-dashoffset="slice.offset"
                                class="pie-slice"
                            />
                        </g>
                        <!-- 中心数值 -->
                        <text x="50" y="45" text-anchor="middle" fill="#fff" font-size="14" font-weight="bold">
                            {{ pie.total }}
                        </text>
                        <text x="50" y="60" text-anchor="middle" fill="#888" font-size="8">
                            {{ pie.unit }}
                        </text>
                    </svg>
                </div>
                <div class="pie-title">{{ pie.title }}</div>
                <div class="pie-legend">
                    <div v-for="item in pie.data" :key="item.name" class="legend-row">
                        <span class="legend-dot" :style="{ background: item.color }"></span>
                        <span class="legend-name">{{ item.name }}</span>
                        <span class="legend-value">{{ item.value }}</span>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <!-- 调度命令 -->
    <div class="bottom-panel command-panel">
        <div class="panel-title">实时调度命令</div>
        <div class="command-list">
            <div v-for="cmd in dispatchCommands" :key="cmd.id"
                :class="['command-item', cmd.status]">
                <div class="cmd-header">
                    <span class="cmd-type">{{ cmd.type }}</span>
                    <span class="cmd-train">{{ cmd.trainId }}</span>
                    <span :class="['cmd-status', cmd.status]">{{ cmd.status }}</span>
                </div>
                <div class="cmd-content">{{ cmd.content }}</div>
                <div class="cmd-meta">

```

```

        <span>{{ cmd.issuer }}</span>
      </span>{{ cmd.time }}</span>
    </div>
  </div>
</div>
</div>

<!-- 调度效率 -->
<div class="bottom-panel efficiency-panel">
  <div class="panel-title">调度效率统计</div>
  <div class="efficiency-metrics">
    <div class="eff-item">
      <div class="eff-ring">
        <svg viewBox="0 0 60 60">
          <circle cx="30" cy="30" r="24" fill="none" stroke="rgba(255,255,255,0.1)" stroke-width="6" />
          <circle cx="30" cy="30" r="24" fill="none" stroke="#00ff88" stroke-width="6"
            stroke-linecap="round" :stroke-dasharray="150.8"
            :stroke-dashoffset="150.8 - (efficiency.successRate / 100 * 150.8)"
            class="eff-circle" />
          <text x="30" y="28" text-anchor="middle" fill="#fff" font-size="12" font-weight="bold">
            {{ efficiency.successRate }}
          </text>
          <text x="30" y="40" text-anchor="middle" fill="#888" font-size="8">%</text>
        </svg>
      </div>
      <div class="eff-label">调度成功率</div>
    </div>
    <div class="eff-item">
      <div class="eff-value">{{ efficiency.avgResponseTime }}<span class="unit">s</span></div>
      <div class="eff-label">平均响应时间</div>
    </div>
    <div class="eff-item">
      <div class="eff-value">{{ efficiency.avgRouteTime }}<span class="unit">s</span></div>
      <div class="eff-label">进路建立时间</div>
    </div>
    <div class="eff-item">
      <div class="eff-value">{{ efficiency.todayStats.totalCommands }}</div>
      <div class="eff-label">今日命令数</div>
    </div>
  </div>
</div>
<!-- 趋势图 -->
<div class="trend-chart">
  <div class="chart-title">近7日调度趋势</div>
  <div class="chart-bars">
    <div v-for="(day, i) in efficiency.weeklyTrend" :key="i" class="bar-group">
      <div class="bar" :style="{ height: (day.trains / 500 * 50) + 'px' }">
        <span class="bar-tooltip">{{ day.trains }}列</span>
      </div>
      <span class="bar-label">{{ day.date }}</span>
    </div>
  </div>
</div>
</div>
</div>
</div>
</template>

<script setup>
import { ref, computed, onMounted, onUnmounted, nextTick, watch } from 'vue'
import { getSignalDispatchData, getSignalDistribution, getDispatchEfficiency } from '../services/mockDataService'

const dispatchData = ref(getSignalDispatchData())
const signalDist = ref(getSignalDistribution())
const efficiency = ref(getDispatchEfficiency())
```

```
const currentTime = ref('')
const updateTimer = ref(null)
const lifeTimer = ref(null)
const lifeScrollRef = ref(null)
const lifeRowsLoop = computed(() => {
  const rows = lifeRows.value || []
  return rows.length ? [...rows, ...rows] : []
})
let lifeAutoScrollRafId = 0
let lifeAutoScrollLastAt = 0
let lifeAutoScrollPaused = false

const stopLifeAutoScroll = () => {
  if (lifeAutoScrollRafId) {
    cancelAnimationFrame(lifeAutoScrollRafId)
    lifeAutoScrollRafId = 0
  }
  lifeAutoScrollLastAt = 0
}

const pauseLifeAutoScroll = () => {
  lifeAutoScrollPaused = true
}

const resumeLifeAutoScroll = () => {
  lifeAutoScrollPaused = false
}

const startLifeAutoScroll = () => {
  stopLifeAutoScroll()
  let initialized = false
  const step = (ts) => {
    const el = lifeScrollRef.value
    if (!el) {
      lifeAutoScrollRafId = requestAnimationFrame(step)
      return
    }
    if (!lifeAutoScrollLastAt) {
      lifeAutoScrollLastAt = ts
      lifeAutoScrollRafId = requestAnimationFrame(step)
      return
    }

    const deltaMs = ts - lifeAutoScrollLastAt
    lifeAutoScrollLastAt = ts
    if (lifeAutoScrollPaused) {
      lifeAutoScrollRafId = requestAnimationFrame(step)
      return
    }

    const maxScrollTop = Math.max(0, el.scrollHeight - el.clientHeight)
    if (maxScrollTop <= 0) {
      lifeAutoScrollRafId = requestAnimationFrame(step)
      return
    }

    const half = Math.floor(el.scrollHeight / 2)
    if (!initialized && half > 0) {
      el.scrollTop = 0
      initialized = true
    }

    const speedPxPerS = 35
    el.scrollTop += (deltaMs / 1000) * speedPxPerS
  }
}
```

```

    if (half > 0 && el.scrollTop >= half) {
      el.scrollTop = 0
    }

    lifeAutoScrollRafId = requestAnimationFrame(step)
  }
  lifeAutoScrollRafId = requestAnimationFrame(step)
}

const toYmd = (date) => {
  const y = date.getFullYear()
  const m = String(date.getMonth() + 1).padStart(2, '0')
  const d = String(date.getDate()).padStart(2, '0')
  return `${y}-${m}-${d}`
}

const formatDuration = (ms) => {
  const totalHours = Math.max(0, Math.floor(ms / 3600000))
  const days = Math.floor(totalHours / 24)
  const hours = totalHours % 24
  return `${days}天${hours}小时`
}

const clamp = (v, min, max) => Math.max(min, Math.min(max, v))

const baseLifeItems = [
  { id: 'filament', name: '信号灯灯丝寿命', note: '热疲劳 / 点亮次数', designDays: 180 },
  { id: 'bracket', name: 'X站101号信号机支架', note: '结构疲劳 / 振动', designDays: 3650 },
  { id: 'shell', name: 'X站101号信号机保护外壳', note: '紫外老化 / 冲击', designDays: 2400 },
  { id: 'transformer', name: 'X站101号信号机变压器', note: '温升老化 / 绝缘退化', designDays: 3000 },
  { id: 'seal', name: 'X站101号信号机防水胶套', note: '材料疲劳 / 密封退化', designDays: 900 }
]

const lifeBase = ref(baseLifeItems.map((it, idx) => {
  // 给每个部件一个不同启用时间与初始磨损, 便于展示
  const enable = new Date(Date.now() - (30 + idx * 120) * 86400000)
  const initWear = clamp(Math.round(12 + idx * 10 + Math.random() * 8), 1, 85)
  // 环境系数: 越大磨损越快(演示用)
  const env = 0.8 + Math.random() * 0.7
  return {
    ...it,
    enableAt: toYmd(enable),
    enableMs: enable.getTime(),
    wear0: initWear,
    env
  }
})))

const lifeRows = ref([])

const updateLifeRows = () => {
  const now = Date.now()
  const t = now * 0.001
  const rows = lifeBase.value.map((it, i) => {
    const usageMs = now - it.enableMs
    const days = usageMs / 86400000

    // 磨损随时间缓慢上升 + 小幅波动(演示), 上限 99
    const drift = (days / Math.max(30, it.designDays)) * 38 * it.env
    const wobble = Math.sin(t * (0.7 + i * 0.15)) * 1.8 + Math.sin(t * 1.9) * 0.6
    const wear = clamp(Math.round(it.wear0 + drift + wobble), 1, 99)

    const wearRatio = wear / 100

```

```

const estimatedTotalDays = Math.max(1, Math.round(it.designDays * (1.05 - it.env * 0.08)))
const remainingDays = clamp(Math.round(estimatedTotalDays * (1 - wearRatio)), 0, estimatedTotalDays)

const level = wear >= 80 ? 'danger' : wear >= 65 ? 'warning' : 'normal'
const status = level === 'danger' ? '需更换' : level === 'warning' ? '建议检修' : '健康'

return {
  id: it.id,
  name: it.name,
  note: it.note,
  enableAt: it.enableAt,
  usage: formatDuration(usageMs),
  wear,
  eta: `${estimatedTotalDays}天`,
  remaining: `${remainingDays}天`,
  level,
  status
}
})

// 高风险置顶，其余按磨损倒序
rows.sort((a, b) => {
  const rank = (x) => (x.level === 'danger' ? 2 : x.level === 'warning' ? 1 : 0)
  const r = rank(b) - rank(a)
  return r !== 0 ? r : b.wear - a.wear
})
lifeRows.value = rows
}

const lifeSummary = computed(() => {
  const rows = lifeRows.value
  const riskCount = rows.filter(r => r.level === 'danger' || r.level === 'warning').length
  const normalCount = rows.filter(r => r.level === 'normal').length
  const avgWear = rows.length
    ? Math.round(rows.reduce((s, r) => s + r.wear, 0) / rows.length)
    : 0
  return { riskCount, normalCount, avgWear }
})

// 缩放和平移状态
const dispatchContainer = ref(null)
const canvasWrapper = ref(null)
const dispatchSvg = ref(null)
const canvasWidth = 800
const canvasHeight = 400

const zoomLevel = ref(1)
const panX = ref(0)
const panY = ref(0)
const isDragging = ref(false)
const dragStart = ref({ x: 0, y: 0 })
const isMinimapDragging = ref(false)

const minZoom = 0.5
const maxZoom = 3
const zoomStep = 0.2

// 画布样式
const canvasStyle = computed(() => ({
  transform: `translate(${panX.value}px, ${panY.value}px) scale(${zoomLevel.value})`,
  transformOrigin: 'center center',
  transition: isDragging.value ? 'none' : 'transform 0.1s ease-out'
}))

```

```

// 小地图视口计算
const viewportWidth = computed(() => {
  if (!dispatchContainer.value) return 800
  return (dispatchContainer.value.clientWidth / zoomLevel.value) * (800 / canvasWidth)
})

const viewportHeight = computed(() => {
  if (!dispatchContainer.value) return 400
  return (dispatchContainer.value.clientHeight / zoomLevel.value) * (400 / canvasHeight)
})

const viewportX = computed(() => {
  return (-panX.value / zoomLevel.value) * (800 / canvasWidth) + 50
})

const viewportY = computed(() => {
  return (-panY.value / zoomLevel.value) * (400 / canvasHeight)
})

// 缩放控制
const zoomIn = () => {
  if (zoomLevel.value < maxZoom) {
    zoomLevel.value = Math.min(maxZoom, zoomLevel.value + zoomStep)
  }
}

const zoomOut = () => {
  if (zoomLevel.value > minZoom) {
    zoomLevel.value = Math.max(minZoom, zoomLevel.value - zoomStep)
  }
}

const resetZoom = () => {
  zoomLevel.value = 1
  panX.value = 0
  panY.value = 0
}

// 鼠标滚轮缩放
const handleWheel = (e) => {
  const delta = e.deltaY > 0 ? -zoomStep : zoomStep
  const newZoom = Math.max(minZoom, Math.min(maxZoom, zoomLevel.value + delta))

  if (newZoom !== zoomLevel.value) {
    // 以鼠标位置为中心缩放
    const rect = dispatchContainer.value.getBoundingClientRect()
    const mouseX = e.clientX - rect.left - rect.width / 2
    const mouseY = e.clientY - rect.top - rect.height / 2

    const zoomRatio = newZoom / zoomLevel.value
    panX.value = mouseX - (mouseX - panX.value) * zoomRatio
    panY.value = mouseY - (mouseY - panY.value) * zoomRatio

    zoomLevel.value = newZoom
  }
}

// 拖拽平移
const startDrag = (e) => {
  if (e.target.closest('.minimap') || e.button !== 0) return
  isDragging.value = true
  dragStart.value = { x: e.clientX - panX.value, y: e.clientY - panY.value }
}

```

```

const onDrag = (e) => {
  if (!isDragging.value) return
  panX.value = e.clientX - dragStart.value.x
  panY.value = e.clientY - dragStart.value.y
}

const endDrag = () => {
  isDragging.value = false
  isMinimapDragging.value = false
}

// 小地图拖拽
const startMinimapDrag = (e) => {
  e.stopPropagation()
  isMinimapDragging.value = true
  updatePanFromMinimap(e)
}

const updatePanFromMinimap = (e) => {
  if (!isMinimapDragging.value || !dispatchContainer.value) return

  const minimap = e.target.closest('.minimap')
  if (!minimap) return

  const rect = minimap.getBoundingClientRect()
  const scaleX = 800 / rect.width
  const scaleY = 400 / rect.height

  const centerX = (e.clientX - rect.left) * scaleX - viewportWidth.value / 2
  const centerY = (e.clientY - rect.top) * scaleY - viewportHeight.value / 2

  panX.value = -centerX * zoomLevel.value * (canvasWidth / 800)
  panY.value = -centerY * zoomLevel.value * (canvasHeight / 400)
}

// 全屏切换
const toggleFullscreen = () => {
  const container = dispatchContainer.value?.parentElement
  if (!container) return

  if (!document.fullscreenElement) {
    container.requestFullscreen?.() || container.webkitRequestFullscreen?.()
  } else {
    document.exitFullscreen?.() || document.webkitExitFullscreen?.()
  }
}

// 核心指标
const keyMetrics = computed(() => [
  { label: '信号开放率', value: ((dispatchData.value.stats.openSignals / dispatchData.value.stats.totalSignals) * 100).toFixed(1), unit: '%', trend: 0.5, icon: '🚦' },
  { label: '列车运行数', value: dispatchData.value.stats.runningTrains, unit: '列', trend: 0.3, icon: '🚂' },
  { label: '进路激活数', value: dispatchData.value.stats.activeRoutes, unit: '条', trend: -0.2, icon: '🚦' },
  { label: '调度准点率', value: efficiency.value.todayStats.punctualityRate, unit: '%', trend: 0.1, icon: '🕒' }
])

// 显示用的线路
const displayLines = computed(() => [
  { id: 'L1', shortName: 'XX1', color: '#00d4ff' },
  { id: 'L2', shortName: 'XX2', color: '#00ff88' },
  { id: 'L3', shortName: 'XX3', color: '#ffaa00' },
  { id: 'L4', shortName: 'XX4', color: '#ff6b6b' },
  { id: 'L5', shortName: 'XX5', color: '#aa88ff' }
])

```

```

// 显示用的轨道区段
const displaySections = computed(() => {
  const sections = []
  for (let line = 0; line < 5; line++) {
    for (let i = 0; i < 10; i++) {
      const states = ['空闲', '空闲', '空闲', '占用', '锁闭']
      sections.push({
        id: `GJ-${line * 10 + i + 1}`.padStart(6, '0'),
        x: 80 + i * 65,
        y: 56 + line * 70,
        width: 55,
        state: states[Math.floor(Math.random() * states.length)]
      })
    }
  }
  return sections
})

// 显示用的信号机
const displaySignals = computed(() => {
  const signals = []
  for (let line = 0; line < 5; line++) {
    for (let i = 0; i < 6; i++) {
      const state = Math.random() > 0.15 ? '开放' : '关闭'
      signals.push({
        id: `XH${line * 6 + i + 1}`.padStart(5, '0'),
        x: 100 + i * 110,
        y: 60 + line * 70,
        state,
        color: state === '开放' ? '#00ff88' : '#ff6b6b'
      })
    }
  }
  return signals
})

// 显示用的道岔
const displaySwitches = computed(() => {
  const switches = []
  for (let line = 0; line < 5; line++) {
    for (let i = 0; i < 3; i++) {
      const states = ['定位', '定位', '定位', '反位', '四开']
      switches.push({
        id: `DC${line * 3 + i + 1}`.padStart(5, '0'),
        x: 200 + i * 180,
        y: 60 + line * 70,
        state: states[Math.floor(Math.random() * states.length)],
        locked: Math.random() > 0.3
      })
    }
  }
  return switches
})

// 显示用的列车
const displayTrains = computed(() => {
  return dispatchData.value.trainDispatch.slice(0, 12).map((train, i) => ({
    ...train,
    x: 100 + (i % 6) * 100 + Math.random() * 50,
    y: 60 + Math.floor(i / 6) * 2 * 70 + (i % 2) * 35
  }))
})

```

```

// 活跃进路
const activeRoutes = computed(() => {
  return [
    { id: 'R1', x1: 100, y1: 60, x2: 300, y2: 60 },
    { id: 'R2', x1: 400, y1: 130, x2: 600, y2: 130 },
    { id: 'R3', x1: 200, y1: 200, x2: 500, y2: 200 }
  ]
})

// 调度统计
const dispatchStats = computed(() => [
  { label: 'X站101号信号机', value: `${dispatchData.value.stats.openSignals}/${dispatchData.value.stats.totalSignals}`, color: '#00ff88' },
  { label: '道岔', value: `${dispatchData.value.stats.normalSwitches}/${dispatchData.value.stats.totalSwitches}`, color: '#00d4ff' },
  { label: '占用区段', value: dispatchData.value.stats.occupiedSections, color: '#ff6b6b' },
  { label: '运行列车', value: dispatchData.value.stats.runningTrains, color: '#ffaa00' },
  { label: '晚点列车', value: dispatchData.value.stats.delayedTrains, color: '#ff6b6b' }
])

// 调度命令
const dispatchCommands = computed(() => dispatchData.value.dispatchCommands.slice(0, 4))

// 饼图数据
const pieData = computed(() => {
  const createPieData = (data, title, unit) => {
    const total = data.reduce((sum, d) => sum + d.value, 0)
    let offset = 0
    const slices = data.map(d => {
      const percent = d.value / total
      const dashArray = `${percent * 220} ${220 - percent * 220}`
      const slice = { ...d, dashArray, offset: -offset * 220 }
      offset += percent
      return slice
    })
    return { title, data: slices, total, unit }
  }

  return [
    createPieData(signalDist.value.signalStatus, 'X站101号信号机状态', '台'),
    createPieData(signalDist.value.switchStatus, '道岔状态', '组'),
    createPieData(signalDist.value.trackStatus, '轨道区段', '段'),
    createPieData(signalDist.value.trainStatus, '列车状态', '列')
  ]
})

// 获取区段颜色
const getSectionColor = (state) => {
  const colors = {
    '空闲': '#00ff88',
    '占用': '#ff6b6b',
    '锁闭': '#ffaa00',
    '故障': '#aa88ff'
  }
  return colors[state] || '#555'
}

// 获取列车状态颜色
const getTrainStatusColor = (state) => {
  const colors = {
    '正常运行': '#00ff88',
    '等待信号': '#ffaa00',
    '减速运行': '#00d4ff',
    '临时停车': '#ff6b6b',
    '晚点运行': '#ff9966'
  }
}

```

```

    return colors[state] || '#888'
  }

  // 显示信号机信息
  const showSignalInfo = (signal) => {
    console.log('Signal info:', signal)
  }

  // 更新时间
  const updateTime = () => {
    currentTime.value = new Date().toLocaleString('zh-CN', {
      year: 'numeric',
      month: '2-digit',
      day: '2-digit',
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit'
    })
  }

  onMounted(async () => {
    updateTime()
    setInterval(updateTime, 1000)

    updateTimer.value = setInterval(() => {
      dispatchData.value = getSignalDispatchData()
      signalDist.value = getSignalDistribution()
      efficiency.value = getDispatchEfficiency()
    }, 5000)

    updateLifeRows()
    lifeTimer.value = setInterval(updateLifeRows, 1500)
    await nextTick()
    startLifeAutoScroll()

    // 全局鼠标移动和释放事件（用于小地图拖拽）
    window.addEventListener('mousemove', handleMinimapMove)
    window.addEventListener('mouseup', endDrag)
  })

  onUnmounted(() => {
    if (updateTimer.value) clearInterval(updateTimer.value)
    if (lifeTimer.value) clearInterval(lifeTimer.value)
    stopLifeAutoScroll()
    window.removeEventListener('mousemove', handleMinimapMove)
    window.removeEventListener('mouseup', endDrag)
  })

  watch(
    () => lifeRowsLoop.value.length,
    async (len) => {
      if (!len) return
      await nextTick()
      startLifeAutoScroll()
    }
  )

  // 小地图移动处理
  const handleMinimapMove = (e) => {
    if (isMinimapDragging.value) {
      updatePanFromMinimap(e)
    }
  }
}
</script>

```

```
<style scoped>
.center-panel {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  gap: 8px;
  padding: 8px;
}

/* 顶部指标 */
.top-metrics {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 12px;
}

.metric-card {
  background: linear-gradient(135deg, rgba(0, 30, 60, 0.9) 0%, rgba(0, 50, 100, 0.6) 100%);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 10px;
  display: flex;
  align-items: center;
  gap: 12px;
}

.metric-icon {
  font-size: 28px;
}

.metric-content {
  flex: 1;
}

.metric-value {
  display: flex;
  align-items: baseline;
  gap: 4px;
}

.metric-value .value {
  font-size: 26px;
  font-weight: bold;
  color: #fff;
}

.metric-value .unit {
  font-size: 13px;
  color: #888;
}

.metric-label {
  font-size: 13px;
  color: #888;
  margin-top: 3px;
}

.metric-trend {
  font-size: 12px;
  margin-top: 3px;
}
```

```
.metric-trend.up { color: #00ff88; }
.metric-trend.down { color: #ff6b6b; }

/* 主可视化区域 */
.main-visualization {
  flex: 1;
  background: linear-gradient(135deg, rgba(0, 30, 60, 0.9) 0%, rgba(0, 50, 100, 0.6) 100%);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 10px;
  display: flex;
  flex-direction: column;
  min-height: 310px;
}

.viz-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 6px;
}

.viz-title {
  font-size: 16px;
  color: #00d4ff;
  font-weight: bold;
  margin: 0;
}

.viz-time {
  font-size: 14px;
  color: #fff;
  font-family: monospace;
}

.dispatch-container {
  flex: 1;
  overflow: hidden;
  position: relative;
  cursor: grab;
  user-select: none;
}

.dispatch-container:active {
  cursor: grabbing;
}

.canvas-wrapper {
  width: 100%;
  height: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
}

/* 设备寿命预测 */
.life-container {
  flex: 1;
  overflow: hidden;
  position: relative;
  user-select: none;
}

.life-table {
```

```
width: 100%;
height: 100%;
display: flex;
flex-direction: column;
gap: 8px;
padding: 8px;
}

.life-table-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}

.life-tip {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  color: rgba(255, 255, 255, 0.75);
  font-size: 12px;
}

.life-tip .dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #00d4ff;
  box-shadow: 0 0 12px rgba(0, 212, 255, 0.6);
}

.life-summary {
  display: flex;
  gap: 10px;
}

.sum-item {
  background: rgba(0, 50, 100, 0.25);
  border: 1px solid rgba(0, 200, 255, 0.18);
  border-radius: 8px;
  padding: 8px 10px;
  min-width: 80px;
  text-align: center;
}

.sum-value {
  font-size: 20px;
  font-weight: 700;
  color: #fff;
  line-height: 1.1;
}

.sum-value.warning {
  color: #ff5555;
  text-shadow: 0 0 10px rgba(255, 80, 80, 0.35);
}

.sum-label {
  margin-top: 4px;
  font-size: 12px;
  color: rgba(255, 255, 255, 0.55);
}

.life-table-scroll {
```

```
flex: 1;
overflow: auto;
border-radius: 8px;
border: 1px solid rgba(255, 255, 255, 0.08);
background: rgba(0, 0, 0, 0.12);
scrollbar-width: none;
}

.life-table-scroll::-webkit-scrollbar {
width: 0;
height: 0;
}

.life-table-grid {
width: 100%;
border-collapse: collapse;
min-width: 780px;
}

.life-table-grid thead th {
position: sticky;
top: 0;
z-index: 2;
text-align: left;
font-size: 13px;
font-weight: 700;
color: rgba(255, 255, 255, 0.85);
padding: 10px 10px;
background: rgba(0, 40, 80, 0.65);
border-bottom: 1px solid rgba(0, 200, 255, 0.18);
}

.life-table-grid tbody td {
padding: 10px 10px;
font-size: 13px;
color: rgba(255, 255, 255, 0.78);
border-bottom: 1px solid rgba(255, 255, 255, 0.06);
vertical-align: middle;
}

.life-table-grid tbody tr:hover {
background: rgba(0, 212, 255, 0.06);
}

.life-table-grid tbody tr.danger {
background: rgba(255, 80, 80, 0.08);
}

.life-table-grid tbody tr.warning {
background: rgba(255, 170, 0, 0.06);
}

.col-name .name-main {
font-weight: 700;
color: rgba(255, 255, 255, 0.92);
}

.col-name .name-sub {
margin-top: 2px;
font-size: 12px;
color: rgba(255, 255, 255, 0.55);
}

.wear-wrap {
```

```
display: flex;
align-items: center;
gap: 10px;
}

.wear-bar {
  flex: 1;
  height: 8px;
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.10);
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.08);
}

.wear-fill {
  height: 100%;
  border-radius: 999px;
  background: linear-gradient(90deg, rgba(0, 255, 136, 0.85) 0%, rgba(0, 212, 255, 0.9) 45%, rgba(255, 170, 0, 0.9) 75%, rgba(255, 80, 80, 0.92) 100%);
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.18);
  transition: width 0.35s ease;
}

.wear-text {
  width: 44px;
  text-align: right;
  font-variant-numeric: tabular-nums;
  color: rgba(255, 255, 255, 0.85);
}

.col-life .life-main {
  font-weight: 700;
  color: rgba(255, 255, 255, 0.92);
}

.col-life .life-sub {
  margin-top: 2px;
  font-size: 12px;
  color: rgba(255, 255, 255, 0.55);
}

.status-pill {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-width: 64px;
  padding: 3px 8px;
  border-radius: 999px;
  font-size: 12px;
  font-weight: 700;
  border: 1px solid rgba(255, 255, 255, 0.10);
  background: rgba(0, 0, 0, 0.12);
}

.status-pill.normal {
  color: #00ff88;
  border-color: rgba(0, 255, 136, 0.28);
  background: rgba(0, 255, 136, 0.08);
}

.status-pill.warning {
  color: #ffaa00;
  border-color: rgba(255, 170, 0, 0.28);
  background: rgba(255, 170, 0, 0.08);
}
```

```
}

.status-pill.danger {
  color: #ff5555;
  border-color: rgba(255, 80, 80, 0.28);
  background: rgba(255, 80, 80, 0.10);
}

/* 缩放控制 */
.viz-controls {
  display: flex;
  align-items: center;
}

.zoom-controls {
  display: flex;
  align-items: center;
  gap: 4px;
  background: rgba(0, 50, 100, 0.5);
  padding: 4px 8px;
  border-radius: 6px;
  border: 1px solid rgba(0, 200, 255, 0.3);
}

.zoom-btn {
  width: 26px;
  height: 26px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgba(0, 100, 150, 0.3);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 4px;
  color: #00d4ff;
  cursor: pointer;
  transition: all 0.2s;
}

.zoom-btn:hover {
  background: rgba(0, 150, 200, 0.5);
  transform: scale(1.05);
}

.zoom-btn:active {
  transform: scale(0.95);
}

.zoom-level {
  font-size: 11px;
  color: #fff;
  min-width: 40px;
  text-align: center;
  font-family: monospace;
}

/* 小地图 */
.minimap {
  position: absolute;
  right: 10px;
  bottom: 10px;
  width: 160px;
  height: 80px;
  background: rgba(0, 20, 40, 0.9);
  border: 1px solid rgba(0, 200, 255, 0.4);
}
```

```
border-radius: 6px;
overflow: hidden;
box-shadow: 0 2px 10px rgba(0, 0, 0, 0.3);
}

.minimap-svg {
width: 100%;
height: 100%;
}

.viewport-indicator {
cursor: move;
transition: fill 0.2s;
}

.viewport-indicator:hover {
fill: rgba(0, 200, 255, 0.3);
}

.dispatch-svg {
width: 100%;
height: 100%;
}

.signal-group, .switch-group, .train-group {
cursor: pointer;
transition: transform 0.2s;
}

.signal-group:hover, .switch-group:hover {
transform: scale(1.2);
}

.track-section.占用 {
animation: pulse 1s infinite;
}

@keyframes pulse {
0%, 100% { opacity: 0.8; }
50% { opacity: 1; }
}

.dispatch-stats {
display: flex;
justify-content: space-around;
margin-top: 10px;
padding-top: 10px;
border-top: 1px solid rgba(0, 200, 255, 0.2);
}

.stat-item {
text-align: center;
}

.stat-value {
font-size: 16px;
font-weight: bold;
display: block;
}

.stat-label {
font-size: 12px;
color: #888;
}
```

```
/* 底部面板 */
.bottom-panels {
  display: grid;
  grid-template-columns: 1.2fr 1fr 1fr;
  gap: 10px;
}

.bottom-panel {
  background: linear-gradient(135deg, rgba(0, 30, 60, 0.9) 0%, rgba(0, 50, 100, 0.6) 100%);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  padding: 12px;
}

.panel-title {
  font-size: 14px;
  color: #00d4ff;
  font-weight: bold;
  margin-bottom: 10px;
  padding-bottom: 6px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.2);
}

/* 饼图 */
.pie-charts {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
}

.pie-item {
  text-align: center;
}

.pie-chart-wrapper {
  width: 70px;
  height: 70px;
  margin: 0 auto;
}

.pie-svg {
  width: 100%;
  height: 100%;
}

.pie-slice {
  transform: rotate(-90deg);
  transform-origin: center;
  transition: stroke-dasharray 0.5s ease;
}

.pie-title {
  font-size: 10px;
  color: #fff;
  margin-top: 5px;
}

.pie-legend {
  margin-top: 5px;
}

.legend-row {
  display: flex;
```

```
    align-items: center;
    gap: 4px;
    font-size: 9px;
    justify-content: center;
}

.legend-dot {
    width: 6px;
    height: 6px;
    border-radius: 50%;
}

.legend-name {
    color: #888;
}

.legend-value {
    color: #fff;
    min-width: 15px;
}

/* 调度命令 */
.command-list {
    display: flex;
    flex-direction: column;
    gap: 8px;
    max-height: 260px;
    overflow-y: auto;
}

.command-item {
    padding: 8px;
    background: rgba(0, 50, 100, 0.3);
    border-radius: 6px;
    border-left: 3px solid;
}

.command-item.已执行 { border-left-color: #00ff88; }
.command-item.执行中 { border-left-color: #00d4ff; }
.command-item.待执行 { border-left-color: #ffaa00; }
.command-item.已取消 { border-left-color: #888; }

.cmd-header {
    display: flex;
    align-items: center;
    gap: 8px;
    margin-bottom: 4px;
}

.cmd-type {
    font-size: 12px;
    padding: 2px 6px;
    background: rgba(0, 200, 255, 0.2);
    border-radius: 3px;
    color: #00d4ff;
}

.cmd-train {
    font-size: 13px;
    color: #fff;
    font-weight: bold;
}

.cmd-status {
```

```
font-size: 11px;
margin-left: auto;
}

.cmd-status.已执行 { color: #00ff88; }
.cmd-status.执行中 { color: #00d4ff; }
.cmd-status.待执行 { color: #ffaa00; }
.cmd-status.已取消 { color: #888; }

.cmd-content {
font-size: 12px;
color: #aaa;
margin-bottom: 4px;
}

.cmd-meta {
display: flex;
justify-content: space-between;
font-size: 11px;
color: #666;
}

/* 调度效率 */
.efficiency-metrics {
display: flex;
justify-content: space-around;
margin-bottom: 10px;
}

.eff-item {
text-align: center;
}

.eff-ring {
width: 50px;
height: 50px;
margin: 0 auto;
}

.eff-circle {
transform: rotate(-90deg);
transform-origin: center;
transition: stroke-dashoffset 0.5s ease;
}

.eff-value {
font-size: 20px;
font-weight: bold;
color: #00d4ff;
}

.eff-value .unit {
font-size: 10px;
color: #888;
}

.eff-label {
font-size: 11px;
color: #888;
margin-top: 3px;
}

.trend-chart {
margin-top: 10px;
}
```

```

}

.chart-title {
  font-size: 12px;
  color: #888;
  margin-bottom: 8px;
}

.chart-bars {
  display: flex;
  justify-content: space-between;
  align-items: flex-end;
  height: 55px;
}

.bar-group {
  display: flex;
  flex-direction: column;
  align-items: center;
  flex: 1;
}

.bar {
  width: 15px;
  background: linear-gradient(to top, #00d4ff, rgba(0, 200, 255, 0.3));
  border-radius: 2px 2px 0 0;
  position: relative;
  transition: height 0.3s;
}

.bar-tooltip {
  position: absolute;
  top: -15px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 8px;
  color: #888;
  opacity: 0;
  transition: opacity 0.2s;
}

.bar:hover .bar-tooltip {
  opacity: 1;
}

.bar-label {
  font-size: 10px;
  color: #666;
  margin-top: 3px;
}
</style>

```

5.6 文件：src/components/datapanel/RightPanel.vue

- 文件说明：右侧设备统计、告警列表和运维概览面板。
- 行数统计：963 行

```

<template>
  <div class="right-panel">
    <div class="panel-section ticker-section">
      <div class="section-header">
        <span class="section-icon">播报</span>
        <span class="section-title">区间滚动播报</span>
        <span class="section-badge">脱敏</span>
      </div>
      <div class="ticker-shell">

```

```

        <div class="ticker-track">
            <span v-for="(item, idx) in tickerItemsLoop" :key="'${idx}-${item}`" class="ticker-item">{{ item }}</span>
        </div>
    </div>
</div>
<!-- 设备监控 -->
<div class="panel-section equipment-section">
    <div class="section-header">
        <span class="section-icon">⚙️</span>
        <span class="section-title">设备监控</span>
        <span class="section-badge">{{ totalDevices }} 台</span>
    </div>
    <div class="equipment-stats">
        <div class="stat-item normal">
            <span class="stat-value">{{ normalDevices }}</span>
            <span class="stat-label">正常</span>
        </div>
        <div class="stat-item warning">
            <span class="stat-value">{{ warningDevices }}</span>
            <span class="stat-label">预警</span>
        </div>
        <div class="stat-item error">
            <span class="stat-value">{{ errorDevices }}</span>
            <span class="stat-label">故障</span>
        </div>
    </div>
    <div class="equipment-list">
        <div
            v-for="eq in equipment"
            :key="eq.type"
            class="equipment-item"
        >
            <div class="eq-header">
                <span class="eq-icon">{{ equipmentIcon(eq.icon) }}</span>
                <span class="eq-name">{{ eq.type }}</span>
                <span class="eq-online">{{ eq.onlineRate }}%</span>
            </div>
            <div class="eq-progress">
                <div class="progress-bar">
                    <div
                        class="progress-fill normal"
                        :style="{ width: (eq.normal / eq.total * 100) + '%' }"
                    ></div>
                    <div
                        class="progress-fill warning"
                        :style="{ width: (eq.warning / eq.total * 100) + '%' }"
                    ></div>
                    <div
                        class="progress-fill error"
                        :style="{ width: (eq.error / eq.total * 100) + '%' }"
                    ></div>
                </div>
            </div>
            <div class="eq-stats">
                <span class="normal">✓ {{ eq.normal }}</span>
                <span class="warning">⚠️ {{ eq.warning }}</span>
                <span class="error">✖️ {{ eq.error }}</span>
            </div>
        </div>
    </div>
</div>
<!-- 告警信息 -->
<div class="panel-section alarm-section">

```

```

<div class="section-header">
  <span class="section-icon">🚨</span>
  <span class="section-title">告警信息</span>
  <span class="section-badge alarm">{{ alarmCount }} 条</span>
</div>
<div class="alarm-tabs">
  <button
    v-for="tab in alarmTabs"
    :key="tab.value"
    :class="['alarm-tab', { active: activeAlarmTab === tab.value }]"
    @click="activeAlarmTab = tab.value"
  >
    {{ tab.label }}
  </button>
</div>
<div class="alarm-list">
  <div
    v-for="alarm in filteredAlarms"
    :key="alarm.id"
    :class="['alarm-item', alarm.level, { handled: alarm.handled }]"
  >
    <div class="alarm-level">
      <span :class="['level-icon', alarm.level]">
        {{ alarm.level === 'critical' ? '🔴' : alarm.level === 'warning' ? '🟡' : '🟢' }}
      </span>
    </div>
    <div class="alarm-content">
      <div class="alarm-title">{{ alarm.title }}</div>
      <div class="alarm-desc">{{ alarm.desc }}</div>
      <div class="alarm-meta">
        <span>📍 {{ alarm.source }}</span>
        <span>🕒 {{ formatTime(alarm.time) }}</span>
      </div>
    </div>
    <div class="alarm-actions">
      <button class="action-btn" v-if="!alarm.handled">处理</button>
      <span class="handled-tag" v-else>已处理</span>
    </div>
  </div>
</div>
</div>

<!-- 安全监控 -->
<div class="panel-section security-section">
  <div class="section-header">
    <span class="section-icon">🛡️</span>
    <span class="section-title">安全监控</span>
  </div>
  <div class="security-overview">
    <div class="security-score">
      <svg viewBox="0 0 100 100" class="score-ring">
        <circle cx="50" cy="50" r="40" fill="none" stroke="rgba(255,255,255,0.1)" stroke-width="8" />
        <circle
          cx="50" cy="50" r="40"
          fill="none"
          :stroke="securityColor"
          stroke-width="8"
          stroke-linecap="round"
          :stroke-dasharray="251"
          :stroke-dashoffset="251 - (security.securityScore / 100 * 251)"
          class="score-circle"
        />
        <text x="50" y="45" text-anchor="middle" fill="#fff" font-size="20" font-weight="bold">
          {{ security.securityScore }}
        </text>
      </svg>
    </div>
  </div>
</div>

```

```
</text>
<text x="50" y="62" text-anchor="middle" fill="#888" font-size="10">
  安全评分
</text>
</svg>
</div>
<div class="security-stats">
  <div class="sec-stat">
    <span class="sec-value">{{ security.intrusionAttempts }}</span>
    <span class="sec-label">入侵尝试</span>
  </div>
  <div class="sec-stat">
    <span class="sec-value">{{ security.blockedIPs }}</span>
    <span class="sec-label">已拦截IP</span>
  </div>
  <div class="sec-stat">
    <span class="sec-value">{{ security.activeUsers }}</span>
    <span class="sec-label">活跃用户</span>
  </div>
</div>
</div>
<div class="vulnerability-stats">
  <div class="vuln-title">漏洞统计</div>
  <div class="vuln-bars">
    <div class="vuln-item critical">
      <span class="vuln-label">严重</span>
      <div class="vuln-bar">
        <div class="vuln-fill" :style="{ width: Math.min(security.vulnerabilities.critical * 20, 100) + '%' }"></div>
      </div>
      <span class="vuln-count">{{ security.vulnerabilities.critical }}</span>
    </div>
    <div class="vuln-item high">
      <span class="vuln-label">高危</span>
      <div class="vuln-bar">
        <div class="vuln-fill" :style="{ width: Math.min(security.vulnerabilities.high * 10, 100) + '%' }"></div>
      </div>
      <span class="vuln-count">{{ security.vulnerabilities.high }}</span>
    </div>
    <div class="vuln-item medium">
      <span class="vuln-label">中危</span>
      <div class="vuln-bar">
        <div class="vuln-fill" :style="{ width: Math.min(security.vulnerabilities.medium * 5, 100) + '%' }"></div>
      </div>
      <span class="vuln-count">{{ security.vulnerabilities.medium }}</span>
    </div>
    <div class="vuln-item low">
      <span class="vuln-label">低危</span>
      <div class="vuln-bar">
        <div class="vuln-fill" :style="{ width: Math.min(security.vulnerabilities.low * 2, 100) + '%' }"></div>
      </div>
      <span class="vuln-count">{{ security.vulnerabilities.low }}</span>
    </div>
  </div>
</div>
<div class="security-events">
  <div class="events-title">最近安全事件</div>
  <div class="event-list">
    <div v-for="event in security.recentEvents.slice(0, 4)" :key="event.time" class="event-item">
      <span class="event-type">{{ event.type }}</span>
      <span class="event-user">{{ event.user }}</span>
      <span :class="['event-status', event.status]">{{ event.status }}</span>
    </div>
  </div>
</div>
</div>
```

```
</div>

<!-- 运维工单 -->
<div class="panel-section workorder-section">
  <div class="section-header">
    <span class="section-icon">📄</span>
    <span class="section-title">运维工单</span>
  </div>
  <div class="workorder-stats">
    <div class="wo-stat pending">
      <span class="wo-value">{{ workOrders.pending }}</span>
      <span class="wo-label">待处理</span>
    </div>
    <div class="wo-stat processing">
      <span class="wo-value">{{ workOrders.processing }}</span>
      <span class="wo-label">处理中</span>
    </div>
    <div class="wo-stat completed">
      <span class="wo-value">{{ workOrders.completed }}</span>
      <span class="wo-label">已完成</span>
    </div>
  </div>
  <div class="wo-chart">
    <div class="pie-chart">
      <svg viewBox="0 0 100 100">
        <circle
          v-for="(slice, i) in pieSlices"
          :key="i"
          cx="50" cy="50" r="40"
          fill="none"
          :stroke="slice.color"
          stroke-width="20"
          :stroke-dasharray="slice.dashArray"
          :stroke-dashoffset="slice.offset"
          class="pie-slice"
        />
      </svg>
      <div class="pie-center">
        <span class="pie-value">{{ workOrders.satisfaction }}%</span>
        <span class="pie-label">满意度</span>
      </div>
    </div>
    <div class="wo-legend">
      <div v-for="item in workOrders.ordersByType" :key="item.type" class="legend-item">
        <span class="legend-dot" :style="{ background: item.color }"></span>
        <span class="legend-text">{{ item.type }}</span>
        <span class="legend-count">{{ item.count }}</span>
      </div>
    </div>
  </div>
  <div class="recent-orders">
    <div class="orders-title">最近工单</div>
    <div class="order-list">
      <div v-for="order in workOrders.recentOrders.slice(0, 3)" :key="order.id" class="order-item">
        <div class="order-header">
          <span :class="['order-priority', order.priority]">{{ order.priority }}</span>
          <span class="order-id">{{ order.id }}</span>
        </div>
        <div class="order-title">{{ order.title }}</div>
        <div class="order-meta">
          <span>{{ order.assignee }}</span>
          <span :class="['order-status', order.status]">{{ order.status }}</span>
        </div>
      </div>
    </div>
  </div>
</div>
```

```

        </div>
      </div>
    </div>
  </div>
</template>

<script setup>
import { ref, computed, onMounted, onUnmounted } from 'vue'
import { getEquipmentData, getAlarmData, getSecurityData, getWorkOrderData } from '../services/mockDataService'

const equipment = ref(getEquipmentData())
const alarms = ref(getAlarmData())
const security = ref(getSecurityData())
const workOrders = ref(getWorkOrderData())
const activeAlarmTab = ref('all')
const updateTimer = ref(null)

const tickerItems = [
  '西南山地上行咽喉区间出现轻雾，当前设备标识已脱敏展示。',
  '北侧桥隧结合段风速维持 3 级以内，轨旁箱体湿度处于正常范围。',
  '东坡隧道口至谷地缓坡区间采用匿名区段编码，不展示真实站名与真实人员信息。',
  '坡脚联络线信号机巡检记录为演示数据，检修班组名称已做泛化处理。',
  '临水弯道区段视频巡检为模拟画面，地理描述保留地貌特征但不对应真实坐标。',
  '上行 K 区段传感器在线率稳定，面板内设备编号为资产脱敏编码。',
  '高填方路基边坡监测数据来自本地仿真，不含真实单位名称与真实位置。'
]

const tickerItemsLoop = computed(() => [...tickerItems, ...tickerItems])

// 告警标签
const alarmTabs = [
  { label: '全部', value: 'all' },
  { label: '严重', value: 'critical' },
  { label: '警告', value: 'warning' },
  { label: '信息', value: 'info' }
]

// 设备统计
const totalDevices = computed(() => equipment.value.reduce((sum, eq) => sum + eq.total, 0))
const normalDevices = computed(() => equipment.value.reduce((sum, eq) => sum + eq.normal, 0))
const warningDevices = computed(() => equipment.value.reduce((sum, eq) => sum + eq.warning, 0))
const errorDevices = computed(() => equipment.value.reduce((sum, eq) => sum + eq.error, 0))

// 告警数量
const alarmCount = computed(() => alarms.value.filter(a => !a.handled).length)

// 过滤告警
const filteredAlarms = computed(() => {
  if (activeAlarmTab.value === 'all') return alarms.value.slice(0, 6)
  return alarms.value.filter(a => a.level === activeAlarmTab.value).slice(0, 6)
})

// 安全评分颜色
const securityColor = computed(() => {
  const score = security.value.securityScore
  if (score >= 90) return '#00ff88'
  if (score >= 70) return '#ffaa00'
  return '#ff6b6b'
})

// 饼图切片
const pieSlices = computed(() => {
  const data = workOrders.value.ordersByType

```

```

const total = data.reduce((sum, item) => sum + item.count, 0)
let offset = 0
return data.map(item => {
  const percent = item.count / total
  const dashArray = `${percent * 251} ${251 - percent * 251}`
  const slice = { color: item.color, dashArray, offset: -offset }
  offset += percent * 251
  return slice
})
})

// 设备图标
const equipmentIcon = (type) => {
  const icons = {
    signal: '📶',
    switch: '🔌',
    track: '🚉',
    battery: '🔋',
    network: '🌐',
    camera: '📷',
    train: '🚂',
    lightning: '⚡'
  }
  return icons[type] || '⚙️'
}

// 格式化时间
const formatTime = (timeStr) => {
  const date = new Date(timeStr)
  return date.toLocaleTimeString('zh-CN', { hour: '2-digit', minute: '2-digit' })
}

// 定时更新
onMounted(() => {
  updateTimer.value = setInterval(() => {
    equipment.value = getEquipmentData()
    alarms.value = getAlarmData()
    security.value = getSecurityData()
    workOrders.value = getWorkOrderData()
  }, 5000)
})

onUnmounted(() => {
  if (updateTimer.value) {
    clearInterval(updateTimer.value)
  }
})
</script>

<style scoped>
.right-panel {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  gap: 15px;
  overflow-y: auto;
  padding-left: 5px;
}

.right-panel::-webkit-scrollbar {
  width: 4px;
}

```

```
.right-panel::-webkit-scrollbar-thumb {
  background: rgba(0, 200, 255, 0.3);
  border-radius: 2px;
}

.panel-section {
  background: linear-gradient(135deg, rgba(0, 30, 60, 0.9) 0%, rgba(0, 50, 100, 0.6) 100%);
  border: 1px solid rgba(0, 200, 255, 0.3);
  border-radius: 10px;
  padding: 15px;
  backdrop-filter: blur(10px);
}

.section-header {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 15px;
  padding-bottom: 10px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.2);
}

.section-icon {
  font-size: 20px;
}

.section-title {
  font-size: 16px;
  font-weight: bold;
  color: #00d4ff;
  flex: 1;
}

.section-badge {
  font-size: 13px;
  padding: 2px 8px;
  background: rgba(0, 200, 255, 0.2);
  border: 1px solid rgba(0, 200, 255, 0.4);
  border-radius: 10px;
  color: #00d4ff;
}

.ticker-shell {
  overflow: hidden;
  border-radius: 8px;
  border: 1px solid rgba(0, 200, 255, 0.22);
  background: rgba(0, 18, 38, 0.6);
}

.ticker-track {
  display: flex;
  align-items: center;
  width: max-content;
  gap: 28px;
  padding: 10px 0;
  animation: ticker-scroll 30s linear infinite;
}

.ticker-item {
  flex: 0 0 auto;
  font-size: 14px;
  color: rgba(230, 247, 255, 0.92);
  white-space: nowrap;
}
```

```
@keyframes ticker-scroll {
  from { transform: translateX(0); }
  to { transform: translateX(-50%); }
}

.section-badge.alarm {
  background: rgba(255, 107, 107, 0.2);
  border-color: rgba(255, 107, 107, 0.4);
  color: #ff6b6b;
}

/* 设备统计 */
.equipment-stats {
  display: flex;
  justify-content: space-around;
  margin-bottom: 15px;
}

.stat-item {
  text-align: center;
  padding: 10px;
  background: rgba(0, 50, 100, 0.3);
  border-radius: 8px;
  min-width: 60px;
}

.stat-value {
  font-size: 28px;
  font-weight: bold;
  display: block;
}

.stat-item.normal .stat-value { color: #00ff88; }
.stat-item.warning .stat-value { color: #ffaa00; }
.stat-item.error .stat-value { color: #ff6b6b; }

.stat-label {
  font-size: 13px;
  color: #888;
}

/* 设备列表 */
.equipment-list {
  display: flex;
  flex-direction: column;
  gap: 10px;
  max-height: 200px;
  overflow-y: auto;
}

.equipment-item {
  background: rgba(0, 50, 100, 0.2);
  border-radius: 8px;
  padding: 10px;
}

.eq-header {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 8px;
}
```

```
.eq-icon {
  font-size: 16px;
}

.eq-name {
  font-size: 14px;
  color: #fff;
  flex: 1;
}

.eq-online {
  font-size: 13px;
  color: #00ff88;
}

.eq-progress {
  margin-bottom: 5px;
}

.progress-bar {
  height: 6px;
  background: rgba(255, 255, 255, 0.1);
  border-radius: 3px;
  display: flex;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  transition: width 0.5s ease;
}

.progress-fill.normal { background: #00ff88; }
.progress-fill.warning { background: #ffaa00; }
.progress-fill.error { background: #ff6b6b; }

.eq-stats {
  display: flex;
  gap: 15px;
  font-size: 10px;
}

.eq-stats .normal { color: #00ff88; }
.eq-stats .warning { color: #ffaa00; }
.eq-stats .error { color: #ff6b6b; }

/* 告警区域 */
.alarm-tabs {
  display: flex;
  gap: 5px;
  margin-bottom: 10px;
}

.alarm-tab {
  padding: 5px 12px;
  background: rgba(0, 50, 100, 0.3);
  border: 1px solid rgba(0, 200, 255, 0.2);
  border-radius: 15px;
  color: #888;
  font-size: 13px;
  cursor: pointer;
  transition: all 0.3s;
}
```

```
.alarm-tab:hover {
  background: rgba(0, 100, 150, 0.3);
  color: #00d4ff;
}

.alarm-tab.active {
  background: rgba(0, 200, 255, 0.2);
  border-color: rgba(0, 200, 255, 0.5);
  color: #00d4ff;
}

.alarm-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
  max-height: 250px;
  overflow-y: auto;
}

.alarm-item {
  display: flex;
  gap: 10px;
  padding: 10px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 8px;
  border-left: 3px solid;
  transition: all 0.3s;
}

.alarm-item.critical { border-left-color: #ff6b6b; }
.alarm-item.warning { border-left-color: #ffaa00; }
.alarm-item.info { border-left-color: #00d4ff; }
.alarm-item.handled { opacity: 0.6; }

.alarm-content {
  flex: 1;
  min-width: 0;
}

.alarm-title {
  font-size: 15px;
  color: #fff;
  margin-bottom: 3px;
}

.alarm-desc {
  font-size: 12px;
  color: #aaa;
  margin-bottom: 5px;
}

.alarm-meta {
  display: flex;
  gap: 15px;
  font-size: 11px;
  color: #666;
}

.alarm-actions {
  display: flex;
  align-items: center;
}

.action-btn {
```

```
padding: 4px 10px;
background: rgba(0, 200, 255, 0.2);
border: 1px solid rgba(0, 200, 255, 0.4);
border-radius: 4px;
color: #00d4ff;
font-size: 12px;
cursor: pointer;
transition: all 0.3s;
}

.action-btn:hover {
  background: rgba(0, 200, 255, 0.3);
}

.handled-tag {
  font-size: 12px;
  color: #00ff88;
}

/* 安全监控 */
.security-overview {
  display: flex;
  gap: 15px;
  margin-bottom: 15px;
}

.security-score {
  width: 80px;
  height: 80px;
}

.score-ring {
  width: 100%;
  height: 100%;
}

.score-circle {
  transform: rotate(-90deg);
  transform-origin: center;
  transition: stroke-dashoffset 1s ease;
}

.security-stats {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 8px;
}

.sec-stat {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.sec-value {
  font-size: 20px;
  font-weight: bold;
  color: #fff;
}

.sec-label {
  font-size: 12px;
```

```
    color: #888;
}

/* 漏洞统计 */
.vulnerability-stats {
    margin-bottom: 15px;
}

.vuln-title, .events-title, .orders-title {
    font-size: 13px;
    color: #888;
    margin-bottom: 8px;
}

.vuln-bars {
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.vuln-item {
    display: flex;
    align-items: center;
    gap: 8px;
}

.vuln-label {
    font-size: 10px;
    color: #888;
    width: 30px;
}

.vuln-bar {
    flex: 1;
    height: 4px;
    background: rgba(255, 255, 255, 0.1);
    border-radius: 2px;
    overflow: hidden;
}

.vuln-fill {
    height: 100%;
    border-radius: 2px;
    transition: width 0.5s ease;
}

.vuln-item.critical .vuln-fill { background: #ff6b6b; }
.vuln-item.high .vuln-fill { background: #ff9966; }
.vuln-item.medium .vuln-fill { background: #ffaa00; }
.vuln-item.low .vuln-fill { background: #00d4ff; }

.vuln-count {
    font-size: 10px;
    color: #aaa;
    width: 20px;
    text-align: right;
}

/* 安全事件 */
.event-list {
    display: flex;
    flex-direction: column;
    gap: 5px;
}
```

```
.event-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 6px 8px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 4px;
  font-size: 12px;
}

.event-type {
  color: #00d4ff;
}

.event-user {
  color: #888;
  flex: 1;
}

.event-status {
  padding: 2px 6px;
  border-radius: 3px;
}

.event-status.成功 { background: rgba(0, 255, 136, 0.2); color: #00ff88; }
.event-status.失败 { background: rgba(255, 107, 107, 0.2); color: #ff6b6b; }
.event-status.待审核 { background: rgba(255, 170, 0, 0.2); color: #ffaa00; }

/* 运维工单 */
.workorder-stats {
  display: flex;
  justify-content: space-around;
  margin-bottom: 15px;
}

.wo-stat {
  text-align: center;
}

.wo-value {
  font-size: 24px;
  font-weight: bold;
  display: block;
}

.wo-stat.pending .wo-value { color: #ffaa00; }
.wo-stat.processing .wo-value { color: #00d4ff; }
.wo-stat.completed .wo-value { color: #00ff88; }

.wo-label {
  font-size: 12px;
  color: #888;
}

.wo-chart {
  display: flex;
  gap: 15px;
  margin-bottom: 15px;
}

.pie-chart {
  width: 80px;
  height: 80px;
}
```

```
    position: relative;
}

.pie-slice {
    transform: rotate(-90deg);
    transform-origin: center;
}

.pie-center {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    text-align: center;
}

.pie-value {
    font-size: 16px;
    font-weight: bold;
    color: #00ff88;
    display: block;
}

.pie-label {
    font-size: 11px;
    color: #888;
}

.wo-legend {
    flex: 1;
    display: flex;
    flex-direction: column;
    justify-content: center;
    gap: 4px;
}

.legend-item {
    display: flex;
    align-items: center;
    gap: 6px;
    font-size: 12px;
}

.legend-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
}

.legend-text {
    flex: 1;
    color: #aaa;
}

.legend-count {
    color: #fff;
}

/* 最近工单 */
.order-list {
    display: flex;
    flex-direction: column;
    gap: 8px;
}
```

```

.order-item {
  padding: 8px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 6px;
}

.order-header {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 5px;
}

.order-priority {
  font-size: 11px;
  padding: 2px 6px;
  border-radius: 3px;
}

.order-priority.紧急 { background: #ff6b6b; color: #fff; }
.order-priority.高 { background: #ff9966; color: #fff; }
.order-priority.中 { background: #ffaa00; color: #000; }
.order-priority.低 { background: #00d4ff; color: #000; }

.order-id {
  font-size: 12px;
  color: #666;
}

.order-title {
  font-size: 13px;
  color: #fff;
  margin-bottom: 5px;
}

.order-meta {
  display: flex;
  justify-content: space-between;
  font-size: 12px;
}

.order-meta span:first-child {
  color: #888;
}

.order-status {
  padding: 2px 6px;
  border-radius: 3px;
}

.order-status.待处理 { background: rgba(255, 170, 0, 0.2); color: #ffaa00; }
.order-status.处理中 { background: rgba(0, 200, 255, 0.2); color: #00d4ff; }
.order-status.已完成 { background: rgba(0, 255, 136, 0.2); color: #00ff88; }
.order-status.已关闭 { background: rgba(255, 255, 255, 0.1); color: #888; }
</style>

```

6. 模块：参考与辅助层

参考与辅助层包括早期独立脚本版本和后端遥测桥接服务，用于保留演进痕迹并支撑外部设备数据接入。

6.1 文件：src/js/cesium.js

- 文件说明：早期 Cesium 独立脚本版本，保留地图能力演进参考。
- 行数统计：544 行

```

import * as Cesium from 'cesium'
import 'cesium/Build/Cesium/Widgets/widgets.css'

// 设置 Cesium Ion 访问令牌
Cesium.Ion.defaultAccessToken =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI1MGE2NjE5OC05YmU5LTRiMTctODYxOC1hZW0YTU0NDJmM2UiLCJpZCI6Mzg5Mzg5LCJpYXQiOiE3NzA3NzE2MjR9.X1fsTR
YlQkm1FzS3Z-rGNLnchNdPlNqZUfzdX4SHtWU'

// 全局变量
let viewer
let beaconPoints = [] // 存储所有信标点位
let beaconPopups = [] // 存储每个信标点的弹窗元素
let selectedBeacon = null // 当前选中的信标

// XX火车站坐标（地点已脱敏）
const LIUZHOU_STATION = {
  lon: 109.38871, // 经度
  lat: 24.30755, // 纬度
  height: 500
}

// 初始化 Cesium
export async function initCesium() {
  try {
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('正在初始化 Cesium...')
    }
    console.log('开始初始化 Cesium...')
    console.log('目标位置: XX火车站', LIUZHOU_STATION)

    // 创建基础 Viewer 配置
    const viewerOptions = {
      animation: false,
      timeline: false,
      baseLayerPicker: false,
      geocoder: false,
      homeButton: true,
      sceneModePicker: true,
      navigationHelpButton: false,
      fullscreenButton: true,
      infoBox: false,
      selectionIndicator: false
      // 不设置地形, 使用默认的椭球体
    }

    // 创建 Viewer
    viewer = new Cesium.Viewer('cesiumContainer', viewerOptions)

    // 启用 Cesium World Terrain 3D 地形
    viewer.terrainProvider = await Cesium.CesiumTerrainProvider.fromIonAssetId(1, {
      requestVertexNormals: true,
      requestWaterMask: true
    })

    console.log('已启用 Cesium World Terrain 3D 地形')

    // 添加 OSM Buildings 3D 建筑图层
    try {
      if (window.updateLoadingStatus) {
        window.updateLoadingStatus('正在加载 3D 建筑图层...')
      }

      const osmBuildings = await Cesium.createOsmBuildings()
      viewer.scene.primitives.add(osmBuildings)
    }
  }
}

```

```

        console.log('✅ 已添加 OSM Buildings 3D 建筑图层')
    } catch (error) {
        console.warn('⚠️ OSM Buildings 加载失败:', error)
        // 建筑图层加载失败不影响其他功能
    }

    console.log('Cesium Viewer 创建成功')
    if (window.updateLoadingStatus) {
        window.updateLoadingStatus('Viewer 创建成功, 正在添加地图图层...')
    }

    // 开启大气效果
    viewer.scene.skyAtmosphere.show = true
    viewer.scene.skyAtmosphere.hueShift = 0.1
    viewer.scene.skyAtmosphere.saturationShift = 0.1
    viewer.scene.skyAtmosphere.brightnessShift = 0.1

    // 开启雾效
    viewer.scene.fog.enabled = true
    viewer.scene.fog.density = 0.0001

    // 启用地形照明 - 增强地形立体感
    viewer.scene.globe.enableLighting = true

    // 增强地形夸张度 - 使山地地形更明显
    viewer.terrainExaggeration = 3.0 // 地形夸张倍数, 默认为1.0
    console.log('地形夸张度设置为: 3.0倍')

    // 启用基于地形的遮挡检测
    viewer.scene.globe.depthTestAgainstTerrain = true

    // 设置地面透明度 (可选, 设置为1表示不透明)
    viewer.scene.globe.alpha = 1.0

    // 启用动态大气光影
    viewer.scene.globe.atmosphereShift = 0.05

    console.log('Cesium 初始化成功')
    if (window.updateLoadingStatus) {
        window.updateLoadingStatus('初始化完成! ');
    }
    return viewer
} catch (error) {
    console.error('Cesium 初始化失败:', error)
    if (window.updateLoadingStatus) {
        window.updateLoadingStatus('初始化失败: ' + error.message);
    }
    throw error
}
}

// 相机飞入到 XX火车站
export function flyToLiuZhou() {
    if (!viewer) return

    console.log('相机飞入到 XX火车站...')

    // 使用 flyTo 实现平滑的飞入效果
    viewer.camera.flyTo({
        destination: Cesium.Cartesian3.fromDegrees(
            LIUZHOU_STATION.lon,
            LIUZHOU_STATION.lat,
            LIUZHOU_STATION.height // 飞行高度 500米

```

```

    ),
    orientation: {
        heading: Cesium.Math.toRadians(0),      // 朝向正北
        pitch: Cesium.Math.toRadians(-45),      // 俯视角 -45度
        roll: 0.0
    },
    duration: 3.0, // 飞行时长 3秒
    easingFunction: Cesium.EasingFunction.CUBIC_IN_OUT // 缓动函数
  })
}

// 复位视角
window.resetView = function() {
  if (!viewer) return

  viewer.camera.flyTo({
    destination: Cesium.Cartesian3.fromDegrees(
      LIUZHOU_STATION.lon,
      LIUZHOU_STATION.lat,
      LIUZHOU_STATION.height
    ),
    orientation: {
      heading: Cesium.Math.toRadians(0),
      pitch: Cesium.Math.toRadians(-45),
      roll: 0.0
    },
    duration: 2.0
  })
}

// 更新位置信息显示
function updatePositionDisplay() {
  const lonEl = document.getElementById('longitude')
  const latEl = document.getElementById('latitude')
  const altEl = document.getElementById('altitude')

  if (lonEl && latEl && altEl) {
    const cameraPosition = viewer.camera.positionCartographic
    const lon = Cesium.Math.toDegrees(cameraPosition.longitude).toFixed(4)
    const lat = Cesium.Math.toDegrees(cameraPosition.latitude).toFixed(4)
    const alt = (cameraPosition.height * 1000).toFixed(0)

    lonEl.textContent = lon + '°E'
    latEl.textContent = lat + '°N'
    altEl.textContent = alt + 'm'
  }
}

// 更新坐标显示
const coordinatesEl = document.getElementById('coordinates')
if (coordinatesEl) {
  const carto = viewer.camera.positionCartographic
  const lon = Cesium.Math.toDegrees(carto.longitude).toFixed(4)
  const lat = Cesium.Math.toDegrees(carto.latitude).toFixed(4)
  coordinatesEl.textContent = `${lon}, ${lat}`
}
}

// 隐藏弹窗函数
window.hidePopup = function(beaconId) {
  if (beaconPopups[beaconId]) {
    beaconPopups[beaconId].style.display = 'none'
  }
  selectedBeacon = null
}

```

```

// 设置交互事件
function setupInteractions() {
    // 添加点击事件监听器
    const handler = new Cesium.ScreenSpaceEventHandler(viewer.scene.canvas)

    handler.setInputAction(function(click) {
        // 拾取场景中的对象
        const pickedObject = viewer.scene.pick(click.position, 0, 0)

        if (Cesium.defined(pickedObject)) {
            // 检查是否点击了信标
            for (let i = 0; i < beaconPoints.length; i++) {
                const beacon = beaconPoints[i]
                if (pickedObject.id === beacon.cylinder.id ||
                    pickedObject.id === beacon.point.id ||
                    pickedObject.id === beacon.wave.id) {
                    // 切换弹窗显示
                    const popup = beaconPopups[i]
                    if (selectedBeacon === i) {
                        // 如果已选中，则隐藏
                        popup.style.display = 'none'
                        selectedBeacon = null
                    } else {
                        // 隐藏其他弹窗
                        beaconPopups.forEach((p, idx) => {
                            if (idx !== i) p.style.display = 'none'
                        })
                        // 显示当前弹窗
                        popup.style.display = 'block'
                        selectedBeacon = i
                    }
                    break
                }
            }
        } else {
            // 点击空白处，隐藏所有弹窗
            beaconPopups.forEach(p => p.style.display = 'none')
            selectedBeacon = null
        }
    }, Cesium.ScreenSpaceEventType.LEFT_CLICK)

    // 监听场景渲染事件，更新弹窗位置和相机信息
    viewer.scene.postRender.addEventListener(() => {
        updatePositionDisplay()
        updatePopupPositions()
    })
}

// 更新弹窗位置，让它们跟随信标移动
function updatePopupPositions() {
    beaconPoints.forEach((beacon, index) => {
        const popup = beaconPopups[index]
        if (!popup || popup.style.display === 'none') {
            return // 弹窗不存在或隐藏，跳过
        }

        // 转换3D坐标到屏幕坐标
        const position = Cesium.Cartesian3.fromDegrees(beacon.lon, beacon.lat, beacon.basePosition.height)
        const windowCoord = Cesium.SceneTransforms.wgs84ToWindowCoordinates(
            viewer.scene,
            position
        )
    })
}

```

```

    if (Cesium.defined(windowCoord)) {
        const x = windowCoord.x - popup.offsetWidth / 2
        const y = windowCoord.y - popup.offsetHeight - 50 // 向上偏移, 避免遮挡信标
        popup.style.transform = `translate3d(${Math.round(x)}px, ${Math.round(y)}px, 0)`
    }
})
}

```

// 更新仪表盘数据

```

function updateDashboardData() {
    const lastUpdate = document.getElementById('lastUpdate')
    if (lastUpdate) {
        const now = new Date()
        lastUpdate.textContent = now.toLocaleString('zh-CN')
    }
}

```

// 创建随机发光柱和光波动画

```

function createBeaconPoints() {
    // 在目标区域周边随机生成5-8个点
    const pointCount = Math.floor(Math.random() * 4) + 5 // 5-8个点

    for (let i = 0; i < pointCount; i++) {
        // 在目标站点周围随机分布
        const lon = LIUZHOU_STATION.lon + (Math.random() - 0.5) * 0.1 // ±0.05度
        const lat = LIUZHOU_STATION.lat + (Math.random() - 0.5) * 0.1 // ±0.05度
        const height = 200 + Math.random() * 300 // 200-500米高度

        const position = Cesium.Cartesian3.fromDegrees(lon, lat, height)

        // 生成随机事件提醒信息
        const eventTypes = ['设备正常', '温度异常', '维护中', '离线', '电压异常']
        const eventMessages = [
            '信号灯运行正常, 所有参数在范围内',
            '温度超过阈值, 当前45°C, 请检查设备',
            '设备正在维护, 预计2小时后恢复',
            '设备离线, 最后检查时间: 10分钟前',
            '电压异常, 当前220V, 需要检查线路'
        ]
        const randomEventIndex = Math.floor(Math.random() * eventTypes.length)
        const eventType = eventTypes[randomEventIndex]
        const eventMessage = eventMessages[randomEventIndex]
        const eventTime = new Date().toLocaleString('zh-CN')
    }
}

```

// 1. 创建发光柱（圆柱体）

```

const cylinder = viewer.entities.add({
    position: position,
    cylinder: {
        length: height,
        topRadius: 2,
        bottomRadius: 2,
        material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.6),
        outline: true,
        outlineColor: Cesium.Color.CYAN,
        outlineWidth: 2
    }
})

```

// 2. 创建顶部发光点

```

const point = viewer.entities.add({
    position: position,
    point: {
        pixelSize: 15,
        color: Cesium.Color.CYAN.withAlpha(0.8),
    }
})

```

```

        outlineColor: Cesium.Color.WHITE,
        outlineWidth: 2
    }
})

// 3. 创建光波动画（圆形扩散） - 定位在地面
const groundPosition = Cesium.Cartesian3.fromDegrees(lon, lat, 0) // 地面位置

const wave = viewer.entities.add({
    position: groundPosition, // 放置在地面而不是顶部
    ellipse: {
        semiMinorAxis: 10, // 初始半径较小
        semiMajorAxis: 10,
        height: 2, // 稍微离地一点，避免z-fighting
        material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.6), // 初始更不透明
        outline: true,
        outlineColor: Cesium.Color.CYAN.withAlpha(0.8),
        outlineWidth: 3,
        rotation: Cesium.Math.toRadians(Math.random() * 360)
    }
})

// 保存到数组中，用于动画更新
beaconPoints.push({
    cylinder: cylinder,
    point: point,
    wave: wave,
    basePosition: { lon, lat, height },
    waveRadius: 10, // 初始半径
    maxRadius: 200, // 最大扩散半径
    waveSpeed: 20 + Math.random() * 30, // 扩散速度（米/秒）
    waveAlpha: 0.6, // 初始透明度
    waveStartTime: Date.now() + Math.random() * 2000, // 随机延迟启动
    id: i, // 信标ID
    lon: lon, // 保存坐标用于弹窗
    lat: lat,
    eventType: eventType, // 事件类型
    eventMessage: eventMessage // 事件消息
})

// 创建事件提醒弹窗
const popupDiv = document.createElement('div')
popupDiv.className = 'beacon-popup'
popupDiv.innerHTML = `
    <div class="popup-content">
        <div class="popup-title">🚦 信号灯 ${i + 1} - ${eventType}</div>
        <div class="popup-time">🕒 ${eventTime}</div>
        <div class="popup-message">${eventMessage}</div>
        <div class="popup-coord">📍 坐标: ${lon.toFixed(4)}, ${lat.toFixed(4)}</div>
        <div class="popup-close" onclick="event.stopPropagation(); hidePopup(${i})">✕</div>
    </div>
`

popupDiv.style.cssText = `
    position: absolute;
    left: 0;
    top: 0;
    display: none;
    z-index: 1000;
    pointer-events: auto;
`

viewer.container.appendChild(popupDiv)
beaconPopups.push(popupDiv)

```

```

    console.log(`创建信标点 ${i + 1}: [${lon.toFixed(4)}, ${lat.toFixed(4)}, ${height.toFixed(0)}m]`)
  }

// 添加CSS样式到页面
const style = document.createElement('style')
style.textContent = `
.beacon-popup {
  background: rgba(0, 20, 40, 0.95);
  border: 2px solid rgba(0, 200, 255, 0.6);
  border-radius: 8px;
  padding: 15px;
  min-width: 280px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.5);
  backdrop-filter: blur(10px);
  font-family: 'Microsoft YaHei', Arial, sans-serif;
}
.popup-content {
  color: #fff;
}
.popup-title {
  font-size: 16px;
  font-weight: bold;
  color: #00d4ff;
  margin-bottom: 10px;
  padding-bottom: 8px;
  border-bottom: 1px solid rgba(0, 200, 255, 0.3);
}
.popup-time {
  font-size: 12px;
  color: #aaa;
  margin-bottom: 8px;
}
.popup-message {
  font-size: 13px;
  line-height: 1.5;
  margin-bottom: 10px;
}
.popup-coord {
  font-size: 11px;
  color: #888;
  font-family: monospace;
}
.popup-close {
  position: absolute;
  top: 8px;
  right: 8px;
  cursor: pointer;
  font-size: 18px;
  color: #ff6666;
  background: rgba(255, 255, 255, 0.1);
  border-radius: 4px;
  width: 24px;
  height: 24px;
  display: flex;
  align-items: center;
  justify-content: center;
}
.popup-close:hover {
  background: rgba(255, 255, 255, 0.2);
  color: #ff3333;
}
`

document.head.appendChild(style)
}

```

```

// 更新光波动画 - 像地震波一样从中心向外发散
function updateWaveAnimation() {
  const currentTime = Date.now()

  beaconPoints.forEach((beacon, index) => {
    // 检查是否到了该光波的启动时间
    if (currentTime < beacon.waveStartTime) {
      return // 还没到启动时间, 跳过
    }

    // 计算从启动开始经过的时间 (秒)
    const elapsedTime = (currentTime - beacon.waveStartTime) * 0.001

    // 计算当前半径: 从10米开始向外扩散
    const currentRadius = beacon.waveRadius + elapsedTime * beacon.waveSpeed

    // 如果超过最大半径, 重置 (循环效果)
    if (currentRadius > beacon.maxRadius) {
      beacon.waveStartTime = currentTime // 重置启动时间
      beacon.wave.ellipse.semiMinorAxis = beacon.waveRadius
      beacon.wave.ellipse.semiMajorAxis = beacon.waveRadius
      beacon.wave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(beacon.waveAlpha)
    } else {
      // 更新光波半径 - 从中心向外扩散
      beacon.wave.ellipse.semiMinorAxis = currentRadius
      beacon.wave.ellipse.semiMajorAxis = currentRadius

      // 透明度随半径增大而减小 (扩散越远越透明)
      const progress = (currentRadius - beacon.waveRadius) / (beacon.maxRadius - beacon.waveRadius)
      const newAlpha = beacon.waveAlpha * (1 - progress * 0.9) // 从0.6渐变到0.06
      beacon.wave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(newAlpha)
    }
  })

  // 顶部发光点脉冲效果 (独立于光波)
  const pulseTime = currentTime * 0.003
  const pulseSize = 12 + Math.sin(pulseTime + index * 2) * 5 // 7-17像素
  beacon.point.point.pixelSize = pulseSize
})
}

// 主初始化函数
export async function init() {
  try {
    console.log('=== Cesium 大地图初始化开始 ===')

    // 初始化 Cesium
    await initCesium()

    // 创建随机发光柱和光波
    createBeaconPoints()

    // 相机飞入到 XX火车站
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('正在飞向XX火车站...')
    }

    setTimeout(() => {
      flyToLiuZhou()
    }, 500)

    // 设置交互
    setupInteractions()
  }
}

```

```

// 启动光波动画循环
viewer.clock.onTick.addEventListener(updateWaveAnimation)

// 定时更新仪表盘数据
setInterval(updateDashboardData, 1000)

// 隐藏加载动画
setTimeout(() => {
  const loading = document.getElementById('loading')
  if (loading) loading.classList.add('hidden')
  console.log('=== Cesium 大地图初始化完成 ===')
}, 2000)

} catch (error) {
  console.error('初始化失败:', error)
  if (window.updateLoadingStatus) {
    window.updateLoadingStatus('初始化失败: ' + error.message);
  }
  const loading = document.getElementById('loading')
  if (loading) {
    loading.innerHTML = `
      <div style="color: #ff6666;">
        <div style="font-size: 24px; margin-bottom: 20px;">✖ 加载失败</div>
        <div style="font-size: 14px;">${error.message}</div>
        <div style="font-size: 12px; margin-top: 20px;">请打开浏览器控制台查看详细错误信息</div>
      </div>
    `
  }
}
}
}

// 启动应用
init()

```

6.2 文件: src/js/threejs.js

- 文件说明: 早期 Three.js 独立脚本版本, 保留三维孪生能力演进参考。
- 行数统计: 726 行

```

import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
import { CSS2DRenderer } from 'three/examples/jsm/renderers/CSS2DRenderer.js'

// 全局变量
let scene, camera, renderer, controls, labelRenderer
let signals = []
let train = null
let isTrainRunning = false
let autoRotate = false
let clock = new THREE.Clock()
let startTime = Date.now()
let mixer = null // 动画混合器
let gltfLoader = null
let modelLabels = [] // 存储所有模型标签

// 进度日志节流: 减少“加载进度”刷屏
function createProgressLogger(label, step = 25) {
  let lastBucket = -1
  return (progress) => {
    if (!progress || !progress.total) return
    const percent = Math.floor((progress.loaded / progress.total) * 100)
    const bucket = Math.floor(percent / step) * step
    if (bucket === lastBucket) return
    lastBucket = bucket
    console.log(` ${label} 加载进度: ${percent}%`)
  }
}

```

```

    }
}

// 初始化 Three.js
function init() {
    // 创建场景
    scene = new THREE.Scene()
    scene.background = new THREE.Color(0xf0f0f0) // 改为白色背景
    scene.fog = new THREE.Fog(0xf0f0f0, 100, 800) // 雾效也改为白色

    // 创建相机
    camera = new THREE.PerspectiveCamera(
        60,
        window.innerWidth / window.innerHeight,
        0.1,
        2000
    )
    camera.position.set(80, 60, 80)

    // 创建渲染器
    renderer = new THREE.WebGLRenderer({ antialias: true })
    renderer.setSize(window.innerWidth, window.innerHeight)
    renderer.setPixelRatio(window.devicePixelRatio)
    renderer.shadowMap.enabled = true
    renderer.shadowMap.type = THREE.PCFSoftShadowMap
    document.getElementById('threeContainer').appendChild(renderer.domElement)

    // 创建CSS2D渲染器用于HTML标签
    labelRenderer = new CSS2DRenderer()
    labelRenderer.setSize(window.innerWidth, window.innerHeight)
    labelRenderer.domElement.style.position = 'absolute'
    labelRenderer.domElement.style.top = '0'
    labelRenderer.domElement.style.pointerEvents = 'none'
    document.body.appendChild(labelRenderer.domElement)

    // 创建控制器
    controls = new OrbitControls(camera, renderer.domElement)
    controls.enableDamping = true
    controls.dampingFactor = 0.05
    controls.maxPolarAngle = Math.PI / 2
    controls.minDistance = 20
    controls.maxDistance = 200

    // 添加光照
    setupLights()

    // 创建场景
    createGround()
    createMountains()
    createRailway()
    createSignalLights()
    createTrain() // 恢复火车头
    // createStation() // 已禁用 - 移除火车站
    createTrees()

    // 事件监听
    window.addEventListener('resize', onWindowResize)

    // 隐藏加载动画
    setTimeout(() => {
        const loading = document.getElementById('loading')
        if (loading) loading.classList.add('hidden')
    }, 1000)
}

```

```
// 开始动画
animate()

// 更新 UI
updateUI()
setInterval(updateUI, 1000)

console.log('Three.js 小地图初始化完成')
}

// 设置光照
function setupLights() {
  // 环境光 - 增强亮度
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.8)
  scene.add(ambientLight)

  // 主光源 - 增强亮度
  const mainLight = new THREE.DirectionalLight(0xffffff, 1.5)
  mainLight.position.set(50, 100, 50)
  mainLight.castShadow = true
  mainLight.shadow.camera.left = -100
  mainLight.shadow.camera.right = 100
  mainLight.shadow.camera.top = 100
  mainLight.shadow.camera.bottom = -100
  mainLight.shadow.mapSize.width = 2048
  mainLight.shadow.mapSize.height = 2048
  scene.add(mainLight)

  // 补光 - 改为暖色
  const fillLight = new THREE.DirectionalLight(0xffeedd, 0.5)
  fillLight.position.set(-50, 50, -50)
  scene.add(fillLight)

  // 半球光 - 提供更自然的照明
  const hemiLight = new THREE.HemisphereLight(0xffffff, 0x444444, 0.6)
  hemiLight.position.set(0, 200, 0)
  scene.add(hemiLight)
}

// 创建地面
function createGround() {
  const groundGeometry = new THREE.PlaneGeometry(500, 500, 50, 50)
  const groundMaterial = new THREE.MeshStandardMaterial({
    color: 0x7a8a6a, // 浅绿色地面
    roughness: 0.9,
    metalness: 0.1
  })

  // 添加地形起伏
  const vertices = groundGeometry.attributes.position.array
  for (let i = 0; i < vertices.length; i += 3) {
    const x = vertices[i]
    const z = vertices[i + 2]
    vertices[i + 1] = Math.sin(x * 0.05) * Math.cos(z * 0.05) * 5
  }
  groundGeometry.computeVertexNormals()

  const ground = new THREE.Mesh(groundGeometry, groundMaterial)
  ground.rotation.x = -Math.PI / 2
  ground.receiveShadow = true
  scene.add(ground)
}

// 创建山脉
```

```

function createMountains() {
  const mountainGeometry = new THREE.ConeGeometry(30, 60, 8)
  const mountainMaterial = new THREE.MeshStandardMaterial({
    color: 0x8a9a8a, // 浅灰色山脉
    roughness: 0.95,
    metalness: 0.05
  })

  const positions = [
    { x: -80, z: -80 },
    { x: -100, z: 50 },
    { x: 80, z: -100 },
    { x: 100, z: 60 },
    { x: -60, z: 120 }
  ]

  positions.forEach(pos => {
    const mountain = new THREE.Mesh(mountainGeometry, mountainMaterial)
    mountain.position.set(pos.x, 30, pos.z)
    mountain.scale.y = 1 + Math.random() * 0.5
    mountain.castShadow = true
    mountain.receiveShadow = true
    scene.add(mountain)
  })
}

// 创建铁道（加载 GLB 模型并拼接）
function createRailway() {
  if (!glTFLoader) {
    glTFLoader = new GLTFLoader()
  }

  // 定义铁道路径点 - 9段轨道在一直线上，首尾相接，统一旋转50度
  const railPositions = [
    { x: -80, z: -60, rotation: Math.PI / 3.6 }, // 轨道1
    { x: -60, z: -45, rotation: Math.PI / 3.6 }, // 轨道2
    { x: -40, z: -30, rotation: Math.PI / 3.6 }, // 轨道3
    { x: -20, z: -15, rotation: Math.PI / 3.6 }, // 轨道4
    { x: 0, z: 0, rotation: Math.PI / 3.6 }, // 轨道5（中心）
    { x: 20, z: 15, rotation: Math.PI / 3.6 }, // 轨道6
    { x: 40, z: 30, rotation: Math.PI / 3.6 }, // 轨道7
    { x: 60, z: 45, rotation: Math.PI / 3.6 }, // 轨道8
    { x: 80, z: 60, rotation: Math.PI / 3.6 } // 轨道9
  ]

  // 加载并放置多个铁轨段
  railPositions.forEach((pos, index) => {
    const logProgress = createProgressLogger(`铁轨段 ${index + 1}`, 25)
    glTFLoader.load(
      '/assets/models/railway.glb',
      (glTF) => {
        const railway = glTF.scene
        railway.scale.set(12, 12, 12) // 放大12倍
        railway.position.set(pos.x, 0, pos.z) // 放在地平线上
        railway.rotation.y = pos.rotation

        // 启用阴影
        railway.traverse((child) => {
          if (child.isMesh) {
            child.castShadow = true
            child.receiveShadow = true
          }
        })
      })
  })
}

```

```

        scene.add(railway)
        console.log(`铁轨段 ${index + 1} 加载成功`)
    },
    (progress) => {
        logProgress(progress)
    },
    (error) => {
        console.error(`铁轨段 ${index + 1} 加载失败:`, error)
    }
)
})

// 创建路径用于火车运动
const curve = new THREE.CatmullRomCurve3([
    new THREE.Vector3(-60, 0.5, -40),
    new THREE.Vector3(-30, 0.5, -20),
    new THREE.Vector3(0, 0.5, 0),
    new THREE.Vector3(30, 0.5, 20),
    new THREE.Vector3(60, 0.5, 40)
])

return curve
}

// 创建信号灯（加载 GLB 模型）
function createSignalLights() {
    if (!glTFLoader) {
        glTFLoader = new GLTFLoader()
    }

    const signalPositions = [
        { x: -40, z: -25 },
        { x: -10, z: -5 },
        { x: 20, z: 15 },
        { x: 40, z: 30 }
    ]

    const signalStates = ['red', 'green', 'yellow', 'green']
    const signalNames = ['进站信号', '出站信号', '区间信号1', '区间信号2']

    signalPositions.forEach((pos, index) => {
        const logProgress = createProgressLogger(`信号灯 ${signalNames[index]}`, 25)
        glTFLoader.load(
            '/assets/models/sign.glb',
            (glTF) => {
                const signGroup = glTF.scene
                signGroup.scale.set(8, 8, 8) // 放大8倍
                signGroup.position.set(pos.x, 0, pos.z) // 放在地平线上

                // 启用阴影
                signGroup.traverse((child) => {
                    if (child.isMesh) {
                        child.castShadow = true
                        child.receiveShadow = true
                    }
                })
            })

        scene.add(signGroup)

        // 添加点光源（根据信号灯状态）
        const color = getColorByState(signalStates[index])
        const light = new THREE.PointLight(color, 3, 40) // 增强光源强度和范围
        light.position.set(pos.x, 5, pos.z)
        scene.add(light)
    })
}

```

```

    signals.push({
      mesh: signGroup,
      light: light,
      state: signalStates[index],
      name: signalNames[index]
    })

    // 创建3D标签
    createModellabel(signGroup, signalNames[index], 15, 65, 45, '109.3887, 24.3076')

    updateSignalUI()
    console.log(`信号灯 ${signalNames[index]} 加载成功`)
  },
  (progress) => {
    logProgress(progress)
  },
  (error) => {
    console.error(`信号灯 ${signalNames[index]} 加载失败:`, error)
  }
)
})
}

```

```

// 根据状态获取颜色
function getColorByState(state) {
  switch (state) {
    case 'red': return 0xff3333
    case 'green': return 0x33ff33
    case 'yellow': return 0xffff33
    default: return 0x666666
  }
}

```

```

// 创建列车（加载 GLB 模型）
function createTrain() {
  if (!glTFLoader) {
    glTFLoader = new GLTFLoader()
  }
}

```

```

const logProgress = createProgressLogger('火车头', 25)
glTFLoader.load(
  '/assets/models/locomotive.glb',
  (glTF) => {
    train = glTF.scene
    train.scale.set(12, 12, 12) // 放大12倍
    train.position.set(-60, 0.5, -40) // 稍微抬高，放在铁轨上
    train.rotation.y = Math.PI / 2 // 调整朝向
  }
)

```

```

// 启用阴影
train.traverse((child) => {
  if (child.isMesh) {
    child.castShadow = true
    child.receiveShadow = true
  }
})

```

```

scene.add(train)

```

```

// 如果有动画，设置动画混合器
if (glTF.animations && glTF.animations.length > 0) {
  mixer = new THREE.AnimationMixer(train)
  const action = mixer.clipAction(glTF.animations[0])
  action.play()
}

```

```

    }

    // 创建3D标签
    createModelLabel(train, '火车头', -5, 75, 60, '109.3900, 24.3100')

    console.log('火车头模型加载成功')
  },
  (progress) => {
    logProgress(progress)
  },
  (error) => {
    console.error('火车头模型加载失败:', error)
    // 如果模型加载失败, 使用简单的几何体作为后备
    createFallbackTrain()
  }
)
}

// 后备火车 (如果 GLB 加载失败)
function createFallbackTrain() {
  const trainGroup = new THREE.Group()

  // 车身
  const bodyGeometry = new THREE.BoxGeometry(8, 3, 3)
  const bodyMaterial = new THREE.MeshStandardMaterial({ color: 0x2244aa })
  const body = new THREE.Mesh(bodyGeometry, bodyMaterial)
  body.position.y = 2
  body.castShadow = true
  trainGroup.add(body)

  // 车头
  const headGeometry = new THREE.BoxGeometry(2, 2.5, 2.8)
  const headMaterial = new THREE.MeshStandardMaterial({ color: 0x3355bb })
  const head = new THREE.Mesh(headGeometry, headMaterial)
  head.position.set(5, 2, 0)
  head.castShadow = true
  trainGroup.add(head)

  // 车窗
  const windowGeometry = new THREE.BoxGeometry(0.1, 1, 2)
  const windowMaterial = new THREE.MeshStandardMaterial({
    color: 0x88ccff,
    emissive: 0x4488cc,
    emissiveIntensity: 0.3
  })
  const trainWindow = new THREE.Mesh(windowGeometry, windowMaterial)
  trainWindow.position.set(6.1, 2.5, 0)
  trainGroup.add(trainWindow)

  // 车轮
  const wheelGeometry = new THREE.CylinderGeometry(0.6, 0.6, 0.3, 16)
  const wheelMaterial = new THREE.MeshStandardMaterial({ color: 0x222222 })
  const wheelPositions = [[-3, 0.6, 1.5], [-3, 0.6, -1.5], [3, 0.6, 1.5], [3, 0.6, -1.5]]

  wheelPositions.forEach(pos => {
    const wheel = new THREE.Mesh(wheelGeometry, wheelMaterial)
    wheel.position.set(...pos)
    wheel.rotation.x = Math.PI / 2
    wheel.castShadow = true
    trainGroup.add(wheel)
  })

  train = trainGroup
  train.position.set(-60, 0, -40)
}

```

```
    scene.add(train)
  }

// 创建火车站（加载 GLB 模型）
function createStation() {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }

  const logProgress = createProgressLogger('火车站', 25)
  gltfLoader.load(
    '/assets/models/station.glb',
    (gltf) => {
      const station = gltf.scene
      station.scale.set(20, 20, 20) // 放大20倍
      station.position.set(30, 0, 20) // 在地平线上，与铁轨、信号灯同高
      station.rotation.y = -Math.PI / 4 // 调整朝向

      // 启用阴影
      station.traverse((child) => {
        if (child.isMesh) {
          child.castShadow = true
          child.receiveShadow = true
        }
      })

      scene.add(station)
      console.log('火车站模型加载成功')

      // 创建3D标签
      createModelLabel(station, 'XX火车站', 30, -5, 85, 'X', 'Y')
    },
    (progress) => {
      logProgress(progress)
    },
    (error) => {
      console.error('火车站模型加载失败:', error)
    }
  )
}

// 创建树木
function createTrees() {
  const treeCount = 30

  for (let i = 0; i < treeCount; i++) {
    const treeGroup = new THREE.Group()

    // 树干
    const trunkGeometry = new THREE.CylinderGeometry(0.3, 0.5, 3, 8)
    const trunkMaterial = new THREE.MeshStandardMaterial({ color: 0x6a5a4a })
    const trunk = new THREE.Mesh(trunkGeometry, trunkMaterial)
    trunk.position.y = 1.5
    trunk.castShadow = true
    treeGroup.add(trunk)

    // 树冠
    const leavesGeometry = new THREE.ConeGeometry(2, 4, 8)
    const leavesMaterial = new THREE.MeshStandardMaterial({ color: 0x4a8a4a })
    const leaves = new THREE.Mesh(leavesGeometry, leavesMaterial)
    leaves.position.y = 5
    leaves.castShadow = true
    treeGroup.add(leaves)
  }
}
```

```

// 随机位置
const x = (Math.random() - 0.5) * 200
const z = (Math.random() - 0.5) * 200
treeGroup.position.set(x, 0, z)

scene.add(treeGroup)
}
}

// 创建3D模型的HTML标签
function createModelLabel(model, name, temperature, humidity, gpsLon, gpsLat) {
  // 创建标签容器div
  const div = document.createElement('div')
  div.className = 'model-label'

  // 根据温度确定颜色
  let tempClass = 'temp-low'
  if (temperature < 0) tempClass = 'temp-low'
  else if (temperature < 20) tempClass = 'temp-medium'
  else tempClass = 'temp-high'

  // 计算温度进度条宽度
  const tempPercent = Math.min(Math.abs(temperature) / 40 * 100, 100)

  div.innerHTML = `
    <div class="label-title">${name}</div>
    <div class="label-row">
      <span>🔥 温度:</span>
      <span class="label-value">${temperature}°C</span>
    </div>
    <div class="label-row">
      <span>💧 湿度:</span>
      <span class="label-value">${humidity}%</span>
    </div>
    <div class="label-row">
      <span>📶 GPS:</span>
      <span class="label-value">${gpsLon}, ${gpsLat}</span>
    </div>
    <div class="label-row">
      <span style="flex: 1; margin-left: 10px;">
        <span>温度:</span>
        <span style="flex: 1; margin-left: auto;">
          <div class="progress-bg">
            <div class="temp-bar ${tempClass}" style="width: ${tempPercent}%></div>
          </div>
        </span>
      </span>
    </div>
  `

  // 添加到场景和标签渲染器
  const label = new THREE.CSS2DObject(div)
  label.position.set(0, 0, 0)
  model.add(label)
  modelLabels.push({ object: model, label: label })
}

// 切换信号灯
window.toggleSignals = function() {
  const states = ['red', 'green', 'yellow']
  signals.forEach(signal => {
    const currentStateIndex = states.indexOf(signal.state)
    signal.state = states[(currentStateIndex + 1) % states.length]
    const color = getColorByState(signal.state)
  })
}

```

```

// 更新光源颜色
signal.light.color.setHex(color)

// 尝试更新模型中 mesh 的颜色（如果模型支持）
if (signal.mesh) {
  signal.mesh.traverse((child) => {
    if (child.isMesh) {
      // 尝试识别信号灯部分（通常名称包含 'light' 或 'signal'）
      if (child.material) {
        // 如果材质有 emissive 属性，修改它
        if (child.material.emissive) {
          child.material.emissive.setHex(color)
        }
        // 如果材质有 color 属性，也修改它
        if (child.material.color) {
          child.material.color.setHex(color)
        }
      }
    }
  })
}
updateSignalUI()
}

// 更新信号灯 UI
function updateSignalUI() {
  const signalList = document.getElementById('signalList')
  if (!signalList) return

  const stateText = {
    'red': '禁止通行',
    'green': '允许通行',
    'yellow': '减速通行'
  }

  signalList.innerHTML = signals.map(signal => `
    <div class="signal-item">
      <div class="signal-light signal-${signal.state}"></div>
      <div class="signal-info">
        <div class="signal-name">${signal.name}</div>
        <div class="signal-status">${stateText[signal.state]}</div>
      </div>
    </div>
  `).join('')

  const countEl = document.getElementById('signalCount')
  if (countEl) countEl.textContent = signals.length
}

// 列车动画
window.trainAnimation = function() {
  isTrainRunning = !isTrainRunning
}

// 自动旋转
window.toggleRotation = function() {
  autoRotate = !autoRotate
}

// 复位视角
window.resetView = function() {
  camera.position.set(80, 60, 80)
}

```

```

controls.target.set(0, 0, 0)
controls.update()
}

// 切换摄像头
window.switchCamera = function(cameraId) {
  // 移除所有tab的active类
  const tabs = document.querySelectorAll('.camera-tab')
  tabs.forEach(tab => tab.classList.remove('active'))

  // 隐藏所有摄像头内容
  const contents = document.querySelectorAll('.camera-content')
  contents.forEach(content => content.classList.remove('active'))

  // 激活选中的tab和内容
  const selectedTab = document.querySelector(`.camera-tab:nth-child(${cameraId})`)
  const selectedContent = document.getElementById(`camera${cameraId}`)

  if (selectedTab) selectedTab.classList.add('active')
  if (selectedContent) selectedContent.classList.add('active')

  console.log(`切换到监控${cameraId}`)
}

// 窗口大小调整
function onWindowResize() {
  camera.aspect = window.innerWidth / window.innerHeight
  camera.updateProjectionMatrix()
  renderer.setSize(window.innerWidth, window.innerHeight)
  labelRenderer.setSize(window.innerWidth, window.innerHeight)
}

// 更新 UI
function updateUI() {
  // 更新系统状态
  const lastUpdate = document.getElementById('lastUpdate')
  if (lastUpdate) {
    lastUpdate.textContent = new Date().toLocaleString('zh-CN')
  }

  // 更新运行时间
  const uptime = document.getElementById('uptime')
  if (uptime) {
    const seconds = Math.floor((Date.now() - startTime) / 1000)
    uptime.textContent = `${seconds}s`
  }

  // 更新相机信息
  const camPos = document.getElementById('cameraPos')
  if (camPos) {
    camPos.textContent = `${camera.position.x.toFixed(0)}, ${camera.position.y.toFixed(0)}, ${camera.position.z.toFixed(0)}`
  }

  const camTarget = document.getElementById('cameraTarget')
  if (camTarget) {
    camTarget.textContent = `${controls.target.x.toFixed(0)}, ${controls.target.y.toFixed(0)}, ${controls.target.z.toFixed(0)}`
  }

  // 更新性能信息
  const fps = document.getElementById('fps')
  if (fps) {
    fps.textContent = '60 FPS'
  }
}

```

```

const triangles = document.getElementById('triangles')
if (triangles) {
  triangles.textContent = renderer.info.render.triangles.toLocaleString()
}

const drawCalls = document.getElementById('drawCalls')
if (drawCalls) {
  drawCalls.textContent = renderer.info.render.calls
}
}

// 动画循环
function animate() {
  requestAnimationFrame(animate)

  const delta = clock.getDelta()

  // 更新控制器
  controls.update()

  // 自动旋转
  if (autoRotate) {
    controls.autoRotate = true
    controls.autoRotateSpeed = 2.0
  } else {
    controls.autoRotate = false
  }

  // 列车动画
  if (isTrainRunning && train) {
    const time = clock.getElapsedTime() * 0.5
    const path = new THREE.CatmullRomCurve3([
      new THREE.Vector3(-60, 0.5, -40),
      new THREE.Vector3(-30, 0.5, -20),
      new THREE.Vector3(0, 0.5, 0),
      new THREE.Vector3(30, 0.5, 20),
      new THREE.Vector3(60, 0.5, 40)
    ])
    const position = path.getPoint((time % 1))
    train.position.copy(position)

    // 计算方向
    const nextPoint = path.getPoint(((time + 0.01) % 1))
    train.lookAt(nextPoint)
  }

  // 更新动画混合器（如果火车模型有动画）
  if (mixer) {
    mixer.update(delta)
  }

  // 渲染
  renderer.render(scene, camera)
  labelRenderer.render(scene, camera) // 渲染HTML标签
}

// 启动应用
init()

```

6.3 文件: server/telemetry-bridge.js

- 文件说明: Node.js 遥测桥接服务, 提供 HTTP 接收与 WebSocket 广播。
- 行数统计: 189 行

```

import http from 'node:http'
import { WebSocketServer } from 'ws'

```

```

const PORT = Number(process.env.TELEMETRY_PORT || 8080)
const WS_PATH = process.env.TELEMETRY_WS_PATH || '/ws'
const UPLOAD_PATH = process.env.TELEMETRY_UPLOAD_PATH || '/upload'
const MAX_BODY_BYTES = Number(process.env.TELEMETRY_MAX_BODY_BYTES || 32 * 1024)

const now = () => new Date().toISOString()
const log = (msg) => console.log(`${now()} [telemetry] ${msg}`)

const writeCommonHeaders = (res) => {
  res.setHeader('Access-Control-Allow-Origin', '*')
  res.setHeader('Access-Control-Allow-Methods', 'POST,OPTIONS,GET')
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type,*')
  res.setHeader('Cache-Control', 'no-store')
}

const safeJsonParse = (text) => {
  try {
    return { ok: true, value: JSON.parse(text) }
  } catch (e) {
    return { ok: false, error: e instanceof Error ? e.message : String(e) }
  }
}

const normalizeTelemetry = (payload) => {
  const distanceCm = payload?.distance_cm
  const distanceCmNum = Number.isFinite(Number(distanceCm)) ? Number(distanceCm) : null
  const distanceM = distanceCmNum == null ? null : distanceCmNum / 100

  const waterActive = payload?.water_active
  const waterActiveNum = Number.isFinite(Number(waterActive)) ? Number(waterActive) : null

  return {
    type: payload?.type || 'telemetry',
    device: payload?.device || 'unknown',
    device_id: payload?.device_id || 'unknown',
    ts_ms: Number.isFinite(Number(payload?.ts_ms)) ? Number(payload.ts_ms) : Date.now(),
    uptime_ms: Number.isFinite(Number(payload?.uptime_ms)) ? Number(payload.uptime_ms) : null,
    wifi_ip: typeof payload?.wifi_ip === 'string' ? payload.wifi_ip : null,
    distance_cm: distanceCmNum == null ? null : Math.round(distanceCmNum * 10) / 10,
    distance_m: distanceM == null ? null : Math.round(distanceM * 100) / 100,
    water_active: waterActiveNum == null ? null : waterActiveNum,
    raw: payload
  }
}

let lastTelemetry = null

const broadcastTelemetry = (msg, meta = {}) => {
  const encoded = JSON.stringify({ topic: 'telemetry', data: msg })
  let sent = 0
  let skipped = 0
  for (const client of wss.clients) {
    if (client.readyState === 1) {
      client.send(encoded)
      sent++
    } else {
      skipped++
    }
  }
  return { sent, skipped, clients: wss.clients.size, ...meta }
}

const server = http.createServer((req, res) => {

```

```

if (!req.url) {
  res.statusCode = 400
  return res.end('Bad request')
}

const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`)
const pathname = url.pathname

if (req.method === 'OPTIONS') {
  writeCommonHeaders(res)
  res.statusCode = 204
  return res.end()
}

if (req.method === 'GET' && pathname === '/telemetry/health') {
  writeCommonHeaders(res)
  res.statusCode = 200
  res.setHeader('Content-Type', 'application/json; charset=utf-8')
  return res.end(
    JSON.stringify({
      ok: true,
      port: PORT,
      ws_path: WS_PATH,
      upload_path: UPLOAD_PATH,
      clients: wss.clients.size,
      last: lastTelemetry
      ? { ts_ms: lastTelemetry.ts_ms, device: lastTelemetry.device, type: lastTelemetry.type }
      : null
    })
  )
}

if (req.method === 'POST' && pathname === UPLOAD_PATH) {
  const remote = req.socket?.remoteAddress || 'unknown'
  const contentType = typeof req.headers['content-type'] === 'string' ? req.headers['content-type'] : 'unknown'
  const contentLength = typeof req.headers['content-length'] === 'string' ? req.headers['content-length'] : 'unknown'
  log(`upload start: from=${remote} ct=${contentType} len=${contentLength}`)
  let body = ''
  let bodyBytes = 0

  req.setEncoding('utf8')
  req.on('data', (chunk) => {
    bodyBytes += Buffer.byteLength(chunk, 'utf8')
    if (bodyBytes > MAX_BODY_BYTES) {
      log(`upload rejected: 413 too large bytes=${bodyBytes} from=${remote}`)
      writeCommonHeaders(res)
      res.statusCode = 413
      res.setHeader('Content-Type', 'application/json; charset=utf-8')
      res.end(JSON.stringify({ ok: false, error: 'payload too large' }))
      req.destroy()
      return
    }
    body += chunk
  })

  req.on('end', () => {
    log(`upload end: bytes=${bodyBytes} from=${remote}`)
    const parsed = safeJsonParse(body || '{}')
    if (!parsed.ok) {
      log(`upload rejected: 400 invalid json from=${remote} detail=${parsed.error}`)
      writeCommonHeaders(res)
      res.statusCode = 400
      res.setHeader('Content-Type', 'application/json; charset=utf-8')
      res.end(JSON.stringify({ ok: false, error: 'invalid json', detail: parsed.error }))
    }
  })
}

```

```

    return
  }

  const msg = normalizeTelemetry(parsed.value)
  lastTelemetry = msg

  const { sent, skipped } = broadcastTelemetry(msg, { from: remote })

  log(
    `upload ok: device=${msg.device} id=${msg.device_id} type=${msg.type} distance_cm=${msg.distance_cm ?? 'null'} water=${msg.water_active ?? 'null'} bytes=${bodyBytes} ws_sent=${sent} ws_skipped=${skipped} clients=${wss.clients.size} from=${remote}`
  )

  writeCommonHeaders(res)
  res.statusCode = 200
  res.setHeader('Content-Type', 'application/json; charset=utf-8')
  res.end(JSON.stringify({ ok: true }))
})

req.on('error', (err) => {
  writeCommonHeaders(res)
  res.statusCode = 500
  res.setHeader('Content-Type', 'application/json; charset=utf-8')
  res.end(JSON.stringify({ ok: false, error: err?.message || 'request error' }))
})

return
}

writeCommonHeaders(res)
res.statusCode = 404
res.setHeader('Content-Type', 'application/json; charset=utf-8')
res.end(JSON.stringify({ ok: false, error: 'Not Found' }))
})

const wss = new WebSocketServer({ server, path: WS_PATH })

wss.on('connection', (ws, req) => {
  log(`ws connected: ${req?.socket?.remoteAddress || 'unknown'} clients=${wss.clients.size}`)
  if (lastTelemetry) {
    ws.send(JSON.stringify({ topic: 'telemetry', data: lastTelemetry }))
  }

  ws.on('message', (data) => {
    // optional ping/pong or client subscriptions
    const text = typeof data === 'string' ? data : data.toString('utf8')
    if (text === 'ping') ws.send('pong')
  })

  ws.on('close', () => {
    log(`ws closed: clients=${wss.clients.size}`)
  })
})

server.listen(PORT, () => {
  log(`listening: http://localhost:${PORT}`)
  log(`upload: POST ${UPLOAD_PATH}`)
  log(`ws: ${WS_PATH}`)
})

```

6.4 文件: package.json

- 文件说明: 项目依赖与开发命令配置。
- 行数统计: 42 行

```

{
  "name": "railway_sign",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "telemetry": "node server/telemetry-bridge.js",
    "dev": "vite",
    "dev:all": "concurrently -n telemetry,web -c magenta,cyan \"npm:telemetry\" \"npm:dev\"",
    "build": "vite build",
    "preview": "vite preview",
    "softdocs": "node scripts/build-soft-copyright-pdfs.mjs"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "cesium": "^1.137.0",
    "echarts": "^6.0.0",
    "gsap": "^3.14.2",
    "lil-gui": "^0.21.0",
    "mqtt": "^5.14.1",
    "three": "^0.182.0",
    "vue": "^3.5.28",
    "ws": "^8.19.0"
  },
  "devDependencies": {
    "@pdf-lib/fontkit": "^1.1.1",
    "@vitejs/plugin-vue": "^6.0.4",
    "concurrently": "^9.2.0",
    "dompurify": "^3.3.2",
    "jsdom": "^28.1.0",
    "marked": "^17.0.4",
    "mermaid": "^11.13.0",
    "pdf-lib": "^1.17.1",
    "typescript": "^5.9.3",
    "vite": "^7.3.1",
    "vite-plugin-cesium": "^1.2.23"
  }
}

```

7. 总行数统计说明

本次纳入整理的真实文件共 20 个，总计 14361 行；其中 `src` 目录代码量 14098 行，已超过软著材料常见的 3000 行整理需求。

8. 纳入材料说明

- 已纳入：前端核心源码、辅助服务源码、工程配置、入口页面、演示场景配置。
- 未纳入：第三方依赖、构建产物、静态资源文件、无关缓存目录。
- 特别说明：`src/js` 目录为历史独立脚本版本，当前单页应用未直接引用，但属于项目真实原创代码，故纳入材料作为参考与演进说明。